

考虑变物性与径向收缩的药材 热湿传递及烘干时长预测

摘 要

针对圆柱形药材热风烘干过程中温湿分布不均、物性变化与径向收缩问题，本文建立由固定域向收缩域递进的热湿传递模型，预测径向温湿分布及全域达标时刻。采用保形分段三次 Hermite 插值（PCHIP）构造环境与半径的时间输入，以**表面加密有限体积法**离散守恒方程，并用**隐式变阶后向差分法（BDF）**求解。

针对问题一，建立常热物性导热方程与含水率相关的**非线性 Fick 扩散方程**，施加中心零通量和表面对流交换边界。1800 s 时，中心、表面温度分别为 33.5758、36.7879 °C，干基含水率分别为 2.5500、1.5102 kg/kg，表明预热期外层先升温、先失水，中心失水滞后。

针对问题二，引入含水率相关的热物性及温度、含水率共同决定的扩散系数，联合求解**双向耦合**的温度场与水分场。3 h 时中心温度为 49.8494 °C，干基含水率仍为 1.7662 kg/kg，说明温度接近稳定不能代表干燥完成。与冻结初始扩散系数的对照相比，前 3 h 含水率最大差为 **0.3981 kg/kg**，说明扩散系数随状态变化会明显影响水分分布。

针对问题三，假设 4 h 后烘房维持 50 °C 和 0.05 kg/kg，以**全域最大干基含水率**严格低于 0.15 kg/kg 为达标判据，结合**事件监测、局部求根与严格达标复核**，得到烘干时长 **57.4731 h**，中心控制最终达标。后期温度 ± 2 °C 情景使临界时长变化为 -6.08% 至 $+6.65\%$ 。在端面与侧面交换系数相等的设定下，**二维轴对称配对校核**的临界时间比一维提前约 **8 s (0.004%)**，表明该情景下端面对达标时长的影响较小，支持径向简化的合理性。

针对问题四，在**比例径向收缩假设**下，结合给定半径轨迹确定材料速度场，再由干物质与水质量守恒建立**材料坐标模型**，将移动区域映射至固定区间。采用附录 4 物性，得到烘干时长 **51.0877 h**、终时半径 1.2000 cm。同物性、同环境下的固定半径对照时长为 129.8448 h，收缩使模型预测时长缩短 **78.7571 h (60.65%)**，量化了给定收缩轨迹对干燥进程的影响。

参考解对照、网格与时间加密支持数值结果在已检验配置下的一致性，四问水分收支相对残差均小于 10^{-6} 。所报时长为给定输入与模型假设下的预测，尚需内部温湿观测验证，未显式计入的潜热及界面平衡关系仍构成适用限制。

关键词：热湿耦合；变物性；有限体积法；全域达标；径向收缩

一 问题重述

1.1 问题背景

干燥是影响中药材成品品质的重要工序，热风烘干是常见的干燥方式，其过程包括预热平衡与恒温干燥两个阶段。工艺参数选取不当可能造成干燥效率低、能耗高和品质不稳定，因而需要通过数学模型刻画内部传热与水分迁移规律。

在题给变物性关系下，升温会增大水分扩散系数，含水率下降则改变密度、比热容、导热系数和扩散系数。表层与内部的失水进程不同，平均含水率不能直接代表所有位置均已达标。同时，失水引起的尺寸变化会改变实际迁移距离，需要在收缩条件下重新建立传递模型。

1.2 问题提出

某中药材近似为长 25 cm、初始半径 2 cm 的圆柱，初始温度为 28 °C、干基含水率为 2.55 kg/kg。附件 1 记录初期烘房温湿环境，附件 2 记录烘干中的半径变化，题目按问题分别给出物性参数。据此研究以下四个问题。

1. **问题一：**采用附录 2 参数，建立预热平衡阶段的温度与含水率模型，预测前 1800 s 的径向分布。在指定七个时刻、五个半径位置列出结果，并按 1 s、0.1 cm 间隔输出完整数据。
2. **问题二：**烘干过程一般持续 2 至 3 天，预热平衡与恒温干燥阶段的参数有所不同。全程统一采用附录 3 经验式建立温湿模型，给出前 3 h 每 0.5 h 的指定位置结果，并按 1 s、0.1 cm 间隔输出。
3. **问题三：**以药材各处含水率均低于 0.15 kg/kg 为干燥标准，确定累计烘干时长。列出每 6 h、每 0.5 cm 及结束时刻的含水率，并按 60 s、0.1 cm 间隔保存完整结果。
4. **问题四：**利用附件 2 的半径轨迹和附录 4 物性，重新确定尺寸变化条件下的烘干时长。按问题三的时间与距离间隔输出含水率，并在论文表格中单列当前药材表面。

四问依次考察预热传递、全过程变物性、全域干燥终点和收缩条件下的传递规律。完整结果分别保存至 `result1.xlsx` 至 `result4.xlsx`。所有结果表保留四位小数，数值推进和达标判定使用未舍入值。

题目所称“水分浓度”即干基含水率，指单位干物质质量所含的水质量，本文正文统一采用“干基含水率”这一表述，并记为 C ；为便于与题目对照，结果表标题保留题目原用语。

二 问题分析

四问的共同核心是内部水分扩散与表面交换的相互作用。集中参数模型只能给出平均状态，无法满足题目对不同径向位置的输出要求，因此本文保留径向空间分布，按题给物性建立有效模型。Hussain 和 Dincer 研究了圆柱湿物体干燥中的二维热湿传递 [1]，Kaya 等讨论了圆柱湿物体强制对流干燥的数值建模 [2]；本文沿用其圆柱径向建模与第三类边界的思路，物性则全部取自题给经验式。

四问按固定域预热、变物性耦合、全域达标、收缩域传递逐步递进，均从题给初始状态独立积分。烘房环境与第四问半径轨迹的处理见第四节。

2.1 问题一的分析

问题一的重点是描述前 1800 s 由表面向内部发展的温湿梯度。根据环层热量与水分收支建立径向方程，以对流换热和等效传质描述表面交换。附录 2 中热物性恒定、扩散系数只依赖于基含水率，在忽略潜热与水分携热的近似下，两场可分别求解，但水分扩散仍为非线性。

2.2 问题二的分析

问题二与问题一的差别在物性关系和计算时间尺度。附录 3 的密度、比热容、导热系数均随干基含水率变化，扩散系数同时依赖于基含水率和局部温度。干基含水率影响温度传播，温度又影响水分迁移，两个场必须联合求解。升温与失水对扩散系数的作用方向相反，沿实际轨迹相互竞争，因此不能把 D 冻结为单一常数。

2.3 问题三的分析

问题三需判断内部最后一处湿区何时达标。沿用问题二模型延长积分，以径向最大干基含水率严格低于 0.15 kg/kg 作为终止条件，并检查最大值位置和事件时间的数值稳定性。长期环境延续及端面影响分别通过情景分析和轴对称校核评估。

2.4 问题四的分析

问题四同时改变物性与几何，难点在于移动边界下的守恒关系和结果定位。以比例径向收缩假设确定材料运动，再引入材料坐标将移动区域化为固定区间，按当前物理半径输出结果。另设附录 4 物性下的固定半径对照，分离收缩对预测时长的影响。

三 模型假设与符号说明

3.1 模型假设及其用途

- (1) 药材及交换条件沿周向均匀，主模型忽略轴向梯度和端面交换，采用一维径向形式。第三问另作轴对称二维校核，假设物性各向同性、两端环境相同、端面交换系数与侧面相等，见第 7.6 节。
- (2) 初始温度与干基含水率均匀；问题一至三半径与长度固定。问题四长度仍取 0.25 m ，骨架按给定 $R(t)$ 作比例径向收缩，干物质不流失。比例收缩用于由外半径轨迹闭合内部速度场。
- (3) 题给 ρc_p 作为有效体积热容量，内部传递按 Fourier 导热和有效 Fick 扩散描述。热方程不显式包含汽化潜热、水分携焓及机械功，该简化用于在题给数据下封闭模型，其适用限制见第十节。
- (4) 沿用 $h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 和 $h_m = 8 \times 10^{-7} \text{ m/s}$ 作为表面交换系数，同类圆柱干燥数值研究亦采用对流换热与对流传质的第三类边界 [2]。由于题目只给出常数系数，未

给出该药材的吸附等温关系，本文把 $h_m(C_s - C_a)$ 作为题给信息下的经验 Robin 闭合；气相与固相的 kg/kg 对应不同干基载体，故 C_a 不解释为药材的平衡含水率。

- (5) 固定域采用均匀、恒定的参考干物质密度，收缩域按干物质守恒更新该密度。它与用于热方程的经验 $\rho(C)$ 分别记账，不要求二者逐点满足湿密度换算关系。
- (6) 附件 1 仅覆盖前 4 h。鉴于末 1 h 烘房温度和水分量已趋于稳定，问题三、四在 4 h 后采用 $T_a = 50\text{ }^\circ\text{C}$ 、 $C_a = 0.05\text{ kg/kg}$ 的恒定环境延续。第三问检验后期温度、水分量及全程传质系数情景；第四问检验固定给定半径轨迹下的后期温度情景，不据此推断温度改变后的实际收缩轨迹。

问题一的径向近似可作尺度说明。初始长度与直径之比为 6.25，两端总面积与侧面积之比为 0.08；附录 2 给出热扩散率 $\alpha = 0.36/(820 \times 2600) = 1.6886 \times 10^{-7}\text{ m}^2/\text{s}$ ，1800 s 内的传播尺度 $\sqrt{\alpha t} = 0.0174\text{ m}$ ，远小于中截面到端面的距离 0.125 m。这支持短时中截面的径向近似；长时间尺度上的端面影响另由第 7.6 节的二维校核评估。

3.2 主要符号

符号	含义	单位
t, r	入炉后的时间、当前径向距离	s, m
$R, R(t), L$	固定半径、随时间变化的半径、长度	m
T, T_a	药材局部温度、烘房温度	$^\circ\text{C}$
T_K	药材局部绝对温度, $T_K = T + 273.15$	K
C, C_a	药材干基含水率、烘房题给水分量	kg/kg
ρ, c_p, k	有效密度、比热容、导热系数	kg/m ³ , J/(kg·K), W/(m·K)
$B = \rho c_p, D$	有效体积热容量、水分扩散系数	J/(m ³ ·K), m ² /s
h, h_m	对流换热系数、有效传质系数	W/(m ² ·K), m/s
s, s_0, v	当前体积干物质密度、初始参考干物质密度、骨架径向速度	kg/m ³ , kg/m ³ , m/s
ξ, U, Θ	材料坐标、该坐标下的干基含水率和温度	1, kg/kg, $^\circ\text{C}$
W_i, w_i	物理域、材料域控制体的径向权重	m ² , 1
M, t_*, t_{rep}	全域最大干基含水率、临界时刻、报告时刻	kg/kg, s, s
$\varepsilon_{C,\text{bal}}$	问题一水分收支相对残差	1
$\varepsilon_{C,\text{margin}}$	问题三阈值经验裕量	kg/kg

下标 s 表示当前表面， i 表示内部计算节点。正文中的时间公式以 s 为单位，时长结果及表 3 至表 8 的时间列换算为 h。

四 数据预处理

为使四问模型使用一致的输入口径，本节统一整理附件 1 的烘房温湿环境和附件 2 的药材半径数据，将离散观测转换为数值求解所需的时间函数，并明确长期计算中的环境延续范围。

4.1 数据整理与单位统一

附件 1 给出时间、烘房温度和水分量，共 241 组观测，覆盖 0 至 14400 s，间隔 60 s。附件 2 给出时间与药材半径，共 145 组观测，覆盖 0 至 259200 s 即 72 h，间隔 1800 s 即 0.5 h。两份附件分别用于构造表面环境输入 $T_a(t), C_a(t)$ 和第四问的几何输入 $R(t)$ 。

计算中时间统一使用 s，半径由 cm 换算为 m，正文表格按题目要求再换算为 h 和 cm。温度观测保留摄氏度，扩散系数经验式中的绝对温度由药材局部温度加 273.15 得到。烘房水分量保留题给 kg/kg 口径，其气固交换解释见假设（4）。

问题一截取附件 1 前 1800 s 的 31 组观测，问题二截取前 10800 s 的 181 组观测，问题三、四使用前 14400 s 的全部 241 组观测。各问均保留相应区间的原始观测及其小幅波动，不作全局平滑。

4.2 烘房环境插值与时间延续

数值求解器需要在观测时刻之间取得表面环境值。为避免普通三次样条在观测点附近产生过冲，本文采用保形分段三次 Hermite 插值（PCHIP）构造 $T_a(t)$ 与 $C_a(t)$ [3]：该插值精确通过全部观测点，在节点处一阶导数连续，且不在单调区间内引入新的极值，插值表达式见附录 B。前 30 min 的环境插值结果见图 1。

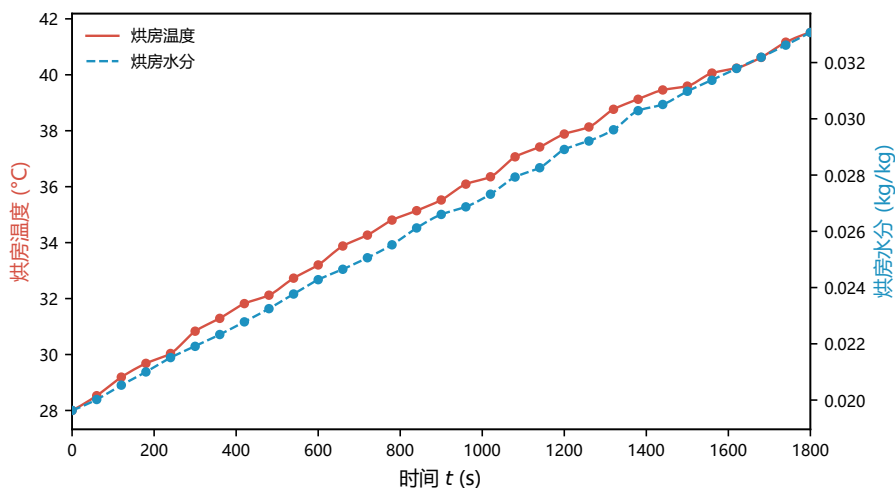


图 1 前 30 min 烘房温湿环境插值曲线

附件 1 没有覆盖完整干燥时段，其最后 1 h 的温度和水分量均值分别约为 49.999 °C 和 0.04999 kg/kg，已接近平台。据此按假设（6），问题三、四在 $t \leq 14400$ s 时保留全部观测输入，其后采用恒定环境延续

$$T_a(t) = 50\text{ }^{\circ}\text{C}, \quad C_a(t) = 0.05\text{ kg/kg}, \quad t > 14400\text{ s}. \quad (1)$$

切换时刻保留原观测终值，其后允许边界输入发生小幅变化，药材内部温湿状态保持连续。该延续假设对达标时长的影响由第七节的情景敏感性分析量化。

4.3 半径轨迹插值与数据覆盖范围

附件 2 的半径由 2 cm 降至 1.198 cm。将其换算为 m 后，采用同一保形插值方法构造

$$R(t) = \text{PCHIP}\{(t_j, R_j)\}, \quad 0 \leq t \leq 259200 \text{ s}. \quad (2)$$

图 2 给出半径观测散点与插值曲线。 $R(t)$ 仅在附件 2 的覆盖区间内使用；第四问的预测终点落在该区间之内，因此无需对半径作数据外推。

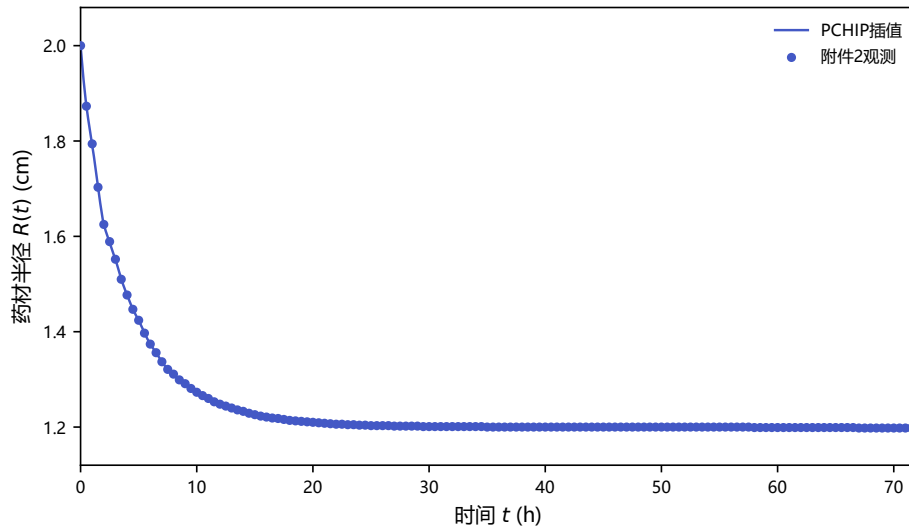


图 2 药材半径观测与 PCHIP 插值曲线

五 问题一：预热阶段的径向热湿传递

5.1 控制方程的建立

(1) 基于热量守恒的径向导热模型

根据轴对称假设，温度记为 $T(r, t)$ 。取半径 r 至 $r + dr$ 、长度 L 的圆柱环层，径向向外热流率为 $Q = -2\pi r L k T_r$ 。在不计内部热源、汽化潜热及水分携热的近似下，由环层蓄热等于净流入热量，得到

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right), \quad 0 < r < R. \quad (3)$$

问题一取 $\rho = 820 \text{ kg/m}^3$ 、 $c_p = 2600 \text{ J/(kg}\cdot\text{K)}$ 、 $k = 0.36 \text{ W/(m}\cdot\text{K)}$ ，故式(3)为线性方程，热扩散率 $\alpha = k/(\rho c_p)$ 。

(2) 基于水分守恒的非线性扩散模型

题目中的 $C(r, t)$ 为干基含水率，即水质量与干物质质量之比。按假设 (5)，参考干物质密度 s_0 恒定，单位体积水质量为 $s_0 C$ ，相对干物质骨架的有效 Fick 通量为

$$j(r, t) = -s_0 D(C) \frac{\partial C}{\partial r},$$

其中 $D(C)$ 为有效水分扩散系数。沿用环层收支，约去恒定的 s_0 和公共几何因子，得到

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[r D(C) \frac{\partial C}{\partial r} \right], \quad D(C) = 7 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right). \quad (4)$$

与常系数扩散不同，式(4)中的 D 随局部含水率变化，水分迁移因而具有非线性。将其展开可得

$$\frac{\partial C}{\partial t} = D(C) \left(\frac{\partial^2 C}{\partial r^2} + \frac{1}{r} \frac{\partial C}{\partial r} \right) + D'(C) \left(\frac{\partial C}{\partial r} \right)^2, \quad D'(C) = \frac{0.89 D(C)}{C^2} > 0.$$

可见扩散系数不能作为常数移出微分算子。后续求解保留式(4)的通量形式，以便统一计算相邻环层之间的水分交换并检验整体水分收支。

由于 D 仅依赖含水率，在上述热学近似下，两场解耦，可分别求解。

图 3 给出径向计算域、内部温湿场及中心与表面的边界关系，几何形状不按实物比例绘制。图中通量以径向向外为正，橙红箭头表示热量传入 ($q_s < 0$)，蓝色箭头表示水分迁出 ($j_s > 0$)。

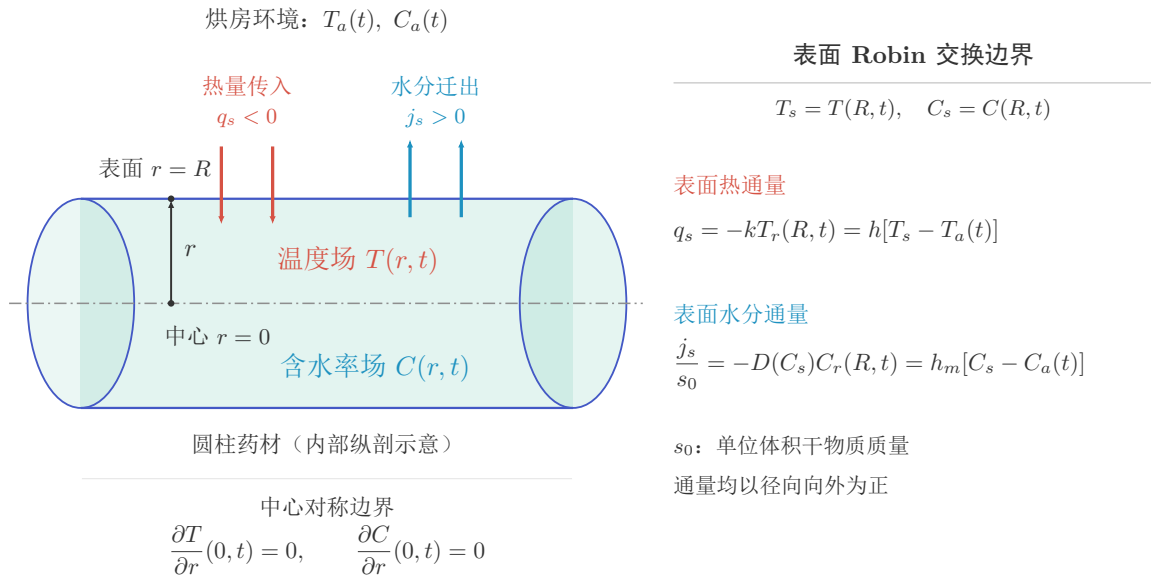


图 3 圆柱药材径向热湿传递模型示意图

5.2 初始条件与边界条件

入炉时药材各处温度和含水率均匀，题给初始状态为

$$T(r, 0) = 28, \quad C(r, 0) = 2.55, \quad 0 \leq r \leq R, \quad (5)$$

单位分别为 $^{\circ}\text{C}$ 和 kg/kg 。由径向对称性，中心处径向梯度为零：

$$\frac{\partial T}{\partial r}(0, t) = \frac{\partial C}{\partial r}(0, t) = 0,$$

中心状态仍随内部传递过程变化。在外表面，导热通量与对流换热通量相平衡，水分扩散通量与假设（4）的等效水分交换通量相平衡，故采用 Robin 边界

$$\begin{aligned} -kT_r(R, t) &= h [T_s - T_a(t)], \\ -D(C_s)C_r(R, t) &= h_m [C_s - C_a(t)]. \end{aligned} \quad (6)$$

其中 $T_s = T(R, t)$ 、 $C_s = C(R, t)$ ， $h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ 。当 $T_a > T_s$ 时向外热流为负，表示热量进入药材； $C_s > C_a$ 时水分向外迁移。表面温度和含水率由内部传递与外部交换共同决定，因而通常与烘房状态不同。环境函数 $T_a(t)$ 、 $C_a(t)$ 取第四节整理的前 1800 s 观测插值。

均匀初始水分使表面初始梯度为零，而交换边界在接触烘房后立即产生失水通量，二者在 $t = 0$ 角点不相容。本文保留题给初值，在 $t > 0$ 施加表面交换，求得初始表面层的瞬态发展，并把前 1 s 的表面结果纳入网格检验。

5.3 模型的数值求解

(1) 空间离散

预热初期表面含水率变化较快，较陡的径向梯度集中在外层。为分辨这一薄层，采用节点型有限体积法，在靠近表面处逐步加密网格，并使题目规定的 0、0.1、...、2.0 cm 输出位置均为实际节点。加密等级记为 G_m ，本问采用 G_8 ，共 2113 个节点，分段细分方案见附录 B。

设全部节点依次为 r_i ($i = 0, \dots, N$)，内部面为 $r_{i+1/2} = (r_i + r_{i+1})/2$ ，控制体左右端点为 a_i, b_i^* ，中心端取 0、表面端取 R ，定义径向权重

$$W_i = \int_{a_i}^{b_i^*} r \, dr = \frac{(b_i^*)^2 - a_i^2}{2}, \quad \sum_{i=0}^N W_i = \frac{R^2}{2}. \quad (7)$$

真实控制体体积为 $2\pi L W_i$ 。式(7)保留了不同半径处环层体积的差异，中心与表面处的控制体均在物理边界截取。

(2) 通量处理与半离散方程

记 $D_i = D(C_i)$ ，相邻节点之间对扩散系数取调和平均，得到已乘半径的向外通量

$$\begin{aligned} F_{i+1/2}^T &= -r_{i+1/2} k \frac{T_{i+1} - T_i}{r_{i+1} - r_i}, \\ F_{i+1/2}^C &= -r_{i+1/2} \frac{2D_i D_{i+1}}{D_i + D_{i+1}} \frac{C_{i+1} - C_i}{r_{i+1} - r_i}. \end{aligned} \quad (8)$$

中心控制体在 $r = 0$ 处的边界通量取零，表面通量直接取 $Rh(T_N - T_a)$ 、 $Rh_m(C_N - C_a)$ 。对式(3)和式(4)作控制体积分，并用节点状态近似体平均，得到

$$\rho c_p W_i \dot{T}_i = F_{i-1/2}^T - F_{i+1/2}^T, \quad W_i \dot{C}_i = F_{i-1/2}^C - F_{i+1/2}^C. \quad (9)$$

同一内部面通量在相邻控制体中以相反符号出现，对所有控制体求和后仅保留表面交换项，这一性质直接用于后文的收支检验。

中心处通过控制体积分避免直接计算 $1/r$ ；表面节点保留蓄积项，由内部通量和外部交换共同确定其状态。

(3) 时间推进与结果输出

空间离散后得到刚性常微分方程组，采用自适应变阶隐式 BDF 方法推进时间 [4]，并按当前状态更新非线性物性。隐式离散与迭代过程见附录 B。

本问相对容差为 10^{-10} ，温度与含水率绝对容差分别为 10^{-10} 、 10^{-12} ，最大步长 0.5 s。完整结果由连续数值输出按 1 s、0.1 cm 间隔保存至 `result1.xlsx`，题定表格从中选取指定时刻与位置。

5.4 计算结果与分析

表 1、表 2 给出题定时刻与位置的温度和含水率。图 4 左、右图分别为两场的时空等值线，纵轴为径向位置，等值线数值的单位分别为 $^{\circ}\text{C}$ 和 kg/kg 。

表 1 30 分钟内药材的温度 ($^{\circ}\text{C}$)

时间/s	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
100	28.0000	28.0003	28.0038	28.0319	28.1795
300	28.0406	28.0634	28.1513	28.3673	28.8486
600	28.4533	28.5361	28.8039	29.3155	30.1641
900	29.3243	29.4583	29.8757	30.6164	31.7295
1200	30.5429	30.7099	31.2225	32.1130	33.4295
1500	31.9960	32.1871	32.7667	33.7476	35.1211
1800	33.5758	33.7724	34.3642	35.3626	36.7879

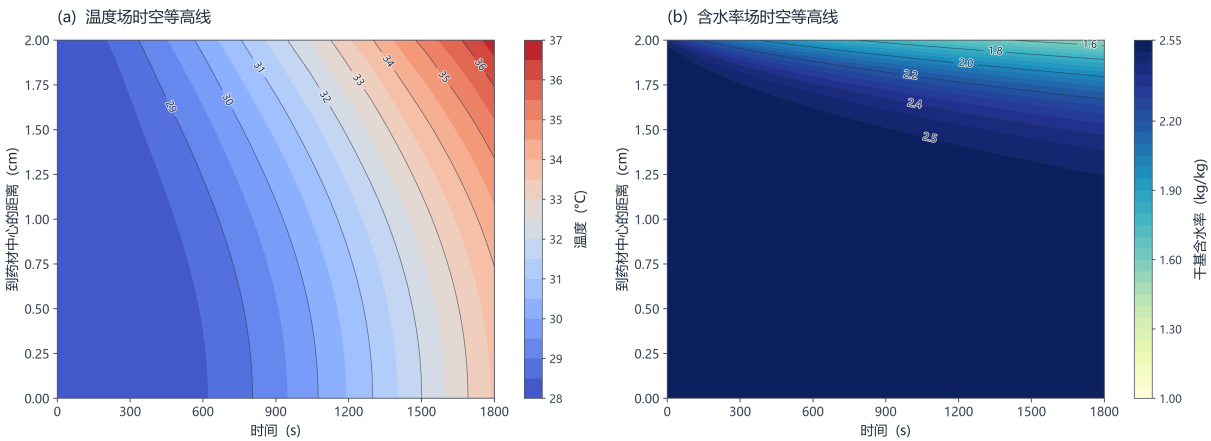


图 4 预热阶段温度与干基含水率时空分布

表2 30分钟内药材的水分浓度 (kg/kg)

时间/s	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
100	2.5500	2.5500	2.5500	2.5500	2.2470
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

温度分布体现了由外向内的升温过程。至 1800 s，表面温度为 36.7879 °C，中心温度为 33.5758 °C，两者相差 3.2121 °C；表面仍低于同期烘房温度 41.5130 °C。热量先经表面对流进入药材，再通过内部导热向中心传播，有限的表面换热和内部导热共同造成了这一温度滞后。

水分变化主要集中在外层。末时刻表面含水率降至 1.5102 kg/kg，而中心未舍入值为 2.5499924 kg/kg，保留四位小数仍为 2.5500 kg/kg。表面持续向烘房排湿，首先形成含水率梯度；在前 30 min 内，内部水分向外迁移尚不充分，因此中心变化较小，径向剖面表现为中心高、表面低。结合中心已经升温而含水率近乎不变的结果，预热阶段的热响应快于中心失水，表层变化不能代表内部干燥程度。

含水率下降还会改变后续迁移速率。由 $D'(C) > 0$ 可知，失水使局部扩散系数减小。本问 D 由初始 4.9377×10^{-9} 降至局部最小 $3.8830 \times 10^{-9} \text{ m}^2/\text{s}$ ，表明在相同含水率梯度下，较干区域的水分传递能力有所降低。与冻结初始 D 的计算相比，含水率最大差达到 0.0419 kg/kg，因此正式求解保留 $D(C)$ 的变化。

5.5 模型检验

采用 G_2 、 G_4 、 G_8 网格，在全部整秒输出与 21 个规定半径上比较解；另在 G_8 上将容差缩至原来的 1/5、最大步长减至 0.25 s。相邻网格的含水率最大差由 1.828×10^{-5} 降至 $4.564 \times 10^{-6} \text{ kg/kg}$ ，均出现在 1 s 时的表面，说明初始薄层最敏感。题定 7 组时刻、5 个位置的两场结果在 G_4/G_8 及时间收紧比较下四位小数一致，汇总见表 12。

温度方程可构造独立于有限体积离散的 Bessel–Duhamel 参考解，完整推导见附录 A。级数由 480 项增至 960 项时，参考解最大变化为 $2.7632 \times 10^{-9} \text{ °C}$ ；960 项参考解与有限体积解在规定输出点上的最大差为 $2.1547 \times 10^{-7} \text{ °C}$ 。冻结 $D = D(C_0)$ 的辅助水分算例与 Bessel 参考解最大差为 $1.5090 \times 10^{-6} \text{ kg/kg}$ ，用于检查圆柱几何和交换边界；正式非线性水分解仍由加密及收支检验评价。

累计表面通量与末态存量的独立比较满足所设 10^{-6} 相对残差阈值，结果见表 11，残差定义见附录 B。这些检验支持所建一维有效模型的数值一致性，物理简化的预测偏差仍需额外观测评估。

六 问题二：变物性条件下的热湿耦合

6.1 统一物性与耦合方程

问题二建立整个烘干过程的变物性模型，全程统一使用附录 3 经验式，本问给出前 3 h 的结果。因此保留问题一的固定几何、原始初值与边界结构，并从 $t = 0$ 重新积分。局部物性为

$$\begin{aligned}\rho(C) &= 650 + 128C, \\ c_p(C) &= 1450 + 2736 \frac{C}{1+C}, \\ k(C) &= 0.21 + 0.38 \frac{C}{1+C}, \\ D(C, T) &= 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left[-\frac{3850}{T + 273.15}\right].\end{aligned}\quad (10)$$

其中 ρ 、 c_p 、 k 和 D 的单位分别为 kg/m^3 、 $\text{J}/(\text{kg}\cdot\text{K})$ 、 $\text{W}/(\text{m}\cdot\text{K})$ 和 m^2/s 。温度 T 按 $^\circ\text{C}$ 表示，扩散系数指数中的 $T + 273.15$ 为药材局部绝对温度，单位为 K 。

问题二沿用与问题一相同的固定圆柱环层控制体，环层几何和水分守恒结构不变，附录 3 只是把热物性与扩散系数改为局部温湿状态的函数。其中 $\rho(C)c_p(C)$ 作为有效体积热容量 $B(C)$ ，固定域水分守恒仍按参考干物质骨架计量；根据假设 (3)，仍不显式引入汽化潜热和水分携焓。将式(10)代入该环层模型，得到

$$\begin{aligned}B(C)T_t &= \frac{1}{r} \partial_r [rk(C)T_r], \\ C_t &= \frac{1}{r} \partial_r [rD(C, T)C_r], \\ B(C) &= \rho(C)c_p(C).\end{aligned}\quad (11)$$

初始条件沿用式(5)，中心仍取零梯度条件，表面边界沿用式(6)并将 k, D 替换为当前表面物性。附录 3 未另给交换系数，按假设 (4) 继续使用前述 h, h_m 。

式(11)中的耦合有两个方向： C 改变 B 和 k ，从而改变温度传播； T 改变 D ，从而改变水分迁移。将水分扩散项展开可得

$$C_t = D \left(C_{rr} + \frac{C_r}{r} \right) + D_C (C_r)^2 + D_T T_r C_r, \quad (12)$$

其中 $D_T T_r C_r$ 项由扩散系数随温度场的空间变化产生，而非额外引入的经验项。采用原扩散通量形式离散，可同时保留上述各非线性项。

6.2 有效热容量与干物质守恒核算

问题二沿用固定参考干物质骨架，水分方程中的参考干密度保持恒定。经验 $\rho(C)$ 只用于构造 $B(C)$ ；若在水分守恒中又取随时间更新的 $s = \rho(C)/(1+C)$ ，则与固定体积内干物质质量不变的假设相矛盾，故两者分别记账。

同样， $B(C)T_t$ 与 $\partial_t[B(C)T]$ 并不等价。取初始参考温度 $T_{\text{ini}} = 28^\circ\text{C}$ ，有

$$\partial_t[B(C)(T - T_{\text{ini}})] = B(C)T_t + B'(C)(T - T_{\text{ini}})C_t, \quad (13)$$

其中第二项反映有效热容量随组成变化而改变。我们采用以温度形式表达的有效热方程，并在收支检验中保留该修正项。

6.3 联合求解与结果

沿用问题一的环形控制体，热导率与扩散系数均取相邻节点的调和平均。将 $\mathbf{y} = (T_0, \dots, T_N, C_0, \dots, C_N)^T$ 作为联合状态，在每次右端函数求值及隐式迭代中按局部温湿状态更新物性；每个节点只与相邻节点的两个场相联系，BDF 积分利用这一稀疏结构。本问采用 G_4 网格，相对容差 10^{-9} ，温度、含水率绝对容差分别为 10^{-8} 、 10^{-10} ，最大步长 1 s。

表 3 和表 4 给出每 0.5 h 的题定结果。图 5 左、右图分别展示前 3 h 的温度与干基含水率时空等值线，数值单位分别为 $^{\circ}\text{C}$ 和 kg/kg 。

表 3 3 小时内药材的温度 ($^{\circ}\text{C}$)

时间/h	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
0.5	32.1896	32.3824	32.9660	33.9608	35.4132
1	40.3822	40.5542	41.0609	41.8783	43.0000
1.5	45.8471	45.9351	46.1932	46.6055	47.1397
2	48.4503	48.4881	48.5985	48.7736	49.0033
2.5	49.4670	49.4793	49.5136	49.5645	49.6617
3	49.8494	49.8553	49.8746	49.9096	49.9645

表 4 3 小时内药材的水分浓度 (kg/kg)

时间/h	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
0.5	2.5499	2.5489	2.5256	2.3257	1.6486
1	2.5257	2.4948	2.3578	2.0230	1.4711
1.5	2.3861	2.3256	2.1344	1.8020	1.3476
2	2.1708	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3	1.7662	1.7165	1.5702	1.3333	1.0081

图 5 中温度逐渐接近烘房水平，含水率仍保持中心高、表面低的径向差异。3 h 时中心温度为 49.8494°C ，但中心含水率仍为 1.7662 kg/kg ，远高于 0.15 kg/kg 的干燥要求。此时升温过程已接近完成，内部失水却仍需持续，故温度接近稳定不能作为停止烘干的条件。

问题一与问题二在相同时刻的结果差异来自物性设定和耦合方式，不能将两问末态

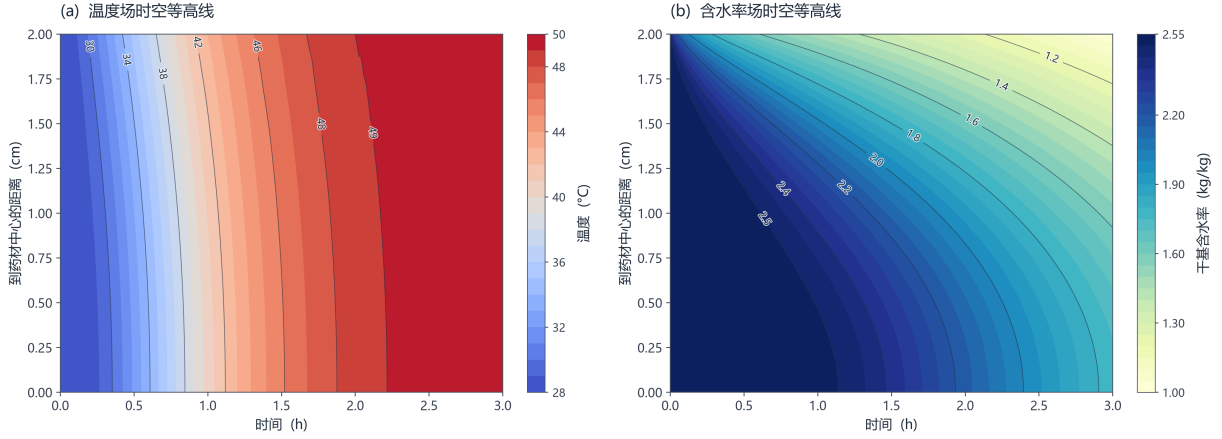


图5 变物性模型前3 h 温度与干基含水率时空分布

直接拼接。按题目要求，本问从1 s 到 10800 s 以 1 s、0.1 cm 采样输出。

6.4 升温与失水对扩散系数的竞争

由式(10)求导可得

$$\frac{\partial \ln D}{\partial C} = \frac{0.45}{C^2} > 0, \quad \frac{\partial \ln D}{\partial T} = \frac{3850}{(T + 273.15)^2} > 0. \quad (14)$$

两个偏导数同号，说明升温使扩散系数增大、失水使其减小，二者沿实际轨迹相互竞争。按经验式代入可见竞争的量级：固定 $C = 2.55$ 时，温度由 28 升至 50 °C 使 D 增至约 2.388 倍；固定 $T = 50$ °C 时， C 由 2.55 降至 0.15 使 D 降至约 5.94%。

数值结果中前 3 h 局部 D 在 5.5476×10^{-9} 至 1.2522×10^{-8} m²/s 之间变化，因各位置升温 and 失水程度不同，无法用单一常数代表整个时空场。与冻结为初始值 D_0 的对照算例相比，3 h 内含水率最大差为 0.3981 kg/kg，说明忽略物性变化会明显改变水分分布。结合图 5 可见，温度趋于稳定后，升温带来的扩散增益逐渐减弱，而持续失水仍使 D 降低，这为问题三的后期减速提供了机制解释。上述偏导数说明两种作用的方向，具体迁移速率还取决于局部含水率梯度，不能仅由 D 的增减判断。

七 问题三：全域达标条件与烘干时间

7.1 模型承接与长期烘干环境

沿用问题二的式(10)–(11)及初边值条件，从入炉初态联合积分温度与干基含水率，固定半径仍为 $R = 0.02$ m。前 4 h 使用附件 1 观测插值，其后按式(1)保持 50 °C、0.05 kg/kg，直至各处干基含水率均低于 0.15 kg/kg。

7.2 全域达标判据的建立

题目要求药材各处均低于阈值。由于表层先干而中心滞后，若以平均值或表面值判断，会在内部仍存在湿区时提前结束计算，因此定义径向最大干基含水率及事件函数

$$M(t) = \max_{0 \leq r \leq R} C(r, t), \quad g(t) = M(t) - 0.15, \quad t_* = \inf\{t \geq 0 : M(t) < 0.15\}, \quad (15)$$

离散判据取全部网格节点的最大值 $M_{G_8}(t) = \max_i C_i(t)$ ，不以题定输出位置代替全网格。保存的径向剖面由中心向表面不增，最大值位于中心。“全域”限于一维径向模型，端面影响见第 7.6 节。

连续向下穿越时，临界根满足 $M(t_*) = 0.15$ 。为满足严格不等式，报告时刻 t_{rep} 取稍晚于 t_* ，并以未舍入结果及加密比较复核，具体见第 7.5 节和图 6。

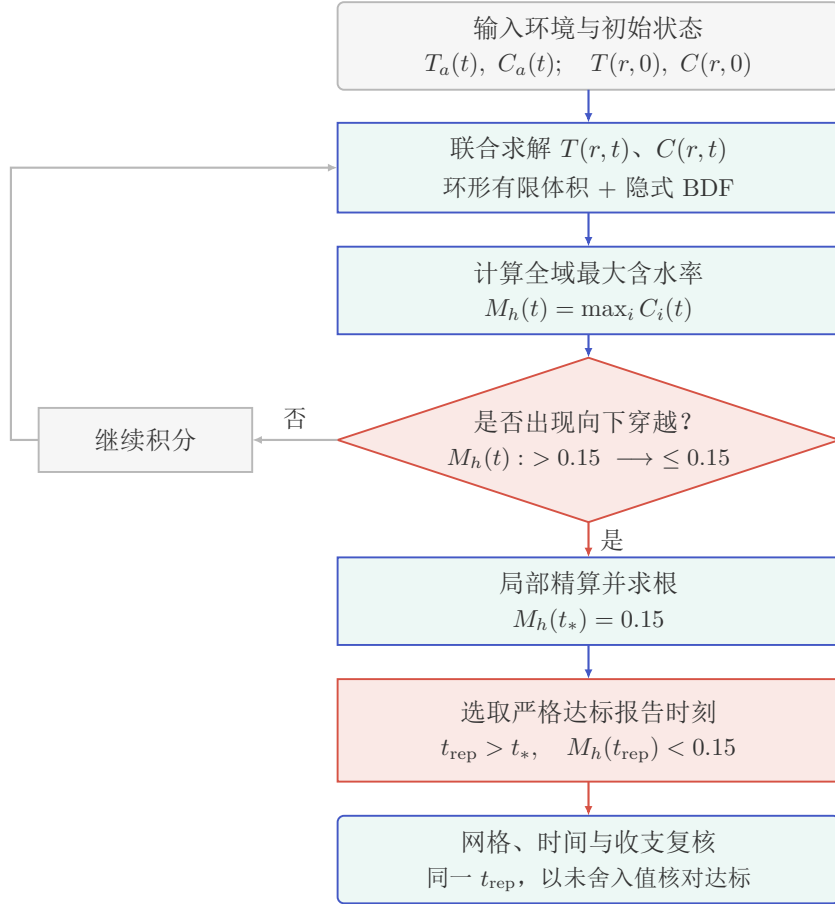


图 6 全域达标事件定位与结果核验流程

7.3 数值求解与达标事件定位

沿用环形有限体积法与 BDF 联合求解，采用 G_8 网格。相对容差为 10^{-9} ，温度和含水率绝对容差分别为 10^{-8} 、 10^{-10} ；前 4 h 最大步长 2 s，其后 60 s，在环境切换处分段接续。

积分过程中监测 $g_{G_8}(t) = M_{G_8}(t) - 0.15$ ，搜索其由正到非正的穿越区间，在阈值附近以最大 0.5 s 步长局部精算，再利用连续数值插值和带括号求根定位 t_* 。因此，每 60 s 的文件输出间隔并不限制事件定位的时间精度。

7.4 烘干时长与水分分布结果

按上述全域判据与严格达标规则，药材所需烘干时长为

$$t_{\text{rep},3} = 57.4731 \text{ h} = 206903.16 \text{ s}, \quad (16)$$

此时中心控制最终达标，未舍入末态核验见第 7.5 节。

为考察后期环境假设的影响，固定前 4 h 观测输入，分别改变后期温度、水分量；另对有效传质系数作全程比例缩放，其余条件不变。水分量 $\pm 10\%$ 为设定情景，幅度大于附件 1 末 1 h 的观测波动。

表 5 问题三长期输入敏感性分析

情景	$T_a / ^\circ\text{C}$	$C_a / \text{kg/kg}$	h_m/h_m^0	临界时间 t_*/h	相对变化
基准	50	0.050	1.0	57.4725	0.00%
$T_a - 2^\circ\text{C}$	48	0.050	1.0	61.2927	+6.65%
$T_a + 2^\circ\text{C}$	52	0.050	1.0	53.9791	-6.08%
$C_a - 10\%$	50	0.045	1.0	57.3815	-0.16%
$C_a + 10\%$	50	0.055	1.0	57.5826	+0.19%
$h_m - 20\%$	50	0.050	0.8	59.1690	+2.95%
$h_m + 20\%$	50	0.050	1.2	56.4333	-1.81%

后期温度情景使达标时间变化约 6%，全程传质系数情景约 2–3%，后期水分量情景约 0.2%，均明显大于已检验的数值加密差异。在本次扰动幅度下，温度情景造成的时长差异最大，说明长期温度延续值得重点核实；各参数的扰动幅度及生效时段不同，不能据此直接作统一的敏感性排序。这些情景结果也不构成实际误差界或置信区间。

表 6 列出每 6 h 及结束时刻的含水率，图 7 展示烘干全过程的含水率分布及阈值等值线。

图 7 中的红色虚线为 $C = 0.15 \text{ kg/kg}$ 等值线，填色采用非等距分级以区分低含水率阶段；图示止于严格达标报告时刻。阈值线由表面向中心推进，反映未达标区域逐步缩小。18 h 时表面已降至 0.0845 kg/kg ，中心仍为 0.2988 kg/kg ，约为阈值的两倍，说明表层达标后内部仍需较长时间排湿。

48 h 时中心含水率为 0.1618 kg/kg ，距阈值仅差约 0.0118 kg/kg ，但还需约 9.47 h，占总时长的 16.5%。这表明接近终点时，较小的含水率降幅仍可能对应较长等待时间。温度接近稳定后，失水使局部扩散系数减小，最大含水率曲线逐渐变平，故不能将前 3 h 的失水速率线性外推至阈值；停止时刻应由剩余湿区的最大含水率确定。

表 6 固定半径模型药材烘干过程的水分浓度 (kg/kg)

时间/h	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4561	0.4432	0.4031	0.3297	0.1642
18	0.2988	0.2915	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1855	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0549
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1514	0.1436	0.1277	0.0530
57.4731	0.1500	0.1477	0.1402	0.1249	0.0527

注：终时中心未舍入值为 0.1499993913 kg/kg，严格低于 0.15；0.1500 为四位显示。

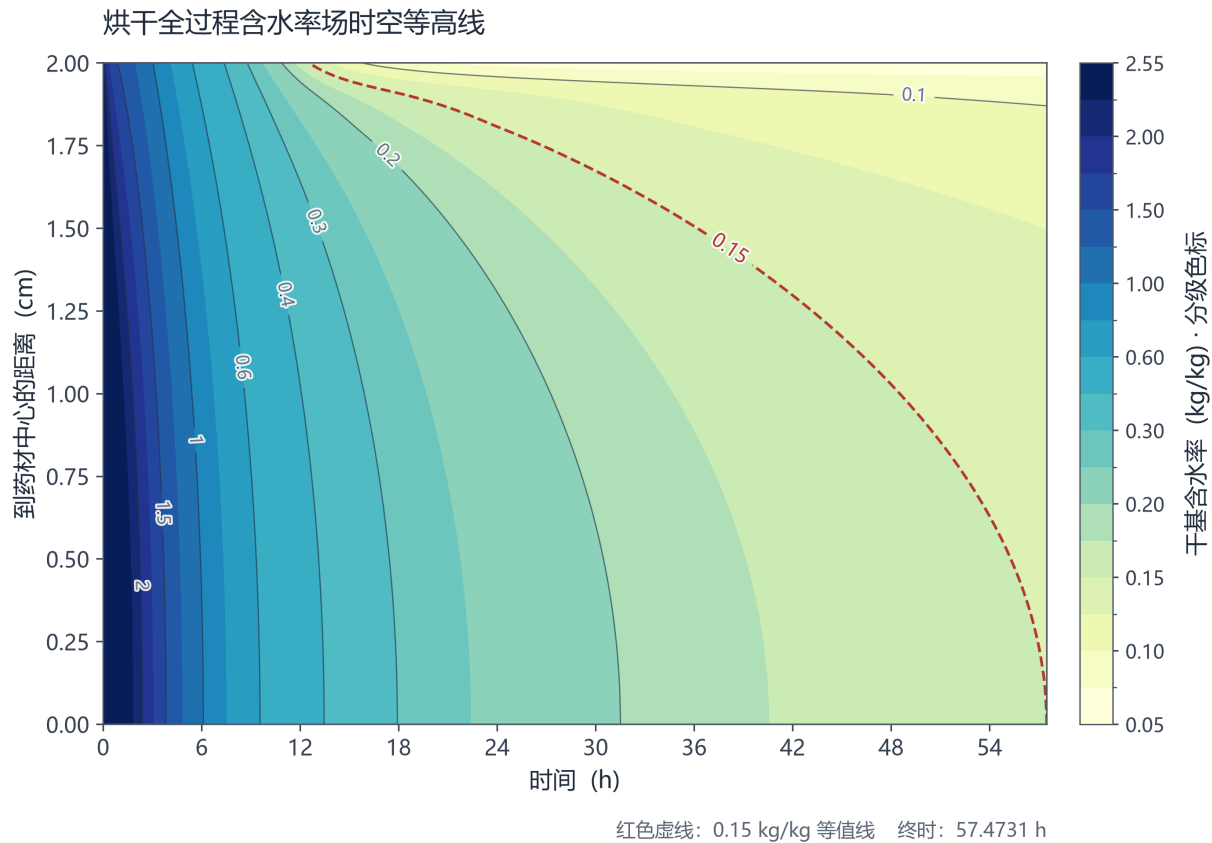


图 7 全程干基含水率时空分布与达标阈值等值线

7.5 数值可靠性与结果输出

基准临界时刻为 $t_* = 57.4725195022 \text{ h}$ 。以 G_{16} 网格和 G_8 时间收紧解复核，临界时间分别变化 0.585132 s 和 $1.7506 \times 10^{-5} \text{ s}$ ；题定表在两类加密下四位小数一致，配置对照见表 13。

报告时刻的选取需覆盖上述离散差异。在同一基准网格临界时刻 $t_{*,h}$ 比较三组解，定义经验裕量

$$\varepsilon_{C,\text{margin}} = 2 \max \{|M_{G_8}(t_{*,G_8}) - M_{G_{16}}(t_{*,G_8})|, |M_{G_8}(t_{*,G_8}) - M_{G_{8,\text{tight}}}(t_{*,G_8})|\} + 10^{-8}, \quad (17)$$

其中 $M_{G_{16}}$ 和 $M_{G_{8,\text{tight}}}$ 分别为细网格解和时间收紧解的最大含水率。本问得到 $\varepsilon_{C,\text{margin}} = 3.5088 \times 10^{-7} \text{ kg/kg}$ ；据此先寻找 $M_{G_8}(t) < 0.15 - \varepsilon_{C,\text{margin}}$ 的时刻，再向上取整秒并向上保留四位小时，即得式(16)。在 $t_{\text{rep},3}$ 时，基准解最大值加上该裕量仍低于 0.15 kg/kg ，细网格解和时间收紧解的最大值分别为 0.1499992209 和 $0.1499993913 \text{ kg/kg}$ ，也均严格达标。该裕量来自已完成的数值加密比较，不是连续方程解的严格误差上界。热湿收支检验见表 11。

报告时刻基准最大干基含水率为 $0.1499993913 \text{ kg/kg}$ ，严格低于阈值；表 6 末行中心的 0.1500 仅为四位舍入显示。完整结果保存至 `result3.xlsx`，按 60 s 、 0.1 cm 采样，并追加式(16)的准确结束时刻。

7.6 径向简化的二维端面校核

短时传播尺度不能直接保证约 57 h 预测中的端面影响仍小。参考 Hussain 和 Dincer 对有限圆柱二维热湿传递的研究 [1]，在第三问原初值、附录 3 物性及环境输入下恢复轴向通量。取 $z = 0$ 为中截面，在 $0 \leq r \leq R$ 、 $0 \leq z \leq L/2$ 上建立

$$\begin{aligned} B(C)T_t &= \frac{1}{r} \frac{\partial}{\partial r} (rk(C)T_r) + \frac{\partial}{\partial z} (k(C)T_z), \\ C_t &= \frac{1}{r} \frac{\partial}{\partial r} (rD(C,T)C_r) + \frac{\partial}{\partial z} (D(C,T)C_z). \end{aligned} \quad (18)$$

中心轴和中截面施加零法向梯度，侧面沿用原边界；端面 $z = L/2$ 取 $-kT_z = h_z(T - T_a)$ 、 $-DC_z = h_{m,z}(C - C_a)$ ，其中 $h_z = h$ 、 $h_{m,z} = h_m$ 为指定对照情景。该模型保留本文的变系数守恒形式，以表面加密有限体积法和隐式 BDF 求解，不移植文献材料参数。

以全部二维节点的 $\max C_{ij}(t) = 0.15$ 定位临界根，并对每个径向网格独立计算一维配对解。表 7 中 N_r, N_z 为区间数， $\Delta t_* = t_{*,2D} - t_{*,1D}$ ；采用同径向网格比较，以分离改变网格与开放端面的作用。

关闭端面交换后，二维与配对一维临界时间相差 0.0015 s ，说明退化情形一致；开放端面时累计水分收支相对残差均低于 1.2×10^{-8} 。径向加密、轴向加密及时间收紧使配对时间差分别变化 0.0102 、 0.5094 和 0.0012 s ，配对差在所检验配置下均约为 8 s ，支持端面影响很小的判断；仅凭两级轴向网格尚不能确定其渐近收敛阶。二维绝对临界时间的径向加密差仍为 80.28 s ，因此本节只用配对比较评估端面的相对作用，正式答案仍取一维 G_8 结果。

图 8 中图为保持实际长径比的 36 h 纵剖面含水率，下图比较同径向网格的全域最大值及其差异。端面促进端部干燥，但采样及临界时刻的全域最大值仍由中心控制。细网

表 7 同径向网格下一、二维临界时间对照

$N_r \times N_z$	时间配置	一维 / h	二维 / h	$\Delta t_*/s$
80×40	普通	57.502060	57.499795	-8.153
160×40	普通	57.479758	57.477496	-8.143
80×80	普通	57.502060	57.499937	-7.644
160×80	普通	57.479758	57.477637	-7.634
160×80	收紧	57.479760	57.477640	-7.635

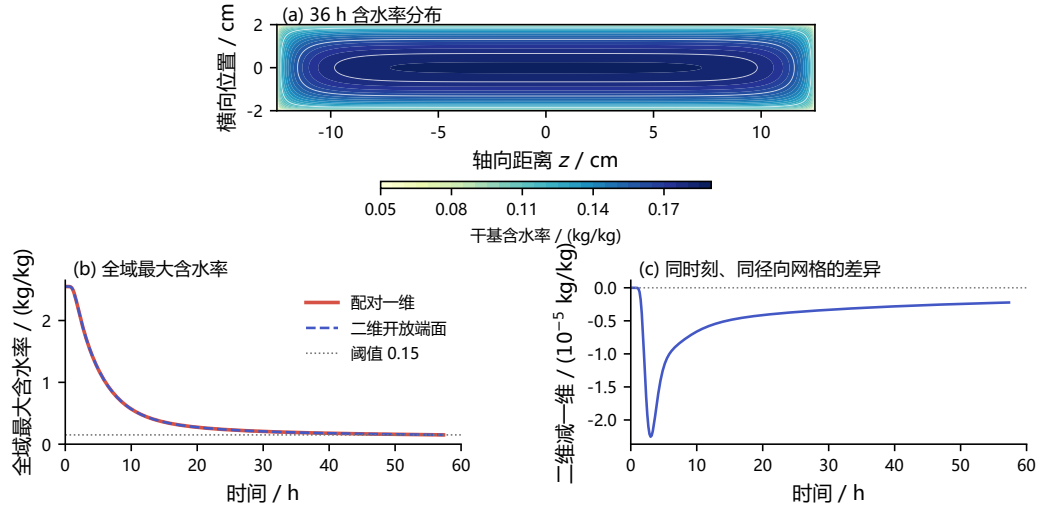


图 8 第三问轴对称二维端面校核

格开放端面后仅提前约 8 s，约占配对一维时长的 0.004%；两端累计排湿却约占总排湿的 5.3%，可见总排湿贡献与中心达标时刻的节时比例并不对应。结果表明，在本文设定的端面交换条件下，二维端面对全域最大含水率及临界达标时刻的影响较小，从而支持采用一维径向模型作为主模型的合理性。

八 问题四：径向收缩条件下的热湿传递

8.1 问题分析与比例收缩假设

第 7.6 节的二维校核表明，在指定端面条件下端面效应对第三问中心控制的达标时长影响很小。问题四忽略轴向缩短，长度仍取 $L = 0.25 \text{ m}$ ；径向收缩使两端总面积与侧面积之比 $R(t)/L$ 减小，从几何上支持继续采用径向近似，但面积比不能直接确定端面通量或达标时长的变化。本问在周向均匀、物性各向同性及相同端面环境的假设下采用一维径向模型，收缩情景下的端面时间误差尚未单独量化，第三问的 0.004% 不作为本问的误差界。

本问将固定半径改为附件 2 的 $R(t)$ ，物性统一改用附录 4，从 $T(r, 0) = 28 \text{ }^\circ\text{C}$ 、 $C(r, 0) = 2.55 \text{ kg/kg}$ 独立积分。环境输入与半径轨迹的处理见第四节。Brasiello 等讨论了食品干燥中收缩与非等温热湿传递的耦合 [5]，其结论支持将材料运动写入传递方程；

本文沿用这一建模思路，具体收缩关系仍由本题数据确定。

题目只给出外半径，没有给出内部各点位移。按假设（2），设初始位置 r_0 的材料点按相同比例收缩：

$$r(t) = \frac{R(t)}{R_0} r_0, \quad (19)$$

相应骨架径向速度为

$$v(r, t) = \frac{dr}{dt} = \frac{\dot{R}(t)}{R(t)} r. \quad (20)$$

式(19)和式(20)由外半径轨迹闭合内部速度场。

图 9 中的 P_0 与 $P(t)$ 表示同一材料点在初始时刻和 t 时刻的位置，收缩比例和长径比仅作示意，不对应特定实测时刻。比例收缩下其轴向位置保持不变，物理半径随 $R(t)$ 同比缩小，而相对位置 $\xi = r_0/R_0 = r(t)/R(t)$ 保持不变，由此可将移动区域映到固定材料区间。

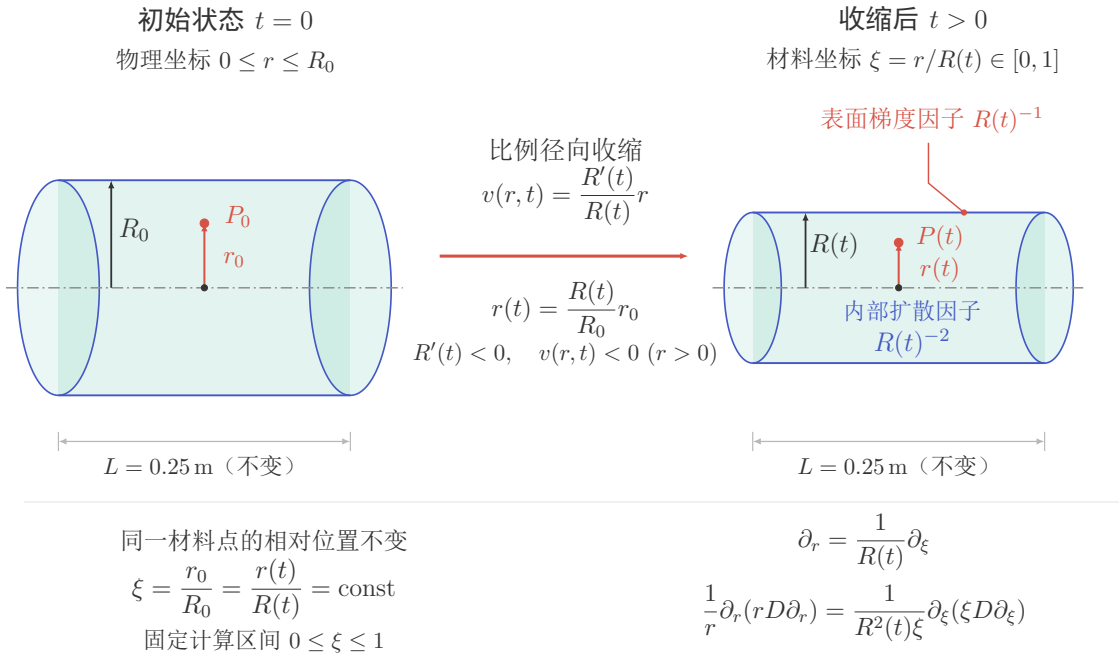


图 9 比例径向收缩与材料坐标变换示意图

8.2 收缩条件下的水分守恒与有效传热方程

直接在移动区间 $0 \leq r \leq R(t)$ 上离散不便于沿用前问网格。本节先在物理坐标中写出传递方程，下一节再映到固定区间。

(1) 干物质守恒。设 $s(r, t)$ 为单位当前体积中的干物质质量。干物质不流失时，

$$s_t + \frac{1}{r} \partial_r(r s v) = 0. \quad (21)$$

将式(20)代入，并设初始干物质密度均匀，则 s 保持空间均匀，满足

$$\begin{aligned}\dot{s} + 2\frac{\dot{R}}{R}s &= 0, \\ s(t) &= s_0 \left[\frac{R_0}{R(t)} \right]^2.\end{aligned}\tag{22}$$

其中因子 2 来自仅考虑径向收缩、长度不变：当前体积 $V(t) = \pi R^2(t)L$ 。半径减小后，同样的干物质被压缩到更小体积，因而当前体积干物质密度按 R^{-2} 增大。

(2) 水分守恒。干基含水率 C 是水质量与干物质质量之比，而非单位体积的水分含量。单位当前体积中，干物质质量为 s ，水质量为 sC ，相对固体骨架的有效 Fick 通量为

$$j = -s D(C, T) C_r.$$

水分守恒为

$$\partial_t(sC) + \frac{1}{r}\partial_r[r(sCv + j)] = 0.\tag{23}$$

由式(21)消去几何压缩项，得到

$$C_t + vC_r = \frac{1}{sr}\partial_r[r s D C_r].\tag{24}$$

在式(22)下 $s = s(t)$ 不依赖 r ，故

$$C_t + vC_r = \frac{1}{r}\partial_r[r D(C, T) C_r].$$

式(24)说明：若材料单元只发生收缩，水和干物质均无进出，二者质量之比不变，因此干基含水率方程不含 $-C\nabla \cdot v$ 形式的体积压缩源项；该项属于按单位体积计量的浓度变量。

(3) 有效传热方程。沿用有效热容量 $B(C) = \rho(C)c_p(C)$ ，不显式计入汽化潜热、水分携焓和收缩机械功。随骨架运动的材料微元，其温度变化率取物质导数 $DT/Dt = T_t + vT_r$ ，由此将有效热方程推广至收缩域，与水分方程组成

$$\begin{cases} B(C)(T_t + vT_r) = \frac{1}{r}\partial_r[rk(C)T_r], \\ C_t + vC_r = \frac{1}{r}\partial_r[rD(C, T)C_r]. \end{cases}\tag{25}$$

收缩通过材料随动项 vT_r 和 vC_r 进入方程。经验密度仅用于 $B(C)$ ，干物质密度由式(22)确定；这项有效近似及其相容性限制见第十节和附录 B。

8.3 材料坐标变换与定解条件

定义

$$\begin{aligned}\xi &= \frac{r}{R(t)}, \quad 0 \leq \xi \leq 1, \\ U(\xi, t) &= C(R(t)\xi, t), \\ \Theta(\xi, t) &= T(R(t)\xi, t).\end{aligned}\tag{26}$$

在比例收缩(19)下, 同一材料点满足 $\xi = r_0/R_0$ 不随时间变化, 因而 ξ 标记随骨架运动的相对位置, 计算区域始终为固定区间 $[0, 1]$ 。

由链式法则, $C_r = U_\xi/R$, $U_t = C_t + \xi \dot{R} C_r$; 由于 $v = \dot{R}\xi$, 有 $U_t = C_t + v C_r$, 温度场同样满足 $\Theta_t = T_t + v T_r$ 。可见材料运动与坐标网格同步, 式(25)中的显式随动项恰被坐标变换吸收。固定半径模型中内部扩散含因子 R_0^{-2} , 收缩后该因子成为 $R(t)^{-2}$; 随着 $R(t)$ 减小, R^{-2} 增大, 表示在相同无量纲梯度下, 中心到表面的实际迁移距离缩短, 这是收缩改变干燥速度的几何机制。

问题四按题意从初始时刻统一使用附录 4 经验式:

$$\begin{aligned}\rho(U) &= 760 + 90U, \\ c_p(U) &= 1850 + 2150 \times \frac{U}{1+U}, \\ k(U) &= 0.12 + 0.20 \times \frac{U}{1+U}, \\ D(U, \Theta) &= 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{U}\right) \times \exp\left[-\frac{3850}{\Theta + 273.15}\right].\end{aligned}\tag{27}$$

扩散系数仍由药材局部温度进入指数。表面梯度由 $C_r = U_\xi/R$ 改写, 因而内部扩散带 R^{-2} 、表面交换带 R^{-1} , 二者必须同时进入定解问题。 $R(t)$ 由附件 2 的 PCHIP 给出, $T_a(t)$ 、 $C_a(t)$ 沿用式(1)。于是在固定区间 $0 \leq \xi \leq 1$ 上, 本问实际积分的定解问题为

$$\begin{cases} U_t = \frac{1}{R^2(t)\xi} \partial_\xi [\xi D(U, \Theta) U_\xi], \\ B(U)\Theta_t = \frac{1}{R^2(t)\xi} \partial_\xi [\xi k(U) \Theta_\xi], \\ U(\xi, 0) = 2.55, \quad \Theta(\xi, 0) = 28, \\ U_\xi(0, t) = \Theta_\xi(0, t) = 0, \\ -\frac{D_s}{R(t)} U_\xi(1, t) = h_m [U_s - C_a(t)], \\ -\frac{k_s}{R(t)} \Theta_\xi(1, t) = h [\Theta_s - T_a(t)]. \end{cases}\tag{28}$$

其中 $B(U) = \rho(U)c_p(U)$, 物性取式(27), 含水率初值以 kg/kg 计、温度初值以 °C 计。物理坐标式(25)只用于说明随动项如何进入方程, 离散求解针对式(28)。

8.4 数值求解、达标判据与输出转换

在 $\xi \in [0, 1]$ 上沿用前三问的环形有限体积结构。将问题一的 G_8 节点除以 R_0 得到固定材料节点 ξ_i , 内部面取相邻节点中点, 径向权重

$$w_i = \int_{\xi_{i-1/2}}^{\xi_{i+1/2}} \xi \, d\xi,$$

端点按 0 和 1 截取。内部水分通量与半离散方程为

$$\begin{aligned}\hat{F}_{i+1/2}^C &= -\xi_{i+1/2} D_{i+1/2} \cdot \frac{U_{i+1} - U_i}{\xi_{i+1} - \xi_i}, \\ R^2(t) w_i \dot{U}_i &= \hat{F}_{i-1/2}^C - \hat{F}_{i+1/2}^C.\end{aligned}\tag{29}$$

温度方程将 D 换为 k ，并乘以 B_i ；表面通量分别取 $Rh_m(U_N - C_a)$ 与 $Rh(\Theta_N - T_a)$ 。时间推进仍采用隐式 BDF，局部物性在每次迭代中按当前温湿状态更新。基准计算使用 2113 个材料节点，相对容差 10^{-8} ，温度与含水率绝对容差分别为 10^{-8} 、 10^{-10} ；前 4 h 最大步长 10 s，其后 60 s。网格加密与时间收紧检验见第九节。

烘干完成的定义沿用式(15)，只把固定半径上的最大值换成材料坐标上的最大值：

$$M(t) = \max_{0 \leq \xi \leq 1} U(\xi, t), \quad t_* = \inf\{t \geq 0 : M(t) < 0.15\},$$

离散求解监测全部材料节点，报告时刻同样取稍晚于 t_* 的时刻并以未舍入结果复核。

题目要求按当前物理距离输出。对 $r_j \leq R(t)$ ，先取 $\xi_j = r_j/R(t)$ ，再在材料网格上对 U 作保形插值；若 $r_j > R(t)$ ，该位置已在药材外，单元格记为“—”。药材表面始终对应 $\xi = 1$ ，其含水率与当时半径分别单列。

8.5 烘干时长与含水率分布

基准解临界时刻为 $t_* = 51.0871229810$ h。按式(15)的严格低于阈值规则，得到

$$t_{\text{rep},4} = 51.0877 \text{ h} = 183915.72 \text{ s}.\tag{30}$$

此时最大含水率为 0.1499989333 kg/kg，严格低于 0.15 kg/kg，最大值位于中心，当时半径为 1.2000 cm，仍在附件 2 的 72 h 覆盖范围内，无需半径外推。表 8 给出每 6 h 及结束时刻的含水率。

表 8 收缩模型药材烘干过程的水分浓度 (kg/kg)

时间/h	到药材中心的距离/cm						$R(t)/\text{cm}$
	0	0.5	1.0	1.5	2.0	药材表面	
6	1.7188	1.5370	1.0221	—	—	0.4205	1.3740
12	0.7374	0.6544	0.4082	—	—	0.1669	1.2480
18	0.4086	0.3686	0.2397	—	—	0.0892	1.2140
24	0.2851	0.2610	0.1790	—	—	0.0673	1.2040
30	0.2263	0.2094	0.1493	—	—	0.0595	1.2010
36	0.1928	0.1796	0.1316	—	—	0.0560	1.2000
42	0.1712	0.1604	0.1200	—	—	0.0541	1.2000
48	0.1561	0.1468	0.1116	—	—	0.0530	1.2000
51.0877	0.1500	0.1413	0.1081	—	—	0.0526	1.2000

注：—表示该固定距离已超出当时半径；药材表面对应当前 $R(t)$ 。终时中心未舍入值为 0.1499989333 kg/kg。

表 8 中 “—” 表示域外位置，表面值与半径分别单列。终时中心的 0.1500 仅为四位舍入显示，达标判断以未舍入值为准。**result4.xlsx** 从 60 s 起每 60 s 输出一次，并追加准确结束时刻；表格与结果文件均由同一源解生成。

收缩条件下的空间非均匀性依然存在：18 h 时表面含水率已降至 0.0892 kg/kg，中心仍为 0.4086 kg/kg；48 h 时中心为 0.1561 kg/kg，仍高于阈值。因此本问同样以空间最大值作为达标判据。

8.6 同物性对照与收缩贡献

第三、四问同时改变了物性和几何，两者的时长差不能单独归因于收缩。为分离几何作用，在附录 4 物性下增加 $R \equiv R_0 = 0.02 \text{ m}$ 的对照组，与实测 $R(t)$ 组保持初值、热湿方程、交换系数及全域达标判据一致；两组前 4 h 均用同一环境插值，其后保持 50 °C、0.05 kg/kg。固定组由式(28)取常半径并联合求解温湿场，沿用收缩组的 G_8 及 BDF 配置，同样以 G_{16} 和时间收紧复核。

表 9 附录 4 物性下固定半径与实测收缩的对照

半径方案	临界时间/h	严格报告时间/h	报告时最大含水率 (kg/kg)
固定 $R = R_0$	129.8442292	129.8448	0.1499996198
实测 $R(t)$	51.0871230	51.0877	0.1499989333

以固定组报告时长为分母，收缩使本模型的预测烘干时间减少

$$\Delta t_{\text{shrink}} = t_{\text{fixed},4} - t_{\text{shrink},4} = 129.8448 - 51.0877 = 78.7571 \text{ h},$$

$$\eta_t = \frac{\Delta t_{\text{shrink}}}{t_{\text{fixed},4}} \times 100\% \approx 60.65\%. \quad (31)$$

图 10 左、右图分别为固定半径组与收缩组，采用相同的非等距分级色标；红色虚线为 $C = 0.15 \text{ kg/kg}$ 等值线，黑线为药材边界。两图均止于各自的严格达标报告时刻，时间轴范围不同，不能直接用图中等值线的视觉斜率比较干燥速度。

在同一物理时刻比较，18 h 时固定组与收缩组中心含水率分别为 0.9691 和 0.4086 kg/kg；48 h 时分别为 0.3243 和 0.1561 kg/kg。此时固定组中心仍超过阈值两倍，收缩组已接近达标，说明最终时长差异对应着持续演变的内部失水差异。收缩虽缩短迁移距离，却未消除中心滞后，不能仅凭半径减小判断烘干完成。

固定组约 129.84 h 虽超过 72 h 半径记录，但其半径为指定常数，无需外推；收缩组则在观测范围内达标。该对照在既定后期环境下量化了扩散距离、表面交换及温湿反馈共同形成的收缩总效应。就方程结构而言，材料坐标中内部扩散含 R^{-2} ，报告时刻 $R = 1.2000 \text{ cm}$ ，相对于初值 2 cm 有

$$\left(\frac{R_0}{R}\right)^2 = \left(\frac{2.0}{1.2}\right)^2 \approx 2.78.$$

该 2.78 倍只是扩散算子几何系数的变化，不等于速度或时长的倍率，实际还受局部 D 、温度场和表面 R^{-1} 交换的共同影响。

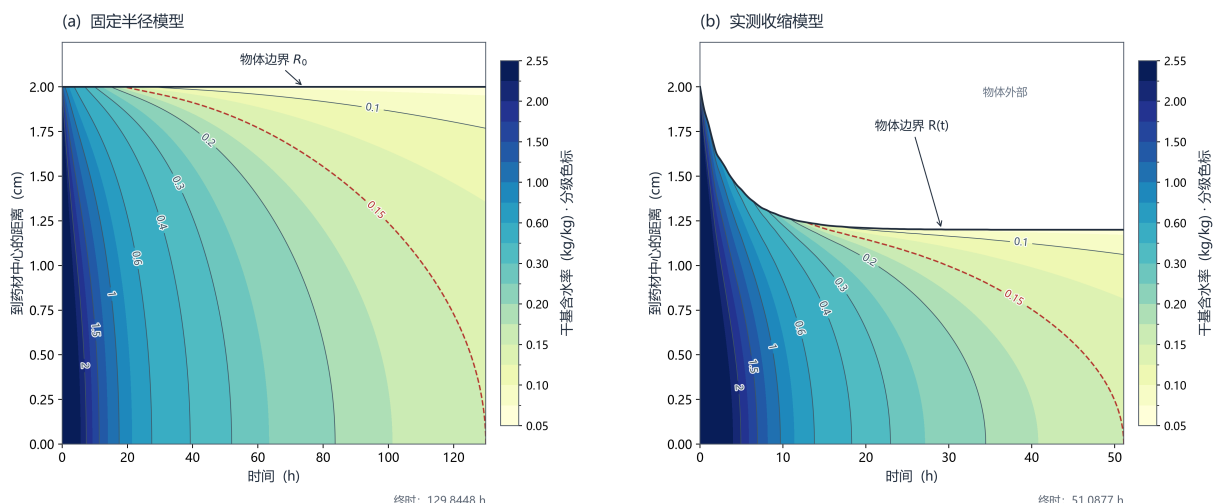


图 10 附录 4 物性下固定半径与收缩模型的含水率时空分布

收缩域的水分总量按干物质质量归一化后满足

$$\frac{M_w}{M_d} = 2 \int_0^1 U \xi d\xi, \quad (32)$$

$$\frac{d}{dt} \left(\frac{M_w}{M_d} \right) = -\frac{2h_m}{R(t)} [U_s - C_a(t)].$$

式(32)由材料域方程积分并使用表面条件得到，无需指定 s_0 的绝对值。令 $R(t) \equiv R_0$ 时，式(28)退化为同一附录 4 物性的固定域方程；关闭表面交换且初值均匀时，纯收缩不改变干基含水率。这两种退化及式(32)的数值收支均已检查，网格与时间加密结果见表 13 和表 11。

8.7 后期温度的条件性敏感性分析

为检验环境延续对第四问及同物性对照的影响，保持前 4 h 观测输入、附录 4 物性和交换系数不变，令后期温度分别为 48、50、52 °C，水分量仍为 0.05 kg/kg。收缩组在各情景下使用同一给定 $R(t)$ ，固定组始终取 R_0 ；该对照仅考察传递方程对温度的响应，不包含温度变化对实际收缩轨迹的反馈。表 10 中，降温与升温使收缩组临界时长分别

表 10 附录 4 同物性模型的后期温度情景；时长均为临界时刻

后期温度/°C	固定半径时长/h	收缩时长/h	临界时长缩短比例/%
48	139.3230	54.4358	60.9284
50	129.8442	51.0871	60.6551
52	121.1586	48.0214	60.3649

增加约 3.35 h、减少约 3.07 h，缩短比例保持在 60.36% 至 60.93% 之间。该范围支持所考察温度情景下收缩组仍明显早于固定组达标，不是其他收缩规律或环境条件下的保证。比例按未舍入临界时刻计算，与表 9 按严格报告时刻计算的比例口径不同，原正式报告时长保持不变。

两组均用 G_4/G_8 复核，全部情景的临界时间网格差小于 2.30 s，远小于温度情景引起的小时级差异；从 4 h 到临界时刻的累计水分相对残差均低于 4.5×10^{-9} 。50 °C 基准复算与原临界时刻的差异小于 0.02 s。收缩组最晚在 54.4358 h 附近达标，未超出 72 h 半径观测范围；该检验不估计潜热或气固界面近似的物理误差。

九 模型检验与误差分析

9.1 水分收支与有效热方程检验

固定域对半离散方程求和，内部通量相消，得到

$$\sum_i W_i [C_i(t) - C_i(0)] = -Rh_m \int_0^t [C_s(\tau) - C_a(\tau)] d\tau. \quad (33)$$

式(33)按参考干密度归一化，不能直接以 kg 标记；收缩域使用式(32)。

问题一热容量恒定，总有效蓄热增量应与累计对流输入一致。问题二、三则令

$$E(t) = 2\pi L \sum_i W_i B(C_i)(T_i - T_{\text{ini}}),$$

由式(13)推导

$$\begin{aligned} E(t) - E(0) &= 2\pi L \int_0^t \sum_i W_i B'(C_i)(T_i - T_{\text{ini}}) \dot{C}_i d\tau \\ &= 2\pi LRh \int_0^t (T_a - T_s) d\tau. \end{aligned} \quad (34)$$

组成修正项不能省略。相对残差用收支差除以累计交换量尺度，近零时设正下限；第一问定义见式(41)。第三问在 BDF 接受步连续解上作 4 点、8 点 Gauss 积分对照。表 11 汇总结果，第四问热量仅检验加权半离散方程及温度收敛。各问残差均远低于 10^{-6} 的检

表 11 各问收支检验：相对残差仅针对所选有效模型

问题	水分相对残差	有效热收支相对残差	检验范围
一	2.4254×10^{-11}	2.0484×10^{-9}	0 至 1800 s
二	4.312×10^{-7}	3.565×10^{-8}	0 至 10800 s
三	4.0221×10^{-11}	3.6640×10^{-8}	至报告时刻
四	4.0558×10^{-9}	不作同类累计比较	基准计算至 72 h

查阈值，说明离散通量与所建方程一致；该检验针对本文的有效模型，不涉及潜热、相平衡或端面简化所对应的物理误差。

9.2 空间与时间加密

在相同物理时刻比较固定域共享节点及收缩域共享材料节点，以各配置的节点最大绝对差为指标。表 12 分别列出温度和含水率结果。前两问覆盖全部规定输出点，后两问还覆盖内部节点，第四问比较至 72 h。第二问另以 G_4/G_8 复核，温度、含水率最大差为

表 12 主要网格与时间加密差异

问题	空间比较	ΔT_{space} °C	ΔC_{space} kg/kg	ΔT_{time} °C	ΔC_{time} kg/kg
一	G_4/G_8	5.895×10^{-7}	4.564×10^{-6}	2.874×10^{-8}	1.179×10^{-9}
二	G_2/G_4	2.452×10^{-6}	1.493×10^{-5}	4.108×10^{-7}	2.381×10^{-8}
三	G_8/G_{16}	9.545×10^{-7}	1.537×10^{-5}	1.002×10^{-6}	7.716×10^{-9}
四	G_8/G_{16}	1.012×10^{-5}	8.681×10^{-7}	1.188×10^{-5}	6.383×10^{-8}

6.244×10^{-7} °C、 3.732×10^{-6} kg/kg，复现方式见支撑材料。六张题定表在相应加密下四位小数一致；表中差值为不同计算配置之间的比较结果，不构成真解的误差上界。各配置下的临界时间见表 13。

表 13 不同计算配置下的临界烘干时间（h）

配置	问题三	问题四
G_4	57.4731739851	51.0872487284
G_8	57.4725195022	51.0871229810
G_{16}	57.4723569654	51.0870914570
G_8 时间收紧	57.4725194974	51.0871221801

9.3 独立对照与输出一致性

问题一的 Bessel–Duhamel 温度对照及冻结扩散系数辅助测试见第 5.5 节。问题二在短时制造解测试中保留变系数导数项，温度和含水率最大误差分别为 6.0949×10^{-8} °C 和 8.9808×10^{-9} kg/kg。制造解用于检验实现，不是药材实验数据。

第三问与第二问在 1800、3600、7200、9000 s 的重叠输出比较中，温度、水分最大差分别为 4.66×10^{-7} °C 和 9.12×10^{-7} kg/kg，说明独立起算的两个过程在共享区间内一致。第四问则采用同一附录 4 物性的固定半径退化检查及关闭边界交换的纯收缩检查，以保证退化参照与本问物性一致。

论文表格、Excel 结果与源数组均由同一未舍入数值解生成，并另行核对第三问的准确终时、第四问的当前半径及域外标记。上述检查针对数值实现，不替代物理验证。

9.4 同物性固定半径对照的检验

固定组通过独立物理环层算子检查。空间加密和时间收紧的临界时间差为 0.498231 s 和 0.013113 s；至 132 h 共同保存时刻的温度、水分空间差分别为 1.3522×10^{-5} °C、 1.9052×10^{-6} kg/kg，时间差分别为 1.4730×10^{-5} °C、 6.4161×10^{-8} kg/kg。三组累计水分相对残差均低于 10^{-6} 。129.8448 h 时三组最大含水率均低于 0.15，生产值加经验裕量 1.9437×10^{-7} 后仍达标，支持表 9 的数值可比性。

9.5 局部敏感性与适用边界

式(14)给出扩散系数对状态变量的局部敏感性。在 $T = 50\text{ }^{\circ}\text{C}$ 、 $C = 0.15$ 时，问题三有

$$\frac{\partial \ln D}{\partial T} \approx 0.0369\text{ K}^{-1}, \quad \frac{\partial \ln D}{\partial C} = 20\text{ (kg/kg)}^{-1}.$$

局部升温 $1\text{ }^{\circ}\text{C}$ 对应 D 约 3.69% 的一阶变化，不能等同于时长变化。问题四有 $\partial \ln D / \partial U = 0.30/U^2$ ，阈值处为 $13.3333\text{ (kg/kg)}^{-1}$ ；几何因子满足 $\partial \ln(R^{-2}) / \partial \ln R = -2$ ，其局部方向仍需全过程对照检验。

若事件附近最大值位置不变且 $M'(t_*) \neq 0$ ，一阶扰动给出

$$\delta t_* \approx -\frac{\delta M(t_*)}{M'(t_*)}. \quad (35)$$

后期曲线越平缓，相同含水率误差越可能放大为时间偏差，故需直接检验事件收敛。表 5 的有限输入扰动量化了指定情景的时长变化，不构成全局稳健性保证。

第三问径向近似的二维端面校核已在第 7.6 节给出；其作用是检验本题条件下的维度简化，与本章的离散收支和数值加密检验相互补充。

十 模型评价与结论

10.1 模型特点

四问以统一的环层通量结构描述热湿传递，有限体积法保留变系数散度与收支关系，表面加密分辨早期薄层。全域最大值判据识别最后达标位置，材料坐标则将收缩域映到固定区间。同物性对照进一步量化了给定收缩轨迹对预测时长的影响。

10.2 局限与可推广方向

第三问二维校核支持指定端面条件下的径向近似；第四问结合收缩后端侧面积比减小沿用该近似，但其端面时间误差尚未单独量化。获得端面交换系数数据后，可将轴对称模型推广至收缩情景。

热方程未显式考虑汽化潜热及水分携焓，因此所得温度场应理解为题给物性条件下的有效传热预测。由于题目未提供相变潜热及界面相平衡参数，本文未进一步引入相关机制，其影响有待实验数据进一步验证。

经验密度仅用于有效热容量，水分守恒另按干物质骨架计量。这是一项模型近似：若把经验密度同时解释为真实湿体积密度，则其与给定收缩轨迹、固定长度和干物质守恒不能同时精确相容，具体核算见附录 B。该处理保证所建方程的质量核算一致，但其热学适用性仍需实验检验。

后期环境采用恒定延续假设，且没有药材内部实测数据可供验证。问题三的后期温度 $\pm 2\text{ }^{\circ}\text{C}$ 情景使时长变化约 3.5–3.8 h，后期水分量 $\pm 10\%$ 情景使其变化约 0.09–0.11 h。第四问固定给定半径轨迹的温度情景使收缩时长变化约 3.1–3.3 h，临界时长缩短比例为 60.36% 至 60.93%，见表 10。这些情景差异不能换算为预测误差界或置信区间；四位小数是题目要求，不代表真实烘干时间具有相应精度。

10.3 结论

题给条件下，预热期失水主要集中在外层，温度接近平衡早于内部达到干燥标准。全域阈值计算给出附录 3 固定半径模型时长 57.4731 h、附录 4 收缩模型时长 51.0877 h，均经未舍入末态和数值加密复核。附录 4 同物性固定半径对照为 129.8448 h，收缩使预测时长缩短 78.7571 h (60.65%)；该结论限于本文所用环境、物性及收缩假设。

AI 工具使用声明

本文工作使用了 AI 工具，主要用于建模材料核对、论文起草、同物性固定半径对照与二维端面校核的程序编写和数值验证，以及图表绘制、公式一致性检查和 LaTeX 排版。具体使用范围、交互方式和核验记录见支撑材料中的《AI 工具使用详情》。

参考文献

- [1] Hussain M M, Dincer I. Two-dimensional heat and moisture transfer analysis of a cylindrical moist object subjected to drying: A finite-difference approach[J]. International Journal of Heat and Mass Transfer, 2003, 46(21): 4033–4039. DOI: 10.1016/S0017-9310(03)00229-1.
- [2] Kaya A, Aydin O, Dincer I. Numerical modeling of forced-convection drying of cylindrical moist objects[J]. Numerical Heat Transfer, Part A: Applications, 2007, 51(9): 843–854. DOI: 10.1080/10407780601112753.
- [3] The SciPy community. PchipInterpolator: SciPy v1.18.0 Manual[EB/OL]. [2026-09-12]. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.PchipInterpolator.html>.
- [4] The SciPy community. BDF: SciPy v1.18.0 Manual[EB/OL]. [2026-09-12]. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.BDF.html>.
- [5] Brasiello A, Venditti C, Adrover A. Non-isothermal Moving-boundary Model for Food Drying[J]. Chemical Engineering Transactions, 2021, 87: 193–198. DOI: 10.3303/CET2187033.

附录 A 问题一温度参考解的推导

本附录补充第 5.5 节温度参考解的构造过程。先求恒定环境下的阶跃响应，再对环境温度的连续变化进行叠加。

当烘房温度恒为 T_f 时，令 $u(r, t) = T(r, t) - T_f$ ，取分离变量形式 $u = X(r)G(t)$ 。将其代入常热物性导热方程，得到

$$\frac{G'}{\alpha G} = \frac{X'' + X'/r}{X} = -\frac{\lambda^2}{R^2}.$$

时间部分为 $G(t) = \exp(-\alpha\lambda^2 t/R^2)$ ，径向部分满足

$$r^2 X'' + rX' + \lambda^2 \frac{r^2}{R^2} X = 0.$$

该方程为零阶 Bessel 方程。中心有界性排除第二类 Bessel 函数，取 $X(r) = J_0(\lambda r/R)$ ，其中 J_0, J_1 为第一类 Bessel 函数。由 $J'_0 = -J_1$ 及表面条件 $-ku_r(R, t) = hu(R, t)$ ，得到

$$\lambda_n J_1(\lambda_n) = \text{Bi} J_0(\lambda_n), \quad \text{Bi} = \frac{hR}{k}. \quad (36)$$

以 r 为权展开均匀初始温差 $28 - T_f$ 。令 $x = r/R$ ，利用径向模态的正交性，得到

$$A_n = \frac{\int_0^1 x J_0(\lambda_n x) dx}{\int_0^1 x J_0^2(\lambda_n x) dx} = \frac{2J_1(\lambda_n)}{\lambda_n [J_0^2(\lambda_n) + J_1^2(\lambda_n)]}.$$

因此，恒定环境下的解为

$$T(r, t) = T_f + (28 - T_f) \sum_{n=1}^{\infty} A_n J_0(\lambda_n r/R) e^{-\beta_n t}, \quad \beta_n = \alpha \lambda_n^2 / R^2.$$

对随时间变化的 $T_a(t)$ ，利用热方程的线性性，将环境变化分解为一系列微小阶跃。单位环境阶跃的温度响应为

$$H(r, t) = 1 - \sum_{n=1}^{\infty} A_n J_0(\lambda_n r/R) e^{-\beta_n t}.$$

由 $T_a(0) = 28$ 及 Duhamel 叠加，有

$$T_{\text{ref}}(r, t) = 28 + \int_0^t H(r, t - \tau) T'_a(\tau) d\tau.$$

代入 H 并利用 $\int_0^t T'_a(\tau) d\tau = T_a(t) - 28$ ，得到

$$T_{\text{ref}}(r, t) = T_a(t) - \sum_{n=1}^{\infty} A_n J_0(\lambda_n r/R) \int_0^t e^{-\beta_n(t-\tau)} T'_a(\tau) d\tau. \quad (37)$$

计算中对 PCHIP 插值函数求导。各阶卷积积分记为

$$I_n(t) = \int_0^t e^{-\beta_n(t-\tau)} T'_a(\tau) d\tau,$$

对时间求导可将其改写为相互独立的模态常微分方程

$$\dot{I}_n = -\beta_n I_n + T'_a(t), \quad I_n(0) = 0.$$

由此得到独立于径向有限体积离散的温度参考计算，并通过增加模态数检查级数截断的影响。

附录 B 数值实现细节

本附录补充插值表达式、网格细分方案、时间离散形式、密度相容性核算及问题一的收支残差定义，供复现与核对使用。

B.1 保形分段三次 Hermite 插值

记相邻观测点为 (t_j, f_j) ，区间长度 $h_j = t_{j+1} - t_j$ ， $z = (t - t_j)/h_j$ ，则第四节所用插值为

$$\begin{aligned} f(t) = & (2z^3 - 3z^2 + 1)f_j + (-2z^3 + 3z^2)f_{j+1} \\ & + (z^3 - 2z^2 + z)h_j m_j + (z^3 - z^2)h_j m_{j+1}, \end{aligned} \quad (38)$$

其中节点导数 m_j 由保形规则确定：当相邻两段差商同号时， m_j 取二者的加权调和平均；异号或任一差商为零时取 $m_j = 0$ 。由此保证插值精确通过全部观测点、节点处一阶导数连续，且不在单调区间内产生新的极值。 f 分别取烘房温度 T_a 、水分量 C_a 和第四问的半径 R 。

B.2 表面加密网格的分段细分方案

以问题一为例，将 $0 \leq r \leq R$ 按题目规定的 0.1 cm 输出位置分段，并在 0 至 1.5 cm、1.5 至 1.8 cm、1.8 至 1.9 cm、1.9 至 2.0 cm 四段内分别细分为 $8m$ 、 $16m$ 、 $32m$ 、 $64m$ 份，所得网格记为 G_m ，共 $264m + 1$ 个节点。该方案使全部规定输出位置保持为实际节点，无需空间插值，同时在表面附近提供更高的分辨率。正文各问采用的加密等级为：问题一、三、四取 G_8 (2113 个节点)，问题二取 G_4 (1057 个节点)，加密检验另用 G_2 、 G_{16} 。第四问的材料节点由问题一的 G_8 节点除以 R_0 得到。

B.3 隐式 BDF 的离散形式

空间离散后的半离散方程组记为 $\dot{\mathbf{y}} = \mathbf{f}(t, \mathbf{y})$ 。以等步长二阶 BDF 为例，下一时刻的状态满足

$$\mathcal{R}(\mathbf{y}^{n+1}) = 3\mathbf{y}^{n+1} - 4\mathbf{y}^n + \mathbf{y}^{n-1} - 2\Delta t \mathbf{f}(t_{n+1}, \mathbf{y}^{n+1}) = 0. \quad (39)$$

其中历史状态 $\mathbf{y}^n$ 、 $\mathbf{y}^{n-1}$ 已知，而通量依赖待求的 $\mathbf{y}^{n+1}$ ，故需求解隐式代数方程组；对水分场， D 随未知含水率变化，方程组还具有非线性。每一步先由历史状态预测新状态，再作牛顿型迭代：按试探状态更新物性和通量，计算残差(39)，求解线性修正方程并更新状态，直至满足收敛要求；随后估计局部时间误差，误差满足容差时接受该步，否则缩小步长重算。实际计算调用 SciPy 的变步长、变阶 BDF 实现，式(39)仅用于说明其隐式离散形式，计算并不固定采用二阶公式。

B.4 经验密度与干物质密度的相容性核算

第十节所述相容性检查的完整过程如下。若把附录 4 的 $\rho(U) = 760 + 90U$ 解释为真实湿体积密度，则单位当前体积中的干物质质量应为 $s = \rho(U)/(1 + U)$ ，初始时

$$s_0 = \frac{760 + 90 \times 2.55}{1 + 2.55} \approx 278.73 \text{ kg/m}^3.$$

在长度固定、干物质守恒的前提下，72 h 时半径由 2 cm 降至 1.198 cm，体积比为 $(1.198/2)^2 = 0.358801$ ，故当时的平均干物质密度应为

$$\bar{s}(72 \text{ h}) = \frac{s_0}{0.358801} \approx 776.84 \text{ kg/m}^3.$$

然而对任意 $U \geq 0$,

$$\frac{760 + 90U}{1 + U} = 90 + \frac{670}{1 + U} \leq 760 \text{ kg/m}^3,$$

上界与 U 的径向分布无关，因此非均匀收缩也不能消除该矛盾，两种解释相差约 2.2%。这一核算针对 72 h 的半径数据与上述特定解释，不是报告时刻的误差估计。据此，本文把 ρ_{c_p} 仅作为有效体积热容量使用，水分守恒另按式(22)的干物质骨架计量，二者分别记账。

B.5 问题一的累计收支残差

对式(9)在全部控制体上求和，内部通量成对抵消。定义有效蓄热增量、累计热输入、归一化水分存量与累计排湿量为

$$\begin{aligned} \mathcal{E}_T(t) &= 2\pi L \rho_{c_p} \sum_i W_i [T_i(t) - T_i(0)], \\ Q_T(t) &= 2\pi R L \int_0^t h [T_a(\tau) - T_s(\tau)] d\tau, \\ M_C(t) &= \sum_i W_i C_i(t), \quad L_C(t) = R \int_0^t h_m [C_s(\tau) - C_a(\tau)] d\tau. \end{aligned} \quad (40)$$

应满足 $\mathcal{E}_T = Q_T$ 和 $\Delta M_C = -L_C$ ，其中 $\Delta M_C = M_C(t) - M_C(0)$ 。 M_C 乘以 $2\pi L s_0$ 后才对应水质量。相对残差定义为

$$\varepsilon_T = \frac{|\mathcal{E}_T - Q_T|}{\max(|\mathcal{E}_T|, |Q_T|, \eta_T)}, \quad \varepsilon_{C, \text{bal}} = \frac{|\Delta M_C + L_C|}{\max(|\Delta M_C|, |L_C|, \eta_C)}, \quad (41)$$

其中 $\eta_T = 10^{-12} \text{ J}$ 、 $\eta_C = 10^{-18} \text{ m}^2 \cdot (\text{kg/kg})$ 为近零分母下限。利用连续数值输出，在 PCHIP 观测分区间上对表面通量自适应积分，再与末态存量独立比较，所得 1800 s 残差见表 11。

B.6 潜热遗漏的条件性量级核算

本项只对第一问正式 G_8 解作后处理。取 $V = \pi R^2 L$ ，仅为质量换算而将 820 kg/m^3 解释为初始湿体积密度，则干物质量 $m_d = 820V/(1 + 2.55)$ 。以环层体积加权的末态均值计算失水 $\Delta m_w = m_d(2.55 - \bar{C})$ ；再假设失水全部蒸发，取自由水潜热量级 $L_v = 2.4 \text{ MJ/kg}$ ，比较 $L_v \Delta m_w$ 与显热增量 $820V c_p (\bar{T} - 28)$ 。1800 s 时预测失水 18.6091 g，对应潜热 44.6619 kJ，而显热增量为 4.8015 kJ，比值约 9.30。该比值提示遗漏机制可能重要，但 L_v 不是该药材的实测参数；纳入潜热后表面温度与对流入热均会变化，因此此后处理不能给出修正解或预测误差界。复现程序与计算记录见支撑材料。

附录 C 支撑材料清单与复现说明

支撑材料包含四问完整 Python 程序 Q1.py–Q4.py、四个正式 Excel 结果文件和 AI 工具使用详情。四份程序均内嵌相应问题所需模块，可独立展开运行时目录；输入、输出相对于程序所在目录定位。

运行环境需安装 NumPy、SciPy、openpyxl 和 Matplotlib，并在支撑材料目录中保留题目附件及结果模板。可按下列命令复算各问；使用 `--check` 参数可先检查运行环境和文件完整性。

```
python Q1.py
python Q2.py
python Q3.py
python Q4.py
```

附录 D 完整源程序

D.1 Q1.py

```
1 # -*- coding: utf-8 -*-
2 """ 问题 1: 从原始附件复算答案数据。
3
4 运行 python Q1.py; 检查环境 python Q1.py --check。
5 本文件内嵌本问所需的全部项目模块，模块源码在下方按文件名分段列出。
6 运行时自动展开到相对目录 _runtime/Q1/code，以支持源码哈希核验。
7 输入、输出均相对于本文件的位置，与当前工作目录无关。
8 方程、物性、网格、容差和验证判据沿用原正式求解程序。
9 """
10 import argparse
11 import importlib
12 import os
13 from pathlib import Path
14 import sys
15
16 PACKAGE_ROOT = Path(__file__).resolve().parent
17 QUESTION = 1
18 SOURCES = {}
19
20 # =====
21 # Module: utils.py
22 # =====
23 SOURCES['utils'] = r'""""Shared paths and attachment readers for Problem A."""'
24 from __future__ import annotations
25
26 from pathlib import Path
27
28 import numpy as np
29 from openpyxl import load_workbook
30 from scipy.interpolate import PchipInterpolator
31
32 ROOT = Path(__file__).resolve().parents[3]
33 DATA_DIR = ROOT / "data"
34 ATTACH_DIR = DATA_DIR / " 附件"
35 RESULTS_DIR = ROOT / "results"
36 FIGURES_DIR = ROOT / "figures"
37 REPORTS_DIR = ROOT / "reports"
38
39 # Appendix 2 (Problem 1)
```

```

40 R = 0.02
41 L = 0.25
42 RHO = 820.0
43 CP = 2600.0
44 K = 0.36
45 H = 25.0
46 HM = 8.0e-7
47 T0 = 28.0
48 C0 = 2.55
49 D_PREFACTOR = 7.0e-9
50 D_EXP_COEF = 0.89
51 T_END = 1800.0
52 ALPHA = K / (RHO * CP)
53
54
55 def moisture_diffusivity(C: np.ndarray | float, *, protect: bool = True) -> np.ndarray | float:
56     """D(C) = 7e-9 exp(-0.89/C). Nonpositive C uses D=0 only as a solver-domain guard."""
57     C_arr = np.asarray(C, dtype=float)
58     out = np.zeros_like(C_arr, dtype=float)
59     positive = C_arr > 0.0
60     if np.any(positive):
61         out[positive] = D_PREFACTOR * np.exp(-D_EXP_COEF / C_arr[positive])
62     if not protect and np.any(~positive):
63         raise ValueError("moisture diffusivity evaluated at nonpositive C without protection")
64     if np.isscalar(C):
65         return float(out)
66     return out
67
68
69 def load_oven_table() -> tuple[np.ndarray, np.ndarray, np.ndarray]:
70     path = ATTACH_DIR / " 附件 1.xlsx"
71     wb = load_workbook(path, read_only=True, data_only=True)
72     rows = list(wb.active.values)
73     wb.close()
74     data = np.asarray(rows[1:], dtype=float)
75     t, Ta, Ca = data[:, 0], data[:, 1], data[:, 2]
76     if t[0] != 0.0:
77         raise ValueError(f"oven table must start at t=0, got {t[0]}")
78     return t, Ta, Ca
79
80
81 class PiecewiseLinear:
82     """Piecewise linear interpolant; kept for comparison, not used in Q1 production."""
83
84     def __init__(self, t: np.ndarray, y: np.ndarray):
85         self.t = np.asarray(t, dtype=float)
86         self.y = np.asarray(y, dtype=float)
87         self.kind = "linear"
88         if self.t.ndim != 1 or self.y.shape != self.t.shape:
89             raise ValueError("interpolant requires matching 1-D arrays")
90         if np.any(np.diff(self.t) <= 0):
91             raise ValueError("breakpoints must be strictly increasing")
92
93     def __call__(self, t: float | np.ndarray) -> float | np.ndarray:
94         return np.interp(t, self.t, self.y)
95
96     def slope(self, t: float) -> float:
97         t = float(t)
98         if t <= self.t[0] or t >= self.t[-1]:
99             if t >= self.t[-1]:
100                 dt = self.t[-1] - self.t[-2]
101                 return (self.y[-1] - self.y[-2]) / dt
102             return (self.y[1] - self.y[0]) / (self.t[1] - self.t[0])
103         i = int(np.searchsorted(self.t, t, side="right") - 1)
104         i = min(i, len(self.t) - 2)

```

```

105         return (self.y[i + 1] - self.y[i]) / (self.t[i + 1] - self.t[i])
106
107
108 class PchipOven:
109     """Monotone piecewise cubic Hermite interpolant (PCHIP) through attachment knots."""
110
111     def __init__(self, t: np.ndarray, y: np.ndarray):
112         self.t = np.asarray(t, dtype=float)
113         self.y = np.asarray(y, dtype=float)
114         self.kind = "pchip"
115         if self.t.ndim != 1 or self.y.shape != self.t.shape:
116             raise ValueError("interpolant requires matching 1-D arrays")
117         if np.any(np.diff(self.t) <= 0):
118             raise ValueError("breakpoints must be strictly increasing")
119         self._pchip = PchipInterpolator(self.t, self.y, extrapolate=False)
120         self._deriv = self._pchip.derivative()
121
122     def __call__(self, t: float | np.ndarray) -> float | np.ndarray:
123         tt = np.clip(np.asarray(t, dtype=float), self.t[0], self.t[-1])
124         val = self._pchip(tt)
125         if np.ndim(t) == 0:
126             return float(val)
127         return np.asarray(val, dtype=float)
128
129     def slope(self, t: float) -> float:
130         tt = float(np.clip(t, self.t[0], self.t[-1]))
131         return float(self._deriv(tt))
132
133
134 # =====
135 # Module: delivery.py
136 # =====
137 SOURCES['delivery'] = r'""""Fail-closed numerical delivery checks shared by Q1 and Q2.""'
138 import json
139 from pathlib import Path
140
141
142 def make_gate(checks):
143     checks = {name: bool(value) for name, value in checks.items()}
144     return {"checks": checks, "failed_checks": [k for k, v in checks.items() if not v],
145           "meets_delivery_gate": bool(checks) and all(checks.values())}
146
147
148 def require_delivery(validation, diagnostic_path):
149     """Persist diagnostics first; never reach any production writer on failure."""
150     path = Path(diagnostic_path)
151     path.parent.mkdir(parents=True, exist_ok=True)
152     path.write_text(json.dumps(validation, ensure_ascii=False, indent=2, default=float), encoding="utf-8")
153     gate = validation["delivery_gate"]
154     checks = gate.get("checks", {})
155     if not checks or not all(value is True for value in checks.values()) or gate.get("meets_delivery_gate")
156     ↪ is not True:
157         raise RuntimeError("Delivery blocked; see " + str(path) + ": " + str(gate.get("failed_checks",
158     ↪ [])))
159
160
161 # =====
162 # Module: q1_analytic.py
163 # =====
164 SOURCES['q1_analytic'] = r'""""Independent Bessel-Duhamel reference for the linear radial heat (or
165     ↪ frozen-D moisture) problem.""'
166 from __future__ import annotations
167
168 from typing import Any
169

```

```

167 import numpy as np
168 from scipy.integrate import solve_ivp
169 from scipy.optimize import brentq
170 from scipy.special import j0, j1
171 from scipy.sparse import diags
172
173
174 def robin_eigenvalues(Bi: float, n_roots: int) -> np.ndarray:
175     """Positive roots of  $\lambda^2 J_1(\lambda) = Bi * J_0(\lambda)$ ."""
176     if Bi <= 0:
177         raise ValueError("Biot number must be positive")
178     n_roots = int(n_roots)
179
180     def f(lam: float) -> float:
181         return lam * j1(lam) - Bi * j0(lam)
182
183     lam_max = (n_roots + 4) * np.pi
184     grid = np.linspace(1e-10, lam_max, max(8000, 200 * n_roots))
185     vals = grid * j1(grid) - Bi * j0(grid)
186     roots: list[float] = []
187     for i in range(len(grid) - 1):
188         v0, v1 = vals[i], vals[i + 1]
189         if not np.isfinite(v0) or not np.isfinite(v1):
190             continue
191         a, b = float(grid[i]), float(grid[i + 1])
192         if v0 == 0.0:
193             cand = a
194         elif v0 * v1 < 0.0:
195             cand = float(brentq(f, a, b, xtol=1e-14, maxiter=200))
196         else:
197             continue
198         if not roots or abs(cand - roots[-1]) > 1e-10:
199             roots.append(cand)
200         if len(roots) >= n_roots:
201             break
202     if len(roots) < n_roots:
203         raise RuntimeError(f"found only {len(roots)} Robin Bessel roots, need {n_roots}")
204     return np.asarray(roots, dtype=float)
205
206
207 def expansion_coefficients(lam: np.ndarray) -> np.ndarray:
208     j0l = j0(lam)
209     j1l = j1(lam)
210     return 2.0 * j1l / (lam * (j0l**2 + j1l**2))
211
212
213 class BesselDuhamel:
214     def __init__(self, Bi: float, diffusivity: float, R: float, n_terms: int = 120):
215         self.Bi = float(Bi)
216         self.alpha = float(diffusivity)
217         self.R = float(R)
218         self.lam = robin_eigenvalues(self.Bi, n_terms)
219         self.A = expansion_coefficients(self.lam)
220         self.beta = self.alpha * self.lam**2 / self.R**2
221
222     def radial_modes(self, r: np.ndarray) -> np.ndarray:
223         """Return array shape (n_terms, n_r) of  $A_n J_0(\lambda_n r/R)$ ."""
224         r = np.asarray(r, dtype=float)
225         return self.A[:, None] * j0(self.lam[:, None] * (r[None, :] / self.R))
226
227     def evaluate(self, r: np.ndarray, t: float, env: Any, u0: float) -> np.ndarray:
228         return self.evaluate_many(r, np.array([float(t)]), env, u0)[0]
229
230     def evaluate_many(self, r: np.ndarray, times: np.ndarray, env: Any, u0: float) -> np.ndarray:
231         """ $T = env(t) - \sum A_n J_0 I_n$ , with  $I' = -\beta I + env'(t)$ ,  $I(0)=env(0)-u_0$ ."""

```

```

232
233     Valid for any C1 interpolant that exposes env.y, env.slope, and env(t).
234     """
235     r = np.asarray(r, dtype=float)
236     times = np.asarray(times, dtype=float)
237     if times.size == 0:
238         return np.empty((0, r.size), dtype=float)
239     modes = self.radial_modes(r)
240     beta = self.beta
241     I0 = np.full(beta.size, float(env.y[0] - u0))
242     t_end = float(np.max(times))
243     out = np.empty((times.size, r.size), dtype=float)
244     zero = times == 0.0
245     if np.all(zero):
246         out[:] = u0
247         return out
248     t_eval = times[~zero]
249     sol = solve_ivp(
250         lambda t, I: -beta * I + env.slope(t),
251         (0.0, max(t_end, 1e-30)),
252         I0,
253         method="BDF",
254         jac=diags(-beta, format="csc"),
255         rtol=1e-11,
256         atol=1e-13,
257         t_eval=t_eval,
258         max_step=1.0,
259         vectorized=False,
260     )
261     if not sol.success:
262         raise RuntimeError(f"Bessel Duhamel ODE failed: {sol.message}")
263     Ihist = sol.y.T
264     env_t = np.array([float(env(t)) for t in t_eval], dtype=float)
265     nonzero_vals = env_t[:, None] - Ihist @ modes
266     out[zero] = u0
267     out[~zero] = nonzero_vals
268     return out
269
270
271 def converged_reference(Bi, diffusivity, R, r, times, env, u0, start_terms=320, tolerance=1e-8):
272     """Double the modal truncation and explicitly check every requested point."""
273     previous = BesselDuhamel(Bi, diffusivity, R, start_terms).evaluate_many(r, times, env, u0)
274     history = []
275     for terms in (start_terms * 2, start_terms * 4, start_terms * 8):
276         current = BesselDuhamel(Bi, diffusivity, R, terms).evaluate_many(r, times, env, u0)
277         error = float(np.max(np.abs(current - previous)))
278         history.append({"terms": terms, "previous_terms": terms // 2, "max_abs_difference": error})
279         if np.isfinite(error) and error <= tolerance:
280             return current, {"passed": True, "tolerance": tolerance, "history": history, "terms": terms}
281         previous = current
282     return current, {"passed": False, "tolerance": tolerance, "history": history, "terms": terms}
283
284
285 # =====
286 # Module: radial_fvm.py
287 # =====
288 SOURCES['radial_fvm'] = r'""Node-centered radial finite-volume operators for Problem 1.""'
289 from __future__ import annotations
290
291 import numpy as np
292 from scipy.sparse import diags
293
294 from utils import CP, H, HM, K, R, RHO, moisture_diffusivity
295
296 OUTPUT_RADII_M = 0.001 * np.arange(21)

```

```

297
298
299 def graded_radial_nodes(level: int) -> np.ndarray:
300     """Nodes denser near the surface; every 0.1 cm output radius remains a node.
301
302     Interval subdivisions (before multiplying by ``level``):
303     0–1.5 cm: 8; 1.5–1.8 cm: 16; 1.8–1.9 cm: 32; 1.9–2.0 cm: 64.
304     Surface spacing at level L is 1e-3 / (64 L) metres.
305     """
306     level = int(level)
307     if level < 1:
308         raise ValueError("graded mesh level must be a positive integer")
309     n_sub = np.full(20, 8 * level, dtype=int)
310     n_sub[15:] = 16 * level
311     n_sub[18] = 32 * level
312     n_sub[19] = 64 * level
313     pts = [0.0]
314     for j in range(20):
315         r0 = 0.001 * j
316         r1 = 0.001 * (j + 1)
317         m = int(n_sub[j])
318         for k in range(1, m + 1):
319             pts.append(r0 + (r1 - r0) * k / m)
320     r = np.asarray(pts, dtype=float)
321     r[0] = 0.0
322     r[-1] = R
323     return r
324
325
326 class RadialFVM:
327     def __init__(self, N: int | None = None, r: np.ndarray | None = None):
328         if r is not None:
329             self.r = np.asarray(r, dtype=float)
330             if self.r.ndim != 1 or self.r.size < 3:
331                 raise ValueError("r must be a 1-D array with at least 3 nodes")
332             if abs(self.r[0]) > 1e-15 or abs(self.r[-1] - R) > 1e-14:
333                 raise ValueError("radial nodes must start at 0 and end at R")
334             if np.any(np.diff(self.r) <= 0.0):
335                 raise ValueError("radial nodes must be strictly increasing")
336         else:
337             if N is None or int(N) < 2:
338                 raise ValueError("N must be at least 2")
339             self.r = np.linspace(0.0, R, int(N) + 1)
340             self.n_nodes = int(self.r.size)
341             self.N = self.n_nodes - 1
342             self.dr_face = np.diff(self.r)
343             self.r_half = 0.5 * (self.r[:-1] + self.r[1:])
344             a = np.empty(self.n_nodes)
345             b = np.empty(self.n_nodes)
346             a[0] = 0.0
347             b[0] = self.r_half[0]
348             a[-1] = self.r_half[-1]
349             b[-1] = R
350             if self.n_nodes > 2:
351                 a[1:-1] = self.r_half[:-1]
352                 b[1:-1] = self.r_half[1:]
353             self.W = 0.5 * (b**2 - a**2)
354             self.dr = float(self.dr_face.min())
355             if abs(self.W.sum() - 0.5 * R**2) > 1e-14:
356                 raise RuntimeError("radial weights do not sum to R^2/2")
357
358             self.M_heat = self._heat_matrix()
359             self.M_heat_sparse = diags(
360                 [np.diag(self.M_heat, -1), np.diag(self.M_heat, 0), np.diag(self.M_heat, 1)],
361                 offsets=[-1, 0, 1],

```

```

362         shape=self.M_heat.shape,
363         format="csc",
364     )
365     self._heat_force_coef = (R * H) / (RHO * CP * self.W[-1])
366     self._heat_force_index = self.n_nodes - 1
367
368     def output_indices(self) -> np.ndarray:
369         """Indices of 0, 0.1, ..., 2.0 cm nodes."""
370         idx = np.empty(OUTPUT_RADII_M.size, dtype=int)
371         for j, rc in enumerate(OUTPUT_RADII_M):
372             k = int(np.argmin(np.abs(self.r - rc)))
373             if abs(self.r[k] - rc) > 1e-12:
374                 raise ValueError(f"grid misses output radius {rc}, nearest {self.r[k]}")
375             idx[j] = k
376         return idx
377
378     def _interior_face_coeff(self) -> np.ndarray:
379         return self.r_half / self.dr_face
380
381     def _heat_matrix(self) -> np.ndarray:
382         n = self.n_nodes
383         M = np.zeros((n, n))
384         face = K * self._interior_face_coeff()
385         den = RHO * CP * self.W
386         M[0, 0] = -face[0] / den[0]
387         M[0, 1] = face[0] / den[0]
388         for i in range(1, n - 1):
389             M[i, i - 1] = face[i - 1] / den[i]
390             M[i, i + 1] = face[i] / den[i]
391             M[i, i] = -(face[i - 1] + face[i]) / den[i]
392         i = n - 1
393         M[i, i - 1] = face[i - 1] / den[i]
394         M[i, i] = (-face[i - 1] - R * H) / den[i]
395         return M
396
397     def heat_rhs(self, t: float, T: np.ndarray, Ta: float) -> np.ndarray:
398         out = self.M_heat @ T
399         out[self._heat_force_index] += self._heat_force_coef * Ta
400         return out
401
402     def heat_jac(self, t: float, T: np.ndarray):
403         return self.M_heat_sparse
404
405     def heat_jac_sparsity(self):
406         n = self.n_nodes
407         ones_m = np.ones(n - 1, dtype=float)
408         ones_d = np.ones(n, dtype=float)
409         return diags([ones_m, ones_d, ones_m], offsets=[-1, 0, 1], shape=(n, n), format="csc")
410
411     def moisture_rhs(self, t: float, C: np.ndarray, Ca: float, D_const: float | None = None) -> np.ndarray:
412         if D_const is None:
413             D_node = moisture_diffusivity(C, protect=True)
414         else:
415             D_node = np.full_like(C, float(D_const))
416         D_face = harmonic_mean(D_node[:-1], D_node[1:])
417         F = -self.r_half * D_face * np.diff(C) / self.dr_face
418         Cdot = np.empty_like(C)
419         Cdot[0] = -F[0] / self.W[0]
420         Cdot[1:-1] = (F[:-1] - F[1:]) / self.W[1:-1]
421         F_surf = R * HM * (C[-1] - Ca)
422         Cdot[-1] = (F[-1] - F_surf) / self.W[-1]
423         return Cdot
424
425     def moisture_jac_sparsity(self):
426         return self.heat_jac_sparsity()

```

```

427
428     def surface_heat_influx_density(self, T_s: float, Ta: float) -> float:
429         """Heat into the solid per unit surface area, W/m^2."""
430         return H * (Ta - T_s)
431
432     def surface_moisture_outflux(self, C_s: float, Ca: float) -> float:
433         """Outward moisture flux in the normalized (no rho_d) units, m/s * kg/kg."""
434         return HM * (C_s - Ca)
435
436
437     def harmonic_mean(a: np.ndarray, b: np.ndarray) -> np.ndarray:
438         out = np.zeros_like(a, dtype=float)
439         both = (a > 0.0) & (b > 0.0)
440         out[both] = 2.0 * a[both] * b[both] / (a[both] + b[both])
441         return out
442
443
444 # =====
445 # Module: reproduction_export.py
446 # =====
447 SOURCES['reproduction_export'] = r'""""Portable Excel export from validated solver arrays; no external
↳ runtime."""'
448 from copy import copy
449 import json
450 from pathlib import Path
451
452 import numpy as np
453 from openpyxl import load_workbook
454
455
456 def write_template(root, question, times, fields):
457     root = Path(root)
458     template = root / 'data' / '附件' / '附件 3' / f'result{question}.xlsx'
459     wb = load_workbook(template)
460     if len(wb.worksheets) != len(fields):
461         raise ValueError('Template sheet count does not match the output fields')
462     times = np.asarray(times)
463     if times.ndim != 1 or not np.isfinite(times).all() or not (np.diff(times) > 0).all():
464         raise ValueError('Invalid output time sequence')
465     for ws, values in zip(wb.worksheets, fields):
466         values = np.asarray(values)
467         expected = 22 if question == 4 else 21
468         if values.shape != (len(times), expected):
469             raise ValueError('Output shape does not match the requested radial grid')
470         if np.isinf(values).any() or (question != 4 and np.isnan(values).any()):
471             raise ValueError('Unexpected nonfinite output')
472         header = ws.cell(1, 1).value
473         surface = ws.cell(1, ws.max_column).value
474         number_style = copy(ws.cell(2, 2)._style)
475         header_style = copy(ws.cell(1, 2)._style)
476         ws.delete_rows(1, ws.max_row)
477         ws.append([header] + [j / 10 for j in range(21)] + ([surface] if question == 4 else []))
478         for cell in ws[1]:
479             cell._style = copy(header_style)
480         for t, row in zip(times, values):
481             ws.append([float(t)] + [None if np.isnan(v) else float(v) for v in row])
482         for row in ws.iter_rows(min_row=2, min_col=2):
483             for cell in row:
484                 cell._style = copy(number_style)
485                 cell.number_format = '0.0000'
486         ws.freeze_panes = 'B2'
487     wb.properties.creator = 'Team'
488     wb.properties.lastModifiedBy = 'Team'
489     dest = root / 'results' / f'result{question}.xlsx'
490     dest.parent.mkdir(parents=True, exist_ok=True)

```

```

491     wb.save(dest)
492     wb.close()
493     print(f'Excel saved: results/result{question}.xlsx', flush=True)
494
495
496 def export_saved(root, question):
497     root = Path(root)
498     validation = json.loads((root / 'results' /
499     ↪ f'q{question}_validation.json').read_text(encoding='utf-8'))
500     if question == 4:
501         if validation.get('pass') is not True or not validation.get('checks') or not all(v is True for v in
502         ↪ validation['checks'].values()):
503             raise RuntimeError('Q4 numerical validation did not pass')
504     else:
505         from delivery import require_delivery
506         require_delivery(validation, root / 'results' / f'q{question}_export_validation.json')
507         if question == 3:
508             from q3_closeout import require_q3_delivery
509             require_q3_delivery(validation, root / 'results' / 'q3_export_validation.json')
510         with np.load(root / 'results' / f'q{question}_solution.npz') as z:
511             if question in (1, 2):
512                 write_template(root, question, z['times'], [z['T_out'], z['C_out']])
513             elif question == 3:
514                 write_template(root, question, z['excel_times'], [z['C_out']])
515             else:
516                 write_template(root, question, z['times_s'][1:], [z['C_physical'][1:]])
517     '''
518
519 # =====
520 # Module: problem1.py
521 # =====
522 SOURCES['problem1'] = r'""""Problem 1: radial FVM + BDF for preheating temperature and moisture.""'
523 from __future__ import annotations
524
525 import json
526 import sys
527 import time
528 from pathlib import Path
529
530 import numpy as np
531 from openpyxl import Workbook
532 from scipy.integrate import solve_ivp, quad
533
534 CODE_DIR = Path(__file__).resolve().parent
535 if str(CODE_DIR) not in sys.path:
536     sys.path.insert(0, str(CODE_DIR))
537
538 from q1_analytic import BesselDuhamel, converged_reference
539 from delivery import make_gate, require_delivery
540 from radial_fvm import RadialFVM
541 from radial_fvm import graded_radial_nodes
542 from utils import (
543     ALPHA,
544     C0,
545     CP,
546     FIGURES_DIR,
547     H,
548     HM,
549     K,
550     L,
551     R,
552     RESULTS_DIR,
553     RHO,
554     T0,
555     T_END,

```

```

554     PchipOven,
555     load_oven_table,
556     moisture_diffusivity,
557 )
558
559 OUTPUT_TIMES = np.arange(1.0, T_END + 0.5, 1.0)
560 TABLE_TIMES = np.array([100, 300, 600, 900, 1200, 1500, 1800], dtype=float)
561 TABLE_RADII_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
562 OUTPUT_RADII_M = 0.001 * np.arange(21) # 0, 0.1, ..., 2.0 cm
563 PAPER_TIMES = TABLE_TIMES.copy()
564
565
566 def load_q1_inputs() -> tuple[PchipOven, PchipOven, np.ndarray]:
567     t, Ta, Ca = load_oven_table()
568     mask = t <= T_END + 1e-12
569     t, Ta, Ca = t[mask], Ta[mask], Ca[mask]
570     if t.size != 31 or abs(t[-1] - T_END) > 1e-12:
571         raise ValueError(f"expected 31 oven records on [0, 1800], got {t.size}, t_end={t[-1]}")
572     if abs(Ta[0] - T0) > 1e-12:
573         raise ValueError(f"Ta(0)={Ta[0]} != T0={T0}")
574     Ta_fun = PchipOven(t, Ta)
575     Ca_fun = PchipOven(t, Ca)
576     for tj, yj in zip(t, Ta):
577         if abs(float(Ta_fun(tj)) - float(yj)) > 1e-12:
578             raise RuntimeError(f"PCHIP misses Ta knot t={tj}")
579     for tj, yj in zip(t, Ca):
580         if abs(float(Ca_fun(tj)) - float(yj)) > 1e-12:
581             raise RuntimeError(f"PCHIP misses Ca knot t={tj}")
582     return Ta_fun, Ca_fun, t
583
584
585 def integrate_mol(
586     rhs,
587     y0: np.ndarray,
588     t_breaks: np.ndarray,
589     *,
590     rtol: float,
591     atol: float,
592     max_step: float,
593     jac=None,
594     jac_sparsity=None,
595     field_name: str = "field",
596 ):
597     y = np.asarray(y0, dtype=float).copy()
598     times = [0.0]
599     states = [y.copy()]
600     pieces: list[tuple[float, float, object]] = []
601     t_end = float(t_breaks[-1])
602     for i in range(len(t_breaks) - 1):
603         t0 = float(t_breaks[i])
604         t1 = float(min(t_breaks[i + 1], t_end))
605         if t1 <= t0:
606             continue
607         t_eval = np.arange(np.floor(t0) + 1.0, np.floor(t1) + 1.0 + 1e-12, 1.0)
608         t_eval = t_eval[(t_eval > t0 + 1e-14) & (t_eval <= t1 + 1e-14)]
609         kwargs = dict(
610             fun=rhs,
611             t_span=(t0, t1),
612             y0=y,
613             method="BDF",
614             rtol=rtol,
615             atol=atol,
616             max_step=max_step,
617             dense_output=True,
618             vectorized=False,

```

```

619     )
620     if t_eval.size:
621         kwargs["t_eval"] = t_eval
622     if jac is not None:
623         kwargs["jac"] = jac
624     if jac_sparsity is not None:
625         kwargs["jac_sparsity"] = jac_sparsity
626     sol = solve_ivp(**kwargs)
627     if not sol.success:
628         raise RuntimeError(f"{field_name} BDF failed on [{t0}, {t1}]: {sol.message}")
629     pieces.append((t0, t1, sol))
630     if sol.t.size:
631         for k, tk in enumerate(sol.t):
632             if times and abs(tk - times[-1]) < 1e-12:
633                 states[-1] = sol.y[:, k].copy()
634                 continue
635             times.append(float(tk))
636             states.append(sol.y[:, k].copy())
637     y = sol.y[:, -1].copy() if sol.y.size else y
638     t_arr = np.asarray(times, dtype=float)
639     Y = np.stack(states, axis=0)
640     return t_arr, Y, pieces
641
642
643 def dense_state(pieces, t: float) -> np.ndarray:
644     t = float(t)
645     for t0, t1, sol in pieces:
646         if t0 - 1e-12 <= t <= t1 + 1e-12:
647             return np.asarray(sol.sol(np.clip(t, t0, t1)), dtype=float)
648     raise ValueError(f"time {t} not covered by dense output")
649
650
651 def extract_output_grid(fvm: RadialFVM, Y: np.ndarray) -> np.ndarray:
652     idx = fvm.output_indices()
653     return Y[:, idx]
654
655
656 def conservation_heat(fvm: RadialFVM, pieces, Ta_fun: PchipOven, T_init: np.ndarray, t: float) -> dict:
657     T_t = dense_state(pieces, t)
658     E = 2.0 * np.pi * L * RHO * CP * np.dot(fvm.W, T_t - T_init)
659     flux = lambda x: float(Ta_fun(x)) - float(dense_state(pieces, x)[-1])
660     Qraw, qerr = quad(flux, 0, t, points=Ta_fun.t[(Ta_fun.t > 0) & (Ta_fun.t < t)], epsabs=1e-9,
661                      ↪ epsrel=1e-10, limit=500)
662     Q = 2.0 * np.pi * R * L * H * Qraw
663     denom = max(abs(Q), abs(E), 1e-12)
664     return {
665         "E_relative_J": E,
666         "Q_surface_J": Q,
667         "relative_residual": abs(E - Q) / denom,
668         "quadrature_abs_error": qerr, "quadrature": "adaptive QUADPACK on PCHIP intervals",
669     }
670
671 def conservation_moisture(fvm: RadialFVM, pieces, Ca_fun: PchipOven, C_init: np.ndarray, t: float) -> dict:
672     C_t = dense_state(pieces, t)
673     MC = float(np.dot(fvm.W, C_t))
674     MC0 = float(np.dot(fvm.W, C_init))
675     flux = lambda x: HM * (float(dense_state(pieces, x)[-1]) - float(Ca_fun(x)))
676     I, qerr = quad(flux, 0, t, points=Ca_fun.t[(Ca_fun.t > 0) & (Ca_fun.t < t)], epsabs=1e-14,
677                  ↪ epsrel=1e-10, limit=500)
678     rhs = MC0 - R * I
679     denom = max(abs(MC0 - MC), abs(R * I), 1e-18)
680     return {
681         "MC": MC,
682         "MC0": MC0,

```

```

682     "minus_R_int_hm": rhs,
683     "relative_residual": abs(MC - rhs) / denom,
684     "quadrature_abs_error": qerr, "quadrature": "adaptive QUADPACK on PCHIP intervals",
685 }
686
687
688 def compare_on_output(A: np.ndarray, B: np.ndarray, times: np.ndarray) -> dict:
689     diff = np.abs(A - B)
690     k = int(np.argmax(diff))
691     n_t, n_r = diff.shape
692     i, j = divmod(k, n_r)
693     return {
694         "max_abs": float(diff.max()),
695         "max_time_s": float(times[i]),
696         "max_radius_cm": 0.1 * j,
697         "mean_abs": float(diff.mean()),
698         "paper_table_max_abs": float(np.abs(A[np.isin(times, TABLE_TIMES)][:, ::5] - B[np.isin(times,
        ↪ TABLE_TIMES)][:, ::5]).max())
699         if A.shape[1] == 21
700         else None,
701     }
702
703
704 def four_decimal_stats(A: np.ndarray, B: np.ndarray, times: np.ndarray) -> dict:
705     Ar = np.round(A, 4)
706     Br = np.round(B, 4)
707     changed = Ar != Br
708     paper_t = np.isin(times, TABLE_TIMES)
709     paper_r = slice(None, None, 5) if A.shape[1] == 21 else slice(None)
710     paper_changed = Ar[paper_t][:, paper_r] != Br[paper_t][:, paper_r]
711     early = times <= 100.0
712     return {
713         "changed_cells": int(np.sum(changed)),
714         "total_cells": int(A.size),
715         "paper_changed_cells": int(np.sum(paper_changed)),
716         "paper_total_cells": int(paper_changed.size),
717         "early100_changed_cells": int(np.sum(changed[early])),
718         "max_abs": float(np.max(np.abs(A - B))),
719         "paper_table_max_abs": float(np.max(np.abs(A[paper_t][:, paper_r] - B[paper_t][:, paper_r]))),
720     }
721
722
723 def solve_temperature(fvm: RadialFVM, Ta_fun: PchipOven, t_breaks, rtol, atol, max_step):
724     T_init = np.full(fvm.n_nodes, T0)
725     def rhs(t, T):
726         return fvm.heat_rhs(t, T, float(Ta_fun(t)))
727     def jac(t, T):
728         return fvm.heat_jac(t, T)
729     return integrate_mol(
730         rhs, T_init, t_breaks,
731         rtol=rtol, atol=atol, max_step=max_step,
732         jac=jac, jac_sparsity=fvm.heat_jac_sparsity(),
733         field_name="temperature",
734     )
735
736
737 def solve_moisture(fvm: RadialFVM, Ca_fun: PchipOven, t_breaks, rtol, atol, max_step, D_const=None):
738     C_init = np.full(fvm.n_nodes, C0)
739     def rhs(t, C):
740         return fvm.moisture_rhs(t, C, float(Ca_fun(t)), D_const=D_const)
741     return integrate_mol(
742         rhs, C_init, t_breaks,
743         rtol=rtol, atol=atol, max_step=max_step,
744         jac_sparsity=fvm.moisture_jac_sparsity(),
745         field_name="moisture",

```

```

746     )
747
748
749 def assert_physical(T_out, C_out, Ta_fun, Ca_fun) -> dict:
750     Ta_max = float(np.max(Ta_fun.y))
751     Ta_min = float(np.min(Ta_fun.y))
752     Ca_min = float(np.min(Ca_fun.y))
753     info = {
754         "T_min": float(T_out.min()),
755         "T_max": float(T_out.max()),
756         "C_min": float(C_out.min()),
757         "C_max": float(C_out.max()),
758         "T_below_28": float(max(0.0, 28.0 - T_out.min())),
759         "T_above-Ta_max": float(max(0.0, T_out.max() - Ta_max)),
760         "C_above-C0": float(max(0.0, C_out.max() - C0)),
761         "C_below-Ca_min": float(max(0.0, Ca_min - C_out.min())),
762         "Ta_range": [Ta_min, Ta_max],
763         "Ca_range": [Ca_min, float(np.max(Ca_fun.y))],
764     }
765     info["T_bounds_ok"] = info["T_below_28"] <= 1e-6 and info["T_above-Ta_max"] <= 1e-6
766     info["C_bounds_ok"] = info["C_above-C0"] <= 1e-8 and info["C_below-Ca_min"] <= 1e-8
767     info["C_nonnegative"] = bool(C_out.min() >= -1e-8)
768     return info
769
770
771 def write_result_workbook(path: Path, T_out: np.ndarray, C_out: np.ndarray, times: np.ndarray):
772     radii_cm = 0.1 * np.arange(21)
773     wb = Workbook()
774     for title, arr in ((" 温度", T_out), (" 水分浓度", C_out)):
775         ws = wb.create_sheet(title)
776         ws["A1"] = " 时间\\到药材中心的距离"
777         for j, rc in enumerate(radii_cm, start=2):
778             ws.cell(1, j, float(rc))
779             for i, t in enumerate(times):
780                 ws.cell(i + 2, 1, int(round(float(t))))
781                 for j in range(21):
782                     cell = ws.cell(i + 2, j + 2, float(arr[i, j]))
783                     cell.number_format = "0.0000"
784     if "Sheet" in wb.sheetnames:
785         del wb["Sheet"]
786     path.parent.mkdir(parents=True, exist_ok=True)
787     wb.save(path)
788
789
790 def write_paper_csv(path: Path, arr_out: np.ndarray, times: np.ndarray, radii_cm=TABLE_RADII_CM):
791     t_idx = {float(t): i for i, t in enumerate(times)}
792     r_idx = {round(0.1 * j, 10): j for j in range(21)}
793     lines = ["t_s," + ",".join(f"r_{rc:g}cm" for rc in radii_cm)]
794     for t in TABLE_TIMES:
795         row = [f"{t:.0f}"]
796         i = t_idx[float(t)]
797         for rc in radii_cm:
798             j = r_idx[float(rc)]
799             row.append(f"{arr_out[i, j]:.8f}")
800         lines.append(",".join(row))
801     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
802
803
804 def setup_mpl():
805     import matplotlib
806     matplotlib.use("Agg")
807     import matplotlib.pyplot as plt
808     plt.rcParams.update({
809         "font.sans-serif": ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "DejaVu Sans"],
810         "axes.unicode_minus": False,

```

```

811         "font.size": 9,
812         "axes.labelsize": 10,
813         "legend.fontsize": 8,
814         "xtick.labelsize": 8,
815         "ytick.labelsize": 8,
816         "axes.linewidth": 0.8,
817         "pdf.fonttype": 42,
818         "ps.fonttype": 42,
819     })
820     return plt
821
822
823 def plot_figures(plt, fvm, times, T_full, C_full, T_out, C_out, Ta_fun, Ca_fun):
824     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
825     r_cm = fvm.r * 100.0
826     r_out_cm = OUTPUT_RADII_M * 100.0
827     t_marks = PAPER_TIMES
828     idx = {float(t): i for i, t in enumerate(times)}
829
830     fig, ax = plt.subplots(figsize=(6.4, 3.8))
831     for t in t_marks:
832         ax.plot(r_cm, T_full[idx[float(t)]], lw=1.2, label=fr"$t={int(t)}\, \mathrm{{{s}}}$")
833     ax.set_xlabel(" 半径 $r$ (cm)")
834     ax.set_ylabel(" 温度 $T$ (°C)")
835     ax.set_xlim(0, 2)
836     ax.legend(frameon=False, ncol=2)
837     fig.tight_layout()
838     fig.savefig(FIGURES_DIR / "q1_T_profiles.pdf")
839     plt.close(fig)
840
841     fig, ax = plt.subplots(figsize=(6.4, 3.8))
842     for t in t_marks:
843         ax.plot(r_cm, C_full[idx[float(t)]], lw=1.2, label=fr"$t={int(t)}\, \mathrm{{{s}}}$")
844     ax.set_xlabel(" 半径 $r$ (cm)")
845     ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
846     ax.set_xlim(0, 2)
847     ax.legend(frameon=False, ncol=2)
848     fig.tight_layout()
849     fig.savefig(FIGURES_DIR / "q1_C_profiles.pdf")
850     plt.close(fig)
851
852     t_plot = np.concatenate([[0.0], times])
853     T_c = np.concatenate([[T0], T_out[:, 0]])
854     T_s = np.concatenate([[T0], T_out[:, -1]])
855     Ta = np.array([Ta_fun(t) for t in t_plot])
856     fig, ax = plt.subplots(figsize=(6.4, 3.8))
857     ax.plot(t_plot, Ta, lw=1.0, ls="--", label=" 烘房 $T_a$")
858     ax.plot(t_plot, T_s, lw=1.2, label=" 表面")
859     ax.plot(t_plot, T_c, lw=1.2, label=" 中心")
860     ax.set_xlabel(" 时间 $t$ (s)")
861     ax.set_ylabel(" 温度 (°C)")
862     ax.set_xlim(0, 1800)
863     ax.legend(frameon=False)
864     fig.tight_layout()
865     fig.savefig(FIGURES_DIR / "q1_T_center_surface.pdf")
866     plt.close(fig)
867
868     C_c = np.concatenate([[C0], C_out[:, 0]])
869     C_s = np.concatenate([[C0], C_out[:, -1]])
870     fig, ax = plt.subplots(figsize=(6.4, 3.8))
871     ax.plot(t_plot, C_s, lw=1.2, label=" 表面")
872     ax.plot(t_plot, C_c, lw=1.2, label=" 中心")
873     ax.set_xlabel(" 时间 $t$ (s)")
874     ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
875     ax.set_xlim(0, 1800)

```

```

876 ax.legend(frameon=False)
877 fig.tight_layout()
878 fig.savefig(FIGURES_DIR / "q1_C_center_surface.pdf")
879 plt.close(fig)
880
881 fig, ax1 = plt.subplots(figsize=(6.4, 3.6))
882 ax2 = ax1.twinx()
883 t_dense = np.linspace(0.0, T_END, 1801)
884 ax1.plot(t_dense, np.array([Ta_fun(t) for t in t_dense]), color="C0", lw=1.2, label=" 烘房温度")
885 ax2.plot(t_dense, np.array([Ca_fun(t) for t in t_dense]), color="C1", lw=1.2, ls="--",
886 ↪ label="烘房水分")
886 ax1.scatter(Ta_fun.t, Ta_fun.y, s=14, color="C0", zorder=3)
887 ax2.scatter(Ca_fun.t, Ca_fun.y, s=14, color="C1", zorder=3)
888 ax1.set_xlabel(" 时间 $t$ (s)")
889 ax1.set_ylabel(" 烘房温度 (°C)", color="C0")
890 ax2.set_ylabel(" 烘房水分 (kg/kg)", color="C1")
891 ax1.set_xlim(0, 1800)
892 h1, l1 = ax1.get_legend_handles_labels()
893 h2, l2 = ax2.get_legend_handles_labels()
894 ax1.legend(h1 + h2, l1 + l2, frameon=False)
895 fig.tight_layout()
896 fig.savefig(FIGURES_DIR / "q1_oven_input.pdf")
897 plt.close(fig)
898 _ = r_out_cm
899
900
901 def main():
902     t_wall0 = time.perf_counter()
903     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
904     Ta_fun, Ca_fun, knots = load_q1_inputs()
905     # PCHIP is C1; integrate as one span rather than 60 s linear kinks.
906     t_breaks = np.array([0.0, T_END], dtype=float)
907     print(
908         "oven points", knots.size,
909         "kind", Ta_fun.kind,
910         "Ta(1800)=", Ta_fun(T_END),
911         "Ca(1800)=", Ca_fun(T_END),
912     )
913     print("alpha", ALPHA, "D(C0)", moisture_diffusivity(C0), "Bi_T", H * R / K)
914
915     mesh_levels = [2, 4, 8]
916     rtol, atol_T, atol_C, max_step = 1e-10, 1e-10, 1e-12, 0.5
917     solutions = {}
918     for level in mesh_levels:
919         r = graded_radial_nodes(level)
920         fvm = RadialFVM(r=r)
921         tag = f"G{level}"
922         print(
923             f"\n=== {tag} nodes={fvm.n_nodes} surface_dr={fvm.dr_face[-1]:.4e} m ==="
924         )
925         print(f"=== {tag} temperature ===")
926         tT, YT, piecesT = solve_temperature(fvm, Ta_fun, t_breaks, rtol, atol_T, max_step)
927         print(f"=== {tag} moisture ===")
928         tC, YC, piecesC = solve_moisture(fvm, Ca_fun, t_breaks, rtol, atol_C, max_step)
929         if not np.allclose(tT, tC):
930             raise RuntimeError("temperature and moisture output times differ")
931         times = tT[tT > 0]
932         YT_pos = YT[tT > 0]
933         YC_pos = YC[tC > 0]
934         T_out = extract_output_grid(fvm, YT_pos)
935         C_out = extract_output_grid(fvm, YC_pos)
936         solutions[level] = dict(
937             tag=tag,
938             level=level,
939             fvm=fvm,

```

```

940         times=times,
941         YT=YT_pos,
942         YC=YC_pos,
943         T_out=T_out,
944         C_out=C_out,
945         piecesT=piecesT,
946         piecesC=piecesC,
947         T_init=np.full(fvm.n_nodes, T0),
948         C_init=np.full(fvm.n_nodes, C0),
949         n_nodes=fvm.n_nodes,
950         surface_dr=float(fvm.dr_face[-1]),
951     )
952     print(
953         "done", tag,
954         "T range", T_out.min(), T_out.max(),
955         "C range", C_out.min(), C_out.max(),
956         "C(t=1,s)", C_out[0, -1],
957         "C(t=100,s)", C_out[99, -1],
958     )
959
960     conv = {}
961     fourdec = {}
962     for coarse, fine in ((2, 4), (4, 8)):
963         t = solutions[fine]["times"]
964         conv[f"T_G{coarse}_vs_G{fine}"] = compare_on_output(
965             solutions[coarse]["T_out"], solutions[fine]["T_out"], t
966         )
967         conv[f"C_G{coarse}_vs_G{fine}"] = compare_on_output(
968             solutions[coarse]["C_out"], solutions[fine]["C_out"], t
969         )
970         fourdec[f"T_G{coarse}_vs_G{fine}"] = four_decimal_stats(
971             solutions[coarse]["T_out"], solutions[fine]["T_out"], t
972         )
973         fourdec[f"C_G{coarse}_vs_G{fine}"] = four_decimal_stats(
974             solutions[coarse]["C_out"], solutions[fine]["C_out"], t
975         )
976         print("conv T", coarse, fine, conv[f"T_G{coarse}_vs_G{fine}"])
977         print("conv C", coarse, fine, conv[f"C_G{coarse}_vs_G{fine}"])
978         print("4dp T", coarse, fine, fourdec[f"T_G{coarse}_vs_G{fine}"])
979         print("4dp C", coarse, fine, fourdec[f"C_G{coarse}_vs_G{fine}"])
980
981     fd_T = fourdec["T_G4_vs_G8"]
982     fd_C = fourdec["C_G4_vs_G8"]
983     paper_4dp_stable = (
984         fd_T["paper_changed_cells"] == 0 and fd_C["paper_changed_cells"] == 0
985     )
986     full_4dp_stable = fd_T["changed_cells"] == 0 and fd_C["changed_cells"] == 0
987     abs_2e5 = (
988         conv["T_G4_vs_G8"]["max_abs"] <= 2e-5
989         and conv["C_G4_vs_G8"]["max_abs"] <= 2e-5
990     )
991     pub_level = 4 if paper_4dp_stable and full_4dp_stable else 8
992     pub = solutions[pub_level]
993     fvm = pub["fvm"]
994     times = pub["times"]
995     print(
996         "publishing", pub["tag"],
997         "n_nodes", pub["n_nodes"],
998         "paper_4dp_stable", paper_4dp_stable,
999         "full_4dp_stable", full_4dp_stable,
1000         "abs_2e5", abs_2e5,
1001     )
1002
1003     print("time refinement on published mesh")
1004     tT2, YT2, _ = solve_temperature(fvm, Ta_fun, t_breaks, rtol / 5, atol_T / 5, 0.25)

```

```

1005     tC2, YC2, _ = solve_moisture(fvm, Ca_fun, t_breaks, rtol / 5, atol_C / 5, 0.25)
1006     T2 = extract_output_grid(fvm, YT2[tT2 > 0])
1007     C2 = extract_output_grid(fvm, YC2[tC2 > 0])
1008     time_sens = {
1009         "T": compare_on_output(pub["T_out"], T2, times),
1010         "C": compare_on_output(pub["C_out"], C2, times),
1011     }
1012     time_4dp = {
1013         "T": four_decimal_stats(pub["T_out"], T2, times),
1014         "C": four_decimal_stats(pub["C_out"], C2, times),
1015     }
1016     print("time sens", time_sens)
1017     print("time 4dp", time_4dp)
1018
1019     print("Bessel temperature reference")
1020     Bi = H * R / K
1021     T_ref, heat_truncation = converged_reference(Bi, ALPHA, R, OUTPUT_RADII_M, times, Ta_fun, T0,
1022         ↪ start_terms=120)
1023     bessel_T = compare_on_output(pub["T_out"], T_ref, times)
1024     print("Bessel T", bessel_T)
1025
1026     print("frozen-D moisture probe")
1027     D0 = float(moisture_diffusivity(C0))
1028     Bi_m = HM * R / D0
1029     C_ref, moisture_truncation = converged_reference(Bi_m, D0, R, OUTPUT_RADII_M, times, Ca_fun, C0)
1030     tCf, YCf, _ = solve_moisture(fvm, Ca_fun, t_breaks, rtol, atol_C, max_step, D_const=D0)
1031     C_frozen = extract_output_grid(fvm, YCf[tCf > 0])
1032     frozen = compare_on_output(C_frozen, C_ref, times)
1033     print("frozen D vs analytic", frozen)
1034     nl_vs_frozen = compare_on_output(pub["C_out"], C_frozen, times)
1035     print("nonlinear vs frozen D", nl_vs_frozen)
1036     D_field = moisture_diffusivity(pub["YC"])
1037     D_stats = {
1038         "D0": D0,
1039         "D_min": float(np.min(D_field)),
1040         "D_max": float(np.max(D_field)),
1041         "D_min_over_D0": float(np.min(D_field) / D0),
1042         "D_max_over_D0": float(np.max(D_field) / D0),
1043         "C_min_field": float(np.min(pub["YC"])),
1044         "C_max_field": float(np.max(pub["YC"])),
1045         "surface_C_min": float(np.min(pub["C_out"][:, -1])),
1046         "center_C_end": float(pub["C_out"][-1, 0]),
1047         "surface_C_end": float(pub["C_out"][-1, -1]),
1048         "surface_C_t1": float(pub["C_out"][0, -1]),
1049         "surface_C_t100": float(pub["C_out"][99, -1]),
1050     }
1051     print("D stats", D_stats)
1052
1053     bounds = assert_physical(pub["YT"], pub["YC"], Ta_fun, Ca_fun)
1054     print("bounds", bounds)
1055
1056     cons_T = conservation_heat(fvm, pub["piecesT"], Ta_fun, pub["T_init"], T_END)
1057     cons_C = conservation_moisture(fvm, pub["piecesC"], Ca_fun, pub["C_init"], T_END)
1058     print("conservation T", cons_T)
1059     print("conservation C", cons_C)
1060
1061     delivery = make_gate({
1062         "space_abs": abs_2e5,
1063         "space_paper": paper_4dp_stable,
1064         "all_finite": np.isfinite(pub["YT"]).all() and np.isfinite(pub["YC"]).all(),
1065         "heat_reference_truncation": heat_truncation["passed"],
1066         "heat_reference_error": bessel_T["max_abs"] <= 2e-5,
1067         "time_abs": time_sens["T"]["max_abs"] <= 2e-5 and time_sens["C"]["max_abs"] <= 2e-5,
1068         "time_paper": time_4dp["T"]["paper_changed_cells"] == 0 and time_4dp["C"]["paper_changed_cells"] ==
1069         ↪ 0,

```

```

1068     "temperature_bounds": bounds["T_bounds_ok"],
1069     "moisture_bounds": bounds["C_bounds_ok"],
1070     "moisture_reference_truncation": moisture_truncation["passed"],
1071     "frozen_reference_error": frozen["max_abs"] <= 2e-5,
1072     "heat_balance": cons_T["relative_residual"] <= 1e-6,
1073     "moisture_balance": cons_C["relative_residual"] <= 1e-6,
1074 })
1075 print("delivery", delivery)
1076
1077 paper_T = {
1078     f"t{int(t)}_r{rc:g}": float(pub["T_out"][int(t) - 1, int(round(rc / 0.1))])
1079     for t in TABLE_TIMES for rc in TABLE_RADII_CM
1080 }
1081 paper_C = {
1082     f"t{int(t)}_r{rc:g}": float(pub["C_out"][int(t) - 1, int(round(rc / 0.1))])
1083     for t in TABLE_TIMES for rc in TABLE_RADII_CM
1084 }
1085
1086 validation = {
1087     "result_version": "q1-closeout-v1",
1088     "interpolation": Ta_fun.kind,
1089     "published_mesh": pub["tag"],
1090     "published_n_nodes": pub["n_nodes"],
1091     "published_surface_dr_m": pub["surface_dr"],
1092     "mesh_levels": mesh_levels,
1093     "mesh_n_nodes": {f"G{lv}": solutions[lv]["n_nodes"] for lv in mesh_levels},
1094     "solver": "FVM+BDF",
1095     "rtol": rtol,
1096     "atol_T": atol_T,
1097     "atol_C": atol_C,
1098     "max_step": max_step,
1099     "t_breaks": t_breaks.tolist(),
1100     "grid_convergence": conv,
1101     "four_decimal_grid": fourdec,
1102     "time_sensitivity": time_sens,
1103     "four_decimal_time": time_4dp,
1104     "delivery_gate": delivery,
1105     "reference_truncation_T": heat_truncation,
1106     "bessel_temperature": bessel_T,
1107     "reference_truncation_C": moisture_truncation,
1108     "frozen_D_moisture_vs_analytic": frozen,
1109     "nonlinear_C_vs_frozen_D": nl_vs_frozen,
1110     "D_variation": D_stats,
1111     "conservation_T_1800s": cons_T,
1112     "conservation_C_1800s": cons_C,
1113     "physical_bounds": bounds,
1114     "paper_table_T": paper_T,
1115     "paper_table_C": paper_C,
1116     "elapsed_s": time.perf_counter() - t_wall0,
1117     "python": sys.version,
1118 }
1119 require_delivery(validation, RESULTS_DIR / "diagnostics/q1_delivery/latest.json")
1120 np.savez(
1121     RESULTS_DIR / "q1_solution.npz",
1122     N=pub["n_nodes"] - 1,
1123     mesh_level=pub_level,
1124     n_nodes=pub["n_nodes"],
1125     times=times,
1126     r=fvm.r,
1127     checkpoint_times=np.r_[0., TABLE_TIMES],
1128     T_checkpoints=np.vstack([pub["T_init"], pub["YT"][TABLE_TIMES.astype(int)-1]]),
1129     C_checkpoints=np.vstack([pub["C_init"], pub["YC"][TABLE_TIMES.astype(int)-1]]),
1130     T_full=pub["YT"],
1131     C_full=pub["YC"],
1132     T_out=pub["T_out"],

```

```

1133     C_out=pub["C_out"],
1134     output_radai_m=OUTPUT_RADII_M,
1135     interpolation=np.array(Ta_fun.kind),
1136     rtol=rtol,
1137     atol_T=atol_T,
1138     atol_C=atol_C,
1139     max_step=max_step,
1140     surface_dr=pub["surface_dr"],
1141 )
1142 write_result_workbook(RESULTS_DIR / "result1.xlsx", pub["T_out"], pub["C_out"], times)
1143 write_paper_csv(RESULTS_DIR / "q1_table_temperature.csv", pub["T_out"], times)
1144 write_paper_csv(RESULTS_DIR / "q1_table_moisture.csv", pub["C_out"], times)
1145
1146 plt = setup_mpl()
1147 plot_figures(plt, fvm, times, pub["YT"], pub["YC"], pub["T_out"], pub["C_out"], Ta_fun, Ca_fun)
1148
1149 (RESULTS_DIR / "q1_validation.json").write_text(
1150     json.dumps(validation, ensure_ascii=False, indent=2, default=float),
1151     encoding="utf-8",
1152 )
1153 print("elapsed_s", validation["elapsed_s"])
1154 print("wrote results to", RESULTS_DIR)
1155
1156
1157
1158 if __name__ == "__main__":
1159     main()
1160 '''
1161
1162 def prepare():
1163     code_dir = PACKAGE_ROOT / '_runtime' / f'Q{QUESTION}' / 'code'
1164     code_dir.mkdir(parents=True, exist_ok=True)
1165     for name, source in SOURCES.items():
1166         (code_dir / (name + '.py')).write_text(source, encoding='utf-8')
1167     sys.path.insert(0, str(code_dir))
1168     for folder in ('results', 'figures', 'reports'):
1169         (PACKAGE_ROOT / folder).mkdir(exist_ok=True)
1170     os.environ.setdefault('MPLBACKEND', 'Agg')
1171     for name in SOURCES:
1172         importlib.import_module(name)
1173
1174
1175 def main():
1176     parser = argparse.ArgumentParser(description=__doc__)
1177     parser.add_argument('--check', action='store_true', help='Check imports and original inputs without
1178 ↪ solving')
1179     parser.add_argument('--export-only', action='store_true', help='Export existing validated NPZ arrays;
1180 ↪ does not solve the model')
1178     args = parser.parse_args()
1179     prepare()
1180     from utils import load_oven_table
1181     load_oven_table()
1182     if QUESTION == 4:
1183         from problem4 import Inputs
1184         Inputs()
1185     if args.check:
1186         print(f'Q{QUESTION}: imports and input checks passed', flush=True)
1187         return
1188     from reproduction_export import export_saved
1189     if args.export_only:
1190         export_saved(PACKAGE_ROOT, QUESTION)
1191         return
1192     if QUESTION == 1:
1193         import problem1
1194         problem1.main()
1195

```

```

1196     elif QUESTION == 2:
1197         import problem2
1198         problem2.main()
1199     elif QUESTION == 3:
1200         # The original delivery gate compares Q3 with independently computed Q2.
1201         if not (PACKAGE_ROOT / 'results' / 'q2_solution.npz').exists():
1202             print('Q3: computing Q2 first for the original overlap check', flush=True)
1203             import problem2
1204             problem2.main()
1205             export_saved(PACKAGE_ROOT, 2)
1206             import q3_closeout
1207             q3_closeout.main(['--no-xlsx'])
1208     else:
1209         import problem4
1210         import verify_q4_results
1211         for level, tight in ((2, False), (4, False), (4, True), (8, False), (16, False), (8, True)):
1212             problem4.run(level, tight)
1213             verify_q4_results.main()
1214         export_saved(PACKAGE_ROOT, QUESTION)
1215         print(f'Q{QUESTION}: full recomputation and Excel export completed', flush=True)
1216
1217
1218 if __name__ == '__main__':
1219     main()

```

D.2 Q2.py

```

1  # -*- coding: utf-8 -*-
2  """ 问题 2: 从原始附件复算答案数据。
3
4  运行 python Q2.py; 检查环境 python Q2.py --check。
5  本文件内嵌本问所需的全部项目模块, 模块源码在下方按文件名分段列出。
6  运行时自动展开到相对目录 _runtime/Q2/code, 以支持源码哈希核验。
7  输入、输出均相对于本文件的位置, 与当前工作目录无关。
8  方程、物性、网格、容差和验证判据沿用原正式求解程序。
9  """
10 import argparse
11 import importlib
12 import os
13 from pathlib import Path
14 import sys
15
16 PACKAGE_ROOT = Path(__file__).resolve().parent
17 QUESTION = 2
18 SOURCES = {}
19
20 # =====
21 # Module: utils.py
22 # =====
23 SOURCES['utils'] = r'""""Shared paths and attachment readers for Problem A."""'
24 from __future__ import annotations
25
26 from pathlib import Path
27
28 import numpy as np
29 from openpyxl import load_workbook
30 from scipy.interpolate import PchipInterpolator
31
32 ROOT = Path(__file__).resolve().parents[3]
33 DATA_DIR = ROOT / "data"
34 ATTACH_DIR = DATA_DIR / " 附件"
35 RESULTS_DIR = ROOT / "results"
36 FIGURES_DIR = ROOT / "figures"
37 REPORTS_DIR = ROOT / "reports"
38

```

```

39 # Appendix 2 (Problem 1)
40 R = 0.02
41 L = 0.25
42 RHO = 820.0
43 CP = 2600.0
44 K = 0.36
45 H = 25.0
46 HM = 8.0e-7
47 T0 = 28.0
48 C0 = 2.55
49 D_PREFACTOR = 7.0e-9
50 D_EXP_COEF = 0.89
51 T_END = 1800.0
52 ALPHA = K / (RHO * CP)
53
54
55 def moisture_diffusivity(C: np.ndarray | float, *, protect: bool = True) -> np.ndarray | float:
56     """D(C) = 7e-9 exp(-0.89/C). Nonpositive C uses D=0 only as a solver-domain guard."""
57     C_arr = np.asarray(C, dtype=float)
58     out = np.zeros_like(C_arr, dtype=float)
59     positive = C_arr > 0.0
60     if np.any(positive):
61         out[positive] = D_PREFACTOR * np.exp(-D_EXP_COEF / C_arr[positive])
62     if not protect and np.any(~positive):
63         raise ValueError("moisture diffusivity evaluated at nonpositive C without protection")
64     if np.isscalar(C):
65         return float(out)
66     return out
67
68
69 def load_oven_table() -> tuple[np.ndarray, np.ndarray, np.ndarray]:
70     path = ATTACH_DIR / " 附件 1.xlsx"
71     wb = load_workbook(path, read_only=True, data_only=True)
72     rows = list(wb.active.values)
73     wb.close()
74     data = np.asarray(rows[1:], dtype=float)
75     t, Ta, Ca = data[:, 0], data[:, 1], data[:, 2]
76     if t[0] != 0.0:
77         raise ValueError(f"oven table must start at t=0, got {t[0]}")
78     return t, Ta, Ca
79
80
81 class PiecewiseLinear:
82     """Piecewise linear interpolant; kept for comparison, not used in Q1 production."""
83
84     def __init__(self, t: np.ndarray, y: np.ndarray):
85         self.t = np.asarray(t, dtype=float)
86         self.y = np.asarray(y, dtype=float)
87         self.kind = "linear"
88         if self.t.ndim != 1 or self.y.shape != self.t.shape:
89             raise ValueError("interpolant requires matching 1-D arrays")
90         if np.any(np.diff(self.t) <= 0):
91             raise ValueError("breakpoints must be strictly increasing")
92
93     def __call__(self, t: float | np.ndarray) -> float | np.ndarray:
94         return np.interp(t, self.t, self.y)
95
96     def slope(self, t: float) -> float:
97         t = float(t)
98         if t <= self.t[0] or t >= self.t[-1]:
99             if t >= self.t[-1]:
100                 dt = self.t[-1] - self.t[-2]
101                 return (self.y[-1] - self.y[-2]) / dt
102             return (self.y[1] - self.y[0]) / (self.t[1] - self.t[0])
103         i = int(np.searchsorted(self.t, t, side="right") - 1)

```

```

104         i = min(i, len(self.t) - 2)
105         return (self.y[i + 1] - self.y[i]) / (self.t[i + 1] - self.t[i])
106
107
108 class PchipOven:
109     """Monotone piecewise cubic Hermite interpolant (PCHIP) through attachment knots."""
110
111     def __init__(self, t: np.ndarray, y: np.ndarray):
112         self.t = np.asarray(t, dtype=float)
113         self.y = np.asarray(y, dtype=float)
114         self.kind = "pchip"
115         if self.t.ndim != 1 or self.y.shape != self.t.shape:
116             raise ValueError("interpolant requires matching 1-D arrays")
117         if np.any(np.diff(self.t) <= 0):
118             raise ValueError("breakpoints must be strictly increasing")
119         self._pchip = PchipInterpolator(self.t, self.y, extrapolate=False)
120         self._deriv = self._pchip.derivative()
121
122     def __call__(self, t: float | np.ndarray) -> float | np.ndarray:
123         tt = np.clip(np.asarray(t, dtype=float), self.t[0], self.t[-1])
124         val = self._pchip(tt)
125         if np.ndim(t) == 0:
126             return float(val)
127         return np.asarray(val, dtype=float)
128
129     def slope(self, t: float) -> float:
130         tt = float(np.clip(t, self.t[0], self.t[-1]))
131         return float(self._deriv(tt))
132
133
134 # =====
135 # Module: delivery.py
136 # =====
137 SOURCES['delivery'] = r'""""Fail-closed numerical delivery checks shared by Q1 and Q2."""'
138 import json
139 from pathlib import Path
140
141
142 def make_gate(checks):
143     checks = {name: bool(value) for name, value in checks.items()}
144     return {"checks": checks, "failed_checks": [k for k, v in checks.items() if not v],
145           "meets_delivery_gate": bool(checks) and all(checks.values())}
146
147
148 def require_delivery(validation, diagnostic_path):
149     """Persist diagnostics first; never reach any production writer on failure."""
150     path = Path(diagnostic_path)
151     path.parent.mkdir(parents=True, exist_ok=True)
152     path.write_text(json.dumps(validation, ensure_ascii=False, indent=2, default=float), encoding="utf-8")
153     gate = validation["delivery_gate"]
154     checks = gate.get("checks", {})
155     if not checks or not all(value is True for value in checks.values()) or gate.get("meets_delivery_gate")
156         ↪ is not True:
157         ↪ raise RuntimeError("Delivery blocked; see " + str(path) + ": " + str(gate.get("failed_checks",
158         ↪ [])))
159
160
161 # =====
162 # Module: q1_analytic.py
163 # =====
164 SOURCES['q1_analytic'] = r'""""Independent Bessel-Duhamel reference for the linear radial heat (or
165 ↪ frozen-D moisture) problem."""'
166 from __future__ import annotations
167
168 from typing import Any

```

```

166
167 import numpy as np
168 from scipy.integrate import solve_ivp
169 from scipy.optimize import brentq
170 from scipy.special import j0, j1
171 from scipy.sparse import diags
172
173
174 def robin_eigenvalues(Bi: float, n_roots: int) -> np.ndarray:
175     """Positive roots of  $\lambda^2 J_1(\lambda) = Bi * J_0(\lambda)$ ."""
176     if Bi <= 0:
177         raise ValueError("Biot number must be positive")
178     n_roots = int(n_roots)
179
180     def f(lam: float) -> float:
181         return lam * j1(lam) - Bi * j0(lam)
182
183     lam_max = (n_roots + 4) * np.pi
184     grid = np.linspace(1e-10, lam_max, max(8000, 200 * n_roots))
185     vals = grid * j1(grid) - Bi * j0(grid)
186     roots: list[float] = []
187     for i in range(len(grid) - 1):
188         v0, v1 = vals[i], vals[i + 1]
189         if not np.isfinite(v0) or not np.isfinite(v1):
190             continue
191         a, b = float(grid[i]), float(grid[i + 1])
192         if v0 == 0.0:
193             cand = a
194         elif v0 * v1 < 0.0:
195             cand = float(brentq(f, a, b, xtol=1e-14, maxiter=200))
196         else:
197             continue
198         if not roots or abs(cand - roots[-1]) > 1e-10:
199             roots.append(cand)
200         if len(roots) >= n_roots:
201             break
202     if len(roots) < n_roots:
203         raise RuntimeError(f"found only {len(roots)} Robin Bessel roots, need {n_roots}")
204     return np.asarray(roots, dtype=float)
205
206
207 def expansion_coefficients(lam: np.ndarray) -> np.ndarray:
208     j0l = j0(lam)
209     j1l = j1(lam)
210     return 2.0 * j1l / (lam * (j0l**2 + j1l**2))
211
212
213 class BesselDuhamel:
214     def __init__(self, Bi: float, diffusivity: float, R: float, n_terms: int = 120):
215         self.Bi = float(Bi)
216         self.alpha = float(diffusivity)
217         self.R = float(R)
218         self.lam = robin_eigenvalues(self.Bi, n_terms)
219         self.A = expansion_coefficients(self.lam)
220         self.beta = self.alpha * self.lam**2 / self.R**2
221
222     def radial_modes(self, r: np.ndarray) -> np.ndarray:
223         """Return array shape (n_terms, n_r) of  $A_n J_0(\lambda_n r/R)$ ."""
224         r = np.asarray(r, dtype=float)
225         return self.A[:, None] * j0(self.lam[:, None] * (r[None, :] / self.R))
226
227     def evaluate(self, r: np.ndarray, t: float, env: Any, u0: float) -> np.ndarray:
228         return self.evaluate_many(r, np.array([float(t)]), env, u0)[0]
229
230     def evaluate_many(self, r: np.ndarray, times: np.ndarray, env: Any, u0: float) -> np.ndarray:

```

```

231     """T = env(t) - sum A_n J0 I_n, with I' = -beta I + env'(t), I(0)=env(0)-u0.
232
233     Valid for any C1 interpolant that exposes env.y, env.slope, and env(t).
234     """
235     r = np.asarray(r, dtype=float)
236     times = np.asarray(times, dtype=float)
237     if times.size == 0:
238         return np.empty((0, r.size), dtype=float)
239     modes = self.radial_modes(r)
240     beta = self.beta
241     I0 = np.full(beta.size, float(env.y[0] - u0))
242     t_end = float(np.max(times))
243     out = np.empty((times.size, r.size), dtype=float)
244     zero = times == 0.0
245     if np.all(zero):
246         out[:] = u0
247         return out
248     t_eval = times[~zero]
249     sol = solve_ivp(
250         lambda t, I: -beta * I + env.slope(t),
251         (0.0, max(t_end, 1e-30)),
252         I0,
253         method="BDF",
254         jac=diags(-beta, format="csc"),
255         rtol=1e-11,
256         atol=1e-13,
257         t_eval=t_eval,
258         max_step=1.0,
259         vectorized=False,
260     )
261     if not sol.success:
262         raise RuntimeError(f"Bessel Duhamel ODE failed: {sol.message}")
263     Ihist = sol.y.T
264     env_t = np.array([float(env(t)) for t in t_eval], dtype=float)
265     nonzero_vals = env_t[:, None] - Ihist @ modes
266     out[zero] = u0
267     out[~zero] = nonzero_vals
268     return out
269
270
271 def converged_reference(Bi, diffusivity, R, r, times, env, u0, start_terms=320, tolerance=1e-8):
272     """Double the modal truncation and explicitly check every requested point."""
273     previous = BesselDuhamel(Bi, diffusivity, R, start_terms).evaluate_many(r, times, env, u0)
274     history = []
275     for terms in (start_terms * 2, start_terms * 4, start_terms * 8):
276         current = BesselDuhamel(Bi, diffusivity, R, terms).evaluate_many(r, times, env, u0)
277         error = float(np.max(np.abs(current - previous)))
278         history.append({"terms": terms, "previous_terms": terms // 2, "max_abs_difference": error})
279         if np.isfinite(error) and error <= tolerance:
280             return current, {"passed": True, "tolerance": tolerance, "history": history, "terms": terms}
281     previous = current
282     return current, {"passed": False, "tolerance": tolerance, "history": history, "terms": terms}
283 '''
284
285 # =====
286 # Module: radial_fvm.py
287 # =====
288 SOURCES['radial_fvm'] = r'""Node-centered radial finite-volume operators for Problem 1.""'
289 from __future__ import annotations
290
291 import numpy as np
292 from scipy.sparse import diags
293
294 from utils import CP, H, HM, K, R, RHO, moisture_diffusivity
295

```

```

296 OUTPUT_RADII_M = 0.001 * np.arange(21)
297
298
299 def graded_radial_nodes(level: int) -> np.ndarray:
300     """Nodes denser near the surface; every 0.1 cm output radius remains a node.
301
302     Interval subdivisions (before multiplying by ``level``):
303     0–1.5 cm: 8; 1.5–1.8 cm: 16; 1.8–1.9 cm: 32; 1.9–2.0 cm: 64.
304     Surface spacing at level L is 1e-3 / (64 L) metres.
305     """
306     level = int(level)
307     if level < 1:
308         raise ValueError("graded mesh level must be a positive integer")
309     n_sub = np.full(20, 8 * level, dtype=int)
310     n_sub[15:] = 16 * level
311     n_sub[18] = 32 * level
312     n_sub[19] = 64 * level
313     pts = [0.0]
314     for j in range(20):
315         r0 = 0.001 * j
316         r1 = 0.001 * (j + 1)
317         m = int(n_sub[j])
318         for k in range(1, m + 1):
319             pts.append(r0 + (r1 - r0) * k / m)
320     r = np.asarray(pts, dtype=float)
321     r[0] = 0.0
322     r[-1] = R
323     return r
324
325
326 class RadialFVM:
327     def __init__(self, N: int | None = None, r: np.ndarray | None = None):
328         if r is not None:
329             self.r = np.asarray(r, dtype=float)
330             if self.r.ndim != 1 or self.r.size < 3:
331                 raise ValueError("r must be a 1-D array with at least 3 nodes")
332             if abs(self.r[0]) > 1e-15 or abs(self.r[-1] - R) > 1e-14:
333                 raise ValueError("radial nodes must start at 0 and end at R")
334             if np.any(np.diff(self.r) <= 0.0):
335                 raise ValueError("radial nodes must be strictly increasing")
336         else:
337             if N is None or int(N) < 2:
338                 raise ValueError("N must be at least 2")
339             self.r = np.linspace(0.0, R, int(N) + 1)
340             self.n_nodes = int(self.r.size)
341             self.N = self.n_nodes - 1
342             self.dr_face = np.diff(self.r)
343             self.r_half = 0.5 * (self.r[:-1] + self.r[1:])
344             a = np.empty(self.n_nodes)
345             b = np.empty(self.n_nodes)
346             a[0] = 0.0
347             b[0] = self.r_half[0]
348             a[-1] = self.r_half[-1]
349             b[-1] = R
350             if self.n_nodes > 2:
351                 a[1:-1] = self.r_half[:-1]
352                 b[1:-1] = self.r_half[1:]
353             self.W = 0.5 * (b**2 - a**2)
354             self.dr = float(self.dr_face.min())
355             if abs(self.W.sum() - 0.5 * R**2) > 1e-14:
356                 raise RuntimeError("radial weights do not sum to R^2/2")
357
358             self.M_heat = self._heat_matrix()
359             self.M_heat_sparse = diags(
360                 [np.diag(self.M_heat, -1), np.diag(self.M_heat, 0), np.diag(self.M_heat, 1)],

```

```

361         offsets=[-1, 0, 1],
362         shape=self.M_heat.shape,
363         format="csc",
364     )
365     self._heat_force_coef = (R * H) / (RHO * CP * self.W[-1])
366     self._heat_force_index = self.n_nodes - 1
367
368     def output_indices(self) -> np.ndarray:
369         """Indices of 0, 0.1, ..., 2.0 cm nodes."""
370         idx = np.empty(OUTPUT_RADII_M.size, dtype=int)
371         for j, rc in enumerate(OUTPUT_RADII_M):
372             k = int(np.argmin(np.abs(self.r - rc)))
373             if abs(self.r[k] - rc) > 1e-12:
374                 raise ValueError(f"grid misses output radius {rc}, nearest {self.r[k]}")
375             idx[j] = k
376         return idx
377
378     def _interior_face_coeff(self) -> np.ndarray:
379         return self.r_half / self.dr_face
380
381     def _heat_matrix(self) -> np.ndarray:
382         n = self.n_nodes
383         M = np.zeros((n, n))
384         face = K * self._interior_face_coeff()
385         den = RHO * CP * self.W
386         M[0, 0] = -face[0] / den[0]
387         M[0, 1] = face[0] / den[0]
388         for i in range(1, n - 1):
389             M[i, i - 1] = face[i - 1] / den[i]
390             M[i, i + 1] = face[i] / den[i]
391             M[i, i] = -(face[i - 1] + face[i]) / den[i]
392         i = n - 1
393         M[i, i - 1] = face[i - 1] / den[i]
394         M[i, i] = (-face[i - 1] - R * H) / den[i]
395         return M
396
397     def heat_rhs(self, t: float, T: np.ndarray, Ta: float) -> np.ndarray:
398         out = self.M_heat @ T
399         out[self._heat_force_index] += self._heat_force_coef * Ta
400         return out
401
402     def heat_jac(self, t: float, T: np.ndarray):
403         return self.M_heat_sparse
404
405     def heat_jac_sparsity(self):
406         n = self.n_nodes
407         ones_m = np.ones(n - 1, dtype=float)
408         ones_d = np.ones(n, dtype=float)
409         return diags([ones_m, ones_d, ones_m], offsets=[-1, 0, 1], shape=(n, n), format="csc")
410
411     def moisture_rhs(self, t: float, C: np.ndarray, Ca: float, D_const: float | None = None) -> np.ndarray:
412         if D_const is None:
413             D_node = moisture_diffusivity(C, protect=True)
414         else:
415             D_node = np.full_like(C, float(D_const))
416         D_face = harmonic_mean(D_node[:-1], D_node[1:])
417         F = -self.r_half * D_face * np.diff(C) / self.dr_face
418         Cdot = np.empty_like(C)
419         Cdot[0] = -F[0] / self.W[0]
420         Cdot[1:-1] = (F[:-1] - F[1:]) / self.W[1:-1]
421         F_surf = R * HM * (C[-1] - Ca)
422         Cdot[-1] = (F[-1] - F_surf) / self.W[-1]
423         return Cdot
424
425     def moisture_jac_sparsity(self):

```

```

426         return self.heat_jac_sparsity()
427
428     def surface_heat_influx_density(self, T_s: float, Ta: float) -> float:
429         """Heat into the solid per unit surface area, W/m^2."""
430         return H * (Ta - T_s)
431
432     def surface_moisture_outflux(self, C_s: float, Ca: float) -> float:
433         """Outward moisture flux in the normalized (no rho_d) units, m/s * kg/kg."""
434         return HM * (C_s - Ca)
435
436
437 def harmonic_mean(a: np.ndarray, b: np.ndarray) -> np.ndarray:
438     out = np.zeros_like(a, dtype=float)
439     both = (a > 0.0) & (b > 0.0)
440     out[both] = 2.0 * a[both] * b[both] / (a[both] + b[both])
441     return out
442 '''
443
444 # =====
445 # Module: reproduction_export.py
446 # =====
447 SOURCES['reproduction_export'] = r''''''Portable Excel export from validated solver arrays; no external
    ↳ runtime.'''
448 from copy import copy
449 import json
450 from pathlib import Path
451
452 import numpy as np
453 from openpyxl import load_workbook
454
455
456 def write_template(root, question, times, fields):
457     root = Path(root)
458     template = root / 'data' / '附件' / '附件 3' / f'result{question}.xlsx'
459     wb = load_workbook(template)
460     if len(wb.worksheets) != len(fields):
461         raise ValueError('Template sheet count does not match the output fields')
462     times = np.asarray(times)
463     if times.ndim != 1 or not np.isfinite(times).all() or not (np.diff(times) > 0).all():
464         raise ValueError('Invalid output time sequence')
465     for ws, values in zip(wb.worksheets, fields):
466         values = np.asarray(values)
467         expected = 22 if question == 4 else 21
468         if values.shape != (len(times), expected):
469             raise ValueError('Output shape does not match the requested radial grid')
470         if np.isinf(values).any() or (question != 4 and np.isnan(values).any()):
471             raise ValueError('Unexpected nonfinite output')
472         header = ws.cell(1, 1).value
473         surface = ws.cell(1, ws.max_column).value
474         number_style = copy(ws.cell(2, 2)._style)
475         header_style = copy(ws.cell(1, 2)._style)
476         ws.delete_rows(1, ws.max_row)
477         ws.append([header] + [j / 10 for j in range(21)] + ([surface] if question == 4 else []))
478         for cell in ws[1]:
479             cell._style = copy(header_style)
480         for t, row in zip(times, values):
481             ws.append([float(t)] + [None if np.isnan(v) else float(v) for v in row])
482         for row in ws.iter_rows(min_row=2, min_col=2):
483             for cell in row:
484                 cell._style = copy(number_style)
485                 cell.number_format = '0.0000'
486         ws.freeze_panes = 'B2'
487     wb.properties.creator = 'Team'
488     wb.properties.lastModifiedBy = 'Team'
489     dest = root / 'results' / f'result{question}.xlsx'

```

```

490     dest.parent.mkdir(parents=True, exist_ok=True)
491     wb.save(dest)
492     wb.close()
493     print(f'Excel saved: results/result{question}.xlsx', flush=True)
494
495
496 def export_saved(root, question):
497     root = Path(root)
498     validation = json.loads((root / 'results' /
499     ↪ f'q{question}_validation.json').read_text(encoding='utf-8'))
500     if question == 4:
501         if validation.get('pass') is not True or not validation.get('checks') or not all(v is True for v in
502         ↪ validation['checks'].values()):
503             raise RuntimeError('Q4 numerical validation did not pass')
504     else:
505         from delivery import require_delivery
506         require_delivery(validation, root / 'results' / f'q{question}_export_validation.json')
507         if question == 3:
508             from q3_closeout import require_q3_delivery
509             require_q3_delivery(validation, root / 'results' / 'q3_export_validation.json')
510         with np.load(root / 'results' / f'q{question}_solution.npz') as z:
511             if question in (1, 2):
512                 write_template(root, question, z['times'], [z['T_out'], z['C_out']])
513             elif question == 3:
514                 write_template(root, question, z['excel_times'], [z['C_out']])
515             else:
516                 write_template(root, question, z['times_s'][1:], [z['C_physical'][1:]])
517
518 '''
519
520 # =====
521 # Module: appendix3.py
522 # =====
523 SOURCES['appendix3'] = r'""Appendix 3 effective properties for Problems 2 and 3.""'
524 from __future__ import annotations
525
526 import numpy as np
527
528 RHO_A = 650.0
529 RHO_B = 128.0
530 CP_A = 1450.0
531 CP_B = 2736.0
532 K_A = 0.21
533 K_B = 0.38
534 D_PREFACTOR = 2.4e-3
535 D_C_COEF = 0.45
536 D_T_COEF = 3850.0
537
538
539 def rho(C: np.ndarray | float) -> np.ndarray | float:
540     return RHO_A + RHO_B * np.asarray(C, dtype=float)
541
542
543 def cp(C: np.ndarray | float) -> np.ndarray | float:
544     C_arr = np.asarray(C, dtype=float)
545     return CP_A + CP_B * C_arr / (1.0 + C_arr)
546
547
548 def conductivity(C: np.ndarray | float) -> np.ndarray | float:
549     C_arr = np.asarray(C, dtype=float)
550     return K_A + K_B * C_arr / (1.0 + C_arr)
551
552
553 def heat_capacity(C: np.ndarray | float) -> np.ndarray | float:
554     return rho(C) * cp(C)
555
556

```

```

553
554 def conductivity_deriv(C: np.ndarray | float) -> np.ndarray | float:
555     C_arr = np.asarray(C, dtype=float)
556     return K_B / (1.0 + C_arr) ** 2
557
558
559 def heat_capacity_deriv(C: np.ndarray | float) -> np.ndarray | float:
560     C_arr = np.asarray(C, dtype=float)
561     return RHO_B * cp(C_arr) + rho(C_arr) * CP_B / (1.0 + C_arr) ** 2
562
563
564 def moisture_diffusivity_q2(
565     C: np.ndarray | float,
566     T_celsius: np.ndarray | float,
567     *,
568     protect: bool = True,
569 ) -> np.ndarray | float:
570     """D(C,T) with T in Celsius; Kelvin is used only inside the Arrhenius factor."""
571     C_arr = np.asarray(C, dtype=float)
572     T_arr = np.asarray(T_celsius, dtype=float)
573     out = np.zeros(np.broadcast(C_arr, T_arr).shape, dtype=float)
574     C_b, T_b = np.broadcast_arrays(C_arr, T_arr)
575     positive = C_b > 0.0
576     if np.any(positive):
577         Tk = T_b[positive] + 273.15
578         out[positive] = D_PREFACTOR * np.exp(-D_C_COEF / C_b[positive]) * np.exp(-D_T_COEF / Tk)
579     if not protect and np.any(~positive):
580         raise ValueError("q2 diffusivity evaluated at nonpositive C without protection")
581     if np.ndim(C) == 0 and np.ndim(T_celsius) == 0:
582         return float(np.asarray(out))
583     return out
584
585
586 def diffusivity_partials(C: np.ndarray, T_celsius: np.ndarray) -> tuple[np.ndarray, np.ndarray,
↪ np.ndarray]:
587     D = np.asarray(moisture_diffusivity_q2(C, T_celsius, protect=True), dtype=float)
588     C_arr = np.asarray(C, dtype=float)
589     T_arr = np.asarray(T_celsius, dtype=float)
590     dD_dC = np.zeros_like(D)
591     dD_dT = np.zeros_like(D)
592     ok = C_arr > 0.0
593     dD_dC[ok] = D[ok] * (D_C_COEF / C_arr[ok] ** 2)
594     Tk = T_arr + 273.15
595     dD_dT[ok] = D[ok] * (D_T_COEF / Tk[ok] ** 2)
596     return D, dD_dC, dD_dT
597 '''
598
599 # =====
600 # Module: radial_coupled.py
601 # =====
602 SOURCES['radial_coupled'] = r'""Coupled variable-property radial FVM for Problem 2.""'
603 from __future__ import annotations
604
605 import numpy as np
606 from scipy.sparse import lil_matrix
607
608 from appendix3 import conductivity, heat_capacity, moisture_diffusivity_q2
609 from radial_fvm import RadialFVM, harmonic_mean
610 from utils import H, HM, R
611
612
613 class CoupledRadialFVM:
614     def __init__(self, r: np.ndarray):
615         geom = RadialFVM(r=r)
616         self.r = geom.r

```

```

617     self.W = geom.W
618     self.r_half = geom.r_half
619     self.dr_face = geom.dr_face
620     self.n_nodes = geom.n_nodes
621     self.N = geom.N
622     self._geom = geom
623
624     def output_indices(self) -> np.ndarray:
625         return self._geom.output_indices()
626
627     def pack(self, T: np.ndarray, C: np.ndarray) -> np.ndarray:
628         return np.concatenate([T, C])
629
630     def unpack(self, y: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
631         n = self.n_nodes
632         return y[:n], y[n:]
633
634     def rhs(
635         self,
636         t: float,
637         y: np.ndarray,
638         Ta: float,
639         Ca: float,
640         *,
641         D_const: float | None = None,
642         ST: np.ndarray | None = None,
643         SC: np.ndarray | None = None,
644     ) -> np.ndarray:
645         T, C = self.unpack(y)
646         kn = np.asarray(conductivity(C), dtype=float)
647         Bn = np.asarray(heat_capacity(C), dtype=float)
648         if D_const is None:
649             Dn = np.asarray(moisture_diffusivity_q2(C, T, protect=True), dtype=float)
650         else:
651             Dn = np.full_like(C, float(D_const))
652         k_face = harmonic_mean(kn[:-1], kn[1:])
653         D_face = harmonic_mean(Dn[:-1], Dn[1:])
654         FT = -self.r_half * k_face * np.diff(T) / self.dr_face
655         FC = -self.r_half * D_face * np.diff(C) / self.dr_face
656         Tdot = np.empty_like(T)
657         Cdot = np.empty_like(C)
658         Tdot[0] = -FT[0] / (Bn[0] * self.W[0])
659         Cdot[0] = -FC[0] / self.W[0]
660         Tdot[1:-1] = (FT[:-1] - FT[1:]) / (Bn[1:-1] * self.W[1:-1])
661         Cdot[1:-1] = (FC[:-1] - FC[1:]) / self.W[1:-1]
662         FTsurf = R * H * (T[-1] - Ta)
663         FCsurf = R * HM * (C[-1] - Ca)
664         Tdot[-1] = (FT[-1] - FTsurf) / (Bn[-1] * self.W[-1])
665         Cdot[-1] = (FC[-1] - FCsurf) / self.W[-1]
666         if ST is not None:
667             Tdot = Tdot + ST / Bn
668         if SC is not None:
669             Cdot = Cdot + SC
670         return self.pack(Tdot, Cdot)
671
672     def jac_sparsity(self):
673         n = self.n_nodes
674         J = lil_matrix((2 * n, 2 * n), dtype=float)
675         for i in range(n):
676             js = [i]
677             if i > 0:
678                 js.append(i - 1)
679             if i + 1 < n:
680                 js.append(i + 1)
681             for j in js:

```

```

682         J[i, j] = 1.0
683         J[i, n + j] = 1.0
684         J[n + i, j] = 1.0
685         J[n + i, n + j] = 1.0
686     return J.tocsc()
687 '''
688
689 # =====
690 # Module: problem2.py
691 # =====
692 SOURCES['problem2'] = r'""""Problem 2: coupled variable-property radial drying for 0-3 h."""'
693 from __future__ import annotations
694
695 import json
696 import sys
697 import time
698 from pathlib import Path
699
700 import numpy as np
701 from openpyxl import Workbook
702 from scipy.integrate import solve_ivp
703
704 CODE_DIR = Path(__file__).resolve().parent
705 if str(CODE_DIR) not in sys.path:
706     sys.path.insert(0, str(CODE_DIR))
707
708 from appendix3 import (
709     conductivity,
710     conductivity_deriv,
711     diffusivity_partials,
712     heat_capacity,
713     heat_capacity_deriv,
714     moisture_diffusivity_q2,
715 )
716 from q1_analytic import BesselDuhamel, converged_reference
717 from delivery import make_gate, require_delivery
718 from radial_coupled import CoupledRadialFVM
719 from radial_fvm import graded_radial_nodes
720 from utils import (
721     C0,
722     FIGURES_DIR,
723     H,
724     HM,
725     L,
726     R,
727     RESULTS_DIR,
728     T0,
729     PchipOven,
730     load_oven_table,
731 )
732
733 T_END = 10800.0
734 OUTPUT_RADII_M = 0.001 * np.arange(21)
735 TABLE_HOURS = np.array([0.5, 1.0, 1.5, 2.0, 2.5, 3.0])
736 TABLE_TIMES = 3600.0 * TABLE_HOURS
737 TABLE_RADII_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
738 TREF = 28.0
739
740
741 def load_q2_inputs() -> tuple[PchipOven, PchipOven, np.ndarray]:
742     t, Ta, Ca = load_oven_table()
743     mask = t <= T_END + 1e-12
744     t, Ta, Ca = t[mask], Ta[mask], Ca[mask]
745     if t.size != 181 or abs(t[-1] - T_END) > 1e-12:
746         raise ValueError(f"expected 181 oven records on [0, 10800], got {t.size}, t_end={t[-1]}")

```

```

747     if abs(Ta[0] - T0) > 1e-12:
748         raise ValueError(f"Ta(0)={Ta[0]} != T0={T0}")
749     Ta_fun = PchipOven(t, Ta)
750     Ca_fun = PchipOven(t, Ca)
751     for tj, yj in zip(t, Ta):
752         if abs(float(Ta_fun(tj)) - float(yj)) > 1e-12:
753             raise RuntimeError(f"PCHIP misses Ta knot t={tj}")
754     for tj, yj in zip(t, Ca):
755         if abs(float(Ca_fun(tj)) - float(yj)) > 1e-12:
756             raise RuntimeError(f"PCHIP misses Ca knot t={tj}")
757     return Ta_fun, Ca_fun, t
758
759
760 def integrate_coupled(fvm: CoupledRadialFVM, y0, t_end, Ta_fun, Ca_fun, rtol, atol, max_step,
761                      D_const=None, source=None, t_eval=None):
762     y0 = np.asarray(y0, dtype=float)
763     n = fvm.n_nodes
764     atol_arr = np.asarray(atol, dtype=float)
765     if atol_arr.ndim == 0:
766         atol_arr = np.concatenate([np.full(n, float(atol_arr)), np.full(n, float(atol_arr) * 0.01)])
767
768     def rhs(t, y):
769         ST = SC = None
770         if source is not None:
771             ST, SC = source(t, fvm.r)
772         return fvm.rhs(t, y, float(Ta_fun(t)), float(Ca_fun(t)), D_const=D_const, ST=ST, SC=SC)
773
774     if t_eval is None:
775         t_eval = np.arange(1.0, t_end + 0.5, 1.0)
776     sol = solve_ivp(
777         rhs,
778         (0.0, float(t_end)),
779         y0,
780         method="BDF",
781         rtol=rtol,
782         atol=atol_arr,
783         max_step=max_step,
784         t_eval=t_eval,
785         dense_output=False,
786         vectorized=False,
787         jac_sparsity=fvm.jac_sparsity(),
788     )
789     if not sol.success:
790         raise RuntimeError(f"coupled BDF failed: {sol.message}")
791     times = np.concatenate([[0.0], sol.t])
792     Y = np.vstack([y0, sol.y.T])
793     return times, Y
794
795
796 def extract_output(fvm: CoupledRadialFVM, Y: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
797     n = fvm.n_nodes
798     idx = fvm.output_indices()
799     T = Y[:, :n][:, idx]
800     C = Y[:, n:][:, idx]
801     return T, C
802
803
804 def compare_on_output(A, B, times) -> dict:
805     diff = np.abs(A - B)
806     k = int(np.argmax(diff))
807     n_t, n_r = diff.shape
808     i, j = divmod(k, n_r)
809     paper_t = np.isin(times, TABLE_TIMES)
810     paper_r = slice(None, None, 5) if n_r == 21 else slice(None)
811     return {

```

```

812         "max_abs": float(diff.max()),
813         "max_time_s": float(times[i]),
814         "max_radius_cm": 0.1 * j,
815         "mean_abs": float(diff.mean()),
816         "paper_table_max_abs": float(np.max(np.abs(A[paper_t][:, paper_r] - B[paper_t][:, paper_r]))),
817     }
818
819
820 def four_decimal_stats(A, B, times) -> dict:
821     Ar, Br = np.round(A, 4), np.round(B, 4)
822     changed = Ar != Br
823     paper_t = np.isin(times, TABLE_TIMES)
824     paper_r = slice(None, None, 5) if A.shape[1] == 21 else slice(None)
825     paper_changed = Ar[paper_t][:, paper_r] != Br[paper_t][:, paper_r]
826     return {
827         "changed_cells": int(np.sum(changed)),
828         "total_cells": int(A.size),
829         "paper_changed_cells": int(np.sum(paper_changed)),
830         "paper_total_cells": int(paper_changed.size),
831         "max_abs": float(np.max(np.abs(A - B))),
832         "paper_table_max_abs": float(np.max(np.abs(A[paper_t][:, paper_r] - B[paper_t][:, paper_r]))),
833     }
834
835
836 def manufactured_fields(r: np.ndarray, t: float) -> tuple[np.ndarray, np.ndarray]:
837     x2 = (r / R) ** 2
838     T = 28.0 + 2.0 * np.exp(-t / 600.0) * (1.0 + x2)
839     C = 1.5 + 0.2 * np.exp(-t / 500.0) * (1.0 + x2)
840     return T, C
841
842
843 def manufactured_env(r_surface: float, t: float) -> tuple[float, float]:
844     T, C = manufactured_fields(np.array([r_surface]), t)
845     Ts, Cs = float(T[0]), float(C[0])
846     u = np.exp(-t / 600.0)
847     v = np.exp(-t / 500.0)
848     Tr = 4.0 / R * u
849     Cr = 4.0 / R * v * 0.1 # C amplitude 0.2 vs T amplitude 2: Cr = 0.2*v*2r/R^2 at r=R -> 0.4 v / R
850     Cr = 0.4 / R * v
851     kn = float(conductivity(Cs))
852     Dn = float(moisture_diffusivity_q2(Cs, Ts))
853     Ta = Ts + kn * Tr / H
854     Ca = Cs + Dn * Cr / HM
855     return Ta, Ca
856
857
858 class ManufacturedOven:
859     def __init__(self, kind: str):
860         self.kind = kind
861
862     def __call__(self, t):
863         Ta, Ca = manufactured_env(R, float(t))
864         return Ta if self.kind == "T" else Ca
865
866
867 def manufactured_sources(t: float, r: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
868     T, C = manufactured_fields(r, t)
869     x2 = (r / R) ** 2
870     u = np.exp(-t / 600.0)
871     v = np.exp(-t / 500.0)
872     Tt = 2.0 * (-1.0 / 600.0) * u * (1.0 + x2)
873     Ct = 0.2 * (-1.0 / 500.0) * v * (1.0 + x2)
874     Tr = 4.0 * r / (R ** 2) * u
875     Trr = np.full_like(r, 4.0 / (R ** 2) * u)
876     Cr = 0.4 * r / (R ** 2) * v

```

```

877 Crr = np.full_like(r, 0.4 / (R ** 2) * v)
878 lapT = np.empty_like(r)
879 lapC = np.empty_like(r)
880 center = r < 1e-16
881 lapT[center] = 2.0 * Trr[center]
882 lapC[center] = 2.0 * Crr[center]
883 away = ~center
884 lapT[away] = Trr[away] + Tr[away] / r[away]
885 lapC[away] = Crr[away] + Cr[away] / r[away]
886 kn = np.asarray(conductivity(C), dtype=float)
887 kp = np.asarray(conductivity_deriv(C), dtype=float)
888 Bn = np.asarray(heat_capacity(C), dtype=float)
889 D, dD_dC, dD_dT = diffusivity_partials(C, T)
890 ST = Bn * Tt - (kn * lapT + kp * Cr * Tr)
891 SC = Ct - (D * lapC + dD_dC * Cr ** 2 + dD_dT * Tr * Cr)
892 return ST, SC
893
894
895 def conservation_moisture(fvm, times, Y, Ca_fun) -> dict:
896     n = fvm.n_nodes
897     C = Y[:, n:]
898     M = C @ fvm.W
899     M0 = float(M[0])
900     CN = C[:, -1]
901     Ca = np.array([float(Ca_fun(t)) for t in times])
902     I = float(np.trapezoid(HM * (CN - Ca), times))
903     rhs = M0 - R * I
904     Mt = float(M[-1])
905     denom = max(abs(M0 - Mt), abs(R * I), 1e-18)
906     return {"M0": M0, "M_end": Mt, "rhs": rhs, "relative_residual": abs(Mt - rhs) / denom}
907
908
909 def conservation_heat(fvm, times, Y, Ta_fun) -> dict:
910     n = fvm.n_nodes
911     T = Y[:, :n]
912     C = Y[:, n:]
913     Bn = np.asarray(heat_capacity(C), dtype=float)
914     Bp = np.asarray(heat_capacity_deriv(C), dtype=float)
915     E = 2.0 * np.pi * L * np.sum(fvm.W[None, :] * Bn * (T - TREF), axis=1)
916     TN = T[:, -1]
917     Ta = np.array([float(Ta_fun(t)) for t in times])
918     Q = 2.0 * np.pi * R * L * H * float(np.trapezoid(Ta - TN, times))
919     dC = np.gradient(C, times, axis=0)
920     corr = 2.0 * np.pi * L * np.sum(fvm.W[None, :] * Bp * (T - TREF) * dC, axis=1)
921     Icorr = float(np.trapezoid(corr, times))
922     lhs = float(E[-1] - E[0] - Icorr)
923     denom = max(abs(Q), abs(lhs), 1e-12)
924     return {
925         "E_end_minus_E0": float(E[-1] - E[0]),
926         "composition_correction": Icorr,
927         "Q_surface": Q,
928         "relative_residual": abs(lhs - Q) / denom,
929     }
930
931
932 def assert_physical(T_out, C_out, Ta_fun, Ca_fun) -> dict:
933     Ta_max = float(np.max(Ta_fun.y))
934     Ca_min = float(np.min(Ca_fun.y))
935     info = {
936         "T_min": float(T_out.min()),
937         "T_max": float(T_out.max()),
938         "C_min": float(C_out.min()),
939         "C_max": float(C_out.max()),
940         "T_below_28": float(max(0.0, 28.0 - T_out.min())),
941         "T_above-Ta_max": float(max(0.0, T_out.max() - Ta_max)),

```

```

942     "C_above_C0": float(max(0.0, C_out.max() - C0)),
943     "C_below_Ca_min": float(max(0.0, Ca_min - C_out.min())),
944     "Ta_range": [float(np.min(Ta_fun.y)), Ta_max],
945     "Ca_range": [Ca_min, float(np.max(Ca_fun.y))],
946 }
947 info["T_bounds_ok"] = info["T_below_28"] <= 1e-6 and info["T_above-Ta_max"] <= 1e-4
948 info["C_bounds_ok"] = info["C_above_C0"] <= 1e-8 and info["C_below_Ca_min"] <= 1e-8
949 return info
950
951
952 def write_result_workbook(path: Path, T_out, C_out, times):
953     radii_cm = 0.1 * np.arange(21)
954     wb = Workbook()
955     for title, arr in ((" 温度", T_out), (" 水分浓度", C_out)):
956         ws = wb.create_sheet(title)
957         ws["A1"] = " 时间\\到药材中心的距离"
958         for j, rc in enumerate(radii_cm, start=2):
959             ws.cell(1, j, float(rc))
960         for i, t in enumerate(times):
961             ws.cell(i + 2, 1, int(round(float(t))))
962             for j in range(21):
963                 cell = ws.cell(i + 2, j + 2, float(arr[i, j]))
964                 cell.number_format = "0.0000"
965     if "Sheet" in wb.sheetnames:
966         del wb["Sheet"]
967     path.parent.mkdir(parents=True, exist_ok=True)
968     wb.save(path)
969
970
971 def write_paper_csv(path: Path, arr_out, times):
972     t_idx = {float(t): i for i, t in enumerate(times)}
973     r_idx = {round(0.1 * j, 10): j for j in range(21)}
974     lines = ["t_h," + ",".join(f"r_{rc:g}cm" for rc in TABLE_RADII_CM)]
975     for th, t in zip(TABLE_HOURS, TABLE_TIMES):
976         row = [f"{th:g}"]
977         i = t_idx[float(t)]
978         for rc in TABLE_RADII_CM:
979             j = r_idx[float(rc)]
980             row.append(f"{arr_out[i, j]:.8f}")
981         lines.append(",".join(row))
982     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
983
984
985 def setup_mpl():
986     import matplotlib
987     matplotlib.use("Agg")
988     import matplotlib.pyplot as plt
989     plt.rcParams.update({
990         "font.sans-serif": ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "DejaVu Sans"],
991         "axes.unicode_minus": False,
992         "font.size": 9,
993         "axes.labelsize": 10,
994         "legend.fontsize": 8,
995         "xtick.labelsize": 8,
996         "ytick.labelsize": 8,
997         "axes.linewidth": 0.8,
998         "pdf.fonttype": 42,
999         "ps.fonttype": 42,
1000     })
1001     return plt
1002
1003
1004 def plot_figures(plt, fvm, times, Y, T_out, C_out, Ta_fun, Ca_fun):
1005     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
1006     n = fvm.n_nodes

```

```

1007     r_cm = fvm.r * 100.0
1008     idx = {float(t): i for i, t in enumerate(times)}
1009     marks = TABLE_TIMES
1010
1011     fig, ax = plt.subplots(figsize=(6.4, 3.8))
1012     for t in marks:
1013         ax.plot(r_cm, Y[idx[float(t)], :n], lw=1.2, label=fr"$t={t/3600:g}\,\mathrm{{h}}$")
1014     ax.set_xlabel(" 半径 $r$ (cm)")
1015     ax.set_ylabel(" 温度 $T$ (°C)")
1016     ax.set_xlim(0, 2)
1017     ax.legend(frameon=False, ncol=3, loc="lower center", bbox_to_anchor=(0.5, 1.02))
1018     fig.tight_layout()
1019     fig.savefig(FIGURES_DIR / "q2_T_profiles.pdf")
1020     plt.close(fig)
1021
1022     fig, ax = plt.subplots(figsize=(6.4, 3.8))
1023     for t in marks:
1024         ax.plot(r_cm, Y[idx[float(t)], n:], lw=1.2, label=fr"$t={t/3600:g}\,\mathrm{{h}}$")
1025     ax.set_xlabel(" 半径 $r$ (cm)")
1026     ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
1027     ax.set_xlim(0, 2)
1028     ax.legend(frameon=False, ncol=2)
1029     fig.tight_layout()
1030     fig.savefig(FIGURES_DIR / "q2_C_profiles.pdf")
1031     plt.close(fig)
1032
1033     t_plot = times
1034     fig, ax = plt.subplots(figsize=(6.4, 3.8))
1035     ax.plot(t_plot / 3600.0, [Ta_fun(t) for t in t_plot], lw=1.0, ls="--", label=" 烘房 $T_a$")
1036     ax.plot(t_plot / 3600.0, T_out[:, -1], lw=1.2, label=" 表面")
1037     ax.plot(t_plot / 3600.0, T_out[:, 0], lw=1.2, label=" 中心")
1038     ax.set_xlabel(" 时间 $t$ (h)")
1039     ax.set_ylabel(" 温度 (°C)")
1040     ax.set_xlim(0, 3)
1041     ax.legend(frameon=False)
1042     fig.tight_layout()
1043     fig.savefig(FIGURES_DIR / "q2_T_center_surface.pdf")
1044     plt.close(fig)
1045
1046     fig, ax = plt.subplots(figsize=(6.4, 3.8))
1047     ax.plot(t_plot / 3600.0, C_out[:, -1], lw=1.2, label=" 表面")
1048     ax.plot(t_plot / 3600.0, C_out[:, 0], lw=1.2, label=" 中心")
1049     ax.set_xlabel(" 时间 $t$ (h)")
1050     ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
1051     ax.set_xlim(0, 3)
1052     ax.legend(frameon=False)
1053     fig.tight_layout()
1054     fig.savefig(FIGURES_DIR / "q2_C_center_surface.pdf")
1055     plt.close(fig)
1056
1057     D_c = moisture_diffusivity_q2(C_out[:, 0], T_out[:, 0])
1058     D_s = moisture_diffusivity_q2(C_out[:, -1], T_out[:, -1])
1059     fig, ax = plt.subplots(figsize=(6.4, 3.6))
1060     ax.plot(t_plot / 3600.0, D_s, lw=1.2, label=" 表面 $D$")
1061     ax.plot(t_plot / 3600.0, D_c, lw=1.2, label=" 中心 $D$")
1062     ax.set_xlabel(" 时间 $t$ (h)")
1063     ax.set_ylabel(r"$D$ (m$^2$/s)")
1064     ax.set_xlim(0, 3)
1065     ax.legend(frameon=False)
1066     fig.tight_layout()
1067     fig.savefig(FIGURES_DIR / "q2_D_center_surface.pdf")
1068     plt.close(fig)
1069
1070
1071 def run_mms(level: int = 4) -> dict:

```

```

1072     r = graded_radial_nodes(level)
1073     fvm = CoupledRadialFVM(r)
1074     T0f, C0f = manufactured_fields(fvm.r, 0.0)
1075     y0 = fvm.pack(T0f, C0f)
1076     Ta_fun = ManufacturedOven("T")
1077     Ca_fun = ManufacturedOven("C")
1078     times, Y = integrate_coupled(
1079         fvm, y0, 100.0, Ta_fun, Ca_fun,
1080         rtol=1e-8, atol=1e-9, max_step=0.5,
1081         source=manufactured_sources,
1082         t_eval=np.arange(1.0, 100.5, 1.0),
1083     )
1084     n = fvm.n_nodes
1085     T_num, C_num = Y[:, :n], Y[:, n:]
1086     T_ex = np.stack([manufactured_fields(fvm.r, t)[0] for t in times])
1087     C_ex = np.stack([manufactured_fields(fvm.r, t)[1] for t in times])
1088     return {
1089         "level": level,
1090         "n_nodes": n,
1091         "T_max_abs": float(np.max(np.abs(T_num - T_ex))),
1092         "C_max_abs": float(np.max(np.abs(C_num - C_ex))),
1093         "T_end_max_abs": float(np.max(np.abs(T_num[-1] - T_ex[-1]))),
1094         "C_end_max_abs": float(np.max(np.abs(C_num[-1] - C_ex[-1]))),
1095     }
1096
1097
1098 def main():
1099     t_wall0 = time.perf_counter()
1100     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
1101     Ta_fun, Ca_fun, knots = load_q2_inputs()
1102     print("oven points", knots.size, "kind", Ta_fun.kind, "Ta(10800)=", Ta_fun(T_END), "Ca(10800)=",
1103         ↪ Ca_fun(T_END))
1103     D0 = float(moisture_diffusivity_q2(C0, T0))
1104     print("initial D", D0, "k", float(conductivity(C0)), "B", float(heat_capacity(C0)))
1105
1106     print("manufactured-solution probe")
1107     mms = run_mms(4)
1108     print("MMS", mms)
1109
1110     mesh_levels = [2, 4]
1111     rtol, atol_T, atol_C, max_step = 1e-9, 1e-8, 1e-10, 1.0
1112     solutions = {}
1113     for level in mesh_levels:
1114         r = graded_radial_nodes(level)
1115         fvm = CoupledRadialFVM(r)
1116         tag = f"G{level}"
1117         print(f"\n=== {tag} nodes={fvm.n_nodes} ===")
1118         y0 = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
1119         atol = np.concatenate([np.full(fvm.n_nodes, atol_T), np.full(fvm.n_nodes, atol_C)])
1120         times, Y = integrate_coupled(fvm, y0, T_END, Ta_fun, Ca_fun, rtol, atol, max_step)
1121         times_pos = times[times > 0]
1122         Y_pos = Y[times > 0]
1123         T_out, C_out = extract_output(fvm, Y_pos)
1124         solutions[level] = dict(
1125             tag=tag, fvm=fvm, times=times_pos, Y=Y_pos, Y_full=Y, times_full=times,
1126             T_out=T_out, C_out=C_out, n_nodes=fvm.n_nodes,
1127         )
1128         print("done", tag, "T", T_out.min(), T_out.max(), "C", C_out.min(), C_out.max(),
1129             "C(t=1,s)", C_out[0, -1], "C(3h,s)", C_out[-1, -1], "C(3h,c)", C_out[-1, 0])
1130
1131     t = solutions[4]["times"]
1132     conv = {
1133         "T_G2_vs_G4": compare_on_output(solutions[2]["T_out"], solutions[4]["T_out"], t),
1134         "C_G2_vs_G4": compare_on_output(solutions[2]["C_out"], solutions[4]["C_out"], t),
1135     }

```

```

1136 fourdec = {
1137     "T_G2_vs_G4": four_decimal_stats(solutions[2]["T_out"], solutions[4]["T_out"], t),
1138     "C_G2_vs_G4": four_decimal_stats(solutions[2]["C_out"], solutions[4]["C_out"], t),
1139 }
1140 print("conv", conv)
1141 print("4dp", fourdec)
1142 paper_ok = fourdec["T_G2_vs_G4"]["paper_changed_cells"] == 0 and
↪ fourdec["C_G2_vs_G4"]["paper_changed_cells"] == 0
1143 abs_ok = conv["T_G2_vs_G4"]["max_abs"] <= 2e-5 and conv["C_G2_vs_G4"]["max_abs"] <= 2e-5
1144 need_g8 = not (paper_ok and abs_ok)
1145 if need_g8:
1146     print("G2 vs G4 not enough; running G8")
1147     level = 8
1148     r = graded_radial_nodes(level)
1149     fvm = CoupledRadialFVM(r)
1150     y0 = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
1151     atol = np.concatenate([np.full(fvm.n_nodes, atol_T), np.full(fvm.n_nodes, atol_C)])
1152     times, Y = integrate_coupled(fvm, y0, T_END, Ta_fun, Ca_fun, rtol, atol, max_step)
1153     times_pos = times[times > 0]
1154     Y_pos = Y[times > 0]
1155     T_out, C_out = extract_output(fvm, Y_pos)
1156     solutions[8] = dict(
1157         tag="G8", fvm=fvm, times=times_pos, Y=Y_pos, Y_full=Y, times_full=times,
1158         T_out=T_out, C_out=C_out, n_nodes=fvm.n_nodes,
1159     )
1160     t = solutions[8]["times"]
1161     conv["T_G4_vs_G8"] = compare_on_output(solutions[4]["T_out"], solutions[8]["T_out"], t)
1162     conv["C_G4_vs_G8"] = compare_on_output(solutions[4]["C_out"], solutions[8]["C_out"], t)
1163     fourdec["T_G4_vs_G8"] = four_decimal_stats(solutions[4]["T_out"], solutions[8]["T_out"], t)
1164     fourdec["C_G4_vs_G8"] = four_decimal_stats(solutions[4]["C_out"], solutions[8]["C_out"], t)
1165     print("conv G4 G8", conv["T_G4_vs_G8"], conv["C_G4_vs_G8"])
1166     pub_level = 8
1167     paper_ok = fourdec["T_G4_vs_G8"]["paper_changed_cells"] == 0 and
↪ fourdec["C_G4_vs_G8"]["paper_changed_cells"] == 0
1168     abs_ok = conv["T_G4_vs_G8"]["max_abs"] <= 2e-5 and conv["C_G4_vs_G8"]["max_abs"] <= 2e-5
1169 else:
1170     pub_level = 4 if fourdec["C_G2_vs_G4"]["changed_cells"] or fourdec["T_G2_vs_G4"]["changed_cells"]
↪ else 2
1171     if pub_level == 2:
1172         pub_level = 4
1173     pub = solutions[pub_level]
1174     fvm = pub["fvm"]
1175     times = pub["times"]
1176     print("publishing", pub["tag"], "paper_ok", paper_ok, "abs_ok", abs_ok)
1177
1178     print("time refinement")
1179     y0 = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
1180     atol = np.concatenate([np.full(fvm.n_nodes, atol_T / 5), np.full(fvm.n_nodes, atol_C / 5)])
1181     t2, Y2 = integrate_coupled(fvm, y0, T_END, Ta_fun, Ca_fun, rtol / 5, atol, 0.5)
1182     T2, C2 = extract_output(fvm, Y2[t2 > 0])
1183     time_sens = {"T": compare_on_output(pub["T_out"], T2, times), "C": compare_on_output(pub["C_out"], C2,
↪ times)}
1184     time_4dp = {"T": four_decimal_stats(pub["T_out"], T2, times), "C": four_decimal_stats(pub["C_out"], C2,
↪ times)}
1185     print("time sens", time_sens)
1186
1187     print("frozen-D moisture probe")
1188     D_freeze = D0
1189     Bi_m = HM * R / D_freeze
1190     C_ref, moisture_truncation = converged_reference(Bi_m, D_freeze, R, OUTPUT_RADII_M, times, Ca_fun, C0)
1191     y0f = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
1192     atol = np.concatenate([np.full(fvm.n_nodes, atol_T), np.full(fvm.n_nodes, atol_C)])
1193     tf, Yf = integrate_coupled(fvm, y0f, T_END, Ta_fun, Ca_fun, rtol, atol, max_step, D_const=D_freeze)
1194     _, C_frozen = extract_output(fvm, Yf[tf > 0])
1195     frozen = compare_on_output(C_frozen, C_ref, times)

```

```

1196 nl_vs_frozen = compare_on_output(pub["C_out"], C_frozen, times)
1197 print("frozen vs analytic", frozen)
1198 print("nonlinear vs frozen", nl_vs_frozen)
1199
1200 n = fvm.n_nodes
1201 D_field = moisture_diffusivity_q2(pub["Y"][:, n:], pub["Y"][:, :n])
1202 D_stats = {
1203     "D0": D0,
1204     "D_min": float(np.min(D_field)),
1205     "D_max": float(np.max(D_field)),
1206     "D_min_over_D0": float(np.min(D_field) / D0),
1207     "D_max_over_D0": float(np.max(D_field) / D0),
1208     "center_C_end": float(pub["C_out"][-1, 0]),
1209     "surface_C_end": float(pub["C_out"][-1, -1]),
1210     "center_T_end": float(pub["T_out"][-1, 0]),
1211     "surface_T_end": float(pub["T_out"][-1, -1]),
1212 }
1213 print("D stats", D_stats)
1214
1215 bounds = assert_physical(pub["Y_full"][:, :n], pub["Y_full"][:, n:], Ta_fun, Ca_fun)
1216 cons_C = conservation_moisture(fvm, pub["times_full"], pub["Y_full"], Ca_fun)
1217 cons_T = conservation_heat(fvm, pub["times_full"], pub["Y_full"], Ta_fun)
1218 print("bounds", bounds)
1219 print("conservation C", cons_C)
1220 print("conservation T", cons_T)
1221
1222 delivery = make_gate({
1223     "space_abs": abs_ok,
1224     "space_paper": paper_ok,
1225     "all_finite": np.isfinite(pub["Y_full"]).all(),
1226     "manufactured_solution": mms["T_max_abs"] <= 2e-5 and mms["C_max_abs"] <= 2e-5,
1227     "time_abs": time_sens["T"]["max_abs"] <= 2e-5 and time_sens["C"]["max_abs"] <= 2e-5,
1228     "time_paper": time_4dp["T"]["paper_changed_cells"] == 0 and time_4dp["C"]["paper_changed_cells"] ==
1229         ↳ 0,
1230     "temperature_bounds": bounds["T_bounds_ok"],
1231     "moisture_bounds": bounds["C_bounds_ok"],
1232     "moisture_reference_truncation": moisture_truncation["passed"],
1233     "frozen_reference_error": frozen["max_abs"] <= 2e-5,
1234     "heat_balance": cons_T["relative_residual"] <= 1e-6,
1235     "moisture_balance": cons_C["relative_residual"] <= 1e-6,
1236 })
1237 print("delivery", delivery)
1238
1239 paper_T = {f"h{th:g}_r{rc:g}": float(pub["T_out"][int(t) - 1, int(round(rc / 0.1))])
1240     for th, t in zip(TABLE_HOURS, TABLE_TIMES) for rc in TABLE_RADII_CM}
1241 paper_C = {f"h{th:g}_r{rc:g}": float(pub["C_out"][int(t) - 1, int(round(rc / 0.1))])
1242     for th, t in zip(TABLE_HOURS, TABLE_TIMES) for rc in TABLE_RADII_CM}
1243
1244 validation = {
1245     "result_version": "q2-closeout-v1",
1246     "interpolation": Ta_fun.kind,
1247     "published_mesh": pub["tag"],
1248     "published_n_nodes": pub["n_nodes"],
1249     "mesh_levels": list(solutions.keys()),
1250     "solver": "coupled-FVM+BDF",
1251     "rtol": rtol, "atol_T": atol_T, "atol_C": atol_C, "max_step": max_step,
1252     "mms": mms,
1253     "grid_convergence": conv,
1254     "four_decimal_grid": fourdec,
1255     "time_sensitivity": time_sens,
1256     "four_decimal_time": time_4dp,
1257     "delivery_gate": delivery,
1258     "reference_truncation_C": moisture_truncation,
1259     "frozen_D_moisture_vs_analytic": frozen,
1260     "nonlinear_C_vs_frozen_D": nl_vs_frozen,

```

```

1260     "D_variation": D_stats,
1261     "conservation_T_10800s": cons_T,
1262     "conservation_C_10800s": cons_C,
1263     "physical_bounds": bounds,
1264     "paper_table_T": paper_T,
1265     "paper_table_C": paper_C,
1266     "elapsed_s": time.perf_counter() - t_wall0,
1267     "python": sys.version,
1268     "end_face_2d": "skipped_by_plan",
1269 }
1270 require_delivery(validation, RESULTS_DIR / "diagnostics/q2_delivery/latest.json")
1271 np.savez(
1272     RESULTS_DIR / "q2_solution.npz",
1273     mesh_level=pub_level,
1274     n_nodes=pub["n_nodes"],
1275     times=times,
1276     r=fvm.r,
1277     output_radii_m=OUTPUT_RADII_M,
1278     checkpoint_times=np.r_[0., 1., 60., 100., TABLE_TIMES],
1279     T_checkpoints=pub["Y_full"][np.r_[0., 1., 60., 100., TABLE_TIMES].astype(int), :n],
1280     C_checkpoints=pub["Y_full"][np.r_[0., 1., 60., 100., TABLE_TIMES].astype(int), n:],
1281     T_out=pub["T_out"],
1282     C_out=pub["C_out"],
1283     interpolation=np.array(Ta_fun.kind),
1284     rtol=rtol, atol_T=atol_T, atol_C=atol_C, max_step=max_step,
1285 )
1286 write_result_workbook(RESULTS_DIR / "result2.xlsx", pub["T_out"], pub["C_out"], times)
1287 write_paper_csv(RESULTS_DIR / "q2_table_temperature.csv", pub["T_out"], times)
1288 write_paper_csv(RESULTS_DIR / "q2_table_moisture.csv", pub["C_out"], times)
1289 plt = setup_mpl()
1290 plot_figures(plt, fvm, times, pub["Y"], pub["T_out"], pub["C_out"], Ta_fun, Ca_fun)
1291
1292 (RESULTS_DIR / "q2_validation.json").write_text(
1293     json.dumps(validation, ensure_ascii=False, indent=2, default=float),
1294     encoding="utf-8",
1295 )
1296 print("elapsed_s", validation["elapsed_s"])
1297 print("wrote results to", RESULTS_DIR)
1298
1299
1300
1301 if __name__ == "__main__":
1302     main()
1303 '''
1304
1305 def prepare():
1306     code_dir = PACKAGE_ROOT / '_runtime' / f'Q{QUESTION}' / 'code'
1307     code_dir.mkdir(parents=True, exist_ok=True)
1308     for name, source in SOURCES.items():
1309         (code_dir / (name + '.py')).write_text(source, encoding='utf-8')
1310     sys.path.insert(0, str(code_dir))
1311     for folder in ('results', 'figures', 'reports'):
1312         (PACKAGE_ROOT / folder).mkdir(exist_ok=True)
1313     os.environ.setdefault('MPLBACKEND', 'Agg')
1314     for name in SOURCES:
1315         importlib.import_module(name)
1316
1317
1318 def main():
1319     parser = argparse.ArgumentParser(description=__doc__)
1320     parser.add_argument('--check', action='store_true', help='Check imports and original inputs without
1321 ↪ solving')
1322     parser.add_argument('--export-only', action='store_true', help='Export existing validated NPZ arrays;
1323 ↪ does not solve the model')
1324     args = parser.parse_args()

```

```

1323     prepare()
1324     from utils import load_oven_table
1325     load_oven_table()
1326     if QUESTION == 4:
1327         from problem4 import Inputs
1328         Inputs()
1329     if args.check:
1330         print(f'Q{QUESTION}: imports and input checks passed', flush=True)
1331         return
1332     from reproduction_export import export_saved
1333     if args.export_only:
1334         export_saved(PACKAGE_ROOT, QUESTION)
1335         return
1336     if QUESTION == 1:
1337         import problem1
1338         problem1.main()
1339     elif QUESTION == 2:
1340         import problem2
1341         problem2.main()
1342     elif QUESTION == 3:
1343         # The original delivery gate compares Q3 with independently computed Q2.
1344         if not (PACKAGE_ROOT / 'results' / 'q2_solution.npz').exists():
1345             print('Q3: computing Q2 first for the original overlap check', flush=True)
1346             import problem2
1347             problem2.main()
1348             export_saved(PACKAGE_ROOT, 2)
1349             import q3_closeout
1350             q3_closeout.main(['--no-xlsx'])
1351     else:
1352         import problem4
1353         import verify_q4_results
1354         for level, tight in ((2, False), (4, False), (4, True), (8, False), (16, False), (8, True)):
1355             problem4.run(level, tight)
1356             verify_q4_results.main()
1357     export_saved(PACKAGE_ROOT, QUESTION)
1358     print(f'Q{QUESTION}: full recomputation and Excel export completed', flush=True)
1359
1360
1361 if __name__ == '__main__':
1362     main()

```

D.3 Q3.py

```

1 # -*- coding: utf-8 -*-
2 """ 问题 3: 从原始附件复算答案数据。
3
4 运行 python Q3.py; 检查环境 python Q3.py --check。
5 本文件内嵌本问所需的全部项目模块, 模块源码在下方按文件名分段列出。
6 运行时自动展开到相对目录 _runtime/Q3/code, 以支持源码哈希核校。
7 输入、输出均相对于本文件的位置, 与当前工作目录无关。
8 方程、物性、网格、容差和验证判据沿用原正式求解程序。
9 """
10 import argparse
11 import importlib
12 import os
13 from pathlib import Path
14 import sys
15
16 PACKAGE_ROOT = Path(__file__).resolve().parent
17 QUESTION = 3
18 SOURCES = {}
19
20 # =====
21 # Module: utils.py
22 # =====

```

```

23 SOURCES['utils'] = r''''''Shared paths and attachment readers for Problem A.'''
24 from __future__ import annotations
25
26 from pathlib import Path
27
28 import numpy as np
29 from openpyxl import load_workbook
30 from scipy.interpolate import PchipInterpolator
31
32 ROOT = Path(__file__).resolve().parents[3]
33 DATA_DIR = ROOT / "data"
34 ATTACH_DIR = DATA_DIR / " 附件"
35 RESULTS_DIR = ROOT / "results"
36 FIGURES_DIR = ROOT / "figures"
37 REPORTS_DIR = ROOT / "reports"
38
39 # Appendix 2 (Problem 1)
40 R = 0.02
41 L = 0.25
42 RHO = 820.0
43 CP = 2600.0
44 K = 0.36
45 H = 25.0
46 HM = 8.0e-7
47 T0 = 28.0
48 C0 = 2.55
49 D_PREFACTOR = 7.0e-9
50 D_EXP_COEF = 0.89
51 T_END = 1800.0
52 ALPHA = K / (RHO * CP)
53
54
55 def moisture_diffusivity(C: np.ndarray | float, *, protect: bool = True) -> np.ndarray | float:
56     """D(C) = 7e-9 exp(-0.89/C). Nonpositive C uses D=0 only as a solver-domain guard."""
57     C_arr = np.asarray(C, dtype=float)
58     out = np.zeros_like(C_arr, dtype=float)
59     positive = C_arr > 0.0
60     if np.any(positive):
61         out[positive] = D_PREFACTOR * np.exp(-D_EXP_COEF / C_arr[positive])
62     if not protect and np.any(~positive):
63         raise ValueError("moisture diffusivity evaluated at nonpositive C without protection")
64     if np.isscalar(C):
65         return float(out)
66     return out
67
68
69 def load_oven_table() -> tuple[np.ndarray, np.ndarray, np.ndarray]:
70     path = ATTACH_DIR / " 附件 1.xlsx"
71     wb = load_workbook(path, read_only=True, data_only=True)
72     rows = list(wb.active.values)
73     wb.close()
74     data = np.asarray(rows[1:], dtype=float)
75     t, Ta, Ca = data[:, 0], data[:, 1], data[:, 2]
76     if t[0] != 0.0:
77         raise ValueError(f"oven table must start at t=0, got {t[0]}")
78     return t, Ta, Ca
79
80
81 class PiecewiseLinear:
82     """Piecewise linear interpolant; kept for comparison, not used in Q1 production."""
83
84     def __init__(self, t: np.ndarray, y: np.ndarray):
85         self.t = np.asarray(t, dtype=float)
86         self.y = np.asarray(y, dtype=float)
87         self.kind = "linear"

```

```

88         if self.t.ndim != 1 or self.y.shape != self.t.shape:
89             raise ValueError("interpolant requires matching 1-D arrays")
90         if np.any(np.diff(self.t) <= 0):
91             raise ValueError("breakpoints must be strictly increasing")
92
93     def __call__(self, t: float | np.ndarray) -> float | np.ndarray:
94         return np.interp(t, self.t, self.y)
95
96     def slope(self, t: float) -> float:
97         t = float(t)
98         if t <= self.t[0] or t >= self.t[-1]:
99             if t >= self.t[-1]:
100                 dt = self.t[-1] - self.t[-2]
101                 return (self.y[-1] - self.y[-2]) / dt
102             return (self.y[1] - self.y[0]) / (self.t[1] - self.t[0])
103         i = int(np.searchsorted(self.t, t, side="right") - 1)
104         i = min(i, len(self.t) - 2)
105         return (self.y[i + 1] - self.y[i]) / (self.t[i + 1] - self.t[i])
106
107
108 class PchipOven:
109     """Monotone piecewise cubic Hermite interpolant (PCHIP) through attachment knots."""
110
111     def __init__(self, t: np.ndarray, y: np.ndarray):
112         self.t = np.asarray(t, dtype=float)
113         self.y = np.asarray(y, dtype=float)
114         self.kind = "pchip"
115         if self.t.ndim != 1 or self.y.shape != self.t.shape:
116             raise ValueError("interpolant requires matching 1-D arrays")
117         if np.any(np.diff(self.t) <= 0):
118             raise ValueError("breakpoints must be strictly increasing")
119         self._pchip = PchipInterpolator(self.t, self.y, extrapolate=False)
120         self._deriv = self._pchip.derivative()
121
122     def __call__(self, t: float | np.ndarray) -> float | np.ndarray:
123         tt = np.clip(np.asarray(t, dtype=float), self.t[0], self.t[-1])
124         val = self._pchip(tt)
125         if np.ndim(t) == 0:
126             return float(val)
127         return np.asarray(val, dtype=float)
128
129     def slope(self, t: float) -> float:
130         tt = float(np.clip(t, self.t[0], self.t[-1]))
131         return float(self._deriv(tt))
132
133
134 # =====
135 # Module: delivery.py
136 # =====
137 SOURCES['delivery'] = r'""""Fail-closed numerical delivery checks shared by Q1 and Q2."""'
138 import json
139 from pathlib import Path
140
141
142 def make_gate(checks):
143     checks = {name: bool(value) for name, value in checks.items()}
144     return {"checks": checks, "failed_checks": [k for k, v in checks.items() if not v],
145           "meets_delivery_gate": bool(checks) and all(checks.values())}
146
147
148 def require_delivery(validation, diagnostic_path):
149     """Persist diagnostics first; never reach any production writer on failure."""
150     path = Path(diagnostic_path)
151     path.parent.mkdir(parents=True, exist_ok=True)
152     path.write_text(json.dumps(validation, ensure_ascii=False, indent=2, default=float), encoding="utf-8")

```

```

153     gate = validation["delivery_gate"]
154     checks = gate.get("checks", {})
155     if not checks or not all(value is True for value in checks.values()) or gate.get("meets_delivery_gate")
        ↪ is not True:
156         raise RuntimeError("Delivery blocked; see " + str(path) + ": " + str(gate.get("failed_checks",
        ↪ [])))
157 '''
158
159 # =====
160 # Module: q1_analytic.py
161 # =====
162 SOURCES['q1_analytic'] = r'""""Independent Bessel-Duhamel reference for the linear radial heat (or
        ↪ frozen-D moisture) problem.""""
163 from __future__ import annotations
164
165 from typing import Any
166
167 import numpy as np
168 from scipy.integrate import solve_ivp
169 from scipy.optimize import brentq
170 from scipy.special import j0, j1
171 from scipy.sparse import diags
172
173
174 def robin_eigenvalues(Bi: float, n_roots: int) -> np.ndarray:
175     """Positive roots of  $\lambda * J_1(\lambda) = Bi * J_0(\lambda)$ ."""
176     if Bi <= 0:
177         raise ValueError("Biot number must be positive")
178     n_roots = int(n_roots)
179
180     def f(lam: float) -> float:
181         return lam * j1(lam) - Bi * j0(lam)
182
183     lam_max = (n_roots + 4) * np.pi
184     grid = np.linspace(1e-10, lam_max, max(8000, 200 * n_roots))
185     vals = grid * j1(grid) - Bi * j0(grid)
186     roots: list[float] = []
187     for i in range(len(grid) - 1):
188         v0, v1 = vals[i], vals[i + 1]
189         if not np.isfinite(v0) or not np.isfinite(v1):
190             continue
191         a, b = float(grid[i]), float(grid[i + 1])
192         if v0 == 0.0:
193             cand = a
194         elif v0 * v1 < 0.0:
195             cand = float(brentq(f, a, b, xtol=1e-14, maxiter=200))
196         else:
197             continue
198         if not roots or abs(cand - roots[-1]) > 1e-10:
199             roots.append(cand)
200         if len(roots) >= n_roots:
201             break
202     if len(roots) < n_roots:
203         raise RuntimeError(f"found only {len(roots)} Robin Bessel roots, need {n_roots}")
204     return np.asarray(roots, dtype=float)
205
206
207 def expansion_coefficients(lam: np.ndarray) -> np.ndarray:
208     j0l = j0(lam)
209     j1l = j1(lam)
210     return 2.0 * j1l / (lam * (j0l**2 + j1l**2))
211
212
213 class BesselDuhamel:
214     def __init__(self, Bi: float, diffusivity: float, R: float, n_terms: int = 120):

```

```

215     self.Bi = float(Bi)
216     self.alpha = float(diffusivity)
217     self.R = float(R)
218     self.lam = robin_eigenvalues(self.Bi, n_terms)
219     self.A = expansion_coefficients(self.lam)
220     self.beta = self.alpha * self.lam**2 / self.R**2
221
222     def radial_modes(self, r: np.ndarray) -> np.ndarray:
223         """Return array shape (n_terms, n_r) of A_n J0(lambda_n r/R)."""
224         r = np.asarray(r, dtype=float)
225         return self.A[:, None] * j0(self.lam[:, None] * (r[None, :] / self.R))
226
227     def evaluate(self, r: np.ndarray, t: float, env: Any, u0: float) -> np.ndarray:
228         return self.evaluate_many(r, np.array([float(t)]), env, u0)[0]
229
230     def evaluate_many(self, r: np.ndarray, times: np.ndarray, env: Any, u0: float) -> np.ndarray:
231         """T = env(t) - sum A_n J0 I_n, with I' = -beta I + env'(t), I(0)=env(0)-u0.
232
233         Valid for any C1 interpolant that exposes env.y, env.slope, and env(t).
234         """
235         r = np.asarray(r, dtype=float)
236         times = np.asarray(times, dtype=float)
237         if times.size == 0:
238             return np.empty((0, r.size), dtype=float)
239         modes = self.radial_modes(r)
240         beta = self.beta
241         I0 = np.full(beta.size, float(env.y[0] - u0))
242         t_end = float(np.max(times))
243         out = np.empty((times.size, r.size), dtype=float)
244         zero = times == 0.0
245         if np.all(zero):
246             out[:] = u0
247             return out
248         t_eval = times[~zero]
249         sol = solve_ivp(
250             lambda t, I: -beta * I + env.slope(t),
251             (0.0, max(t_end, 1e-30)),
252             I0,
253             method="BDF",
254             jac=diags(-beta, format="csc"),
255             rtol=1e-11,
256             atol=1e-13,
257             t_eval=t_eval,
258             max_step=1.0,
259             vectorized=False,
260         )
261         if not sol.success:
262             raise RuntimeError(f"Bessel Duhamel ODE failed: {sol.message}")
263         Ihist = sol.y.T
264         env_t = np.array([float(env(t)) for t in t_eval], dtype=float)
265         nonzero_vals = env_t[:, None] - Ihist @ modes
266         out[zero] = u0
267         out[~zero] = nonzero_vals
268         return out
269
270
271     def converged_reference(Bi, diffusivity, R, r, times, env, u0, start_terms=320, tolerance=1e-8):
272         """Double the modal truncation and explicitly check every requested point."""
273         previous = BesselDuhamel(Bi, diffusivity, R, start_terms).evaluate_many(r, times, env, u0)
274         history = []
275         for terms in (start_terms * 2, start_terms * 4, start_terms * 8):
276             current = BesselDuhamel(Bi, diffusivity, R, terms).evaluate_many(r, times, env, u0)
277             error = float(np.max(np.abs(current - previous)))
278             history.append({"terms": terms, "previous_terms": terms // 2, "max_abs_difference": error})
279             if np.isfinite(error) and error <= tolerance:

```

```

280         return current, {"passed": True, "tolerance": tolerance, "history": history, "terms": terms}
281     previous = current
282     return current, {"passed": False, "tolerance": tolerance, "history": history, "terms": terms}
283 '''
284
285 # =====
286 # Module: radial_fvm.py
287 # =====
288 SOURCES['radial_fvm'] = r''''''Node-centered radial finite-volume operators for Problem 1.'''
289 from __future__ import annotations
290
291 import numpy as np
292 from scipy.sparse import diags
293
294 from utils import CP, H, HM, K, R, RH0, moisture_diffusivity
295
296 OUTPUT_RADII_M = 0.001 * np.arange(21)
297
298
299 def graded_radial_nodes(level: int) -> np.ndarray:
300     """Nodes denser near the surface; every 0.1 cm output radius remains a node.
301
302     Interval subdivisions (before multiplying by ``level``):
303     0-1.5 cm: 8; 1.5-1.8 cm: 16; 1.8-1.9 cm: 32; 1.9-2.0 cm: 64.
304     Surface spacing at level L is 1e-3 / (64 L) metres.
305     """
306     level = int(level)
307     if level < 1:
308         raise ValueError("graded mesh level must be a positive integer")
309     n_sub = np.full(20, 8 * level, dtype=int)
310     n_sub[15:] = 16 * level
311     n_sub[18] = 32 * level
312     n_sub[19] = 64 * level
313     pts = [0.0]
314     for j in range(20):
315         r0 = 0.001 * j
316         r1 = 0.001 * (j + 1)
317         m = int(n_sub[j])
318         for k in range(1, m + 1):
319             pts.append(r0 + (r1 - r0) * k / m)
320     r = np.asarray(pts, dtype=float)
321     r[0] = 0.0
322     r[-1] = R
323     return r
324
325
326 class RadialFVM:
327     def __init__(self, N: int | None = None, r: np.ndarray | None = None):
328         if r is not None:
329             self.r = np.asarray(r, dtype=float)
330             if self.r.ndim != 1 or self.r.size < 3:
331                 raise ValueError("r must be a 1-D array with at least 3 nodes")
332             if abs(self.r[0]) > 1e-15 or abs(self.r[-1] - R) > 1e-14:
333                 raise ValueError("radial nodes must start at 0 and end at R")
334             if np.any(np.diff(self.r) <= 0.0):
335                 raise ValueError("radial nodes must be strictly increasing")
336         else:
337             if N is None or int(N) < 2:
338                 raise ValueError("N must be at least 2")
339             self.r = np.linspace(0.0, R, int(N) + 1)
340             self.n_nodes = int(self.r.size)
341             self.N = self.n_nodes - 1
342             self.dr_face = np.diff(self.r)
343             self.r_half = 0.5 * (self.r[:-1] + self.r[1:])
344             a = np.empty(self.n_nodes)

```

```

345     b = np.empty(self.n_nodes)
346     a[0] = 0.0
347     b[0] = self.r_half[0]
348     a[-1] = self.r_half[-1]
349     b[-1] = R
350     if self.n_nodes > 2:
351         a[1:-1] = self.r_half[1:-1]
352         b[1:-1] = self.r_half[1:-1]
353     self.W = 0.5 * (b**2 - a**2)
354     self.dr = float(self.dr_face.min())
355     if abs(self.W.sum() - 0.5 * R**2) > 1e-14:
356         raise RuntimeError("radial weights do not sum to R^2/2")
357
358     self.M_heat = self._heat_matrix()
359     self.M_heat_sparse = diags(
360         [np.diag(self.M_heat, -1), np.diag(self.M_heat, 0), np.diag(self.M_heat, 1)],
361         offsets=[-1, 0, 1],
362         shape=self.M_heat.shape,
363         format="csc",
364     )
365     self._heat_force_coef = (R * H) / (RHO * CP * self.W[-1])
366     self._heat_force_index = self.n_nodes - 1
367
368     def output_indices(self) -> np.ndarray:
369         """Indices of 0, 0.1, ..., 2.0 cm nodes."""
370         idx = np.empty(OUTPUT_RADII_M.size, dtype=int)
371         for j, rc in enumerate(OUTPUT_RADII_M):
372             k = int(np.argmin(np.abs(self.r - rc)))
373             if abs(self.r[k] - rc) > 1e-12:
374                 raise ValueError(f"grid misses output radius {rc}, nearest {self.r[k]}")
375             idx[j] = k
376         return idx
377
378     def _interior_face_coeff(self) -> np.ndarray:
379         return self.r_half / self.dr_face
380
381     def _heat_matrix(self) -> np.ndarray:
382         n = self.n_nodes
383         M = np.zeros((n, n))
384         face = K * self._interior_face_coeff()
385         den = RHO * CP * self.W
386         M[0, 0] = -face[0] / den[0]
387         M[0, 1] = face[0] / den[0]
388         for i in range(1, n - 1):
389             M[i, i - 1] = face[i - 1] / den[i]
390             M[i, i + 1] = face[i] / den[i]
391             M[i, i] = -(face[i - 1] + face[i]) / den[i]
392         i = n - 1
393         M[i, i - 1] = face[i - 1] / den[i]
394         M[i, i] = (-face[i - 1] - R * H) / den[i]
395         return M
396
397     def heat_rhs(self, t: float, T: np.ndarray, Ta: float) -> np.ndarray:
398         out = self.M_heat @ T
399         out[self._heat_force_index] += self._heat_force_coef * Ta
400         return out
401
402     def heat_jac(self, t: float, T: np.ndarray):
403         return self.M_heat_sparse
404
405     def heat_jac_sparsity(self):
406         n = self.n_nodes
407         ones_m = np.ones(n - 1, dtype=float)
408         ones_d = np.ones(n, dtype=float)
409         return diags([ones_m, ones_d, ones_m], offsets=[-1, 0, 1], shape=(n, n), format="csc")

```

```

410
411 def moisture_rhs(self, t: float, C: np.ndarray, Ca: float, D_const: float | None = None) -> np.ndarray:
412     if D_const is None:
413         D_node = moisture_diffusivity(C, protect=True)
414     else:
415         D_node = np.full_like(C, float(D_const))
416     D_face = harmonic_mean(D_node[:-1], D_node[1:])
417     F = -self.r_half * D_face * np.diff(C) / self.dr_face
418     Cdot = np.empty_like(C)
419     Cdot[0] = -F[0] / self.W[0]
420     Cdot[1:-1] = (F[:-1] - F[1:]) / self.W[1:-1]
421     F_surf = R * HM * (C[-1] - Ca)
422     Cdot[-1] = (F[-1] - F_surf) / self.W[-1]
423     return Cdot
424
425 def moisture_jac_sparsity(self):
426     return self.heat_jac_sparsity()
427
428 def surface_heat_influx_density(self, T_s: float, Ta: float) -> float:
429     """Heat into the solid per unit surface area, W/m^2."""
430     return H * (Ta - T_s)
431
432 def surface_moisture_outflux(self, C_s: float, Ca: float) -> float:
433     """Outward moisture flux in the normalized (no rho_d) units, m/s * kg/kg."""
434     return HM * (C_s - Ca)
435
436
437 def harmonic_mean(a: np.ndarray, b: np.ndarray) -> np.ndarray:
438     out = np.zeros_like(a, dtype=float)
439     both = (a > 0.0) & (b > 0.0)
440     out[both] = 2.0 * a[both] * b[both] / (a[both] + b[both])
441     return out
442 '''
443
444 # =====
445 # Module: reproduction_export.py
446 # =====
447 SOURCES['reproduction_export'] = r''''''Portable Excel export from validated solver arrays; no external
↳ runtime.'''
448 from copy import copy
449 import json
450 from pathlib import Path
451
452 import numpy as np
453 from openpyxl import load_workbook
454
455
456 def write_template(root, question, times, fields):
457     root = Path(root)
458     template = root / 'data' / '附件' / '附件 3' / f'result{question}.xlsx'
459     wb = load_workbook(template)
460     if len(wb.worksheets) != len(fields):
461         raise ValueError('Template sheet count does not match the output fields')
462     times = np.asarray(times)
463     if times.ndim != 1 or not np.isfinite(times).all() or not (np.diff(times) > 0).all():
464         raise ValueError('Invalid output time sequence')
465     for ws, values in zip(wb.worksheets, fields):
466         values = np.asarray(values)
467         expected = 22 if question == 4 else 21
468         if values.shape != (len(times), expected):
469             raise ValueError('Output shape does not match the requested radial grid')
470         if np.isinf(values).any() or (question != 4 and np.isnan(values).any()):
471             raise ValueError('Unexpected nonfinite output')
472         header = ws.cell(1, 1).value
473         surface = ws.cell(1, ws.max_column).value

```

```

474     number_style = copy(ws.cell(2, 2)._style)
475     header_style = copy(ws.cell(1, 2)._style)
476     ws.delete_rows(1, ws.max_row)
477     ws.append([header] + [j / 10 for j in range(21)] + ([surface] if question == 4 else []))
478     for cell in ws[1]:
479         cell._style = copy(header_style)
480     for t, row in zip(times, values):
481         ws.append([float(t)] + [None if np.isnan(v) else float(v) for v in row])
482     for row in ws.iter_rows(min_row=2, min_col=2):
483         for cell in row:
484             cell._style = copy(number_style)
485             cell.number_format = '0.0000'
486     ws.freeze_panes = 'B2'
487     wb.properties.creator = 'Team'
488     wb.properties.lastModifiedBy = 'Team'
489     dest = root / 'results' / f'result{question}.xlsx'
490     dest.parent.mkdir(parents=True, exist_ok=True)
491     wb.save(dest)
492     wb.close()
493     print(f'Excel saved: results/result{question}.xlsx', flush=True)
494
495
496 def export_saved(root, question):
497     root = Path(root)
498     validation = json.loads((root / 'results' /
499     ↪ f'q{question}_validation.json').read_text(encoding='utf-8'))
500     if question == 4:
501         if validation.get('pass') is not True or not validation.get('checks') or not all(v is True for v in
502         ↪ validation['checks'].values()):
503             raise RuntimeError('Q4 numerical validation did not pass')
504     else:
505         from delivery import require_delivery
506         require_delivery(validation, root / 'results' / f'q{question}_export_validation.json')
507         if question == 3:
508             from q3_closeout import require_q3_delivery
509             require_q3_delivery(validation, root / 'results' / 'q3_export_validation.json')
510     with np.load(root / 'results' / f'q{question}_solution.npz') as z:
511         if question in (1, 2):
512             write_template(root, question, z['times'], [z['T_out'], z['C_out']])
513         elif question == 3:
514             write_template(root, question, z['excel_times'], [z['C_out']])
515         else:
516             write_template(root, question, z['times_s'][1:], [z['C_physical'][1:]])
517     '''
518
519 # =====
520 # Module: appendix3.py
521 # =====
522 SOURCES['appendix3'] = r''''''Appendix 3 effective properties for Problems 2 and 3.''''
523 from __future__ import annotations
524
525 import numpy as np
526
527 RHO_A = 650.0
528 RHO_B = 128.0
529 CP_A = 1450.0
530 CP_B = 2736.0
531 K_A = 0.21
532 K_B = 0.38
533 D_PREFACTOR = 2.4e-3
534 D_C_COEF = 0.45
535 D_T_COEF = 3850.0
536
537 def rho(C: np.ndarray | float) -> np.ndarray | float:

```

```

537     return RHO_A + RHO_B * np.asarray(C, dtype=float)
538
539
540 def cp(C: np.ndarray | float) -> np.ndarray | float:
541     C_arr = np.asarray(C, dtype=float)
542     return CP_A + CP_B * C_arr / (1.0 + C_arr)
543
544
545 def conductivity(C: np.ndarray | float) -> np.ndarray | float:
546     C_arr = np.asarray(C, dtype=float)
547     return K_A + K_B * C_arr / (1.0 + C_arr)
548
549
550 def heat_capacity(C: np.ndarray | float) -> np.ndarray | float:
551     return rho(C) * cp(C)
552
553
554 def conductivity_deriv(C: np.ndarray | float) -> np.ndarray | float:
555     C_arr = np.asarray(C, dtype=float)
556     return K_B / (1.0 + C_arr) ** 2
557
558
559 def heat_capacity_deriv(C: np.ndarray | float) -> np.ndarray | float:
560     C_arr = np.asarray(C, dtype=float)
561     return RHO_B * cp(C_arr) + rho(C_arr) * CP_B / (1.0 + C_arr) ** 2
562
563
564 def moisture_diffusivity_q2(
565     C: np.ndarray | float,
566     T_celsius: np.ndarray | float,
567     *,
568     protect: bool = True,
569 ) -> np.ndarray | float:
570     """D(C,T) with T in Celsius; Kelvin is used only inside the Arrhenius factor."""
571     C_arr = np.asarray(C, dtype=float)
572     T_arr = np.asarray(T_celsius, dtype=float)
573     out = np.zeros(np.broadcast(C_arr, T_arr).shape, dtype=float)
574     C_b, T_b = np.broadcast_arrays(C_arr, T_arr)
575     positive = C_b > 0.0
576     if np.any(positive):
577         Tk = T_b[positive] + 273.15
578         out[positive] = D_PREFACTOR * np.exp(-D_C_COEF / C_b[positive]) * np.exp(-D_T_COEF / Tk)
579     if not protect and np.any(~positive):
580         raise ValueError("q2 diffusivity evaluated at nonpositive C without protection")
581     if np.ndim(C) == 0 and np.ndim(T_celsius) == 0:
582         return float(np.asarray(out))
583     return out
584
585
586 def diffusivity_partials(C: np.ndarray, T_celsius: np.ndarray) -> tuple[np.ndarray, np.ndarray,
↪ np.ndarray]:
587     D = np.asarray(moisture_diffusivity_q2(C, T_celsius, protect=True), dtype=float)
588     C_arr = np.asarray(C, dtype=float)
589     T_arr = np.asarray(T_celsius, dtype=float)
590     dD_dC = np.zeros_like(D)
591     dD_dT = np.zeros_like(D)
592     ok = C_arr > 0.0
593     dD_dC[ok] = D[ok] * (D_C_COEF / C_arr[ok] ** 2)
594     Tk = T_arr + 273.15
595     dD_dT[ok] = D[ok] * (D_T_COEF / Tk[ok] ** 2)
596     return D, dD_dC, dD_dT
597 '''
598
599 # =====
600 # Module: radial_coupled.py

```

```

601 # =====
602 SOURCES['radial_coupled'] = r''''Coupled variable-property radial FVM for Problem 2.'''
603 from __future__ import annotations
604
605 import numpy as np
606 from scipy.sparse import lil_matrix
607
608 from appendix3 import conductivity, heat_capacity, moisture_diffusivity_q2
609 from radial_fvm import RadialFVM, harmonic_mean
610 from utils import H, HM, R
611
612
613 class CoupledRadialFVM:
614     def __init__(self, r: np.ndarray):
615         geom = RadialFVM(r=r)
616         self.r = geom.r
617         self.W = geom.W
618         self.r_half = geom.r_half
619         self.dr_face = geom.dr_face
620         self.n_nodes = geom.n_nodes
621         self.N = geom.N
622         self._geom = geom
623
624     def output_indices(self) -> np.ndarray:
625         return self._geom.output_indices()
626
627     def pack(self, T: np.ndarray, C: np.ndarray) -> np.ndarray:
628         return np.concatenate([T, C])
629
630     def unpack(self, y: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
631         n = self.n_nodes
632         return y[:n], y[n:]
633
634     def rhs(
635         self,
636         t: float,
637         y: np.ndarray,
638         Ta: float,
639         Ca: float,
640         *,
641         D_const: float | None = None,
642         ST: np.ndarray | None = None,
643         SC: np.ndarray | None = None,
644     ) -> np.ndarray:
645         T, C = self.unpack(y)
646         kn = np.asarray(conductivity(C), dtype=float)
647         Bn = np.asarray(heat_capacity(C), dtype=float)
648         if D_const is None:
649             Dn = np.asarray(moisture_diffusivity_q2(C, T, protect=True), dtype=float)
650         else:
651             Dn = np.full_like(C, float(D_const))
652         k_face = harmonic_mean(kn[:-1], kn[1:])
653         D_face = harmonic_mean(Dn[:-1], Dn[1:])
654         FT = -self.r_half * k_face * np.diff(T) / self.dr_face
655         FC = -self.r_half * D_face * np.diff(C) / self.dr_face
656         Tdot = np.empty_like(T)
657         Cdot = np.empty_like(C)
658         Tdot[0] = -FT[0] / (Bn[0] * self.W[0])
659         Cdot[0] = -FC[0] / self.W[0]
660         Tdot[1:-1] = (FT[:-1] - FT[1:]) / (Bn[1:-1] * self.W[1:-1])
661         Cdot[1:-1] = (FC[:-1] - FC[1:]) / self.W[1:-1]
662         FTsurf = R * H * (T[-1] - Ta)
663         FCsurf = R * HM * (C[-1] - Ca)
664         Tdot[-1] = (FT[-1] - FTsurf) / (Bn[-1] * self.W[-1])
665         Cdot[-1] = (FC[-1] - FCsurf) / self.W[-1]

```

```

666         if ST is not None:
667             Tdot = Tdot + ST / Bn
668         if SC is not None:
669             Cdot = Cdot + SC
670         return self.pack(Tdot, Cdot)
671
672     def jac_sparsity(self):
673         n = self.n_nodes
674         J = lil_matrix((2 * n, 2 * n), dtype=float)
675         for i in range(n):
676             js = [i]
677             if i > 0:
678                 js.append(i - 1)
679             if i + 1 < n:
680                 js.append(i + 1)
681             for j in js:
682                 J[i, j] = 1.0
683                 J[i, n + j] = 1.0
684                 J[n + i, j] = 1.0
685                 J[n + i, n + j] = 1.0
686         return J.tocsc()
687     '''
688
689 # =====
690 # Module: problem2.py
691 # =====
692 SOURCES['problem2'] = r''''''Problem 2: coupled variable-property radial drying for 0-3 h.'''
693 from __future__ import annotations
694
695 import json
696 import sys
697 import time
698 from pathlib import Path
699
700 import numpy as np
701 from openpyxl import Workbook
702 from scipy.integrate import solve_ivp
703
704 CODE_DIR = Path(__file__).resolve().parent
705 if str(CODE_DIR) not in sys.path:
706     sys.path.insert(0, str(CODE_DIR))
707
708 from appendix3 import (
709     conductivity,
710     conductivity_deriv,
711     diffusivity_partials,
712     heat_capacity,
713     heat_capacity_deriv,
714     moisture_diffusivity_q2,
715 )
716 from q1_analytic import BesselDuhamel, converged_reference
717 from delivery import make_gate, require_delivery
718 from radial_coupled import CoupledRadialFVM
719 from radial_fvm import graded_radial_nodes
720 from utils import (
721     C0,
722     FIGURES_DIR,
723     H,
724     HM,
725     L,
726     R,
727     RESULTS_DIR,
728     T0,
729     PchipOven,
730     load_oven_table,

```

```

731 )
732
733 T_END = 10800.0
734 OUTPUT_RADII_M = 0.001 * np.arange(21)
735 TABLE_HOURS = np.array([0.5, 1.0, 1.5, 2.0, 2.5, 3.0])
736 TABLE_TIMES = 3600.0 * TABLE_HOURS
737 TABLE_RADII_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
738 TREF = 28.0
739
740
741 def load_q2_inputs() -> tuple[PchipOven, PchipOven, np.ndarray]:
742     t, Ta, Ca = load_oven_table()
743     mask = t <= T_END + 1e-12
744     t, Ta, Ca = t[mask], Ta[mask], Ca[mask]
745     if t.size != 181 or abs(t[-1] - T_END) > 1e-12:
746         raise ValueError(f"expected 181 oven records on [0, 10800], got {t.size}, t_end={t[-1]}")
747     if abs(Ta[0] - T0) > 1e-12:
748         raise ValueError(f"Ta(0)={Ta[0]} != T0={T0}")
749     Ta_fun = PchipOven(t, Ta)
750     Ca_fun = PchipOven(t, Ca)
751     for tj, yj in zip(t, Ta):
752         if abs(float(Ta_fun(tj)) - float(yj)) > 1e-12:
753             raise RuntimeError(f"PCHIP misses Ta knot t={tj}")
754     for tj, yj in zip(t, Ca):
755         if abs(float(Ca_fun(tj)) - float(yj)) > 1e-12:
756             raise RuntimeError(f"PCHIP misses Ca knot t={tj}")
757     return Ta_fun, Ca_fun, t
758
759
760 def integrate_coupled(fvm: CoupledRadialFVM, y0, t_end, Ta_fun, Ca_fun, rtol, atol, max_step,
761                      D_const=None, source=None, t_eval=None):
762     y0 = np.asarray(y0, dtype=float)
763     n = fvm.n_nodes
764     atol_arr = np.asarray(atol, dtype=float)
765     if atol_arr.ndim == 0:
766         atol_arr = np.concatenate([np.full(n, float(atol_arr)), np.full(n, float(atol_arr) * 0.01)])
767
768     def rhs(t, y):
769         ST = SC = None
770         if source is not None:
771             ST, SC = source(t, fvm.r)
772         return fvm.rhs(t, y, float(Ta_fun(t)), float(Ca_fun(t)), D_const=D_const, ST=ST, SC=SC)
773
774     if t_eval is None:
775         t_eval = np.arange(1.0, t_end + 0.5, 1.0)
776     sol = solve_ivp(
777         rhs,
778         (0.0, float(t_end)),
779         y0,
780         method="BDF",
781         rtol=rtol,
782         atol=atol_arr,
783         max_step=max_step,
784         t_eval=t_eval,
785         dense_output=False,
786         vectorized=False,
787         jac_sparsity=fvm.jac_sparsity(),
788     )
789     if not sol.success:
790         raise RuntimeError(f"coupled BDF failed: {sol.message}")
791     times = np.concatenate([[0.0], sol.t])
792     Y = np.vstack([y0, sol.y.T])
793     return times, Y
794
795

```

```

796 def extract_output(fvm: CoupledRadialFVM, Y: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
797     n = fvm.n_nodes
798     idx = fvm.output_indices()
799     T = Y[:, :n][:, idx]
800     C = Y[:, n:][:, idx]
801     return T, C
802
803
804 def compare_on_output(A, B, times) -> dict:
805     diff = np.abs(A - B)
806     k = int(np.argmax(diff))
807     n_t, n_r = diff.shape
808     i, j = divmod(k, n_r)
809     paper_t = np.isin(times, TABLE_TIMES)
810     paper_r = slice(None, None, 5) if n_r == 21 else slice(None)
811     return {
812         "max_abs": float(diff.max()),
813         "max_time_s": float(times[i]),
814         "max_radius_cm": 0.1 * j,
815         "mean_abs": float(diff.mean()),
816         "paper_table_max_abs": float(np.max(np.abs(A[paper_t][:, paper_r] - B[paper_t][:, paper_r]))),
817     }
818
819
820 def four_decimal_stats(A, B, times) -> dict:
821     Ar, Br = np.round(A, 4), np.round(B, 4)
822     changed = Ar != Br
823     paper_t = np.isin(times, TABLE_TIMES)
824     paper_r = slice(None, None, 5) if A.shape[1] == 21 else slice(None)
825     paper_changed = Ar[paper_t][:, paper_r] != Br[paper_t][:, paper_r]
826     return {
827         "changed_cells": int(np.sum(changed)),
828         "total_cells": int(A.size),
829         "paper_changed_cells": int(np.sum(paper_changed)),
830         "paper_total_cells": int(paper_changed.size),
831         "max_abs": float(np.max(np.abs(A - B))),
832         "paper_table_max_abs": float(np.max(np.abs(A[paper_t][:, paper_r] - B[paper_t][:, paper_r]))),
833     }
834
835
836 def manufactured_fields(r: np.ndarray, t: float) -> tuple[np.ndarray, np.ndarray]:
837     x2 = (r / R) ** 2
838     T = 28.0 + 2.0 * np.exp(-t / 600.0) * (1.0 + x2)
839     C = 1.5 + 0.2 * np.exp(-t / 500.0) * (1.0 + x2)
840     return T, C
841
842
843 def manufactured_env(r_surface: float, t: float) -> tuple[float, float]:
844     T, C = manufactured_fields(np.array([r_surface]), t)
845     Ts, Cs = float(T[0]), float(C[0])
846     u = np.exp(-t / 600.0)
847     v = np.exp(-t / 500.0)
848     Tr = 4.0 / R * u
849     Cr = 4.0 / R * v * 0.1 # C amplitude 0.2 vs T amplitude 2: Cr = 0.2*v*2r/R^2 at r=R -> 0.4 v / R
850     Cr = 0.4 / R * v
851     kn = float(conductivity(Cs))
852     Dn = float(moisture_diffusivity_q2(Cs, Ts))
853     Ta = Ts + kn * Tr / H
854     Ca = Cs + Dn * Cr / HM
855     return Ta, Ca
856
857
858 class ManufacturedOven:
859     def __init__(self, kind: str):
860         self.kind = kind

```

```

861
862     def __call__(self, t):
863         Ta, Ca = manufactured_env(R, float(t))
864         return Ta if self.kind == "T" else Ca
865
866
867 def manufactured_sources(t: float, r: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
868     T, C = manufactured_fields(r, t)
869     x2 = (r / R) ** 2
870     u = np.exp(-t / 600.0)
871     v = np.exp(-t / 500.0)
872     Tt = 2.0 * (-1.0 / 600.0) * u * (1.0 + x2)
873     Ct = 0.2 * (-1.0 / 500.0) * v * (1.0 + x2)
874     Tr = 4.0 * r / (R ** 2) * u
875     Trr = np.full_like(r, 4.0 / (R ** 2) * u)
876     Cr = 0.4 * r / (R ** 2) * v
877     Crr = np.full_like(r, 0.4 / (R ** 2) * v)
878     lapT = np.empty_like(r)
879     lapC = np.empty_like(r)
880     center = r < 1e-16
881     lapT[center] = 2.0 * Trr[center]
882     lapC[center] = 2.0 * Crr[center]
883     away = ~center
884     lapT[away] = Trr[away] + Tr[away] / r[away]
885     lapC[away] = Crr[away] + Cr[away] / r[away]
886     kn = np.asarray(conductivity(C), dtype=float)
887     kp = np.asarray(conductivity_deriv(C), dtype=float)
888     Bn = np.asarray(heat_capacity(C), dtype=float)
889     D, dD_dC, dD_dT = diffusivity_partials(C, T)
890     ST = Bn * Tt - (kn * lapT + kp * Cr * Tr)
891     SC = Ct - (D * lapC + dD_dC * Cr ** 2 + dD_dT * Tr * Cr)
892     return ST, SC
893
894
895 def conservation_moisture(fvm, times, Y, Ca_fun) -> dict:
896     n = fvm.n_nodes
897     C = Y[:, n:]
898     M = C @ fvm.W
899     M0 = float(M[0])
900     CN = C[:, -1]
901     Ca = np.array([float(Ca_fun(t)) for t in times])
902     I = float(np.trapezoid(HM * (CN - Ca), times))
903     rhs = M0 - R * I
904     Mt = float(M[-1])
905     denom = max(abs(M0 - Mt), abs(R * I), 1e-18)
906     return {"M0": M0, "M_end": Mt, "rhs": rhs, "relative_residual": abs(Mt - rhs) / denom}
907
908
909 def conservation_heat(fvm, times, Y, Ta_fun) -> dict:
910     n = fvm.n_nodes
911     T = Y[:, :n]
912     C = Y[:, n:]
913     Bn = np.asarray(heat_capacity(C), dtype=float)
914     Bp = np.asarray(heat_capacity_deriv(C), dtype=float)
915     E = 2.0 * np.pi * L * np.sum(fvm.W[None, :] * Bn * (T - TREF), axis=1)
916     TN = T[:, -1]
917     Ta = np.array([float(Ta_fun(t)) for t in times])
918     Q = 2.0 * np.pi * R * L * H * float(np.trapezoid(Ta - TN, times))
919     dC = np.gradient(C, times, axis=0)
920     corr = 2.0 * np.pi * L * np.sum(fvm.W[None, :] * Bp * (T - TREF) * dC, axis=1)
921     Icorr = float(np.trapezoid(corr, times))
922     lhs = float(E[-1] - E[0] - Icorr)
923     denom = max(abs(Q), abs(lhs), 1e-12)
924     return {
925         "E_end_minus_E0": float(E[-1] - E[0]),

```

```

926         "composition_correction": Icorr,
927         "Q_surface": Q,
928         "relative_residual": abs(lhs - Q) / denom,
929     }
930
931
932 def assert_physical(T_out, C_out, Ta_fun, Ca_fun) -> dict:
933     Ta_max = float(np.max(Ta_fun.y))
934     Ca_min = float(np.min(Ca_fun.y))
935     info = {
936         "T_min": float(T_out.min()),
937         "T_max": float(T_out.max()),
938         "C_min": float(C_out.min()),
939         "C_max": float(C_out.max()),
940         "T_below_28": float(max(0.0, 28.0 - T_out.min())),
941         "T_above-Ta_max": float(max(0.0, T_out.max() - Ta_max)),
942         "C_above_C0": float(max(0.0, C_out.max() - C0)),
943         "C_below-Ca_min": float(max(0.0, Ca_min - C_out.min())),
944         "Ta_range": [float(np.min(Ta_fun.y)), Ta_max],
945         "Ca_range": [Ca_min, float(np.max(Ca_fun.y))],
946     }
947     info["T_bounds_ok"] = info["T_below_28"] <= 1e-6 and info["T_above-Ta_max"] <= 1e-4
948     info["C_bounds_ok"] = info["C_above_C0"] <= 1e-8 and info["C_below-Ca_min"] <= 1e-8
949     return info
950
951
952 def write_result_workbook(path: Path, T_out, C_out, times):
953     radii_cm = 0.1 * np.arange(21)
954     wb = Workbook()
955     for title, arr in ((" 温度", T_out), (" 水分浓度", C_out)):
956         ws = wb.create_sheet(title)
957         ws["A1"] = " 时间\\到药材中心的距离"
958         for j, rc in enumerate(radii_cm, start=2):
959             ws.cell(1, j, float(rc))
960             for i, t in enumerate(times):
961                 ws.cell(i + 2, 1, int(round(float(t))))
962                 for j in range(21):
963                     cell = ws.cell(i + 2, j + 2, float(arr[i, j]))
964                     cell.number_format = "0.0000"
965     if "Sheet" in wb.sheetnames:
966         del wb["Sheet"]
967     path.parent.mkdir(parents=True, exist_ok=True)
968     wb.save(path)
969
970
971 def write_paper_csv(path: Path, arr_out, times):
972     t_idx = {float(t): i for i, t in enumerate(times)}
973     r_idx = {round(0.1 * j, 10): j for j in range(21)}
974     lines = ["t_h," + ",".join(f"r_{rc:g}cm" for rc in TABLE_RADII_CM)]
975     for th, t in zip(TABLE_HOURS, TABLE_TIMES):
976         row = [f"{th:g}"]
977         i = t_idx[float(t)]
978         for rc in TABLE_RADII_CM:
979             j = r_idx[float(rc)]
980             row.append(f"{arr_out[i, j]:.8f}")
981         lines.append(",".join(row))
982     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
983
984
985 def setup_mpl():
986     import matplotlib
987     matplotlib.use("Agg")
988     import matplotlib.pyplot as plt
989     plt.rcParams.update({
990         "font.sans-serif": ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "DejaVu Sans"],

```

```

991         "axes.unicode_minus": False,
992         "font.size": 9,
993         "axes.labelsize": 10,
994         "legend.fontsize": 8,
995         "xtick.labelsize": 8,
996         "ytick.labelsize": 8,
997         "axes.linewidth": 0.8,
998         "pdf.fonttype": 42,
999         "ps.fonttype": 42,
1000     })
1001     return plt
1002
1003
1004 def plot_figures(plt, fvm, times, Y, T_out, C_out, Ta_fun, Ca_fun):
1005     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
1006     n = fvm.n_nodes
1007     r_cm = fvm.r * 100.0
1008     idx = {float(t): i for i, t in enumerate(times)}
1009     marks = TABLE_TIMES
1010
1011     fig, ax = plt.subplots(figsize=(6.4, 3.8))
1012     for t in marks:
1013         ax.plot(r_cm, Y[idx[float(t)], :n], lw=1.2, label=fr"$t={t/3600:g}\,\mathrm{{h}}$")
1014     ax.set_xlabel(" 半径 $r$ (cm)")
1015     ax.set_ylabel(" 温度 $T$ (°C)")
1016     ax.set_xlim(0, 2)
1017     ax.legend(frameon=False, ncol=3, loc="lower center", bbox_to_anchor=(0.5, 1.02))
1018     fig.tight_layout()
1019     fig.savefig(FIGURES_DIR / "q2_T_profiles.pdf")
1020     plt.close(fig)
1021
1022     fig, ax = plt.subplots(figsize=(6.4, 3.8))
1023     for t in marks:
1024         ax.plot(r_cm, Y[idx[float(t)], n:], lw=1.2, label=fr"$t={t/3600:g}\,\mathrm{{h}}$")
1025     ax.set_xlabel(" 半径 $r$ (cm)")
1026     ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
1027     ax.set_xlim(0, 2)
1028     ax.legend(frameon=False, ncol=2)
1029     fig.tight_layout()
1030     fig.savefig(FIGURES_DIR / "q2_C_profiles.pdf")
1031     plt.close(fig)
1032
1033     t_plot = times
1034     fig, ax = plt.subplots(figsize=(6.4, 3.8))
1035     ax.plot(t_plot / 3600.0, [Ta_fun(t) for t in t_plot], lw=1.0, ls="--", label=" 烘房 $T_a$")
1036     ax.plot(t_plot / 3600.0, T_out[:, -1], lw=1.2, label=" 表面")
1037     ax.plot(t_plot / 3600.0, T_out[:, 0], lw=1.2, label=" 中心")
1038     ax.set_xlabel(" 时间 $t$ (h)")
1039     ax.set_ylabel(" 温度 (°C)")
1040     ax.set_xlim(0, 3)
1041     ax.legend(frameon=False)
1042     fig.tight_layout()
1043     fig.savefig(FIGURES_DIR / "q2_T_center_surface.pdf")
1044     plt.close(fig)
1045
1046     fig, ax = plt.subplots(figsize=(6.4, 3.8))
1047     ax.plot(t_plot / 3600.0, C_out[:, -1], lw=1.2, label=" 表面")
1048     ax.plot(t_plot / 3600.0, C_out[:, 0], lw=1.2, label=" 中心")
1049     ax.set_xlabel(" 时间 $t$ (h)")
1050     ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
1051     ax.set_xlim(0, 3)
1052     ax.legend(frameon=False)
1053     fig.tight_layout()
1054     fig.savefig(FIGURES_DIR / "q2_C_center_surface.pdf")
1055     plt.close(fig)

```

```

1056
1057 D_c = moisture_diffusivity_q2(C_out[:, 0], T_out[:, 0])
1058 D_s = moisture_diffusivity_q2(C_out[:, -1], T_out[:, -1])
1059 fig, ax = plt.subplots(figsize=(6.4, 3.6))
1060 ax.plot(t_plot / 3600.0, D_s, lw=1.2, label="表面 $D_s$")
1061 ax.plot(t_plot / 3600.0, D_c, lw=1.2, label="中心 $D_c$")
1062 ax.set_xlabel("时间 $t$ (h)")
1063 ax.set_ylabel(r"$D$ (m$^2$/s)")
1064 ax.set_xlim(0, 3)
1065 ax.legend(frameon=False)
1066 fig.tight_layout()
1067 fig.savefig(FIGURES_DIR / "q2_D_center_surface.pdf")
1068 plt.close(fig)
1069
1070
1071 def run_mms(level: int = 4) -> dict:
1072     r = graded_radial_nodes(level)
1073     fvm = CoupledRadialFVM(r)
1074     T0f, C0f = manufactured_fields(fvm.r, 0.0)
1075     y0 = fvm.pack(T0f, C0f)
1076     Ta_fun = ManufacturedOven("T")
1077     Ca_fun = ManufacturedOven("C")
1078     times, Y = integrate_coupled(
1079         fvm, y0, 100.0, Ta_fun, Ca_fun,
1080         rtol=1e-8, atol=1e-9, max_step=0.5,
1081         source=manufactured_sources,
1082         t_eval=np.arange(1.0, 100.5, 1.0),
1083     )
1084     n = fvm.n_nodes
1085     T_num, C_num = Y[:, :n], Y[:, n:]
1086     T_ex = np.stack([manufactured_fields(fvm.r, t)[0] for t in times])
1087     C_ex = np.stack([manufactured_fields(fvm.r, t)[1] for t in times])
1088     return {
1089         "level": level,
1090         "n_nodes": n,
1091         "T_max_abs": float(np.max(np.abs(T_num - T_ex))),
1092         "C_max_abs": float(np.max(np.abs(C_num - C_ex))),
1093         "T_end_max_abs": float(np.max(np.abs(T_num[-1] - T_ex[-1]))),
1094         "C_end_max_abs": float(np.max(np.abs(C_num[-1] - C_ex[-1]))),
1095     }
1096
1097
1098 def main():
1099     t_wall0 = time.perf_counter()
1100     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
1101     Ta_fun, Ca_fun, knots = load_q2_inputs()
1102     print("oven points", knots.size, "kind", Ta_fun.kind, "Ta(10800)=", Ta_fun(T_END), "Ca(10800)=",
1103           ↪ Ca_fun(T_END))
1103     D0 = float(moisture_diffusivity_q2(C0, T0))
1104     print("initial D", D0, "k", float(conductivity(C0)), "B", float(heat_capacity(C0)))
1105
1106     print("manufactured-solution probe")
1107     mms = run_mms(4)
1108     print("MMS", mms)
1109
1110     mesh_levels = [2, 4]
1111     rtol, atol_T, atol_C, max_step = 1e-9, 1e-8, 1e-10, 1.0
1112     solutions = {}
1113     for level in mesh_levels:
1114         r = graded_radial_nodes(level)
1115         fvm = CoupledRadialFVM(r)
1116         tag = f"G{level}"
1117         print(f"\n=== {tag} nodes={fvm.n_nodes} ===")
1118         y0 = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
1119         atol = np.concatenate([np.full(fvm.n_nodes, atol_T), np.full(fvm.n_nodes, atol_C)])

```

```

1120     times, Y = integrate_coupled(fvm, y0, T_END, Ta_fun, Ca_fun, rtol, atol, max_step)
1121     times_pos = times[times > 0]
1122     Y_pos = Y[times > 0]
1123     T_out, C_out = extract_output(fvm, Y_pos)
1124     solutions[level] = dict(
1125         tag=tag, fvm=fvm, times=times_pos, Y=Y_pos, Y_full=Y, times_full=times,
1126         T_out=T_out, C_out=C_out, n_nodes=fvm.n_nodes,
1127     )
1128     print("done", tag, "T", T_out.min(), T_out.max(), "C", C_out.min(), C_out.max(),
1129           "C(t=1,s)", C_out[0, -1], "C(3h,s)", C_out[-1, -1], "C(3h,c)", C_out[-1, 0])
1130
1131     t = solutions[4]["times"]
1132     conv = {
1133         "T_G2_vs_G4": compare_on_output(solutions[2]["T_out"], solutions[4]["T_out"], t),
1134         "C_G2_vs_G4": compare_on_output(solutions[2]["C_out"], solutions[4]["C_out"], t),
1135     }
1136     fourdec = {
1137         "T_G2_vs_G4": four_decimal_stats(solutions[2]["T_out"], solutions[4]["T_out"], t),
1138         "C_G2_vs_G4": four_decimal_stats(solutions[2]["C_out"], solutions[4]["C_out"], t),
1139     }
1140     print("conv", conv)
1141     print("4dp", fourdec)
1142     paper_ok = fourdec["T_G2_vs_G4"]["paper_changed_cells"] == 0 and
1143     ↪ fourdec["C_G2_vs_G4"]["paper_changed_cells"] == 0
1144     abs_ok = conv["T_G2_vs_G4"]["max_abs"] <= 2e-5 and conv["C_G2_vs_G4"]["max_abs"] <= 2e-5
1145     need_g8 = not (paper_ok and abs_ok)
1146     if need_g8:
1147         print("G2 vs G4 not enough; running G8")
1148         level = 8
1149         r = graded_radial_nodes(level)
1150         fvm = CoupledRadialFVM(r)
1151         y0 = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
1152         atol = np.concatenate([np.full(fvm.n_nodes, atol_T), np.full(fvm.n_nodes, atol_C)])
1153         times, Y = integrate_coupled(fvm, y0, T_END, Ta_fun, Ca_fun, rtol, atol, max_step)
1154         times_pos = times[times > 0]
1155         Y_pos = Y[times > 0]
1156         T_out, C_out = extract_output(fvm, Y_pos)
1157         solutions[8] = dict(
1158             tag="G8", fvm=fvm, times=times_pos, Y=Y_pos, Y_full=Y, times_full=times,
1159             T_out=T_out, C_out=C_out, n_nodes=fvm.n_nodes,
1160         )
1161         t = solutions[8]["times"]
1162         conv["T_G4_vs_G8"] = compare_on_output(solutions[4]["T_out"], solutions[8]["T_out"], t)
1163         conv["C_G4_vs_G8"] = compare_on_output(solutions[4]["C_out"], solutions[8]["C_out"], t)
1164         fourdec["T_G4_vs_G8"] = four_decimal_stats(solutions[4]["T_out"], solutions[8]["T_out"], t)
1165         fourdec["C_G4_vs_G8"] = four_decimal_stats(solutions[4]["C_out"], solutions[8]["C_out"], t)
1166         print("conv G4 G8", conv["T_G4_vs_G8"], conv["C_G4_vs_G8"])
1167         pub_level = 8
1168         paper_ok = fourdec["T_G4_vs_G8"]["paper_changed_cells"] == 0 and
1169         ↪ fourdec["C_G4_vs_G8"]["paper_changed_cells"] == 0
1170         abs_ok = conv["T_G4_vs_G8"]["max_abs"] <= 2e-5 and conv["C_G4_vs_G8"]["max_abs"] <= 2e-5
1171     else:
1172         pub_level = 4 if fourdec["C_G2_vs_G4"]["changed_cells"] or fourdec["T_G2_vs_G4"]["changed_cells"]
1173         ↪ else 2
1174         if pub_level == 2:
1175             pub_level = 4
1176     pub = solutions[pub_level]
1177     fvm = pub["fvm"]
1178     times = pub["times"]
1179     print("publishing", pub["tag"], "paper_ok", paper_ok, "abs_ok", abs_ok)
1180
1181     print("time refinement")
1182     y0 = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
1183     atol = np.concatenate([np.full(fvm.n_nodes, atol_T / 5), np.full(fvm.n_nodes, atol_C / 5)])
1184     t2, Y2 = integrate_coupled(fvm, y0, T_END, Ta_fun, Ca_fun, rtol / 5, atol, 0.5)

```

```

1182 T2, C2 = extract_output(fvm, Y2[t2 > 0])
1183 time_sens = {"T": compare_on_output(pub["T_out"], T2, times), "C": compare_on_output(pub["C_out"], C2,
↪ times)}
1184 time_4dp = {"T": four_decimal_stats(pub["T_out"], T2, times), "C": four_decimal_stats(pub["C_out"], C2,
↪ times)}
1185 print("time sens", time_sens)
1186
1187 print("frozen-D moisture probe")
1188 D_freeze = D0
1189 Bi_m = HM * R / D_freeze
1190 C_ref, moisture_truncation = converged_reference(Bi_m, D_freeze, R, OUTPUT_RADII_M, times, Ca_fun, C0)
1191 y0f = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
1192 atol = np.concatenate([np.full(fvm.n_nodes, atol_T), np.full(fvm.n_nodes, atol_C)])
1193 tf, Yf = integrate_coupled(fvm, y0f, T_END, Ta_fun, Ca_fun, rtol, atol, max_step, D_const=D_freeze)
1194 _, C_frozen = extract_output(fvm, Yf[tf > 0])
1195 frozen = compare_on_output(C_frozen, C_ref, times)
1196 nl_vs_frozen = compare_on_output(pub["C_out"], C_frozen, times)
1197 print("frozen vs analytic", frozen)
1198 print("nonlinear vs frozen", nl_vs_frozen)
1199
1200 n = fvm.n_nodes
1201 D_field = moisture_diffusivity_q2(pub["Y"][:, n:], pub["Y"][:, :n])
1202 D_stats = {
1203     "D0": D0,
1204     "D_min": float(np.min(D_field)),
1205     "D_max": float(np.max(D_field)),
1206     "D_min_over_D0": float(np.min(D_field) / D0),
1207     "D_max_over_D0": float(np.max(D_field) / D0),
1208     "center_C_end": float(pub["C_out"][-1, 0]),
1209     "surface_C_end": float(pub["C_out"][-1, -1]),
1210     "center_T_end": float(pub["T_out"][-1, 0]),
1211     "surface_T_end": float(pub["T_out"][-1, -1]),
1212 }
1213 print("D stats", D_stats)
1214
1215 bounds = assert_physical(pub["Y_full"][:, :n], pub["Y_full"][:, n:], Ta_fun, Ca_fun)
1216 cons_C = conservation_moisture(fvm, pub["times_full"], pub["Y_full"], Ca_fun)
1217 cons_T = conservation_heat(fvm, pub["times_full"], pub["Y_full"], Ta_fun)
1218 print("bounds", bounds)
1219 print("conservation C", cons_C)
1220 print("conservation T", cons_T)
1221
1222 delivery = make_gate({
1223     "space_abs": abs_ok,
1224     "space_paper": paper_ok,
1225     "all_finite": np.isfinite(pub["Y_full"]).all(),
1226     "manufactured_solution": mms["T_max_abs"] <= 2e-5 and mms["C_max_abs"] <= 2e-5,
1227     "time_abs": time_sens["T"]["max_abs"] <= 2e-5 and time_sens["C"]["max_abs"] <= 2e-5,
1228     "time_paper": time_4dp["T"]["paper_changed_cells"] == 0 and time_4dp["C"]["paper_changed_cells"] ==
↪ 0,
1229     "temperature_bounds": bounds["T_bounds_ok"],
1230     "moisture_bounds": bounds["C_bounds_ok"],
1231     "moisture_reference_truncation": moisture_truncation["passed"],
1232     "frozen_reference_error": frozen["max_abs"] <= 2e-5,
1233     "heat_balance": cons_T["relative_residual"] <= 1e-6,
1234     "moisture_balance": cons_C["relative_residual"] <= 1e-6,
1235 })
1236 print("delivery", delivery)
1237
1238 paper_T = {f"h{th:g}_r{rc:g}": float(pub["T_out"][int(t) - 1, int(round(rc / 0.1))])
1239             for th, t in zip(TABLE_HOURS, TABLE_TIMES) for rc in TABLE_RADII_CM}
1240 paper_C = {f"h{th:g}_r{rc:g}": float(pub["C_out"][int(t) - 1, int(round(rc / 0.1))])
1241             for th, t in zip(TABLE_HOURS, TABLE_TIMES) for rc in TABLE_RADII_CM}
1242
1243 validation = {

```

```

1244     "result_version": "q2-closeout-v1",
1245     "interpolation": Ta_fun.kind,
1246     "published_mesh": pub["tag"],
1247     "published_n_nodes": pub["n_nodes"],
1248     "mesh_levels": list(solutions.keys()),
1249     "solver": "coupled-FVM+BDF",
1250     "rtol": rtol, "atol_T": atol_T, "atol_C": atol_C, "max_step": max_step,
1251     "mms": mms,
1252     "grid_convergence": conv,
1253     "four_decimal_grid": fourdec,
1254     "time_sensitivity": time_sens,
1255     "four_decimal_time": time_4dp,
1256     "delivery_gate": delivery,
1257     "reference_truncation_C": moisture_truncation,
1258     "frozen_D_moisture_vs_analytic": frozen,
1259     "nonlinear_C_vs_frozen_D": nl_vs_frozen,
1260     "D_variation": D_stats,
1261     "conservation_T_10800s": cons_T,
1262     "conservation_C_10800s": cons_C,
1263     "physical_bounds": bounds,
1264     "paper_table_T": paper_T,
1265     "paper_table_C": paper_C,
1266     "elapsed_s": time.perf_counter() - t_wall0,
1267     "python": sys.version,
1268     "end_face_2d": "skipped_by_plan",
1269 }
1270 require_delivery(validation, RESULTS_DIR / "diagnostics/q2_delivery/latest.json")
1271 np.savez(
1272     RESULTS_DIR / "q2_solution.npz",
1273     mesh_level=pub_level,
1274     n_nodes=pub["n_nodes"],
1275     times=times,
1276     r=fvm.r,
1277     output_radii_m=OUTPUT_RADII_M,
1278     checkpoint_times=np.r_[0., 1., 60., 100., TABLE_TIMES],
1279     T_checkpoints=pub["Y_full"][np.r_[0., 1., 60., 100., TABLE_TIMES].astype(int), :n],
1280     C_checkpoints=pub["Y_full"][np.r_[0., 1., 60., 100., TABLE_TIMES].astype(int), n:],
1281     T_out=pub["T_out"],
1282     C_out=pub["C_out"],
1283     interpolation=np.array(Ta_fun.kind),
1284     rtol=rtol, atol_T=atol_T, atol_C=atol_C, max_step=max_step,
1285 )
1286 write_result_workbook(RESULTS_DIR / "result2.xlsx", pub["T_out"], pub["C_out"], times)
1287 write_paper_csv(RESULTS_DIR / "q2_table_temperature.csv", pub["T_out"], times)
1288 write_paper_csv(RESULTS_DIR / "q2_table_moisture.csv", pub["C_out"], times)
1289 plt = setup_mpl()
1290 plot_figures(plt, fvm, times, pub["Y"], pub["T_out"], pub["C_out"], Ta_fun, Ca_fun)
1291
1292 (RESULTS_DIR / "q2_validation.json").write_text(
1293     json.dumps(validation, ensure_ascii=False, indent=2, default=float),
1294     encoding="utf-8",
1295 )
1296 print("elapsed_s", validation["elapsed_s"])
1297 print("wrote results to", RESULTS_DIR)
1298
1299
1300
1301 if __name__ == "__main__":
1302     main()
1303 '''
1304
1305 # =====
1306 # Module: q3_balance.py
1307 # =====

```

```

1308 SOURCES['q3_balance'] = r''''''Continuous-solution balance integrals for the accepted Q3 effective
    ↪ model.'''
1309 import numpy as np
1310 from numpy.polynomial.legendre import leggauss
1311 from appendix3 import heat_capacity, heat_capacity_deriv, moisture_diffusivity_q2
1312 from utils import R, L, H, HM
1313
1314
1315 def audit_segment(fvm, sol, Ta_fun, Ca_fun, t_end=None, orders=(4,8)):
1316     if sol.sol is None:
1317         raise ValueError("balance audit requires a dense solution")
1318     n = fvm.n_nodes
1319     start = float(sol.sol.ts[0])
1320     end = float(sol.sol.ts[-1]) if t_end is None else float(t_end)
1321     if not start < end <= sol.sol.ts[-1]+1e-9:
1322         raise ValueError("audit outside dense-solution interval")
1323     edges = np.r_[sol.sol.ts[sol.sol.ts < end],end]
1324     edges = np.unique(edges)
1325     z0, z1 = sol.sol(start), sol.sol(end)
1326     factor = 2*np.pi*L
1327     def energy(z):
1328         return float(factor*np.dot(fvm.W,heat_capacity(z[n:]))*(z[n:]-28.))
1329     def quadrature(order):
1330         gx,gw=leggauss(order)
1331         total=np.zeros(3)
1332         for j in range(0,len(edges)-1,64):
1333             aa,bb=edges[j:-1][:64],edges[j+1:][:64]
1334             tt=((aa+bb)[:,None]/2+(bb-aa)[:,None]/2*gx).ravel()
1335             Y=sol.sol(tt);T,C=Y[:n],Y[n:]
1336             Ta,Ca=np.asarray(Ta_fun(tt)),np.asarray(Ca_fun(tt))
1337             D=moisture_diffusivity_q2(C,T)
1338             Df=2*D[:,-1]*D[1:]/np.maximum(D[:,-1]+D[1:],1e-300)
1339             F=-fvm.r_half[:,-1]*Df*np.diff(C,axis=0)/fvm.dr_face[:,-1]
1340             flows=np.vstack([np.zeros((1,len(tt))),F,R*HM*(C[-1]-Ca)])
1341             Cdot=-np.diff(flows,axis=0)/fvm.W[:,-1]
1342             vals=np.vstack([R*HM*(C[-1]-Ca),factor*R*H*(Ta-T[-1]),
1343                             factor*np.sum(fvm.W[:,-1]*heat_capacity_deriv(C)*(T-28.)*Cdot,axis=0)])
1344             weights=((bb-aa)[:,None]/2*gw).ravel()
1345             total+=vals@weights
1346         return total
1347     integrals={str(o):quadrature(o).tolist() for o in orders}
1348     return {"start_s":start,"end_s":end,"accepted_intervals":len(edges)-1,
1349            "M_start":float(fvm.W@z0[n:]),"M_end":float(fvm.W@z1[n:]),
1350            "E_start":energy(z0),"E_end":energy(z1),"integrals_by_order":integrals}
1351
1352
1353 def combine(parts):
1354     if not parts:
1355         raise ValueError('no balance segments')
1356     for left, right in zip(parts, parts[1:]):
1357         if abs(left['end_s']-right['start_s']) > 1e-9:
1358             raise ValueError('balance segments have a time gap or overlap')
1359         for field in ('M', 'E'):
1360             if not np.isclose(left[field+'_end'], right[field+'_start'], rtol=1e-12, atol=1e-14):
1361                 raise ValueError('balance segments have a state discontinuity')
1362     I=np.sum([p['integrals_by_order']['8'] for p in parts],axis=0)
1363     I4=np.sum([p['integrals_by_order']['4'] for p in parts],axis=0)
1364     M0,M1=parts[0]['M_start'],parts[-1]['M_end']
1365     E0,E1=parts[0]['E_start'],parts[-1]['E_end']
1366     mass_denom=max(abs(M0-M1),abs(I[0]),1e-18)
1367     heat_denom=max(abs(E1-E0-I[2]),abs(I[1]),1e-12)
1368     return {
1369         'moisture':{'M0':M0,'M_end':M1,'surface_integral':float(I[0]),
1370                    'relative_residual':float(abs(M1-M0+I[0])/mass_denom)},
1371         'heat':{'E_end_minus_E0':E1-E0,'composition_correction':float(I[2]),'Q_surface':float(I[1]),

```

```

1372         'relative_residual':float(abs(E1-E0-I[2]-I[1])/heat_denom)),
1373         'quadrature_relative_difference':{'moisture':float(abs(I[0]-I4[0])/mass_denom),
1374         'heat_and_composition':float((abs(I[1]-I4[1])+abs(I[2]-I4[2]))/heat_denom)},
1375         'parts':parts,
1376         'interpretation':'Integrated effective equation with B_prime(C)*(T-28)*Cdot correction; not full
        ↪ multiphase energy conservation.'}
1377     '''
1378
1379     # =====
1380     # Module: problem3.py
1381     # =====
1382     SOURCES['problem3'] = r''''''Problem 3: full coupled drying until every interior node is below 0.15
        ↪ kg/kg.'''
1383     from __future__ import annotations
1384
1385     import argparse
1386     import json
1387     import sys
1388     import time
1389     from pathlib import Path
1390
1391     import numpy as np
1392     from scipy.integrate import solve_ivp
1393     from scipy.optimize import brentq
1394
1395     CODE_DIR = Path(__file__).resolve().parent
1396     if str(CODE_DIR) not in sys.path:
1397         sys.path.insert(0, str(CODE_DIR))
1398
1399     from appendix3 import moisture_diffusivity_q2
1400     from delivery import make_gate, require_delivery
1401     from problem2 import conservation_heat, conservation_moisture, extract_output, setup_mpl
1402     from radial_coupled import CoupledRadialFVM
1403     from radial_fvm import graded_radial_nodes
1404     from utils import (
1405         C0,
1406         FIGURES_DIR,
1407         RESULTS_DIR,
1408         T0,
1409         PchipOven,
1410         load_oven_table,
1411     )
1412
1413     T_SWITCH = 14400.0
1414     TA_AFTER = 50.0
1415     CA_AFTER = 0.05
1416     C_THRESH = 0.15
1417     T_HARD_CAP = 21.0 * 24.0 * 3600.0
1418     OUTPUT_RADII_M = 0.001 * np.arange(21)
1419     TABLE_RADII_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
1420     EXCEL_DT = 60.0
1421     TABLE_DT_H = 6.0
1422     OVEN_N_EXPECTED = 241
1423
1424     # First-round baseline; recorded after the run, not treated as a prior user decision.
1425     RTOL = 1e-9
1426     ATOL_T = 1e-8
1427     ATOL_C = 1e-10
1428     MAX_STEP_EARLY = 2.0
1429     MAX_STEP_LATE = 60.0
1430
1431
1432     class ExtendedOven:
1433         """PCHIP through attachment 1 on [0, 4 h]; constant value strictly after 4 h."""
1434

```

```

1435     def __init__(self, pchip: PchipOven, after_value: float, t_switch: float = T_SWITCH):
1436         self.pchip = pchip
1437         self.after = float(after_value)
1438         self.t_switch = float(t_switch)
1439         self.kind = pchip.kind
1440         self.t = pchip.t
1441         self.y = pchip.y
1442
1443     def __call__(self, t):
1444         if np.ndim(t) == 0:
1445             tt = float(t)
1446             if tt <= self.t_switch:
1447                 return float(self.pchip(tt))
1448             return self.after
1449         tt = np.asarray(t, dtype=float)
1450         out = np.empty_like(tt, dtype=float)
1451         inside = tt <= self.t_switch
1452         if np.any(inside):
1453             out[inside] = np.asarray(self.pchip(tt[inside]), dtype=float)
1454         out[~inside] = self.after
1455         return out
1456
1457 def load_q3_inputs() -> tuple[ExtendedOven, ExtendedOven, np.ndarray]:
1458     t, Ta, Ca = load_oven_table()
1459     if t.size != OVEN_N_EXPECTED or abs(t[-1] - T_SWITCH) > 1e-12:
1460         raise ValueError(f"expected {OVEN_N_EXPECTED} oven records ending at {T_SWITCH}, got {t.size},
1461             ↳ t_end={t[-1]}")
1462     if abs(Ta[0] - T0) > 1e-12:
1463         raise ValueError(f"Ta(0)={Ta[0]} != T0={T0}")
1464     Ta_pchip = PchipOven(t, Ta)
1465     Ca_pchip = PchipOven(t, Ca)
1466     for tj, yj in zip(t, Ta):
1467         if abs(float(Ta_pchip(tj)) - float(yj)) > 1e-12:
1468             raise RuntimeError(f"PCHIP misses Ta knot t={tj}")
1469     for tj, yj in zip(t, Ca):
1470         if abs(float(Ca_pchip(tj)) - float(yj)) > 1e-12:
1471             raise RuntimeError(f"PCHIP misses Ca knot t={tj}")
1472     Ta_fun = ExtendedOven(Ta_pchip, TA_AFTER)
1473     Ca_fun = ExtendedOven(Ca_pchip, CA_AFTER)
1474     jump = {
1475         "Ta_at_4h": float(Ta_fun(T_SWITCH)),
1476         "Ca_at_4h": float(Ca_fun(T_SWITCH)),
1477         "Ta_after": float(Ta_fun(T_SWITCH + 1e-9)),
1478         "Ca_after": float(Ca_fun(T_SWITCH + 1e-9)),
1479     }
1480     print("oven 4h jump", jump, flush=True)
1481     return Ta_fun, Ca_fun, t
1482
1483 def atol_vector(n: int, atol_T: float = ATOL_T, atol_C: float = ATOL_C) -> np.ndarray:
1484     return np.concatenate([np.full(n, float(atol_T)), np.full(n, float(atol_C))])
1485
1486 def max_moisture_event(n_nodes: int):
1487     def event(t, y):
1488         return float(np.max(y[n_nodes:]) - C_THRESH)
1489
1490     event.terminal = True
1491     event.direction = -1.0
1492     return event
1493
1494 def integrate_segment(
1495     fvm: CoupledRadialFVM,

```

```

1499     y0,
1500     t0: float,
1501     t1: float,
1502     Ta_fun,
1503     Ca_fun,
1504     rtol: float,
1505     atol: np.ndarray,
1506     max_step: float,
1507     *,
1508     t_eval=None,
1509     events=None,
1510     dense_output: bool = False,
1511     first_step=None,
1512     label: str = "",
1513     balance_audit: bool = False,
1514 ):
1515     n = fvm.n_nodes
1516     y0 = np.asarray(y0, dtype=float)
1517     t0 = float(t0)
1518     t1 = float(t1)
1519     if t1 <= t0 + 1e-15:
1520         raise ValueError(f"empty interval [{t0}, {t1}]")
1521     nfev = [0]
1522     t_print = [t0]
1523     t_wall0 = time.perf_counter()
1524
1525     def rhs(t, y):
1526         nfev[0] += 1
1527         interval = 3600.0 if t < T_SWITCH + 12.0 * 3600.0 else 6.0 * 3600.0
1528         if t - t_print[0] >= interval:
1529             C = y[n:]
1530             imax = int(np.argmax(C))
1531             elapsed = time.perf_counter() - t_wall0
1532             print(
1533                 f"    {label}t={t/3600.0:.3f} h    maxC={float(C[imax]):.6f} "
1534                 f"rmax={fvm.r[imax]*100:.4f} cm    nfev={nfev[0]}    wall={elapsed:.1f}s",
1535                 flush=True,
1536             )
1537             t_print[0] = float(t)
1538         return fvm.rhs(t, y, float(Ta_fun(t)), float(Ca_fun(t)))
1539
1540     kwargs = dict(
1541         fun=rhs,
1542         t_span=(t0, t1),
1543         y0=y0,
1544         method="BDF",
1545         rtol=rtol,
1546         atol=np.asarray(atol, dtype=float),
1547         max_step=max_step,
1548         dense_output=dense_output or balance_audit,
1549         vectorized=False,
1550         jac_sparsity=fvm.jac_sparsity(),
1551     )
1552     if t_eval is not None:
1553         te = np.asarray(t_eval, dtype=float)
1554         te = te[(te > t0 + 1e-14) & (te <= t1 + 1e-12)]
1555         if te.size:
1556             kwargs["t_eval"] = te
1557     if events is not None:
1558         kwargs["events"] = events
1559     if first_step is not None:
1560         kwargs["first_step"] = min(float(first_step), 0.5 * max_step, t1 - t0)
1561
1562     sol = solve_ivp(**kwargs)
1563     if not sol.success:

```

```

1564         raise RuntimeError(f"coupled BDF failed on [{t0}, {t1}] {label}: {sol.message}")
1565     if balance_audit:
1566         from q3_balance import audit_segment
1567         sol.balance_audit = audit_segment(fvm, sol, Ta_fun, Ca_fun)
1568         if not dense_output:
1569             sol.sol = None
1570     return sol
1571
1572
1573 def excel_times(t_rep: float) -> np.ndarray:
1574     t_rep = float(t_rep)
1575     grid = np.arange(EXCEL_DT, np.floor(t_rep / EXCEL_DT) * EXCEL_DT + 0.5 * EXCEL_DT, EXCEL_DT)
1576     if grid.size == 0 or grid[-1] < t_rep - 1e-12:
1577         times = np.concatenate([grid, [t_rep]]) if grid.size else np.array([t_rep])
1578     elif abs(grid[-1] - t_rep) <= 1e-9:
1579         times = grid
1580     else:
1581         times = np.concatenate([grid, [t_rep]])
1582     return times
1583
1584
1585 def table_hours(t_rep_h: float) -> np.ndarray:
1586     t_rep_h = float(t_rep_h)
1587     hrs = np.arange(TABLE_DT_H, t_rep_h + 1e-12, TABLE_DT_H)
1588     if hrs.size == 0 or abs(hrs[-1] - t_rep_h) > 1e-10:
1589         hrs = np.concatenate([hrs, [t_rep_h]]) if hrs.size else np.array([t_rep_h])
1590     return hrs
1591
1592
1593 def monitor_from_Y(fvm: CoupledRadialFVM, times: np.ndarray, Y: np.ndarray) -> dict:
1594     n = fvm.n_nodes
1595     C = Y[:, n:]
1596     imax = np.argmax(C, axis=1)
1597     M = C[np.arange(C.shape[0]), imax]
1598     rmax = fvm.r[imax]
1599     dC = np.diff(C, axis=1)
1600     # Allow tiny positive bumps from rounding; count clear interior increases.
1601     n_increase = int(np.sum(dC > 1e-10))
1602     center_is_max = imax == 0
1603     return {
1604         "times": times,
1605         "M": M,
1606         "rmax_m": rmax,
1607         "imax": imax,
1608         "center_is_max_frac": float(np.mean(center_is_max)),
1609         "n_radial_increases": n_increase,
1610         "C_center": C[:, 0].copy(),
1611         "C_surface": C[:, -1].copy(),
1612         "T_center": Y[:, 0].copy(),
1613         "T_surface": Y[:, n - 1].copy(),
1614     }
1615
1616
1617 def locate_threshold(
1618     fvm: CoupledRadialFVM,
1619     Ta_fun,
1620     Ca_fun,
1621     t_left: float,
1622     y_left: np.ndarray,
1623     t_right: float,
1624     rtol: float,
1625     atol: np.ndarray,
1626 ):
1627     """Bracketed root of max_i C_i(t) - 0.15 on a locally refined dense interpolant."""
1628     n = fvm.n_nodes

```

```

1629     event = max_moisture_event(n)
1630     t_pad = max(30.0, 2.0 * (t_right - t_left))
1631     t_hi = t_right + t_pad
1632     sol = integrate_segment(
1633         fvm, y_left, t_left, t_hi, Ta_fun, Ca_fun, rtol, atol,
1634         max_step=0.5, events=event, dense_output=True, first_step=0.05,
1635         label="zoom ",
1636     )
1637     if sol.t_events is None or len(sol.t_events[0]) == 0:
1638         C_end = sol.y[n:, -1]
1639         raise RuntimeError(
1640             f"zoom missed threshold on [{t_left}, {t_hi}]; "
1641             f"maxC(end)={float(np.max(C_end))}"
1642         )
1643     t_solver = float(sol.t_events[0][0])
1644     y_solver = np.asarray(sol.y_events[0][0], dtype=float)
1645
1646     def g(t):
1647         return float(np.max(sol.sol(float(t))[n:])) - C_THRESH
1648
1649     g_left = g(t_left)
1650     t_lo = t_left
1651     t_hi_root = float(sol.t[-1])
1652     g_hi = g(t_hi_root)
1653     if g_left <= 0.0:
1654         raise RuntimeError(f"zoom left end already <= threshold: t={t_left}, g={g_left}")
1655     if g_hi > 0.0 and abs(g_hi) <= 1e-8:
1656         t_star = t_solver
1657     elif g_hi > 0.0:
1658         raise RuntimeError(f"zoom right end still above threshold: t={t_hi_root}, g={g_hi}")
1659     else:
1660         t_star = float(brentq(g, t_lo, t_hi_root, xtol=1e-4, rtol=1e-12, maxiter=200))
1661     y_star = np.asarray(sol.sol(t_star), dtype=float)
1662     C_star = y_star[n:]
1663     imax = int(np.argmax(C_star))
1664     info = {
1665         "t_star_s": t_star,
1666         "t_star_h": t_star / 3600.0,
1667         "t_solver_event_s": t_solver,
1668         "event_solver_minus_brent_s": t_solver - t_star,
1669         "M_star": float(C_star[imax]),
1670         "g_star": float(C_star[imax] - C_THRESH),
1671         "rmax_star_m": float(fvm.r[imax]),
1672         "rmax_star_cm": float(fvm.r[imax] * 100.0),
1673         "imax_star": imax,
1674         "center_is_max": bool(imax == 0),
1675         "C_center_star": float(C_star[0]),
1676         "C_surface_star": float(C_star[-1]),
1677         "T_center_star": float(y_star[0]),
1678         "T_surface_star": float(y_star[n - 1]),
1679         "bracket_s": [float(t_left), float(t_hi_root)],
1680         "zoom_nfev": int(sol.nfev),
1681     }
1682     return t_star, y_star, sol, info, y_solver
1683
1684
1685 def find_strict_report(eval_M, t_star: float, t_hi: float, margin: float = 0.0) -> dict:
1686     target = C_THRESH - float(margin)
1687     t_star = float(t_star)
1688     t_hi = float(t_hi)
1689     if not (np.isfinite([t_star, t_hi, margin]).all() and 0 <= margin < C_THRESH and t_hi > t_star):
1690         raise ValueError('invalid strict-report interval or margin')
1691
1692     original_eval = eval_M
1693     def eval_M(t):

```

```

1694         if not t_star <= t <= t_hi:
1695             raise RuntimeError('strict report requested outside verified continuation')
1696         value = float(original_eval(t))
1697         if not np.isfinite(value):
1698             raise RuntimeError('nonfinite moisture in strict report')
1699         return value
1700
1701     def h(t):
1702         return float(eval_M(t) - target)
1703
1704     h0 = h(t_star)
1705     h1 = h(t_hi)
1706     if h1 >= 0.0:
1707         raise RuntimeError(f"M(t_hi)={eval_M(t_hi)} still >= {target}")
1708     if h0 < 0.0:
1709         t_cross = t_star
1710     else:
1711         t_cross = float(brentq(h, t_star, t_hi, xtol=1e-4, rtol=1e-12, maxiter=200))
1712     t_rep = float(np.floor(max(t_cross, t_star)) + 1.0)
1713     # Walk forward in seconds until unrounded moisture is strictly below the target.
1714     for _ in range(10000):
1715         if float(eval_M(t_rep)) < target:
1716             break
1717         t_rep += 1.0
1718     else:
1719         raise RuntimeError("failed to find integer-second report time below threshold")
1720     t_hour4 = np.ceil((t_rep / 3600.0) * 10000.0) / 10000.0
1721     t_hour4_s = 3600.0 * t_hour4
1722     if float(eval_M(t_hour4_s)) >= target:
1723         raise RuntimeError("four-decimal hour report does not satisfy the target")
1724     t_grid60 = EXCEL_DT * np.ceil(t_star / EXCEL_DT)
1725     if t_grid60 < t_rep:
1726         t_grid60 = EXCEL_DT * np.ceil(t_rep / EXCEL_DT)
1727     return {
1728         "t_cross_s": t_cross,
1729         "t_rep_s": float(t_rep),
1730         "t_rep_h": float(t_rep) / 3600.0,
1731         "M_rep": float(eval_M(t_rep)),
1732         "t_hour4_h": float(t_hour4),
1733         "t_hour4_s": float(t_hour4_s),
1734         "M_hour4": float(eval_M(t_hour4_s)),
1735         "t_grid60_s": float(t_grid60),
1736         "M_grid60": float(eval_M(t_grid60)),
1737         "margin": float(margin),
1738         "strict_below_rep": bool(float(eval_M(t_rep)) < C_THRESH),
1739         "strict_below_hour4": bool(float(eval_M(t_hour4_s)) < C_THRESH),
1740     }
1741
1742
1743 def slope_at(eval_M, t: float, h: float = 1.0) -> float:
1744     return float(eval_M(t + h) - eval_M(t - h)) / (2.0 * h)
1745
1746
1747 def run_case(
1748     level: int,
1749     Ta_fun,
1750     Ca_fun,
1751     t_max: float,
1752     rtol: float,
1753     atol_T: float,
1754     atol_C: float,
1755     max_step_early: float,
1756     max_step_late: float,
1757     tag: str,
1758     audit_balances: bool = False,

```

```

1759 ):
1760     r = graded_radial_nodes(level)
1761     fvm = CoupledRadialFVM(r)
1762     n = fvm.n_nodes
1763     print(f"\n=== {tag} nodes={n} t_max={t_max/3600:.2f} h ===", flush=True)
1764     y0 = fvm.pack(np.full(n, T0), np.full(n, C0))
1765     atol = atol_vector(n, atol_T, atol_C)
1766     t_eval_early = np.arange(EXCEL_DT, T_SWITCH + 0.5 * EXCEL_DT, EXCEL_DT)
1767     t_wall0 = time.perf_counter()
1768     sol1 = integrate_segment(
1769         fvm, y0, 0.0, T_SWITCH, Ta_fun, Ca_fun, rtol, atol,
1770         max_step=max_step_early, t_eval=t_eval_early, first_step=0.05, label=f"{tag} early ",
1771         ↪ balance_audit=audit_balances,
1772     )
1773     y_switch = sol1.y[:, -1]
1774     times1 = np.concatenate([[0.0], sol1.t])
1775     Y1 = np.vstack([y0, sol1.y.T])
1776     M_switch = float(np.max(y_switch[n:]))
1777     print(f" {tag} 4h maxC={M_switch:.6f} C_center={y_switch[n]:.6f}
1778     ↪ wall={time.perf_counter()-t_wall0:.1f}s", flush=True)
1779     if M_switch <= C_THRESH:
1780         raise RuntimeError("threshold already reached at 4 h; unexpected for this model")
1781     event = max_moisture_event(n)
1782     t_eval_late = np.unique(np.r_[np.arange(T_SWITCH + EXCEL_DT, t_max, EXCEL_DT), t_max])
1783     sol2 = integrate_segment(
1784         fvm, y_switch, T_SWITCH, t_max, Ta_fun, Ca_fun, rtol, atol,
1785         max_step=max_step_late, t_eval=t_eval_late, events=event,
1786         first_step=0.2, label=f"{tag} late ", dense_output=audit_balances,
1787     )
1788     if sol2.t_events is None or len(sol2.t_events[0]) == 0:
1789         M_end = float(np.max(sol2.y[n:, -1]))
1790         print(f" {tag} no event by {t_max/3600:.2f} h, maxC={M_end:.6f}; extending to hard cap",
1791             ↪ flush=True)
1792         if t_max >= T_HARD_CAP - 1.0:
1793             raise RuntimeError(f"{tag} still above threshold at {T_HARD_CAP/3600:.1f} h, maxC={M_end}")
1794         y_cont = sol2.y[:, -1]
1795         t_cont = float(sol2.t[-1])
1796         t_eval_cap = np.arange(t_cont + EXCEL_DT, T_HARD_CAP + 0.5 * EXCEL_DT, EXCEL_DT)
1797         sol2b = integrate_segment(
1798             fvm, y_cont, t_cont, T_HARD_CAP, Ta_fun, Ca_fun, rtol, atol,
1799             max_step=max_step_late, t_eval=t_eval_cap, events=event,
1800             first_step=0.2, label=f"{tag} cap ", dense_output=audit_balances,
1801         )
1802         if sol2b.t_events is None or len(sol2b.t_events[0]) == 0:
1803             raise RuntimeError(
1804                 f"{tag} not dry at {T_HARD_CAP/3600:.1f} h, maxC={float(np.max(sol2b.y[n:, -1]))}"
1805             )
1806         times2 = np.concatenate([sol2.t[sol2.t > T_SWITCH + 1e-12], sol2b.t[sol2b.t > t_cont + 1e-12]])
1807         Y2 = np.vstack([sol2.y[:, sol2.t > T_SWITCH + 1e-12].T, sol2b.y[:, sol2b.t > t_cont + 1e-12].T])
1808         t_event_raw = float(sol2b.t_events[0][0])
1809         y_event_raw = np.asarray(sol2b.y_events[0][0], dtype=float)
1810     else:
1811         times2 = sol2.t[sol2.t > T_SWITCH + 1e-12]
1812         Y2 = sol2.y[:, sol2.t > T_SWITCH + 1e-12].T
1813         t_event_raw = float(sol2.t_events[0][0])
1814         y_event_raw = np.asarray(sol2.y_events[0][0], dtype=float)
1815     times = np.concatenate([times1, times2])
1816     Y = np.vstack([Y1, Y2])
1817     order = np.argsort(times)
1818     times, Y = times[order], Y[order]
1819     uniq = np.concatenate([[True], np.diff(times) > 1e-12])
1820     times, Y = times[uniq], Y[uniq]

```

```

1821 M_hist = np.max(Y[:, n:], axis=1)
1822 above = np.where(M_hist > C_THRESH + 1e-14)[0]
1823 if above.size == 0:
1824     raise RuntimeError(f"{tag} never above threshold")
1825 i_left = int(above[-1])
1826 t_left = float(times[i_left])
1827 y_left = Y[i_left]
1828 if t_left >= t_event_raw:
1829     i_left = max(0, i_left - 1)
1830     t_left = float(times[i_left])
1831     y_left = Y[i_left]
1832 t_star, y_star, sol_zoom, event_info, y_solver = locate_threshold(
1833     fvm, Ta_fun, Ca_fun, t_left, y_left, t_event_raw, rtol, atol,
1834 )
1835 event_info["t_event_raw_s"] = t_event_raw
1836 event_info["M_event_raw"] = float(np.max(y_event_raw[n:]))
1837
1838 balance_parts = []
1839 if audit_balances:
1840     from q3_balance import audit_segment
1841     # Follow the same trajectory used for reporting, including the zoom restart.
1842     balance_parts = [sol1.balance_audit]
1843     if 'sol2b' in locals():
1844         balance_parts.append(audit_segment(fvm, sol2, Ta_fun, Ca_fun))
1845         late_sol = sol2b
1846     else:
1847         late_sol = sol2
1848     balance_parts.append(audit_segment(fvm, late_sol, Ta_fun, Ca_fun, t_end=t_left))
1849     balance_parts.append(audit_segment(fvm, sol_zoom, Ta_fun, Ca_fun, t_end=t_star))
1850     sol2.sol = None
1851     if 'sol2b' in locals():
1852         sol2b.sol = None
1853
1854 t_pad = 3600.0
1855 sol_after = integrate_segment(
1856     fvm, y_star, t_star, t_star + t_pad, Ta_fun, Ca_fun, rtol, atol,
1857     max_step=min(max_step_late, 5.0), dense_output=True, first_step=0.05,
1858     label=f"{tag} after ",
1859 )
1860
1861 def eval_M(t):
1862     return float(np.max(eval_y(float(t))[n:]))
1863
1864 def eval_y(t):
1865     tt = float(t)
1866     if abs(tt - t_star) <= 1e-12:
1867         return y_star.copy()
1868     if tt < t_star:
1869         if tt < times[0]:
1870             raise ValueError(f"state requested before initial time: {tt}")
1871         if tt >= t_left:
1872             return np.asarray(sol_zoom.sol(tt), dtype=float)
1873         k = max(0, int(np.searchsorted(times, tt, side="right"))-1)
1874         if abs(times[k]-tt)<1e-8:
1875             return Y[k].copy()
1876         short = integrate_segment(fvm, Y[k], float(times[k]), tt, Ta_fun, Ca_fun,
1877             min(rtol, 1e-10), atol/10, max_step=1.0, first_step=0.05, label="matched time ")
1878         return short.y[:, -1]
1879     if tt > float(sol_after.t[-1])+1e-9:
1880         raise ValueError(f"state requested beyond continuation: {tt}")
1881     return np.asarray(sol_after.sol(tt), dtype=float)
1882
1883 dMdt = slope_at(eval_M, min(t_star + 2.0, t_star + 0.5 * t_pad), h=1.0)
1884 event_info["dMdt_per_s"] = dMdt
1885 event_info["dMdt_per_h"] = dMdt * 3600.0

```

```

1886
1887 T_out, C_out = extract_output(fvm, Y)
1888 mon = monitor_from_Y(fvm, times, Y)
1889 elapsed = time.perf_counter() - t_wall0
1890 print(
1891     f" {tag} t*={t_star/3600:.6f} h rmax={event_info['rmax_star_cm']:.4f} cm "
1892     f"M*={event_info['M_star']:.8f} dM/dt={dMdt:.3e}/s wall={elapsed:.1f}s",
1893     flush=True,
1894 )
1895 return dict(
1896     tag=tag, level=level, fvm=fvm, n_nodes=n, times=times, Y=Y,
1897     T_out=T_out, C_out=C_out, t_star=t_star, y_star=y_star,
1898     eval_M=eval_M, eval_y=eval_y, sol_after=sol_after, event=event_info,
1899     monitor=mon, elapsed_s=elapsed, rtol=rtol, atol_T=atol_T, atol_C=atol_C,
1900     max_step_early=max_step_early, max_step_late=max_step_late, t_max=t_max,
1901     y_switch=y_switch,
1902     balance_parts=balance_parts,
1903 )
1904
1905
1906 def run_probe(Ta_fun, Ca_fun) -> dict:
1907     """Cheap uniform-mesh estimate of t* to size the production interval."""
1908     r = np.linspace(0.0, 0.02, 41)
1909     fvm = CoupledRadialFVM(r)
1910     n = fvm.n_nodes
1911     y0 = fvm.pack(np.full(n, T0), np.full(n, C0))
1912     atol = atol_vector(n, 1e-6, 1e-8)
1913     print("\n=== probe N=41 ===", flush=True)
1914     sol1 = integrate_segment(
1915         fvm, y0, 0.0, T_SWITCH, Ta_fun, Ca_fun, 1e-6, atol, max_step=30.0,
1916         first_step=0.2, label="probe early ",
1917     )
1918     event = max_moisture_event(n)
1919     sol2 = integrate_segment(
1920         fvm, sol1.y[:, -1], T_SWITCH, T_HARD_CAP, Ta_fun, Ca_fun, 1e-6, atol,
1921         max_step=120.0, events=event, first_step=1.0, label="probe late ",
1922     )
1923     if sol2.t_events is None or len(sol2.t_events[0]) == 0:
1924         raise RuntimeError(f"probe not dry by 14 d, maxC={float(np.max(sol2.y[n:, -1]))}")
1925     t_star = float(sol2.t_events[0][0])
1926     y_e = np.asarray(sol2.y_events[0][0], dtype=float)
1927     imax = int(np.argmax(y_e[n:]))
1928     info = {
1929         "n_nodes": n,
1930         "t_star_s": t_star,
1931         "t_star_h": t_star / 3600.0,
1932         "M_star": float(y_e[n + imax]),
1933         "rmax_cm": float(fvm.r[imax] * 100.0),
1934         "C_center": float(y_e[n]),
1935         "C_surface": float(y_e[-1]),
1936         "nfev": int(sol1.nfev + sol2.nfev),
1937     }
1938     print("probe", info, flush=True)
1939     return info
1940
1941
1942 def overlap_q2(case) -> dict:
1943     path = RESULTS_DIR / "q2_solution.npz"
1944     if not path.exists():
1945         return {"available": False}
1946     z = np.load(path)
1947     fvm = case["fvm"]
1948     n = fvm.n_nodes
1949     idx = fvm.output_indices()
1950     t_check = np.array([1800.0, 3600.0, 7200.0, 9000.0])

```

```

1951 q2_times = np.asarray(z["times"], dtype=float)
1952 T_q2 = np.asarray(z["T_out"], dtype=float)
1953 C_q2 = np.asarray(z["C_out"], dtype=float)
1954 rows = {}
1955 for t in t_check:
1956     k = int(np.argmin(np.abs(case["times"] - t)))
1957     if abs(case["times"][k] - t) > 1e-8:
1958         rows[f"t{int(t)}"] = {"matched": False}
1959         continue
1960     jq = int(np.argmin(np.abs(q2_times - t)))
1961     if abs(q2_times[jq] - t) > 1e-8:
1962         rows[f"t{int(t)}"] = {"matched": False}
1963         continue
1964     T = case["Y"][k, :n][idx]
1965     C = case["Y"][k, n:][idx]
1966     rows[f"t{int(t)}"] = {
1967         "matched": True,
1968         "T_max_abs": float(np.max(np.abs(T - T_q2[jq]))),
1969         "C_max_abs": float(np.max(np.abs(C - C_q2[jq]))),
1970         "T_center_diff": float(T[0] - T_q2[jq, 0]),
1971         "C_center_diff": float(C[0] - C_q2[jq, 0]),
1972         "C_surface_diff": float(C[-1] - C_q2[jq, -1]),
1973     }
1974 return {"available": True, "note": "PCHIP stencils differ near 3 h endpoint", "checks": rows}
1975
1976
1977 def sample_state(case, t: float) -> tuple[np.ndarray, np.ndarray]:
1978     fvm = case["fvm"]
1979     n = fvm.n_nodes
1980     y = case["eval_y"](float(t))
1981     T, C = y[:n], y[n:]
1982     return T, C
1983
1984
1985 def output_row(fvm, C: np.ndarray) -> np.ndarray:
1986     return np.asarray(C[fvm.output_indices()], dtype=float)
1987
1988
1989 def build_output_tables(case, t_rep: float) -> dict:
1990     fvm = case["fvm"]
1991     times_x = excel_times(t_rep)
1992     C_excel = np.empty((times_x.size, 21), dtype=float)
1993     T_excel = np.empty_like(C_excel)
1994     n = fvm.n_nodes
1995     idx = fvm.output_indices()
1996     t_star = case["t_star"]
1997     times = case["times"]
1998     Y = case["Y"]
1999     for i, t in enumerate(times_x):
2000         if t <= times[-1] + 1e-12 and t < t_star - 1e-9:
2001             j = int(np.argmin(np.abs(times - t)))
2002             if abs(times[j] - t) > 1.0:
2003                 raise RuntimeError(f"missing 60 s sample at t={t}, nearest {times[j]}")
2004             T_excel[i] = Y[j, :n][idx]
2005             C_excel[i] = Y[j, n:][idx]
2006         else:
2007             y = case["eval_y"](float(t))
2008             T_excel[i] = y[:n][idx]
2009             C_excel[i] = y[n:][idx]
2010     hrs = table_hours(t_rep / 3600.0)
2011     C_table = np.empty((hrs.size, TABLE_RADII_CM.size), dtype=float)
2012     paper_map = {}
2013     r_idx = {round(0.1 * j, 10): j for j in range(21)}
2014     for i, th in enumerate(hrs):
2015         t = 3600.0 * float(th)

```

```

2016     k = int(np.argmin(np.abs(times_x - t)))
2017     if abs(times_x[k] - t) > 1.0 and abs(t - t_rep) > 1.0:
2018         # 6 h marks lie on the 60 s grid; end time may be the extra row.
2019         y = case["eval_y"](t) if t >= t_star - 1e-9 else None
2020         if y is None:
2021             j = int(np.argmin(np.abs(times - t)))
2022             row = Y[j, n:][idx]
2023         else:
2024             row = y[n:][idx]
2025     else:
2026         row = C_excel[k]
2027     for rc, col in zip(TABLE_RADII_CM, range(5)):
2028         C_table[i, col] = float(row[r_idx[float(rc)]])
2029         paper_map[f"h{th:.6g}_r{rc:g}"] = float(row[r_idx[float(rc)]])
2030     return {
2031         "excel_times": times_x,
2032         "C_excel": C_excel,
2033         "T_excel": T_excel,
2034         "table_hours": hrs,
2035         "C_table": C_table,
2036         "paper_C": paper_map,
2037     }
2038
2039
2040 def write_result3_xlsx(path: Path, C_out: np.ndarray, times: np.ndarray, *, validation):
2041     """All numerical checks must pass before calling the workbook author."""
2042     from q3_closeout import export_workbook, require_q3_delivery
2043     require_q3_delivery(validation, RESULTS_DIR / "diagnostics/q3_closeout/pre_export.json")
2044     export_workbook(path, C_out, times, validation)
2045
2046
2047 def write_table5_csv(path: Path, hours: np.ndarray, C_table: np.ndarray):
2048     lines = ["t_h," + ",".join(f"r_{rc:g}cm" for rc in TABLE_RADII_CM)]
2049     for th, row in zip(hours, C_table):
2050         bits = [f"{float(th):.8f}" + [f"{float(v):.10f}" for v in row]
2051         lines.append(",".join(bits))
2052     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
2053
2054
2055 def physical_bounds(case, Ta_fun, Ca_fun) -> dict:
2056     n = case["n_nodes"]
2057     T = case["Y"][:, :n]
2058     C = case["Y"][:, n:]
2059     Ta_max = max(float(np.max(Ta_fun.y)), TA_AFTER)
2060     info = {
2061         "T_min": float(T.min()),
2062         "T_max": float(T.max()),
2063         "C_min": float(C.min()),
2064         "C_max": float(C.max()),
2065         "T_below_28": float(max(0.0, 28.0 - T.min())),
2066         "T_above_Ta_max": float(max(0.0, T.max() - Ta_max)),
2067         "C_above_C0": float(max(0.0, C.max() - C0)),
2068         "C_negative": float(max(0.0, -C.min())),
2069         "Ta_max": Ta_max,
2070         "Ca_range_knots": [float(np.min(Ca_fun.y)), float(np.max(Ca_fun.y))],
2071     }
2072     info["T_bounds_ok"] = info["T_below_28"] <= 1e-6 and info["T_above_Ta_max"] <= 0.05
2073     info["C_bounds_ok"] = info["C_above_C0"] <= 1e-8 and info["C_negative"] <= 1e-12
2074     return info
2075
2076
2077 def plot_figures(plt, pub, tables, t_rep: float, Ta_fun):
2078     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
2079     fvm = pub["fvm"]
2080     n = fvm.n_nodes

```

```

2081     mon = pub["monitor"]
2082     t_h = mon["times"] / 3600.0
2083     t_star_h = pub["t_star"] / 3600.0
2084     t_rep_h = t_rep / 3600.0
2085
2086     fig, ax = plt.subplots(figsize=(6.4, 3.8))
2087     ax.axhline(C_THRESH, color="0.4", lw=1.0, ls="--", label=r" 阈值 $0.15$")
2088     ax.plot(t_h, mon["M"], lw=1.2, label=r" 全网格 $\max C$")
2089     ax.plot(t_h, mon["C_center"], lw=1.2, ls="-. ", label=" 中心")
2090     ax.plot(t_h, mon["C_surface"], lw=1.2, ls=":", label=" 表面")
2091     ax.axvline(t_star_h, color="0.2", lw=0.8, ls=":")
2092     ax.set_xlabel(" 时间 $t$ (h)")
2093     ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
2094     ax.set_xlim(0, t_rep_h)
2095     ax.legend(frameon=False)
2096     fig.tight_layout()
2097     fig.savefig(FIGURES_DIR / "q3_C_max_history.pdf")
2098     plt.close(fig)
2099
2100     fig, ax = plt.subplots(figsize=(6.4, 3.6))
2101     ax.plot(t_h, mon["rmax_m"] * 100.0, lw=1.2)
2102     ax.set_xlabel(" 时间 $t$ (h)")
2103     ax.set_ylabel(r" 最大含水率位置 $r$ (cm)")
2104     ax.set_xlim(0, t_rep_h)
2105     ax.set_ylim(-0.02, 2.05)
2106     fig.tight_layout()
2107     fig.savefig(FIGURES_DIR / "q3_max_location.pdf")
2108     plt.close(fig)
2109
2110     fig, ax = plt.subplots(figsize=(6.4, 3.8))
2111     r_cm = fvm.r * 100.0
2112     hrs = tables["table_hours"]
2113     t_end_h = float(hrs[-1])
2114     if t_end_h > 72.0:
2115         show = np.arange(24.0, t_end_h + 1e-9, 24.0)
2116         if abs(show[-1] - t_end_h) > 1e-6:
2117             show = np.concatenate([show, [t_end_h]])
2118     elif hrs.size > 10:
2119         show = np.unique(np.concatenate([hrs[:,2], hrs[-1:]]))
2120     else:
2121         show = hrs
2122     for th in show:
2123         t = 3600.0 * float(th)
2124         if t < pub["t_star"] - 1e-9:
2125             j = int(np.argmin(np.abs(pub["times"] - t)))
2126             C = pub["Y"][j, n:]
2127         else:
2128             C = pub["eval_y"](t)[n:]
2129         ax.plot(r_cm, C, lw=1.2, label=fr"$t={th:g}\, \mathrm{{{h}}}$")
2130     ax.set_xlabel(" 半径 $r$ (cm)")
2131     ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
2132     ax.set_xlim(0, 2)
2133     ax.legend(frameon=False, ncol=2)
2134     fig.tight_layout()
2135     fig.savefig(FIGURES_DIR / "q3_C_profiles.pdf")
2136     plt.close(fig)
2137
2138     fig, ax = plt.subplots(figsize=(6.4, 3.6))
2139     mask = mon["times"] >= pub["t_star"] - 6 * 3600.0
2140     if np.count_nonzero(mask) < 5:
2141         mask = np.ones_like(mon["times"], dtype=bool)
2142     ax.axhline(C_THRESH, color="0.4", lw=1.0, ls="--", label=r" 阈值 $0.15$")
2143     ax.plot(t_h[mask], mon["M"][mask], lw=1.2, label=r"$\max C$")
2144     ax.axvline(t_star_h, color="0.2", lw=0.8, ls=":", label=fr"$t_*={t_star_h:.3f}\, \mathrm{{{h}}}$")

```

```

2145     ax.axvline(t_rep_h, color="0.2", lw=0.8, ls="-.",
↪     label=fr"$t_{\mathrm{{rep}}}\}=\{t_{rep_h:.3f}\},\mathrm{{h}}\}$")
2146     ax.set_xlabel(" 时间 $t$ (h)")
2147     ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
2148     ax.legend(frameon=False)
2149     fig.tight_layout()
2150     fig.savefig(FIGURES_DIR / "q3_event_zoom.pdf")
2151     plt.close(fig)
2152
2153     fig, ax = plt.subplots(figsize=(6.4, 3.6))
2154     ax.plot(t_h, [Ta_fun(t) for t in mon["times"]], lw=1.0, ls="--", label=" 烘房 $T_a$")
2155     ax.plot(t_h, mon["T_surface"], lw=1.2, label=" 表面")
2156     ax.plot(t_h, mon["T_center"], lw=1.2, label=" 中心")
2157     ax.set_xlabel(" 时间 $t$ (h)")
2158     ax.set_ylabel(" 温度 (°C)")
2159     ax.set_xlim(0, t_rep_h)
2160     ax.legend(frameon=False)
2161     fig.tight_layout()
2162     fig.savefig(FIGURES_DIR / "q3_T_center_surface.pdf")
2163     plt.close(fig)
2164
2165
2166 def json_safe(obj):
2167     if isinstance(obj, dict):
2168         return {str(k): json_safe(v) for k, v in obj.items()}
2169     if isinstance(obj, (list, tuple)):
2170         return [json_safe(v) for v in obj]
2171     if isinstance(obj, np.ndarray):
2172         return obj.tolist()
2173     if isinstance(obj, (np.floating, np.integer)):
2174         return obj.item()
2175     if isinstance(obj, (float, int, str, bool)) or obj is None:
2176         return obj
2177     return str(obj)
2178
2179
2180 def main(argv=None):
2181     from q3_closeout import main as closeout_main
2182     return closeout_main(argv)
2183
2184
2185 if __name__ == '__main__':
2186     main()
2187 '''
2188
2189 # =====
2190 # Module: q3_closeout.py
2191 # =====
2192 SOURCES['q3_closeout'] = r'""""Q3 closeout: matched-time convergence, continuous balances and gated
↪ delivery.""""
2193 from __future__ import annotations
2194 import argparse
2195 import hashlib
2196 import json
2197 import math
2198 import subprocess
2199 import sys
2200 import time
2201 from pathlib import Path
2202 import numpy as np
2203 from delivery import make_gate, require_delivery
2204 from q3_balance import audit_segment, combine
2205 from utils import RESULTS_DIR, REPORTS_DIR, FIGURES_DIR, ROOT
2206
2207 DIAG=RESULTS_DIR/'diagnostics/q3_closeout'

```

```

2208 REQUIRED_CHECKS = frozenset({
2209     'event_found', 'finite', 'strict_below_all_verified_grids',
2210     'strict_below_with_empirical_margin', 'moisture_bounds', 'temperature_bounds',
2211     'moisture_balance_1e-6', 'heat_balance_1e-6', 'quadrature_convergence_1e-7',
2212     'space_event_60s', 'time_event_60s', 'space_full_fields_2e-5',
2213     'time_full_fields_2e-5', 'table_four_decimals', 'q2_overlap',
2214 })
2215
2216
2217 def require_q3_delivery(validation, diagnostic_path):
2218     require_delivery(validation, diagnostic_path)
2219     if validation.get('result_version') != 'q3-closeout-v1' or not
2220         ↪ REQUIRED_CHECKS.issubset(validation['delivery_gate']['checks']):
2221         raise RuntimeError('Delivery blocked: missing Q3 checks or wrong result version')
2222
2223 def export_workbook(path, C_out, times, validation):
2224     require_q3_delivery(validation, DIAG / 'pre_export.json')
2225     from reproduction_export import write_template
2226     write_template(ROOT, 3, times, [C_out])
2227
2228
2229 def save_case(case):
2230     f=case['fvm'];n=f.n_nodes
2231     np.savez_compressed(DIAG/(case['tag']+'.npz'),r=f.r,times=case['times'],Y=case['Y'],
2232         y_star=case['y_star'],t_star=case['t_star'])
2233     info={k:case[k] for k in ('tag','level','n_nodes','rtol','atol_T','atol_C','max_step_early','max_step_')
2234         ↪ late','elapsed_s','event')}
2235     (DIAG/(case['tag']+'.json')).write_text(json.dumps(info,indent=2),encoding='utf-8')
2236
2237
2238 def compare_fields(a,b):
2239     """Shared physical nodes and exactly matching stored times."""
2240     times,ia,ib=np.intersect1d(a['times'],b['times'],return_indices=True)
2241     ra,rb=a['fvm'].r,b['fvm'].r
2242     j=np.searchsorted(rb,ra)
2243     if np.max(abs(rb[j]-ra))>1e-13:
2244         raise RuntimeError('spatial meshes are not nested')
2245     na,nb=a['n_nodes'],b['n_nodes']
2246     # Batch to bound temporary array sizes for fine meshes.
2247     maxT=maxC=0.
2248     for start in range(0,len(times),128):
2249         ai,bi=ia[start:start+128],ib[start:start+128]
2250         maxT=max(maxT,float(np.max(abs(a['Y'][ai,:na]-b['Y'][bi,:nb][:,j]))))
2251         maxC=max(maxC,float(np.max(abs(a['Y'][ai,na:]-b['Y'][bi,nb:][:,j]))))
2252     return {'matched_times':len(times),'max_T':maxT,'max_C':maxC}
2253
2254
2255 def main(argv=None):
2256     from problem3 import (load_q3_inputs,run_case,run_probe,RTOL,ATOL_T,ATOL_C,MAX_STEP_EARLY,
2257         MAX_STEP_LATE,find_strict_report,build_output_tables,physical_bounds,overlap_q2,
2258         write_result3_xlsx,write_table5_csv,plot_figures,setup_mpl,conservation_heat,conservation_moisture)
2259     parser=argparse.ArgumentParser();parser.add_argument('--probe-only',action='store_true')
2260     parser.add_argument('--no-xlsx',action='store_true',help='Save gated numerical evidence; export
2261         ↪ separately later.')
2262     args=parser.parse_args(argv)
2263     DIAG.mkdir(parents=True,exist_ok=True)
2264     start=time.perf_counter()
2265     Ta,Ca,knots=load_q3_inputs()
2266     if args.probe_only:
2267         print(json.dumps(run_probe(Ta,Ca)));return
2268     cases={}
2269     for level in (4,8,16):
2270         cases[level]=run_case(level,Ta,Ca,72*3600.,RTOL,ATOL_T,ATOL_C,MAX_STEP_EARLY,MAX_STEP_LATE,
2271             tag=f'G{level}',audit_balances=(level==8))

```

```

2270     save_case(cases[level])
2271     pub,ref=cases[8],cases[16]
2272     tight=run_case(8,Ta,Ca,72*3600.,RTOL/5,ATOL_T/5,ATOL_C/5,MAX_STEP_EARLY/2,MAX_STEP_LATE/2,tag='G8t')
2273     save_case(tight)
2274     space=compare_fields(pub,ref);temporal=compare_fields(pub,tight)
2275     coarse_space=compare_fields(cases[4],pub)
2276     critical=pub['t_star']
2277     same={name:float(case['eval_M'](critical)) for name,case in
↪    [(['G4',cases[4]),('G8',pub),('G16',ref),('G8t',tight))]}
2278     # The maximum differences are measured at the same physical time.
2279     err=max(abs(same['G8']-same['G16']),abs(same['G8']-same['G8t']))
2280     margin=2*err+1e-8
2281     zero_report=find_strict_report(pub['eval_M'],critical,critical+3500,margin=0.)
2282     report=find_strict_report(pub['eval_M'],critical,critical+3500,margin=margin)
2283     t_rep=report['t_hour4_s'] # Upward four-decimal-hour rounding retains the safety margin.
2284     report_M={name:float(case['eval_M'](t_rep)) for name,case in [(['G8',pub),('G16',ref),('G8t',tight))]}
2285     tables=build_output_tables(pub,t_rep);other=build_output_tables(ref,t_rep)
2286     tight_tables=build_output_tables(tight,t_rep)
2287     table_changed=int(np.count_nonzero(np.round(tables['C_table'],4)!=np.round(other['C_table'],4)))
2288     table_time_changed=int(np.count_nonzero(np.round(tables['C_table'],4)!=np.round(tight_tables['C_table']
↪    [4])))
2289     output_space=float(np.max(abs(tables['C_excel']-other['C_excel'])))
2290     tail=audit_segment(pub['fvm'],pub['sol_after'],Ta,Ca,t_end=t_rep)
2291     balances=combine(pub['balance_parts']+tail)
2292     checked_states = np.vstack([pub['Y'], pub['y_star'], pub['eval_y'](t_rep)])
2293     bounds=physical_bounds(dict(pub,Y=checked_states),Ta,Ca);overlap=overlap_q2(pub)
2294     overlap_ok=overlap.get('available',False) and all(c.get('matched',False) and c['T_max_abs']<2e-5 and
↪    c['C_max_abs']<2e-5 for c in overlap['checks'].values())
2295     event_space=abs(pub['t_star']-ref['t_star']);event_time=abs(pub['t_star']-tight['t_star'])
2296     gates={
2297         'event_found':abs(pub['event']['g_star'])<1e-8,
2298         'finite':bool(np.isfinite(checked_states).all() and np.isfinite(tables['C_excel']).all()
↪    and np.isfinite(tables['T_excel']).all()),
2299         'strict_below_all_verified_grids':all(v<.15 for v in report_M.values()),
2300         'strict_below_with_empirical_margin':report_M['G8']+margin<.15,
2301         'moisture_bounds':bounds['C_bounds_ok'],'temperature_bounds':bounds['T_bounds_ok'],
2302         'moisture_balance_1e-6':balances['moisture']['relative_residual']<1e-6,
2303         'heat_balance_1e-6':balances['heat']['relative_residual']<1e-6,
2304         'quadrature_convergence_1e-7':max(balances['quadrature_relative_difference'].values())<1e-7,
2305         'space_event_60s':event_space<=60., 'time_event_60s':event_time<=60.,
2306         'space_full_fields_2e-5':max(space['max_T'],space['max_C'],output_space)<2e-5,
2307         'time_full_fields_2e-5':max(temporal['max_T'],temporal['max_C'])<2e-5,
2308         'table_four_decimals':table_changed==0 and table_time_changed==0,
2309         'q2_overlap':overlap_ok,
2310     }
2311 }
2312 v={ 'result_version':'q3-closeout-v1', 'model':'q3-model-v1', 'result_status':'PUBLISHED',
2313     'published_mesh':'G8', 'published_n_nodes':pub['n_nodes'], 'rtol':RTOL, 'atol_T':ATOL_T, 'atol_C':ATOL_
↪    C,
2314     'max_step_early':MAX_STEP_EARLY, 'max_step_late':MAX_STEP_LATE,
2315     't_star_s':critical, 't_star_h':critical/3600., 't_rep_s':t_rep, 't_rep_h':t_rep/3600.,
2316     'M_star':pub['event']['M_star'], 'M_rep':report_M['G8'], 'report_maxima':report_M,
2317     'matched_critical_time_maxima':same, 'moisture_margin':margin, 'margin_type':'empirical, not a
↪    rigorous PDE error bound',
2318     'report_zero_margin':zero_report, 'report_with_margin':report,
2319     'space_fields':space, 'time_fields':temporal, 'coarse_space_fields':coarse_space, 'output_space_max_C'
↪    :output_space,
2320     'dt_space_s':event_space, 'dt_time_s':event_time,
2321     'conservation_C':balances['moisture'], 'conservation_T':balances['heat'], 'continuous_balances':balan
↪    ces,
2322     'old_sampled_conservation_C':conservation_moisture(pub['fvm'],pub['times'],pub['Y'],Ca),
2323     'old_sampled_conservation_T':conservation_heat(pub['fvm'],pub['times'],pub['Y'],Ta),
2324     'physical_bounds':bounds, 'q2_overlap':overlap, 'table_four_decimal_changes':table_changed,
2325     'events':{f'G{k}':c['event'] for k,c in cases.items()}[f'G8t':tight['event']],
2326     'delivery_gate':make_gate(gates), 'table_hours':tables['table_hours'].tolist(),

```

```

2327     'excel_nrows':len(tables['excel_times']), 'paper_table_C':tables['paper_C'],
2328     'max_location_cm':pub['event']['rmax_star_cm'], 'center_is_max_at_event':pub['event']['center_is_max']
↪     ' '),
2329     'n_radial_increases':pub['monitor']['n_radial_increases'],
2330     'elapsed_s':time.perf_counter()-start, 'python':sys.version,
2331     'source_sha256':{p:hashlib.sha256(((Path(__file__).parent / Path(p).name) if p.startswith('code/')
↪     else ROOT / p).read_bytes()).hexdigest() for p in
2332         ['code/problem3.py', 'code/q3_balance.py', 'code/q3_closeout.py', 'code/delivery.py',
2333         'code/reproduction_export.py',
2334         'code/radial_coupled.py', 'code/appendix3.py', 'data/附件/附件 1.xlsx']})
2335 print('CLOSEOUT', json.dumps({k:v[k] for k in ('t_star_h', 't_rep_h', 'M_rep', 'dt_space_s', 'dt_time_s', 'c'
↪     onservations_C', 'conservation_T', 'delivery_gate')), ensure_ascii=False), flush=True)
2336 publish_results(pub, tables, v, Ta, no_xlsx=args.no_xlsx)
2337 print('Q3 CLOSEOUT NUMERICAL PASS', flush=True)
2338
2339
2340 def publish_results(pub, tables, v, Ta, *, no_xlsx=False):
2341     from problem3 import write_result3_xlsx, write_table5_csv, plot_figures, setup_mpl
2342     require_q3_delivery(v, DIAG/'latest.json')
2343     t_rep, critical = v['t_rep_s'], v['t_star_s']
2344     # Commit production outputs only after every required numerical check passed.
2345     yend=pub['eval_y'](t_rep)
2346     np.savez_compressed(RESULTS_DIR/'q3_solution.npz', mesh_level=8, n_nodes=pub['n_nodes'], r=pub['fvm'].r,
2347         times=pub['times'], Y=pub['Y'], excel_times=tables['excel_times'], T_out=tables['T_excel'], C_out=tabl
↪     es['C_excel'],
2348         table_hours=tables['table_hours'], C_table=tables['C_table'], t_star=critical, t_rep=t_rep,
2349         y_star=pub['y_star'], y_end=yend, M_hist=pub['monitor']['M'], rmax_m=pub['monitor']['rmax_m'],
2350         C_center=pub['monitor']['C_center'], C_surface=pub['monitor']['C_surface'],
2351         T_center=pub['monitor']['T_center'], T_surface=pub['monitor']['T_surface'],
2352         rtol=v['rtol'], atol_T=v['atol_T'], atol_C=v['atol_C'], max_step_early=v['max_step_early'], max_step_l
↪     ate=v['max_step_late'],
2353         interpolation=np.array(['pchip_then_constant']))
2354     (RESULTS_DIR/'q3_validation.json').write_text(json.dumps(v, ensure_ascii=False, indent=2), encoding='utf-
↪     8')
2355     write_table5_csv(RESULTS_DIR/'q3_table_moisture.csv', tables['table_hours'], tables['C_table'])
2356     if not no_xlsx:
2357         write_result3_xlsx(RESULTS_DIR/'result3.xlsx', tables['C_excel'], tables['excel_times'], validation=v)
2358     plot_figures(setup_mpl(), pub, tables, t_rep, Ta)
2359
2360
2361 if __name__=='__main__':
2362     main()
2363 '''
2364
2365 def prepare():
2366     code_dir = PACKAGE_ROOT / '_runtime' / f'Q{QUESTION}' / 'code'
2367     code_dir.mkdir(parents=True, exist_ok=True)
2368     for name, source in SOURCES.items():
2369         (code_dir / (name + '.py')).write_text(source, encoding='utf-8')
2370     sys.path.insert(0, str(code_dir))
2371     for folder in ('results', 'figures', 'reports'):
2372         (PACKAGE_ROOT / folder).mkdir(exist_ok=True)
2373     os.environ.setdefault('MPLBACKEND', 'Agg')
2374     for name in SOURCES:
2375         importlib.import_module(name)
2376
2377
2378 def main():
2379     parser = argparse.ArgumentParser(description=__doc__)
2380     parser.add_argument('--check', action='store_true', help='Check imports and original inputs without
↪     solving')
2381     parser.add_argument('--export-only', action='store_true', help='Export existing validated NPZ arrays;
↪     does not solve the model')
2382     args = parser.parse_args()

```

```

2383     prepare()
2384     from utils import load_oven_table
2385     load_oven_table()
2386     if QUESTION == 4:
2387         from problem4 import Inputs
2388         Inputs()
2389     if args.check:
2390         print(f'Q{QUESTION}: imports and input checks passed', flush=True)
2391         return
2392     from reproduction_export import export_saved
2393     if args.export_only:
2394         export_saved(PACKAGE_ROOT, QUESTION)
2395         return
2396     if QUESTION == 1:
2397         import problem1
2398         problem1.main()
2399     elif QUESTION == 2:
2400         import problem2
2401         problem2.main()
2402     elif QUESTION == 3:
2403         # The original delivery gate compares Q3 with independently computed Q2.
2404         if not (PACKAGE_ROOT / 'results' / 'q2_solution.npz').exists():
2405             print('Q3: computing Q2 first for the original overlap check', flush=True)
2406             import problem2
2407             problem2.main()
2408             export_saved(PACKAGE_ROOT, 2)
2409             import q3_closeout
2410             q3_closeout.main(['--no-xlsx'])
2411     else:
2412         import problem4
2413         import verify_q4_results
2414         for level, tight in ((2, False), (4, False), (4, True), (8, False), (16, False), (8, True)):
2415             problem4.run(level, tight)
2416             verify_q4_results.main()
2417     export_saved(PACKAGE_ROOT, QUESTION)
2418     print(f'Q{QUESTION}: full recomputation and Excel export completed', flush=True)
2419
2420
2421 if __name__ == '__main__':
2422     main()

```

D.4 Q4.py

```

1 # -*- coding: utf-8 -*-
2 """ 问题 4: 从原始附件复算答案数据。
3
4 运行 python Q4.py; 检查环境 python Q4.py --check。
5 本文件内嵌本问所需的全部项目模块, 模块源码在下方按文件名分段列出。
6 运行时自动展开到相对目录 _runtime/Q4/code, 以支持源码哈希核校。
7 输入、输出均相对于本文件的位置, 与当前工作目录无关。
8 方程、物性、网格、容差和验证判据沿用原正式求解程序。
9 """
10 import argparse
11 import importlib
12 import os
13 from pathlib import Path
14 import sys
15
16 PACKAGE_ROOT = Path(__file__).resolve().parent
17 QUESTION = 4
18 SOURCES = {}
19
20 # =====
21 # Module: utils.py
22 # =====

```

```

23 SOURCES['utils'] = r''''''Shared paths and attachment readers for Problem A.'''
24 from __future__ import annotations
25
26 from pathlib import Path
27
28 import numpy as np
29 from openpyxl import load_workbook
30 from scipy.interpolate import PchipInterpolator
31
32 ROOT = Path(__file__).resolve().parents[3]
33 DATA_DIR = ROOT / "data"
34 ATTACH_DIR = DATA_DIR / " 附件"
35 RESULTS_DIR = ROOT / "results"
36 FIGURES_DIR = ROOT / "figures"
37 REPORTS_DIR = ROOT / "reports"
38
39 # Appendix 2 (Problem 1)
40 R = 0.02
41 L = 0.25
42 RHO = 820.0
43 CP = 2600.0
44 K = 0.36
45 H = 25.0
46 HM = 8.0e-7
47 T0 = 28.0
48 C0 = 2.55
49 D_PREFACTOR = 7.0e-9
50 D_EXP_COEF = 0.89
51 T_END = 1800.0
52 ALPHA = K / (RHO * CP)
53
54
55 def moisture_diffusivity(C: np.ndarray | float, *, protect: bool = True) -> np.ndarray | float:
56     """D(C) = 7e-9 exp(-0.89/C). Nonpositive C uses D=0 only as a solver-domain guard."""
57     C_arr = np.asarray(C, dtype=float)
58     out = np.zeros_like(C_arr, dtype=float)
59     positive = C_arr > 0.0
60     if np.any(positive):
61         out[positive] = D_PREFACTOR * np.exp(-D_EXP_COEF / C_arr[positive])
62     if not protect and np.any(~positive):
63         raise ValueError("moisture diffusivity evaluated at nonpositive C without protection")
64     if np.isscalar(C):
65         return float(out)
66     return out
67
68
69 def load_oven_table() -> tuple[np.ndarray, np.ndarray, np.ndarray]:
70     path = ATTACH_DIR / " 附件 1.xlsx"
71     wb = load_workbook(path, read_only=True, data_only=True)
72     rows = list(wb.active.values)
73     wb.close()
74     data = np.asarray(rows[1:], dtype=float)
75     t, Ta, Ca = data[:, 0], data[:, 1], data[:, 2]
76     if t[0] != 0.0:
77         raise ValueError(f"oven table must start at t=0, got {t[0]}")
78     return t, Ta, Ca
79
80
81 class PiecewiseLinear:
82     """Piecewise linear interpolant; kept for comparison, not used in Q1 production."""
83
84     def __init__(self, t: np.ndarray, y: np.ndarray):
85         self.t = np.asarray(t, dtype=float)
86         self.y = np.asarray(y, dtype=float)
87         self.kind = "linear"

```

```

88         if self.t.ndim != 1 or self.y.shape != self.t.shape:
89             raise ValueError("interpolant requires matching 1-D arrays")
90         if np.any(np.diff(self.t) <= 0):
91             raise ValueError("breakpoints must be strictly increasing")
92
93     def __call__(self, t: float | np.ndarray) -> float | np.ndarray:
94         return np.interp(t, self.t, self.y)
95
96     def slope(self, t: float) -> float:
97         t = float(t)
98         if t <= self.t[0] or t >= self.t[-1]:
99             if t >= self.t[-1]:
100                 dt = self.t[-1] - self.t[-2]
101                 return (self.y[-1] - self.y[-2]) / dt
102             return (self.y[1] - self.y[0]) / (self.t[1] - self.t[0])
103         i = int(np.searchsorted(self.t, t, side="right") - 1)
104         i = min(i, len(self.t) - 2)
105         return (self.y[i + 1] - self.y[i]) / (self.t[i + 1] - self.t[i])
106
107
108 class PchipOven:
109     """Monotone piecewise cubic Hermite interpolant (PCHIP) through attachment knots."""
110
111     def __init__(self, t: np.ndarray, y: np.ndarray):
112         self.t = np.asarray(t, dtype=float)
113         self.y = np.asarray(y, dtype=float)
114         self.kind = "pchip"
115         if self.t.ndim != 1 or self.y.shape != self.t.shape:
116             raise ValueError("interpolant requires matching 1-D arrays")
117         if np.any(np.diff(self.t) <= 0):
118             raise ValueError("breakpoints must be strictly increasing")
119         self._pchip = PchipInterpolator(self.t, self.y, extrapolate=False)
120         self._deriv = self._pchip.derivative()
121
122     def __call__(self, t: float | np.ndarray) -> float | np.ndarray:
123         tt = np.clip(np.asarray(t, dtype=float), self.t[0], self.t[-1])
124         val = self._pchip(tt)
125         if np.ndim(t) == 0:
126             return float(val)
127         return np.asarray(val, dtype=float)
128
129     def slope(self, t: float) -> float:
130         tt = float(np.clip(t, self.t[0], self.t[-1]))
131         return float(self._deriv(tt))
132
133
134 # =====
135 # Module: delivery.py
136 # =====
137 SOURCES['delivery'] = r'""""Fail-closed numerical delivery checks shared by Q1 and Q2."""'
138 import json
139 from pathlib import Path
140
141
142 def make_gate(checks):
143     checks = {name: bool(value) for name, value in checks.items()}
144     return {"checks": checks, "failed_checks": [k for k, v in checks.items() if not v],
145           "meets_delivery_gate": bool(checks) and all(checks.values())}
146
147
148 def require_delivery(validation, diagnostic_path):
149     """Persist diagnostics first; never reach any production writer on failure."""
150     path = Path(diagnostic_path)
151     path.parent.mkdir(parents=True, exist_ok=True)
152     path.write_text(json.dumps(validation, ensure_ascii=False, indent=2, default=float), encoding="utf-8")

```

```

153     gate = validation["delivery_gate"]
154     checks = gate.get("checks", {})
155     if not checks or not all(value is True for value in checks.values()) or gate.get("meets_delivery_gate")
        ↪ is not True:
156         raise RuntimeError("Delivery blocked; see " + str(path) + ": " + str(gate.get("failed_checks",
        ↪ [])))
157 '''
158
159 # =====
160 # Module: q1_analytic.py
161 # =====
162 SOURCES['q1_analytic'] = r'""""Independent Bessel-Duhamel reference for the linear radial heat (or
        ↪ frozen-D moisture) problem.""""
163 from __future__ import annotations
164
165 from typing import Any
166
167 import numpy as np
168 from scipy.integrate import solve_ivp
169 from scipy.optimize import brentq
170 from scipy.special import j0, j1
171 from scipy.sparse import diags
172
173
174 def robin_eigenvalues(Bi: float, n_roots: int) -> np.ndarray:
175     """Positive roots of  $\lambda * J_1(\lambda) = Bi * J_0(\lambda)$ ."""
176     if Bi <= 0:
177         raise ValueError("Biot number must be positive")
178     n_roots = int(n_roots)
179
180     def f(lam: float) -> float:
181         return lam * j1(lam) - Bi * j0(lam)
182
183     lam_max = (n_roots + 4) * np.pi
184     grid = np.linspace(1e-10, lam_max, max(8000, 200 * n_roots))
185     vals = grid * j1(grid) - Bi * j0(grid)
186     roots: list[float] = []
187     for i in range(len(grid) - 1):
188         v0, v1 = vals[i], vals[i + 1]
189         if not np.isfinite(v0) or not np.isfinite(v1):
190             continue
191         a, b = float(grid[i]), float(grid[i + 1])
192         if v0 == 0.0:
193             cand = a
194         elif v0 * v1 < 0.0:
195             cand = float(brentq(f, a, b, xtol=1e-14, maxiter=200))
196         else:
197             continue
198         if not roots or abs(cand - roots[-1]) > 1e-10:
199             roots.append(cand)
200         if len(roots) >= n_roots:
201             break
202     if len(roots) < n_roots:
203         raise RuntimeError(f"found only {len(roots)} Robin Bessel roots, need {n_roots}")
204     return np.asarray(roots, dtype=float)
205
206
207 def expansion_coefficients(lam: np.ndarray) -> np.ndarray:
208     j0l = j0(lam)
209     j1l = j1(lam)
210     return 2.0 * j1l / (lam * (j0l**2 + j1l**2))
211
212
213 class BesselDuhamel:
214     def __init__(self, Bi: float, diffusivity: float, R: float, n_terms: int = 120):

```

```

215     self.Bi = float(Bi)
216     self.alpha = float(diffusivity)
217     self.R = float(R)
218     self.lam = robin_eigenvalues(self.Bi, n_terms)
219     self.A = expansion_coefficients(self.lam)
220     self.beta = self.alpha * self.lam**2 / self.R**2
221
222     def radial_modes(self, r: np.ndarray) -> np.ndarray:
223         """Return array shape (n_terms, n_r) of A_n J0(lambda_n r/R)."""
224         r = np.asarray(r, dtype=float)
225         return self.A[:, None] * j0(self.lam[:, None] * (r[None, :] / self.R))
226
227     def evaluate(self, r: np.ndarray, t: float, env: Any, u0: float) -> np.ndarray:
228         return self.evaluate_many(r, np.array([float(t)]), env, u0)[0]
229
230     def evaluate_many(self, r: np.ndarray, times: np.ndarray, env: Any, u0: float) -> np.ndarray:
231         """T = env(t) - sum A_n J0 I_n, with I' = -beta I + env'(t), I(0)=env(0)-u0.
232
233         Valid for any C1 interpolant that exposes env.y, env.slope, and env(t).
234         """
235         r = np.asarray(r, dtype=float)
236         times = np.asarray(times, dtype=float)
237         if times.size == 0:
238             return np.empty((0, r.size), dtype=float)
239         modes = self.radial_modes(r)
240         beta = self.beta
241         I0 = np.full(beta.size, float(env.y[0] - u0))
242         t_end = float(np.max(times))
243         out = np.empty((times.size, r.size), dtype=float)
244         zero = times == 0.0
245         if np.all(zero):
246             out[:] = u0
247             return out
248         t_eval = times[~zero]
249         sol = solve_ivp(
250             lambda t, I: -beta * I + env.slope(t),
251             (0.0, max(t_end, 1e-30)),
252             I0,
253             method="BDF",
254             jac=diags(-beta, format="csc"),
255             rtol=1e-11,
256             atol=1e-13,
257             t_eval=t_eval,
258             max_step=1.0,
259             vectorized=False,
260         )
261         if not sol.success:
262             raise RuntimeError(f"Bessel Duhamel ODE failed: {sol.message}")
263         Ihist = sol.y.T
264         env_t = np.array([float(env(t)) for t in t_eval], dtype=float)
265         nonzero_vals = env_t[:, None] - Ihist @ modes
266         out[zero] = u0
267         out[~zero] = nonzero_vals
268         return out
269
270
271     def converged_reference(Bi, diffusivity, R, r, times, env, u0, start_terms=320, tolerance=1e-8):
272         """Double the modal truncation and explicitly check every requested point."""
273         previous = BesselDuhamel(Bi, diffusivity, R, start_terms).evaluate_many(r, times, env, u0)
274         history = []
275         for terms in (start_terms * 2, start_terms * 4, start_terms * 8):
276             current = BesselDuhamel(Bi, diffusivity, R, terms).evaluate_many(r, times, env, u0)
277             error = float(np.max(np.abs(current - previous)))
278             history.append({"terms": terms, "previous_terms": terms // 2, "max_abs_difference": error})
279             if np.isfinite(error) and error <= tolerance:

```

```

280         return current, {"passed": True, "tolerance": tolerance, "history": history, "terms": terms}
281     previous = current
282     return current, {"passed": False, "tolerance": tolerance, "history": history, "terms": terms}
283 '''
284
285 # =====
286 # Module: radial_fvm.py
287 # =====
288 SOURCES['radial_fvm'] = r''''''Node-centered radial finite-volume operators for Problem 1.'''
289 from __future__ import annotations
290
291 import numpy as np
292 from scipy.sparse import diags
293
294 from utils import CP, H, HM, K, R, RH0, moisture_diffusivity
295
296 OUTPUT_RADII_M = 0.001 * np.arange(21)
297
298
299 def graded_radial_nodes(level: int) -> np.ndarray:
300     """Nodes denser near the surface; every 0.1 cm output radius remains a node.
301
302     Interval subdivisions (before multiplying by ``level``):
303     0–1.5 cm: 8; 1.5–1.8 cm: 16; 1.8–1.9 cm: 32; 1.9–2.0 cm: 64.
304     Surface spacing at level L is 1e-3 / (64 L) metres.
305     """
306     level = int(level)
307     if level < 1:
308         raise ValueError("graded mesh level must be a positive integer")
309     n_sub = np.full(20, 8 * level, dtype=int)
310     n_sub[15:] = 16 * level
311     n_sub[18] = 32 * level
312     n_sub[19] = 64 * level
313     pts = [0.0]
314     for j in range(20):
315         r0 = 0.001 * j
316         r1 = 0.001 * (j + 1)
317         m = int(n_sub[j])
318         for k in range(1, m + 1):
319             pts.append(r0 + (r1 - r0) * k / m)
320     r = np.asarray(pts, dtype=float)
321     r[0] = 0.0
322     r[-1] = R
323     return r
324
325
326 class RadialFVM:
327     def __init__(self, N: int | None = None, r: np.ndarray | None = None):
328         if r is not None:
329             self.r = np.asarray(r, dtype=float)
330             if self.r.ndim != 1 or self.r.size < 3:
331                 raise ValueError("r must be a 1-D array with at least 3 nodes")
332             if abs(self.r[0]) > 1e-15 or abs(self.r[-1] - R) > 1e-14:
333                 raise ValueError("radial nodes must start at 0 and end at R")
334             if np.any(np.diff(self.r) <= 0.0):
335                 raise ValueError("radial nodes must be strictly increasing")
336         else:
337             if N is None or int(N) < 2:
338                 raise ValueError("N must be at least 2")
339             self.r = np.linspace(0.0, R, int(N) + 1)
340             self.n_nodes = int(self.r.size)
341             self.N = self.n_nodes - 1
342             self.dr_face = np.diff(self.r)
343             self.r_half = 0.5 * (self.r[:-1] + self.r[1:])
344             a = np.empty(self.n_nodes)

```

```

345     b = np.empty(self.n_nodes)
346     a[0] = 0.0
347     b[0] = self.r_half[0]
348     a[-1] = self.r_half[-1]
349     b[-1] = R
350     if self.n_nodes > 2:
351         a[1:-1] = self.r_half[1:-1]
352         b[1:-1] = self.r_half[1:-1]
353     self.W = 0.5 * (b**2 - a**2)
354     self.dr = float(self.dr_face.min())
355     if abs(self.W.sum() - 0.5 * R**2) > 1e-14:
356         raise RuntimeError("radial weights do not sum to R^2/2")
357
358     self.M_heat = self._heat_matrix()
359     self.M_heat_sparse = diags(
360         [np.diag(self.M_heat, -1), np.diag(self.M_heat, 0), np.diag(self.M_heat, 1)],
361         offsets=[-1, 0, 1],
362         shape=self.M_heat.shape,
363         format="csc",
364     )
365     self._heat_force_coef = (R * H) / (RHO * CP * self.W[-1])
366     self._heat_force_index = self.n_nodes - 1
367
368     def output_indices(self) -> np.ndarray:
369         """Indices of 0, 0.1, ..., 2.0 cm nodes."""
370         idx = np.empty(OUTPUT_RADII_M.size, dtype=int)
371         for j, rc in enumerate(OUTPUT_RADII_M):
372             k = int(np.argmin(np.abs(self.r - rc)))
373             if abs(self.r[k] - rc) > 1e-12:
374                 raise ValueError(f"grid misses output radius {rc}, nearest {self.r[k]}")
375             idx[j] = k
376         return idx
377
378     def _interior_face_coeff(self) -> np.ndarray:
379         return self.r_half / self.dr_face
380
381     def _heat_matrix(self) -> np.ndarray:
382         n = self.n_nodes
383         M = np.zeros((n, n))
384         face = K * self._interior_face_coeff()
385         den = RHO * CP * self.W
386         M[0, 0] = -face[0] / den[0]
387         M[0, 1] = face[0] / den[0]
388         for i in range(1, n - 1):
389             M[i, i - 1] = face[i - 1] / den[i]
390             M[i, i + 1] = face[i] / den[i]
391             M[i, i] = -(face[i - 1] + face[i]) / den[i]
392         i = n - 1
393         M[i, i - 1] = face[i - 1] / den[i]
394         M[i, i] = (-face[i - 1] - R * H) / den[i]
395         return M
396
397     def heat_rhs(self, t: float, T: np.ndarray, Ta: float) -> np.ndarray:
398         out = self.M_heat @ T
399         out[self._heat_force_index] += self._heat_force_coef * Ta
400         return out
401
402     def heat_jac(self, t: float, T: np.ndarray):
403         return self.M_heat_sparse
404
405     def heat_jac_sparsity(self):
406         n = self.n_nodes
407         ones_m = np.ones(n - 1, dtype=float)
408         ones_d = np.ones(n, dtype=float)
409         return diags([ones_m, ones_d, ones_m], offsets=[-1, 0, 1], shape=(n, n), format="csc")

```

```

410
411 def moisture_rhs(self, t: float, C: np.ndarray, Ca: float, D_const: float | None = None) -> np.ndarray:
412     if D_const is None:
413         D_node = moisture_diffusivity(C, protect=True)
414     else:
415         D_node = np.full_like(C, float(D_const))
416     D_face = harmonic_mean(D_node[:-1], D_node[1:])
417     F = -self.r_half * D_face * np.diff(C) / self.dr_face
418     Cdot = np.empty_like(C)
419     Cdot[0] = -F[0] / self.W[0]
420     Cdot[1:-1] = (F[:-1] - F[1:]) / self.W[1:-1]
421     F_surf = R * HM * (C[-1] - Ca)
422     Cdot[-1] = (F[-1] - F_surf) / self.W[-1]
423     return Cdot
424
425 def moisture_jac_sparsity(self):
426     return self.heat_jac_sparsity()
427
428 def surface_heat_influx_density(self, T_s: float, Ta: float) -> float:
429     """Heat into the solid per unit surface area, W/m^2."""
430     return H * (Ta - T_s)
431
432 def surface_moisture_outflux(self, C_s: float, Ca: float) -> float:
433     """Outward moisture flux in the normalized (no rho_d) units, m/s * kg/kg."""
434     return HM * (C_s - Ca)
435
436
437 def harmonic_mean(a: np.ndarray, b: np.ndarray) -> np.ndarray:
438     out = np.zeros_like(a, dtype=float)
439     both = (a > 0.0) & (b > 0.0)
440     out[both] = 2.0 * a[both] * b[both] / (a[both] + b[both])
441     return out
442 '''
443
444 # =====
445 # Module: reproduction_export.py
446 # =====
447 SOURCES['reproduction_export'] = r''''''Portable Excel export from validated solver arrays; no external
↳ runtime.'''
448 from copy import copy
449 import json
450 from pathlib import Path
451
452 import numpy as np
453 from openpyxl import load_workbook
454
455
456 def write_template(root, question, times, fields):
457     root = Path(root)
458     template = root / 'data' / '附件' / '附件 3' / f'result{question}.xlsx'
459     wb = load_workbook(template)
460     if len(wb.worksheets) != len(fields):
461         raise ValueError('Template sheet count does not match the output fields')
462     times = np.asarray(times)
463     if times.ndim != 1 or not np.isfinite(times).all() or not (np.diff(times) > 0).all():
464         raise ValueError('Invalid output time sequence')
465     for ws, values in zip(wb.worksheets, fields):
466         values = np.asarray(values)
467         expected = 22 if question == 4 else 21
468         if values.shape != (len(times), expected):
469             raise ValueError('Output shape does not match the requested radial grid')
470         if np.isinf(values).any() or (question != 4 and np.isnan(values).any()):
471             raise ValueError('Unexpected nonfinite output')
472         header = ws.cell(1, 1).value
473         surface = ws.cell(1, ws.max_column).value

```

```

474     number_style = copy(ws.cell(2, 2)._style)
475     header_style = copy(ws.cell(1, 2)._style)
476     ws.delete_rows(1, ws.max_row)
477     ws.append([header] + [j / 10 for j in range(21)] + ([surface] if question == 4 else []))
478     for cell in ws[1]:
479         cell._style = copy(header_style)
480     for t, row in zip(times, values):
481         ws.append([float(t)] + [None if np.isnan(v) else float(v) for v in row])
482     for row in ws.iter_rows(min_row=2, min_col=2):
483         for cell in row:
484             cell._style = copy(number_style)
485             cell.number_format = '0.0000'
486     ws.freeze_panes = 'B2'
487     wb.properties.creator = 'Team'
488     wb.properties.lastModifiedBy = 'Team'
489     dest = root / 'results' / f'result{question}.xlsx'
490     dest.parent.mkdir(parents=True, exist_ok=True)
491     wb.save(dest)
492     wb.close()
493     print(f'Excel saved: results/result{question}.xlsx', flush=True)
494
495
496 def export_saved(root, question):
497     root = Path(root)
498     validation = json.loads((root / 'results' /
499     ↪ f'q{question}_validation.json').read_text(encoding='utf-8'))
500     if question == 4:
501         if validation.get('pass') is not True or not validation.get('checks') or not all(v is True for v in
502         ↪ validation['checks'].values()):
503             raise RuntimeError('Q4 numerical validation did not pass')
504     else:
505         from delivery import require_delivery
506         require_delivery(validation, root / 'results' / f'q{question}_export_validation.json')
507         if question == 3:
508             from q3_closeout import require_q3_delivery
509             require_q3_delivery(validation, root / 'results' / 'q3_export_validation.json')
510     with np.load(root / 'results' / f'q{question}_solution.npz') as z:
511         if question in (1, 2):
512             write_template(root, question, z['times'], [z['T_out'], z['C_out']])
513         elif question == 3:
514             write_template(root, question, z['excel_times'], [z['C_out']])
515         else:
516             write_template(root, question, z['times_s'][1:], [z['C_physical'][1:]])
517     '''
518
519 # =====
520 # Module: appendix3.py
521 # =====
522 SOURCES['appendix3'] = r''''''Appendix 3 effective properties for Problems 2 and 3.'''
523 from __future__ import annotations
524
525 import numpy as np
526
527 RHO_A = 650.0
528 RHO_B = 128.0
529 CP_A = 1450.0
530 CP_B = 2736.0
531 K_A = 0.21
532 K_B = 0.38
533 D_PREFACTOR = 2.4e-3
534 D_C_COEF = 0.45
535 D_T_COEF = 3850.0
536
537 def rho(C: np.ndarray | float) -> np.ndarray | float:

```

```

537     return RHO_A + RHO_B * np.asarray(C, dtype=float)
538
539
540 def cp(C: np.ndarray | float) -> np.ndarray | float:
541     C_arr = np.asarray(C, dtype=float)
542     return CP_A + CP_B * C_arr / (1.0 + C_arr)
543
544
545 def conductivity(C: np.ndarray | float) -> np.ndarray | float:
546     C_arr = np.asarray(C, dtype=float)
547     return K_A + K_B * C_arr / (1.0 + C_arr)
548
549
550 def heat_capacity(C: np.ndarray | float) -> np.ndarray | float:
551     return rho(C) * cp(C)
552
553
554 def conductivity_deriv(C: np.ndarray | float) -> np.ndarray | float:
555     C_arr = np.asarray(C, dtype=float)
556     return K_B / (1.0 + C_arr) ** 2
557
558
559 def heat_capacity_deriv(C: np.ndarray | float) -> np.ndarray | float:
560     C_arr = np.asarray(C, dtype=float)
561     return RHO_B * cp(C_arr) + rho(C_arr) * CP_B / (1.0 + C_arr) ** 2
562
563
564 def moisture_diffusivity_q2(
565     C: np.ndarray | float,
566     T_celsius: np.ndarray | float,
567     *,
568     protect: bool = True,
569 ) -> np.ndarray | float:
570     """D(C,T) with T in Celsius; Kelvin is used only inside the Arrhenius factor."""
571     C_arr = np.asarray(C, dtype=float)
572     T_arr = np.asarray(T_celsius, dtype=float)
573     out = np.zeros(np.broadcast(C_arr, T_arr).shape, dtype=float)
574     C_b, T_b = np.broadcast_arrays(C_arr, T_arr)
575     positive = C_b > 0.0
576     if np.any(positive):
577         Tk = T_b[positive] + 273.15
578         out[positive] = D_PREFACTOR * np.exp(-D_C_COEF / C_b[positive]) * np.exp(-D_T_COEF / Tk)
579     if not protect and np.any(~positive):
580         raise ValueError("q2 diffusivity evaluated at nonpositive C without protection")
581     if np.ndim(C) == 0 and np.ndim(T_celsius) == 0:
582         return float(np.asarray(out))
583     return out
584
585
586 def diffusivity_partials(C: np.ndarray, T_celsius: np.ndarray) -> tuple[np.ndarray, np.ndarray,
↪ np.ndarray]:
587     D = np.asarray(moisture_diffusivity_q2(C, T_celsius, protect=True), dtype=float)
588     C_arr = np.asarray(C, dtype=float)
589     T_arr = np.asarray(T_celsius, dtype=float)
590     dD_dC = np.zeros_like(D)
591     dD_dT = np.zeros_like(D)
592     ok = C_arr > 0.0
593     dD_dC[ok] = D[ok] * (D_C_COEF / C_arr[ok] ** 2)
594     Tk = T_arr + 273.15
595     dD_dT[ok] = D[ok] * (D_T_COEF / Tk[ok] ** 2)
596     return D, dD_dC, dD_dT
597 '''
598
599 # =====
600 # Module: radial_coupled.py

```

```

601 # =====
602 SOURCES['radial_coupled'] = r''''Coupled variable-property radial FVM for Problem 2.'''
603 from __future__ import annotations
604
605 import numpy as np
606 from scipy.sparse import lil_matrix
607
608 from appendix3 import conductivity, heat_capacity, moisture_diffusivity_q2
609 from radial_fvm import RadialFVM, harmonic_mean
610 from utils import H, HM, R
611
612
613 class CoupledRadialFVM:
614     def __init__(self, r: np.ndarray):
615         geom = RadialFVM(r=r)
616         self.r = geom.r
617         self.W = geom.W
618         self.r_half = geom.r_half
619         self.dr_face = geom.dr_face
620         self.n_nodes = geom.n_nodes
621         self.N = geom.N
622         self._geom = geom
623
624     def output_indices(self) -> np.ndarray:
625         return self._geom.output_indices()
626
627     def pack(self, T: np.ndarray, C: np.ndarray) -> np.ndarray:
628         return np.concatenate([T, C])
629
630     def unpack(self, y: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
631         n = self.n_nodes
632         return y[:n], y[n:]
633
634     def rhs(
635         self,
636         t: float,
637         y: np.ndarray,
638         Ta: float,
639         Ca: float,
640         *,
641         D_const: float | None = None,
642         ST: np.ndarray | None = None,
643         SC: np.ndarray | None = None,
644     ) -> np.ndarray:
645         T, C = self.unpack(y)
646         kn = np.asarray(conductivity(C), dtype=float)
647         Bn = np.asarray(heat_capacity(C), dtype=float)
648         if D_const is None:
649             Dn = np.asarray(moisture_diffusivity_q2(C, T, protect=True), dtype=float)
650         else:
651             Dn = np.full_like(C, float(D_const))
652         k_face = harmonic_mean(kn[:-1], kn[1:])
653         D_face = harmonic_mean(Dn[:-1], Dn[1:])
654         FT = -self.r_half * k_face * np.diff(T) / self.dr_face
655         FC = -self.r_half * D_face * np.diff(C) / self.dr_face
656         Tdot = np.empty_like(T)
657         Cdot = np.empty_like(C)
658         Tdot[0] = -FT[0] / (Bn[0] * self.W[0])
659         Cdot[0] = -FC[0] / self.W[0]
660         Tdot[1:-1] = (FT[:-1] - FT[1:]) / (Bn[1:-1] * self.W[1:-1])
661         Cdot[1:-1] = (FC[:-1] - FC[1:]) / self.W[1:-1]
662         FTsurf = R * H * (T[-1] - Ta)
663         FCsurf = R * HM * (C[-1] - Ca)
664         Tdot[-1] = (FT[-1] - FTsurf) / (Bn[-1] * self.W[-1])
665         Cdot[-1] = (FC[-1] - FCsurf) / self.W[-1]

```

```

666         if ST is not None:
667             Tdot = Tdot + ST / Bn
668         if SC is not None:
669             Cdot = Cdot + SC
670         return self.pack(Tdot, Cdot)
671
672     def jac_sparsity(self):
673         n = self.n_nodes
674         J = lil_matrix((2 * n, 2 * n), dtype=float)
675         for i in range(n):
676             js = [i]
677             if i > 0:
678                 js.append(i - 1)
679             if i + 1 < n:
680                 js.append(i + 1)
681             for j in js:
682                 J[i, j] = 1.0
683                 J[i, n + j] = 1.0
684                 J[n + i, j] = 1.0
685                 J[n + i, n + j] = 1.0
686         return J.tocsc()
687     '''
688
689 # =====
690 # Module: problem2.py
691 # =====
692 SOURCES['problem2'] = r''''''Problem 2: coupled variable-property radial drying for 0-3 h.'''
693 from __future__ import annotations
694
695 import json
696 import sys
697 import time
698 from pathlib import Path
699
700 import numpy as np
701 from openpyxl import Workbook
702 from scipy.integrate import solve_ivp
703
704 CODE_DIR = Path(__file__).resolve().parent
705 if str(CODE_DIR) not in sys.path:
706     sys.path.insert(0, str(CODE_DIR))
707
708 from appendix3 import (
709     conductivity,
710     conductivity_deriv,
711     diffusivity_partials,
712     heat_capacity,
713     heat_capacity_deriv,
714     moisture_diffusivity_q2,
715 )
716 from q1_analytic import BesselDuhamel, converged_reference
717 from delivery import make_gate, require_delivery
718 from radial_coupled import CoupledRadialFVM
719 from radial_fvm import graded_radial_nodes
720 from utils import (
721     C0,
722     FIGURES_DIR,
723     H,
724     HM,
725     L,
726     R,
727     RESULTS_DIR,
728     T0,
729     PchipOven,
730     load_oven_table,

```

```

731 )
732
733 T_END = 10800.0
734 OUTPUT_RADII_M = 0.001 * np.arange(21)
735 TABLE_HOURS = np.array([0.5, 1.0, 1.5, 2.0, 2.5, 3.0])
736 TABLE_TIMES = 3600.0 * TABLE_HOURS
737 TABLE_RADII_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
738 TREF = 28.0
739
740
741 def load_q2_inputs() -> tuple[PchipOven, PchipOven, np.ndarray]:
742     t, Ta, Ca = load_oven_table()
743     mask = t <= T_END + 1e-12
744     t, Ta, Ca = t[mask], Ta[mask], Ca[mask]
745     if t.size != 181 or abs(t[-1] - T_END) > 1e-12:
746         raise ValueError(f"expected 181 oven records on [0, 10800], got {t.size}, t_end={t[-1]}")
747     if abs(Ta[0] - T0) > 1e-12:
748         raise ValueError(f"Ta(0)={Ta[0]} != T0={T0}")
749     Ta_fun = PchipOven(t, Ta)
750     Ca_fun = PchipOven(t, Ca)
751     for tj, yj in zip(t, Ta):
752         if abs(float(Ta_fun(tj)) - float(yj)) > 1e-12:
753             raise RuntimeError(f"PCHIP misses Ta knot t={tj}")
754     for tj, yj in zip(t, Ca):
755         if abs(float(Ca_fun(tj)) - float(yj)) > 1e-12:
756             raise RuntimeError(f"PCHIP misses Ca knot t={tj}")
757     return Ta_fun, Ca_fun, t
758
759
760 def integrate_coupled(fvm: CoupledRadialFVM, y0, t_end, Ta_fun, Ca_fun, rtol, atol, max_step,
761                      D_const=None, source=None, t_eval=None):
762     y0 = np.asarray(y0, dtype=float)
763     n = fvm.n_nodes
764     atol_arr = np.asarray(atol, dtype=float)
765     if atol_arr.ndim == 0:
766         atol_arr = np.concatenate([np.full(n, float(atol_arr)), np.full(n, float(atol_arr) * 0.01)])
767
768     def rhs(t, y):
769         ST = SC = None
770         if source is not None:
771             ST, SC = source(t, fvm.r)
772         return fvm.rhs(t, y, float(Ta_fun(t)), float(Ca_fun(t)), D_const=D_const, ST=ST, SC=SC)
773
774     if t_eval is None:
775         t_eval = np.arange(1.0, t_end + 0.5, 1.0)
776     sol = solve_ivp(
777         rhs,
778         (0.0, float(t_end)),
779         y0,
780         method="BDF",
781         rtol=rtol,
782         atol=atol_arr,
783         max_step=max_step,
784         t_eval=t_eval,
785         dense_output=False,
786         vectorized=False,
787         jac_sparsity=fvm.jac_sparsity(),
788     )
789     if not sol.success:
790         raise RuntimeError(f"coupled BDF failed: {sol.message}")
791     times = np.concatenate([[0.0], sol.t])
792     Y = np.vstack([y0, sol.y.T])
793     return times, Y
794
795

```

```

796 def extract_output(fvm: CoupledRadialFVM, Y: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
797     n = fvm.n_nodes
798     idx = fvm.output_indices()
799     T = Y[:, :n][:, idx]
800     C = Y[:, n:][:, idx]
801     return T, C
802
803
804 def compare_on_output(A, B, times) -> dict:
805     diff = np.abs(A - B)
806     k = int(np.argmax(diff))
807     n_t, n_r = diff.shape
808     i, j = divmod(k, n_r)
809     paper_t = np.isin(times, TABLE_TIMES)
810     paper_r = slice(None, None, 5) if n_r == 21 else slice(None)
811     return {
812         "max_abs": float(diff.max()),
813         "max_time_s": float(times[i]),
814         "max_radius_cm": 0.1 * j,
815         "mean_abs": float(diff.mean()),
816         "paper_table_max_abs": float(np.max(np.abs(A[paper_t][:, paper_r] - B[paper_t][:, paper_r]))),
817     }
818
819
820 def four_decimal_stats(A, B, times) -> dict:
821     Ar, Br = np.round(A, 4), np.round(B, 4)
822     changed = Ar != Br
823     paper_t = np.isin(times, TABLE_TIMES)
824     paper_r = slice(None, None, 5) if A.shape[1] == 21 else slice(None)
825     paper_changed = Ar[paper_t][:, paper_r] != Br[paper_t][:, paper_r]
826     return {
827         "changed_cells": int(np.sum(changed)),
828         "total_cells": int(A.size),
829         "paper_changed_cells": int(np.sum(paper_changed)),
830         "paper_total_cells": int(paper_changed.size),
831         "max_abs": float(np.max(np.abs(A - B))),
832         "paper_table_max_abs": float(np.max(np.abs(A[paper_t][:, paper_r] - B[paper_t][:, paper_r]))),
833     }
834
835
836 def manufactured_fields(r: np.ndarray, t: float) -> tuple[np.ndarray, np.ndarray]:
837     x2 = (r / R) ** 2
838     T = 28.0 + 2.0 * np.exp(-t / 600.0) * (1.0 + x2)
839     C = 1.5 + 0.2 * np.exp(-t / 500.0) * (1.0 + x2)
840     return T, C
841
842
843 def manufactured_env(r_surface: float, t: float) -> tuple[float, float]:
844     T, C = manufactured_fields(np.array([r_surface]), t)
845     Ts, Cs = float(T[0]), float(C[0])
846     u = np.exp(-t / 600.0)
847     v = np.exp(-t / 500.0)
848     Tr = 4.0 / R * u
849     Cr = 4.0 / R * v * 0.1 # C amplitude 0.2 vs T amplitude 2: Cr = 0.2*v*2r/R^2 at r=R -> 0.4 v / R
850     Cr = 0.4 / R * v
851     kn = float(conductivity(Cs))
852     Dn = float(moisture_diffusivity_q2(Cs, Ts))
853     Ta = Ts + kn * Tr / H
854     Ca = Cs + Dn * Cr / HM
855     return Ta, Ca
856
857
858 class ManufacturedOven:
859     def __init__(self, kind: str):
860         self.kind = kind

```

```

861
862     def __call__(self, t):
863         Ta, Ca = manufactured_env(R, float(t))
864         return Ta if self.kind == "T" else Ca
865
866
867 def manufactured_sources(t: float, r: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
868     T, C = manufactured_fields(r, t)
869     x2 = (r / R) ** 2
870     u = np.exp(-t / 600.0)
871     v = np.exp(-t / 500.0)
872     Tt = 2.0 * (-1.0 / 600.0) * u * (1.0 + x2)
873     Ct = 0.2 * (-1.0 / 500.0) * v * (1.0 + x2)
874     Tr = 4.0 * r / (R ** 2) * u
875     Trr = np.full_like(r, 4.0 / (R ** 2) * u)
876     Cr = 0.4 * r / (R ** 2) * v
877     Crr = np.full_like(r, 0.4 / (R ** 2) * v)
878     lapT = np.empty_like(r)
879     lapC = np.empty_like(r)
880     center = r < 1e-16
881     lapT[center] = 2.0 * Trr[center]
882     lapC[center] = 2.0 * Crr[center]
883     away = ~center
884     lapT[away] = Trr[away] + Tr[away] / r[away]
885     lapC[away] = Crr[away] + Cr[away] / r[away]
886     kn = np.asarray(conductivity(C), dtype=float)
887     kp = np.asarray(conductivity_deriv(C), dtype=float)
888     Bn = np.asarray(heat_capacity(C), dtype=float)
889     D, dD_dC, dD_dT = diffusivity_partials(C, T)
890     ST = Bn * Tt - (kn * lapT + kp * Cr * Tr)
891     SC = Ct - (D * lapC + dD_dC * Cr ** 2 + dD_dT * Tr * Cr)
892     return ST, SC
893
894
895 def conservation_moisture(fvm, times, Y, Ca_fun) -> dict:
896     n = fvm.n_nodes
897     C = Y[:, n:]
898     M = C @ fvm.W
899     M0 = float(M[0])
900     CN = C[:, -1]
901     Ca = np.array([float(Ca_fun(t)) for t in times])
902     I = float(np.trapezoid(HM * (CN - Ca), times))
903     rhs = M0 - R * I
904     Mt = float(M[-1])
905     denom = max(abs(M0 - Mt), abs(R * I), 1e-18)
906     return {"M0": M0, "M_end": Mt, "rhs": rhs, "relative_residual": abs(Mt - rhs) / denom}
907
908
909 def conservation_heat(fvm, times, Y, Ta_fun) -> dict:
910     n = fvm.n_nodes
911     T = Y[:, :n]
912     C = Y[:, n:]
913     Bn = np.asarray(heat_capacity(C), dtype=float)
914     Bp = np.asarray(heat_capacity_deriv(C), dtype=float)
915     E = 2.0 * np.pi * L * np.sum(fvm.W[None, :] * Bn * (T - TREF), axis=1)
916     TN = T[:, -1]
917     Ta = np.array([float(Ta_fun(t)) for t in times])
918     Q = 2.0 * np.pi * R * L * H * float(np.trapezoid(Ta - TN, times))
919     dC = np.gradient(C, times, axis=0)
920     corr = 2.0 * np.pi * L * np.sum(fvm.W[None, :] * Bp * (T - TREF) * dC, axis=1)
921     Icorr = float(np.trapezoid(corr, times))
922     lhs = float(E[-1] - E[0] - Icorr)
923     denom = max(abs(Q), abs(lhs), 1e-12)
924     return {
925         "E_end_minus_E0": float(E[-1] - E[0]),

```

```

926         "composition_correction": Icorr,
927         "Q_surface": Q,
928         "relative_residual": abs(lhs - Q) / denom,
929     }
930
931
932 def assert_physical(T_out, C_out, Ta_fun, Ca_fun) -> dict:
933     Ta_max = float(np.max(Ta_fun.y))
934     Ca_min = float(np.min(Ca_fun.y))
935     info = {
936         "T_min": float(T_out.min()),
937         "T_max": float(T_out.max()),
938         "C_min": float(C_out.min()),
939         "C_max": float(C_out.max()),
940         "T_below_28": float(max(0.0, 28.0 - T_out.min())),
941         "T_above-Ta_max": float(max(0.0, T_out.max() - Ta_max)),
942         "C_above_C0": float(max(0.0, C_out.max() - C0)),
943         "C_below-Ca_min": float(max(0.0, Ca_min - C_out.min())),
944         "Ta_range": [float(np.min(Ta_fun.y)), Ta_max],
945         "Ca_range": [Ca_min, float(np.max(Ca_fun.y))],
946     }
947     info["T_bounds_ok"] = info["T_below_28"] <= 1e-6 and info["T_above-Ta_max"] <= 1e-4
948     info["C_bounds_ok"] = info["C_above_C0"] <= 1e-8 and info["C_below-Ca_min"] <= 1e-8
949     return info
950
951
952 def write_result_workbook(path: Path, T_out, C_out, times):
953     radii_cm = 0.1 * np.arange(21)
954     wb = Workbook()
955     for title, arr in ((" 温度", T_out), (" 水分浓度", C_out)):
956         ws = wb.create_sheet(title)
957         ws["A1"] = " 时间\\到药材中心的距离"
958         for j, rc in enumerate(radii_cm, start=2):
959             ws.cell(1, j, float(rc))
960             for i, t in enumerate(times):
961                 ws.cell(i + 2, 1, int(round(float(t))))
962                 for j in range(21):
963                     cell = ws.cell(i + 2, j + 2, float(arr[i, j]))
964                     cell.number_format = "0.0000"
965     if "Sheet" in wb.sheetnames:
966         del wb["Sheet"]
967     path.parent.mkdir(parents=True, exist_ok=True)
968     wb.save(path)
969
970
971 def write_paper_csv(path: Path, arr_out, times):
972     t_idx = {float(t): i for i, t in enumerate(times)}
973     r_idx = {round(0.1 * j, 10): j for j in range(21)}
974     lines = ["t_h," + ",".join(f"r_{rc:g}cm" for rc in TABLE_RADII_CM)]
975     for th, t in zip(TABLE_HOURS, TABLE_TIMES):
976         row = [f"{th:g}"]
977         i = t_idx[float(t)]
978         for rc in TABLE_RADII_CM:
979             j = r_idx[float(rc)]
980             row.append(f"{arr_out[i, j]:.8f}")
981         lines.append(",".join(row))
982     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
983
984
985 def setup_mpl():
986     import matplotlib
987     matplotlib.use("Agg")
988     import matplotlib.pyplot as plt
989     plt.rcParams.update({
990         "font.sans-serif": ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "DejaVu Sans"],

```

```

991         "axes.unicode_minus": False,
992         "font.size": 9,
993         "axes.labelsize": 10,
994         "legend.fontsize": 8,
995         "xtick.labelsize": 8,
996         "ytick.labelsize": 8,
997         "axes.linewidth": 0.8,
998         "pdf.fonttype": 42,
999         "ps.fonttype": 42,
1000     })
1001     return plt
1002
1003
1004 def plot_figures(plt, fvm, times, Y, T_out, C_out, Ta_fun, Ca_fun):
1005     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
1006     n = fvm.n_nodes
1007     r_cm = fvm.r * 100.0
1008     idx = {float(t): i for i, t in enumerate(times)}
1009     marks = TABLE_TIMES
1010
1011     fig, ax = plt.subplots(figsize=(6.4, 3.8))
1012     for t in marks:
1013         ax.plot(r_cm, Y[idx[float(t)], :n], lw=1.2, label=fr"$t={t/3600:g}\,\mathrm{{h}}$")
1014     ax.set_xlabel(" 半径 $r$ (cm)")
1015     ax.set_ylabel(" 温度 $T$ (°C)")
1016     ax.set_xlim(0, 2)
1017     ax.legend(frameon=False, ncol=3, loc="lower center", bbox_to_anchor=(0.5, 1.02))
1018     fig.tight_layout()
1019     fig.savefig(FIGURES_DIR / "q2_T_profiles.pdf")
1020     plt.close(fig)
1021
1022     fig, ax = plt.subplots(figsize=(6.4, 3.8))
1023     for t in marks:
1024         ax.plot(r_cm, Y[idx[float(t)], n:], lw=1.2, label=fr"$t={t/3600:g}\,\mathrm{{h}}$")
1025     ax.set_xlabel(" 半径 $r$ (cm)")
1026     ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
1027     ax.set_xlim(0, 2)
1028     ax.legend(frameon=False, ncol=2)
1029     fig.tight_layout()
1030     fig.savefig(FIGURES_DIR / "q2_C_profiles.pdf")
1031     plt.close(fig)
1032
1033     t_plot = times
1034     fig, ax = plt.subplots(figsize=(6.4, 3.8))
1035     ax.plot(t_plot / 3600.0, [Ta_fun(t) for t in t_plot], lw=1.0, ls="--", label=" 烘房 $T_a$")
1036     ax.plot(t_plot / 3600.0, T_out[:, -1], lw=1.2, label=" 表面")
1037     ax.plot(t_plot / 3600.0, T_out[:, 0], lw=1.2, label=" 中心")
1038     ax.set_xlabel(" 时间 $t$ (h)")
1039     ax.set_ylabel(" 温度 (°C)")
1040     ax.set_xlim(0, 3)
1041     ax.legend(frameon=False)
1042     fig.tight_layout()
1043     fig.savefig(FIGURES_DIR / "q2_T_center_surface.pdf")
1044     plt.close(fig)
1045
1046     fig, ax = plt.subplots(figsize=(6.4, 3.8))
1047     ax.plot(t_plot / 3600.0, C_out[:, -1], lw=1.2, label=" 表面")
1048     ax.plot(t_plot / 3600.0, C_out[:, 0], lw=1.2, label=" 中心")
1049     ax.set_xlabel(" 时间 $t$ (h)")
1050     ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
1051     ax.set_xlim(0, 3)
1052     ax.legend(frameon=False)
1053     fig.tight_layout()
1054     fig.savefig(FIGURES_DIR / "q2_C_center_surface.pdf")
1055     plt.close(fig)

```

```

1056
1057 D_c = moisture_diffusivity_q2(C_out[:, 0], T_out[:, 0])
1058 D_s = moisture_diffusivity_q2(C_out[:, -1], T_out[:, -1])
1059 fig, ax = plt.subplots(figsize=(6.4, 3.6))
1060 ax.plot(t_plot / 3600.0, D_s, lw=1.2, label="表面 $D_s$")
1061 ax.plot(t_plot / 3600.0, D_c, lw=1.2, label="中心 $D_c$")
1062 ax.set_xlabel("时间 $t$ (h)")
1063 ax.set_ylabel(r"$D$ (m$^2$/s)")
1064 ax.set_xlim(0, 3)
1065 ax.legend(frameon=False)
1066 fig.tight_layout()
1067 fig.savefig(FIGURES_DIR / "q2_D_center_surface.pdf")
1068 plt.close(fig)
1069
1070
1071 def run_mms(level: int = 4) -> dict:
1072     r = graded_radial_nodes(level)
1073     fvm = CoupledRadialFVM(r)
1074     T0f, C0f = manufactured_fields(fvm.r, 0.0)
1075     y0 = fvm.pack(T0f, C0f)
1076     Ta_fun = ManufacturedOven("T")
1077     Ca_fun = ManufacturedOven("C")
1078     times, Y = integrate_coupled(
1079         fvm, y0, 100.0, Ta_fun, Ca_fun,
1080         rtol=1e-8, atol=1e-9, max_step=0.5,
1081         source=manufactured_sources,
1082         t_eval=np.arange(1.0, 100.5, 1.0),
1083     )
1084     n = fvm.n_nodes
1085     T_num, C_num = Y[:, :n], Y[:, n:]
1086     T_ex = np.stack([manufactured_fields(fvm.r, t)[0] for t in times])
1087     C_ex = np.stack([manufactured_fields(fvm.r, t)[1] for t in times])
1088     return {
1089         "level": level,
1090         "n_nodes": n,
1091         "T_max_abs": float(np.max(np.abs(T_num - T_ex))),
1092         "C_max_abs": float(np.max(np.abs(C_num - C_ex))),
1093         "T_end_max_abs": float(np.max(np.abs(T_num[-1] - T_ex[-1]))),
1094         "C_end_max_abs": float(np.max(np.abs(C_num[-1] - C_ex[-1]))),
1095     }
1096
1097
1098 def main():
1099     t_wall0 = time.perf_counter()
1100     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
1101     Ta_fun, Ca_fun, knots = load_q2_inputs()
1102     print("oven points", knots.size, "kind", Ta_fun.kind, "Ta(10800)=", Ta_fun(T_END), "Ca(10800)=",
1103           ↪ Ca_fun(T_END))
1103     D0 = float(moisture_diffusivity_q2(C0, T0))
1104     print("initial D", D0, "k", float(conductivity(C0)), "B", float(heat_capacity(C0)))
1105
1106     print("manufactured-solution probe")
1107     mms = run_mms(4)
1108     print("MMS", mms)
1109
1110     mesh_levels = [2, 4]
1111     rtol, atol_T, atol_C, max_step = 1e-9, 1e-8, 1e-10, 1.0
1112     solutions = {}
1113     for level in mesh_levels:
1114         r = graded_radial_nodes(level)
1115         fvm = CoupledRadialFVM(r)
1116         tag = f"G{level}"
1117         print(f"\n=== {tag} nodes={fvm.n_nodes} ===")
1118         y0 = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
1119         atol = np.concatenate([np.full(fvm.n_nodes, atol_T), np.full(fvm.n_nodes, atol_C)])

```

```

1120     times, Y = integrate_coupled(fvm, y0, T_END, Ta_fun, Ca_fun, rtol, atol, max_step)
1121     times_pos = times[times > 0]
1122     Y_pos = Y[times > 0]
1123     T_out, C_out = extract_output(fvm, Y_pos)
1124     solutions[level] = dict(
1125         tag=tag, fvm=fvm, times=times_pos, Y=Y_pos, Y_full=Y, times_full=times,
1126         T_out=T_out, C_out=C_out, n_nodes=fvm.n_nodes,
1127     )
1128     print("done", tag, "T", T_out.min(), T_out.max(), "C", C_out.min(), C_out.max(),
1129           "C(t=1,s)", C_out[0, -1], "C(3h,s)", C_out[-1, -1], "C(3h,c)", C_out[-1, 0])
1130
1131     t = solutions[4]["times"]
1132     conv = {
1133         "T_G2_vs_G4": compare_on_output(solutions[2]["T_out"], solutions[4]["T_out"], t),
1134         "C_G2_vs_G4": compare_on_output(solutions[2]["C_out"], solutions[4]["C_out"], t),
1135     }
1136     fourdec = {
1137         "T_G2_vs_G4": four_decimal_stats(solutions[2]["T_out"], solutions[4]["T_out"], t),
1138         "C_G2_vs_G4": four_decimal_stats(solutions[2]["C_out"], solutions[4]["C_out"], t),
1139     }
1140     print("conv", conv)
1141     print("4dp", fourdec)
1142     paper_ok = fourdec["T_G2_vs_G4"]["paper_changed_cells"] == 0 and
1143     ↪ fourdec["C_G2_vs_G4"]["paper_changed_cells"] == 0
1144     abs_ok = conv["T_G2_vs_G4"]["max_abs"] <= 2e-5 and conv["C_G2_vs_G4"]["max_abs"] <= 2e-5
1145     need_g8 = not (paper_ok and abs_ok)
1146     if need_g8:
1147         print("G2 vs G4 not enough; running G8")
1148         level = 8
1149         r = graded_radial_nodes(level)
1150         fvm = CoupledRadialFVM(r)
1151         y0 = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
1152         atol = np.concatenate([np.full(fvm.n_nodes, atol_T), np.full(fvm.n_nodes, atol_C)])
1153         times, Y = integrate_coupled(fvm, y0, T_END, Ta_fun, Ca_fun, rtol, atol, max_step)
1154         times_pos = times[times > 0]
1155         Y_pos = Y[times > 0]
1156         T_out, C_out = extract_output(fvm, Y_pos)
1157         solutions[8] = dict(
1158             tag="G8", fvm=fvm, times=times_pos, Y=Y_pos, Y_full=Y, times_full=times,
1159             T_out=T_out, C_out=C_out, n_nodes=fvm.n_nodes,
1160         )
1161         t = solutions[8]["times"]
1162         conv["T_G4_vs_G8"] = compare_on_output(solutions[4]["T_out"], solutions[8]["T_out"], t)
1163         conv["C_G4_vs_G8"] = compare_on_output(solutions[4]["C_out"], solutions[8]["C_out"], t)
1164         fourdec["T_G4_vs_G8"] = four_decimal_stats(solutions[4]["T_out"], solutions[8]["T_out"], t)
1165         fourdec["C_G4_vs_G8"] = four_decimal_stats(solutions[4]["C_out"], solutions[8]["C_out"], t)
1166         print("conv G4 G8", conv["T_G4_vs_G8"], conv["C_G4_vs_G8"])
1167         pub_level = 8
1168         paper_ok = fourdec["T_G4_vs_G8"]["paper_changed_cells"] == 0 and
1169         ↪ fourdec["C_G4_vs_G8"]["paper_changed_cells"] == 0
1170         abs_ok = conv["T_G4_vs_G8"]["max_abs"] <= 2e-5 and conv["C_G4_vs_G8"]["max_abs"] <= 2e-5
1171     else:
1172         pub_level = 4 if fourdec["C_G2_vs_G4"]["changed_cells"] or fourdec["T_G2_vs_G4"]["changed_cells"]
1173         ↪ else 2
1174         if pub_level == 2:
1175             pub_level = 4
1176     pub = solutions[pub_level]
1177     fvm = pub["fvm"]
1178     times = pub["times"]
1179     print("publishing", pub["tag"], "paper_ok", paper_ok, "abs_ok", abs_ok)
1180
1181     print("time refinement")
1182     y0 = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
1183     atol = np.concatenate([np.full(fvm.n_nodes, atol_T / 5), np.full(fvm.n_nodes, atol_C / 5)])
1184     t2, Y2 = integrate_coupled(fvm, y0, T_END, Ta_fun, Ca_fun, rtol / 5, atol, 0.5)

```

```

1182 T2, C2 = extract_output(fvm, Y2[t2 > 0])
1183 time_sens = {"T": compare_on_output(pub["T_out"], T2, times), "C": compare_on_output(pub["C_out"], C2,
↪ times)}
1184 time_4dp = {"T": four_decimal_stats(pub["T_out"], T2, times), "C": four_decimal_stats(pub["C_out"], C2,
↪ times)}
1185 print("time sens", time_sens)
1186
1187 print("frozen-D moisture probe")
1188 D_freeze = D0
1189 Bi_m = HM * R / D_freeze
1190 C_ref, moisture_truncation = converged_reference(Bi_m, D_freeze, R, OUTPUT_RADII_M, times, Ca_fun, C0)
1191 y0f = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
1192 atol = np.concatenate([np.full(fvm.n_nodes, atol_T), np.full(fvm.n_nodes, atol_C)])
1193 tf, Yf = integrate_coupled(fvm, y0f, T_END, Ta_fun, Ca_fun, rtol, atol, max_step, D_const=D_freeze)
1194 _, C_frozen = extract_output(fvm, Yf[tf > 0])
1195 frozen = compare_on_output(C_frozen, C_ref, times)
1196 nl_vs_frozen = compare_on_output(pub["C_out"], C_frozen, times)
1197 print("frozen vs analytic", frozen)
1198 print("nonlinear vs frozen", nl_vs_frozen)
1199
1200 n = fvm.n_nodes
1201 D_field = moisture_diffusivity_q2(pub["Y"][:, n:], pub["Y"][:, :n])
1202 D_stats = {
1203     "D0": D0,
1204     "D_min": float(np.min(D_field)),
1205     "D_max": float(np.max(D_field)),
1206     "D_min_over_D0": float(np.min(D_field) / D0),
1207     "D_max_over_D0": float(np.max(D_field) / D0),
1208     "center_C_end": float(pub["C_out"][-1, 0]),
1209     "surface_C_end": float(pub["C_out"][-1, -1]),
1210     "center_T_end": float(pub["T_out"][-1, 0]),
1211     "surface_T_end": float(pub["T_out"][-1, -1]),
1212 }
1213 print("D stats", D_stats)
1214
1215 bounds = assert_physical(pub["Y_full"][:, :n], pub["Y_full"][:, n:], Ta_fun, Ca_fun)
1216 cons_C = conservation_moisture(fvm, pub["times_full"], pub["Y_full"], Ca_fun)
1217 cons_T = conservation_heat(fvm, pub["times_full"], pub["Y_full"], Ta_fun)
1218 print("bounds", bounds)
1219 print("conservation C", cons_C)
1220 print("conservation T", cons_T)
1221
1222 delivery = make_gate({
1223     "space_abs": abs_ok,
1224     "space_paper": paper_ok,
1225     "all_finite": np.isfinite(pub["Y_full"]).all(),
1226     "manufactured_solution": mms["T_max_abs"] <= 2e-5 and mms["C_max_abs"] <= 2e-5,
1227     "time_abs": time_sens["T"]["max_abs"] <= 2e-5 and time_sens["C"]["max_abs"] <= 2e-5,
1228     "time_paper": time_4dp["T"]["paper_changed_cells"] == 0 and time_4dp["C"]["paper_changed_cells"] ==
↪ 0,
1229     "temperature_bounds": bounds["T_bounds_ok"],
1230     "moisture_bounds": bounds["C_bounds_ok"],
1231     "moisture_reference_truncation": moisture_truncation["passed"],
1232     "frozen_reference_error": frozen["max_abs"] <= 2e-5,
1233     "heat_balance": cons_T["relative_residual"] <= 1e-6,
1234     "moisture_balance": cons_C["relative_residual"] <= 1e-6,
1235 })
1236 print("delivery", delivery)
1237
1238 paper_T = {f"h{th:g}_r{rc:g}": float(pub["T_out"][int(t) - 1, int(round(rc / 0.1))])
1239             for th, t in zip(TABLE_HOURS, TABLE_TIMES) for rc in TABLE_RADII_CM}
1240 paper_C = {f"h{th:g}_r{rc:g}": float(pub["C_out"][int(t) - 1, int(round(rc / 0.1))])
1241             for th, t in zip(TABLE_HOURS, TABLE_TIMES) for rc in TABLE_RADII_CM}
1242
1243 validation = {

```

```

1244     "result_version": "q2-closeout-v1",
1245     "interpolation": Ta_fun.kind,
1246     "published_mesh": pub["tag"],
1247     "published_n_nodes": pub["n_nodes"],
1248     "mesh_levels": list(solutions.keys()),
1249     "solver": "coupled-FVM+BDF",
1250     "rtol": rtol, "atol_T": atol_T, "atol_C": atol_C, "max_step": max_step,
1251     "mms": mms,
1252     "grid_convergence": conv,
1253     "four_decimal_grid": fourdec,
1254     "time_sensitivity": time_sens,
1255     "four_decimal_time": time_4dp,
1256     "delivery_gate": delivery,
1257     "reference_truncation_C": moisture_truncation,
1258     "frozen_D_moisture_vs_analytic": frozen,
1259     "nonlinear_C_vs_frozen_D": nl_vs_frozen,
1260     "D_variation": D_stats,
1261     "conservation_T_10800s": cons_T,
1262     "conservation_C_10800s": cons_C,
1263     "physical_bounds": bounds,
1264     "paper_table_T": paper_T,
1265     "paper_table_C": paper_C,
1266     "elapsed_s": time.perf_counter() - t_wall0,
1267     "python": sys.version,
1268     "end_face_2d": "skipped_by_plan",
1269 }
1270 require_delivery(validation, RESULTS_DIR / "diagnostics/q2_delivery/latest.json")
1271 np.savez(
1272     RESULTS_DIR / "q2_solution.npz",
1273     mesh_level=pub_level,
1274     n_nodes=pub["n_nodes"],
1275     times=times,
1276     r=fvm.r,
1277     output_radii_m=OUTPUT_RADII_M,
1278     checkpoint_times=np.r_[0., 1., 60., 100., TABLE_TIMES],
1279     T_checkpoints=pub["Y_full"][np.r_[0., 1., 60., 100., TABLE_TIMES].astype(int), :n],
1280     C_checkpoints=pub["Y_full"][np.r_[0., 1., 60., 100., TABLE_TIMES].astype(int), n:],
1281     T_out=pub["T_out"],
1282     C_out=pub["C_out"],
1283     interpolation=np.array(Ta_fun.kind),
1284     rtol=rtol, atol_T=atol_T, atol_C=atol_C, max_step=max_step,
1285 )
1286 write_result_workbook(RESULTS_DIR / "result2.xlsx", pub["T_out"], pub["C_out"], times)
1287 write_paper_csv(RESULTS_DIR / "q2_table_temperature.csv", pub["T_out"], times)
1288 write_paper_csv(RESULTS_DIR / "q2_table_moisture.csv", pub["C_out"], times)
1289 plt = setup_mpl()
1290 plot_figures(plt, fvm, times, pub["Y"], pub["T_out"], pub["C_out"], Ta_fun, Ca_fun)
1291
1292 (RESULTS_DIR / "q2_validation.json").write_text(
1293     json.dumps(validation, ensure_ascii=False, indent=2, default=float),
1294     encoding="utf-8",
1295 )
1296 print("elapsed_s", validation["elapsed_s"])
1297 print("wrote results to", RESULTS_DIR)
1298
1299
1300
1301 if __name__ == "__main__":
1302     main()
1303
1304
1305 # =====
1306 # Module: problem4.py
1307 # =====
1308 SOURCES['problem4'] = r'""Q4: prescribed radial shrinkage, material-coordinate FVM and BDF.

```

```

1309
1310 Run a bounded, source-preserving baseline with --level 2/4/8 [--tight].
1311 No radius extrapolation; output here is internal numerical evidence, not Excel.
1312 """
1313 from __future__ import annotations
1314
1315 import argparse
1316 import hashlib
1317 import json
1318 import platform
1319 import time
1320 from pathlib import Path
1321
1322 import numpy as np
1323 import scipy
1324 from numpy.polynomial.legendre import leggauss
1325 from openpyxl import load_workbook
1326 from scipy.integrate import solve_ivp
1327 from scipy.interpolate import PchipInterpolator
1328 from scipy.sparse import lil_matrix
1329
1330 from radial_fvm import graded_radial_nodes, harmonic_mean
1331 from utils import ATTACH_DIR, RESULTS_DIR, C0, T0, H, HM, R, load_oven_table
1332
1333 CAP = 259200.0
1334 SWITCH = 14400.0
1335 THRESHOLD = 0.15
1336 OUT = RESULTS_DIR / "q4_baseline"
1337
1338
1339 def properties(C, T):
1340     C, T = np.asarray(C), np.asarray(T)
1341     # Only protect nonphysical implicit-iteration probes; stored states are audited.
1342     Cp = np.maximum(C, 1e-12)
1343     rho = 760.0 + 90.0 * Cp
1344     cp = 1850.0 + 2150.0 * Cp / (1.0 + Cp)
1345     k = 0.12 + 0.20 * Cp / (1.0 + Cp)
1346     D = 4.2e-4 * np.exp(-0.30 / Cp) * np.exp(-3850.0 / (T + 273.15))
1347     return rho * cp, k, D
1348
1349
1350 class Inputs:
1351     def __init__(self):
1352         t, Ta, Ca = load_oven_table()
1353         if t.size != 241 or t[0] != 0 or t[-1] != SWITCH:
1354             raise ValueError("unexpected oven input coverage")
1355         self.Ta = PchipInterpolator(t, Ta, extrapolate=False)
1356         self.Ca = PchipInterpolator(t, Ca, extrapolate=False)
1357         wb = load_workbook(ATTACH_DIR / "附件 2.xlsx", read_only=True, data_only=True)
1358         radius_rows = list(wb.active.values)
1359         wb.close()
1360         a = np.asarray(radius_rows[1:], dtype=float)
1361         self.rt, self.rv = a[:, 0], a[:, 1] / 100.0
1362         if (a.shape != (145, 2) or not np.all(np.isfinite(a))
1363             or np.any(np.diff(self.rt) <= 0) or np.any(np.diff(self.rv) > 0)
1364             or self.rt[0] != 0 or self.rt[-1] != CAP or self.rv[0] != R):
1365             raise ValueError("unexpected radius input")
1366         self.radius = PchipInterpolator(self.rt, self.rv, extrapolate=False)
1367         self.audit = {"radius_records": len(a), "radius_range_m": [float(self.rv[0]), float(self.rv[-1])],
1368             "radius_cap_s": CAP, "oven_records": len(t), "radius_input_headers":
1369             ↪ list(radius_rows[0]),
1370             "radius_node_error_m": float(np.max(abs(self.radius(self.rt) - self.rv))),
1371             "oven_node_error": float(max(np.max(abs(self.Ta(t)-Ta)),
1372             ↪ np.max(abs(self.Ca(t)-Ca))))}
1373         self.audit["source_sha256"] = {

```

```

1372         name: hashlib.sha256((ATTACH_DIR / name).read_bytes()).hexdigest()
1373         for name in (" 附件 1.xlsx", " 附件 2.xlsx")}
1374
1375     def environment(self, t, late=False):
1376         if late or t > SWITCH:
1377             return 50.0, 0.05
1378         return float(self.Ta(t)), float(self.Ca(t))
1379
1380
1381 class ShrinkingFVM:
1382     def __init__(self, xi):
1383         self.xi = np.asarray(xi, dtype=float)
1384         if self.xi[0] != 0 or self.xi[-1] != 1 or np.any(np.diff(self.xi) <= 0):
1385             raise ValueError("invalid material mesh")
1386         self.n = len(xi)
1387         self.faces = (self.xi[:-1] + self.xi[1:]) / 2
1388         edges = np.r_[0., self.faces, 1.]
1389         self.w = np.diff(edges**2) / 2
1390         self.dx = np.diff(self.xi)
1391
1392     def sparsity(self):
1393         J = lil_matrix((2*self.n, 2*self.n))
1394         for i in range(self.n):
1395             for j in range(max(0, i-1), min(self.n, i+2)):
1396                 J[i, j] = J[j, self.n+i] = J[self.n+i, j] = J[self.n+i, self.n+j] = 1
1397         return J.tocsc()
1398
1399     def rhs(self, y, radius, Ta, Ca, hm=HM, ht=H):
1400         T, C = y[:self.n], y[self.n:]
1401         B, k, D = properties(C, T)
1402         FT = np.r_[0., -self.faces*harmonic_mean(k[:-1], k[1:])*np.diff(T)/self.dx,
1403                 radius*ht*(T[-1]-Ta)]
1404         FC = np.r_[0., -self.faces*harmonic_mean(D[:-1], D[1:])*np.diff(C)/self.dx,
1405                 radius*hm*(C[-1]-Ca)]
1406         Tdot = -np.diff(FT) / (radius**2 * self.w * B)
1407         Cdot = -np.diff(FC) / (radius**2 * self.w)
1408         return np.r_[Tdot, Cdot]
1409
1410
1411 def model_checks(inputs):
1412     fvm = ShrinkingFVM(np.linspace(0., 1., 41))
1413     x = fvm.xi
1414     y = np.r_[35.+10*x*x, 1.7-1.2*x*x]
1415     rr = float(inputs.radius(21600.))
1416     dy = fvm.rhs(y, rr, 50., 0.05)
1417     mass_error = abs(2*np.dot(fvm.w, dy[fvm.n:])) + 2*HM/rr*(y[-1]-0.05)
1418     B, k, D = properties(y[fvm.n:], y[:fvm.n])
1419     heat_error = abs(np.dot(fvm.w*B, dy[:fvm.n]) + H/rr*(y[fvm.n-1]-50.))
1420     # Independent dimensional-volume assembly on the same physical annuli.
1421     rp = rr*x
1422     rf = (rp[:-1]+rp[1:])/2
1423     volumes = np.diff(np.r_[0., rf, rr]**2)/2
1424     flows_T = np.r_[0., -rf*harmonic_mean(k[:-1], k[1:])*np.diff(y[:fvm.n])/np.diff(rp),
1425     ↪ rr*H*(y[fvm.n-1]-50.))
1426     flows_C = np.r_[0., -rf*harmonic_mean(D[:-1], D[1:])*np.diff(y[fvm.n:])/np.diff(rp),
1427     ↪ rr*HM*(y[-1]-0.05)]
1428     reference = np.r_[-np.diff(flows_T)/(volumes*B), -np.diff(flows_C)/volumes]
1429     fixed_error = float(np.max(abs(dy-reference)))
1430     uniform = np.r_[np.full(fvm.n, T0), np.full(fvm.n, C0)]
1431     pure = solve_ivp(lambda t, z: fvm.rhs(z, float(inputs.radius(t)), T0, C0, hm=0., ht=0.),
1432     (0., CAP), uniform, method="BDF", jac_sparsity=fvm.sparsity(),
1433     rtol=1e-9, atol=1e-11)
1434     pure_error = float(np.max(abs(pure.y-uniform[:,None])))
1435     checks = {"instant_mass_abs": float(mass_error), "instant_heat_abs": float(heat_error),
1436     "fixed_physical_operator_abs": fixed_error, "pure_shrinkage_uniform_abs": pure_error,

```

```

1435         "pure_shrinkage_success": bool(pure.success)}
1436     checks["pass"] = bool(mass_error < 1e-12 and heat_error < 1e-6 and fixed_error < 1e-10
1437                           and pure_error < 1e-10 and pure.success)
1438     return checks
1439
1440
1441 def run(level=2, tight=False):
1442     OUT.mkdir(parents=True, exist_ok=True)
1443     inputs = Inputs()
1444     basic = model_checks(inputs)
1445     if not basic["pass"]:
1446         raise RuntimeError(f"model checks failed: {basic}")
1447     fvm = ShrinkingFVM(graded_radial_nodes(level)/R)
1448     n = fvm.n
1449     rtol = 1e-9 if tight else 1e-8
1450     atol = np.r_[np.full(n, 1e-9 if tight else 1e-8), np.full(n, 1e-11 if tight else 1e-10)]
1451     label = f"G{level}" + ("_tight" if tight else "")
1452     y0 = np.r_[np.full(n, T0), np.full(n, C0)]
1453     times, states = [np.array([0.])], [y0[None,:]]
1454     segments, events, event_states = [], [], []
1455     mass_loss = 0.
1456     max_mass_error = 0.
1457     bounds = [float("inf"), -float("inf"), float("inf"), -float("inf")]
1458     radial_increase = 0.
1459     heat_residual_max = 0.
1460     wall = time.perf_counter()
1461     event = lambda t,y: float(np.max(y[n:])-THRESHOLD)
1462     event.direction = -1.
1463     event.terminal = False
1464     breaks = np.unique(np.r_[0., SWITCH, np.arange(21600., CAP+1., 21600.)])
1465     gx, gw = leggauss(4)
1466     for a,b in zip(breaks[:-1], breaks[1:]):
1467         late = a >= SWITCH
1468         def rhs(t,y):
1469             Ta,Ca = inputs.environment(t, late=late)
1470             return fvm.rhs(y, float(inputs.radius(t)), Ta, Ca)
1471         max_step = (5. if tight else 10.) if not late else (30. if tight else 60.)
1472         sol = solve_ivp(rhs, (a,b), y0, method="BDF", rtol=rtol, atol=atol,
1473                        jac_sparsity=fvm.sparsity(), max_step=max_step,
1474                        dense_output=True, events=event)
1475         if not sol.success:
1476             raise RuntimeError(sol.message)
1477         ts = np.arange(a+60., b+0.1, 60.)
1478         ys = sol.sol(ts).T
1479         times.append(ts)
1480         states.append(ys)
1481         if len(sol.t_events[0]):
1482             events.extend(sol.t_events[0].tolist())
1483             event_states.extend(sol.y_events[0].tolist())
1484         # Quadrature on accepted BDF step intervals; 4-point Gaussian rule.
1485         integral = 0.
1486         for start in range(0, len(sol.t)-1, 128):
1487             aa, bb = sol.t[start:-1][:128], sol.t[start+1:][:128]
1488             qq = ((aa+bb)[:None]/2 + (bb-aa)[:None]/2*gx).ravel()
1489             surface = sol.sol(qq)[-1]
1490             ca = np.full_like(qq, 0.05) if late else inputs.Ca(qq)
1491             loss = 2*HM/inputs.radius(qq)*(surface-ca)
1492             integral += float(np.sum(loss.reshape(-1,4)*gw*(bb-aa)[:None]/2))
1493         mass_loss += integral
1494         y0 = sol.y[:,-1]
1495         mass_error = abs(float(2*np.dot(fvm.w,y0[n:])-C0+mass_loss))
1496         max_mass_error = max(max_mass_error, mass_error)
1497         TT, CC = sol.y[:n], sol.y[n:]
1498         bounds = [min(bounds[0],float(TT.min())), max(bounds[1],float(TT.max())),
1499                 min(bounds[2],float(CC.min())), max(bounds[3],float(CC.max()))]

```

```

1500     radial_increase = max(radial_increase, float(np.max(np.diff(CC, axis=0))))
1501     # Audit sampled heat-equation weighted residual, not  $d(\rho \cdot c_p \cdot T)/dt$ .
1502     for tcheck in (a, (a+b)/2, b):
1503         z = sol.sol(tcheck)
1504         dz = rhs(tcheck, z)
1505         B, _, _ = properties(z[n:], z[:n])
1506         Ta, _ = inputs.environment(tcheck, late=late)
1507         heat_error = abs(float(np.dot(fvm.w*B, dz[:n]) + H/float(inputs.radius(tcheck))*(z[n]-Ta)))
1508         heat_residual_max = max(heat_residual_max, heat_error)
1509         segments.append({"start_s": float(a), "end_s": float(b), "steps": len(sol.t)-1,
1510                         "nfev": sol.nfev, "nlu": sol.nlu, "mass_abs_error": mass_error})
1511         print(f"{label}: {b/3600:g}h maxC={y0[n:].max():.8f} mass_err={mass_error:.2e}
1512         ↪ wall={time.perf_counter()-wall:.1f}s", flush=True)
1512     tt, yy = np.concatenate(times), np.vstack(states)
1513     meta = {"label": label, "level": level, "tight": tight, "nodes": n, "rtol": rtol,
1514            "atol_T": float(atol[0]), "atol_C": float(atol[n]), "early_max_step_s": 5. if tight else 10.,
1515            "late_max_step_s": 30. if tight else 60., "event_s": events[0] if events else None,
1516            "event_h": events[0]/3600 if events else None, "end_maxC": float(yy[-1, n:].max()),
1517            "max_mass_abs_error": max_mass_error, "mass_relative_error": max_mass_error/C0,
1518            "heat_equation_residual_max": heat_residual_max, "bounds_Tmin_Tmax_Cmin_Cmax": bounds,
1519            "max_outward_C_increase": radial_increase, "segments": segments, "input_audit": inputs.audit,
1520            "model_checks": basic, "wall_s": time.perf_counter()-wall,
1521            "python": platform.python_version(), "numpy": np.__version__, "scipy": scipy.__version__,
1522            "model": "q4-model-v1", "radius_extrapolation": False,
1523            "source_code_sha256": hashlib.sha256(Path(__file__).read_bytes()).hexdigest()}
1524     np.savez_compressed(OUT/f"{label}.npz", times_s=tt, Y=yy, xi=fvm.xi, w=fvm.w,
1525                        radius_m=inputs.radius(tt), events_s=np.array(events), event_states=np.array(event_s)
1526                        ↪ tates))
1526     (OUT/f"{label}.json").write_text(json.dumps(meta, ensure_ascii=False, indent=2), encoding="utf-8")
1527     print(json.dumps({k: meta[k] for k in
1528                       ↪ ("label", "event_h", "end_maxC", "mass_relative_error", "wall_s")}), flush=True)
1528     return meta
1529
1530
1531 if __name__ == "__main__":
1532     parser = argparse.ArgumentParser()
1533     parser.add_argument("--level", type=int, default=2)
1534     parser.add_argument("--tight", action="store_true")
1535     args = parser.parse_args()
1536     run(args.level, args.tight)
1537
1538
1539 # =====
1540 # Module: verify_q4_results.py
1541 # =====
1542 SOURCES['verify_q4_results'] = r'""""Validate Q4 at matched times; produce source tables and figures after
1543 ↪ gates.""""
1544 from __future__ import annotations
1545 import csv
1546 import json
1547 import math
1548 from pathlib import Path
1549 import numpy as np
1550 from scipy.integrate import solve_ivp
1551 from scipy.interpolate import PchipInterpolator
1552 from problem4 import OUT, Inputs, ShrinkingFVM, THRESHOLD, SWITCH
1553 from problem2 import setup_mpl
1554 from utils import RESULTS_DIR, FIGURES_DIR, REPORTS_DIR
1555
1556 def load(label):
1557     return np.load(OUT/f"{label}.npz"), json.loads((OUT/f"{label}.json").read_text(encoding="utf-8"))
1558
1559
1560 def state_at(data, meta, t, inputs):

```

```

1561     """Match a new time via short BDF restart from a saved full state, not nearest-time comparison."""
1562     times = data["times_s"]
1563     i = int(np.searchsorted(times,t,side="right")-1)
1564     if abs(times[i]-t) < 1e-8:
1565         return data["Y"][i].copy()
1566     f = ShrinkingFVM(data["xi"])
1567     n = f.n
1568     assert times[i] >= SWITCH # New event/report times are in the late constant environment.
1569     sol = solve_ivp(lambda tt,y:f.rhs(y,float(inputs.radius(tt)),50.,0.05),
1570                     (times[i],t),data["Y"][i],method="BDF",jac_sparsity=f.sparsity(),
1571                     rtol=min(meta["rtol"],1e-10),atol=np.r_[np.full(n,1e-10),np.full(n,1e-12)],
1572                     max_step=1.)
1573     if not sol.success:
1574         raise RuntimeError(sol.message)
1575     return sol.y[:, -1]
1576
1577
1578 def physical_values(xi, Y, times, inputs, spacing=.001):
1579     n = len(xi)
1580     fixed = np.arange(21)*.001 if spacing == .001 else np.arange(5)*.005
1581     radii = inputs.radius(times)
1582     out = np.full((len(times),len(fixed)+1),np.nan)
1583     for i,(row,rr) in enumerate(zip(Y,radii)):
1584         good = fixed <= rr+1e-14
1585         out[i,:len(fixed)][good] =
1586             ↳ PchipInterpolator(xi,row[n:],extrapolate=False)(np.minimum(fixed[good]/rr,1.))
1587     out[i,-1] = row[-1]
1588     return out, radii, fixed
1589
1590 def main():
1591     inputs = Inputs()
1592     labels = ["G2","G4","G8","G4_tight","G16","G8_tight"]
1593     datasets = {k:load(k) for k in labels}
1594     production, reference, tightened = "G8", "G16", "G8_tight"
1595     a,ma = datasets[production]
1596     b,mb = datasets[reference]
1597     c,mc = datasets[tightened]
1598     n,m = len(a["xi"]),len(b["xi"])
1599     if not np.array_equal(a["times_s"],b["times_s"]) or not
1600         ↳ np.allclose(a["xi"],b["xi"][:,2],atol=1e-14,rtol=0):
1601         raise RuntimeError("grids or sampling times are not nested")
1602     spatial_T = float(np.max(abs(a["Y"][:,n]-b["Y"][:,m:2])))
1603     spatial_C = float(np.max(abs(a["Y"][:,n]-b["Y"][:,m:2])))
1604     temporal_T = float(np.max(abs(a["Y"][:,n]-c["Y"][:,n])))
1605     temporal_C = float(np.max(abs(a["Y"][:,n]-c["Y"][:,n])))
1606     # Production was refined after one table cell straddled a rounding boundary.
1607     critical = ma["event_s"]
1608     t_rep = math.ceil((max(ma["event_s"],mb["event_s"],mc["event_s"])+2.)/.36)*.36
1609     checked = (production,reference,tightened)
1610     at_critical = {k:state_at(d,md,critical,inputs) for k,(d,md) in datasets.items() if k in checked}
1611     at_report = {k:state_at(d,md,t_rep,inputs) for k,(d,md) in datasets.items() if k in checked}
1612     maxima = {k:float(z[len(datasets[k][0]["xi"]):].max()) for k,z in at_report.items()}
1613     critical_max = {k:float(z[len(datasets[k][0]["xi"]):].max()) for k,z in at_critical.items()}
1614     event_C_error = max(abs(critical_max[production]-critical_max[reference]),
1615                         abs(critical_max[production]-critical_max[tightened]))
1616     empirical_margin = 2*event_C_error + 1e-8
1617     idx = a["times_s"] < t_rep
1618     tt = np.r_[a["times_s"][idx],t_rep]
1619     YY = np.vstack([a["Y"][idx],at_report[production]])
1620     C_phys,radii,fixed = physical_values(a["xi"],YY,tt,inputs)
1621     C8,_ = physical_values(b["xi"],np.vstack([b["Y"][idx],at_report[reference]]),tt,inputs)
1622     physical_C_error = float(np.nanmax(abs(C_phys-C8)))
1623     table_times = np.r_[np.arange(21600.,t_rep,21600.),t_rep]
1624     table_ids = [int(np.argmin(abs(tt-t))) for t in table_times]

```

```

1624 table_C,table_R,table_fixed = physical_values(a["xi"],YY[table_ids],table_times,inputs,spacing=.005)
1625 # Dedicated table comparison includes the actual moving surface and ignores paired domain blanks.
1626 table8,_,_ = physical_values(b["xi"],np.vstack([b["Y"][idx],at_report[reference]])(table_ids),table_ti
    ↪ mes,inputs,spacing=.005)
1627 finite_table = np.isfinite(table_C)
1628 table_changed =
    ↪ int(np.count_nonzero(np.round(table_C[finite_table],4)!=np.round(table8[finite_table],4)))
1629 bounds_pass = all(md["bounds_Tmin_Tmax_Cmin_Cmax"][2]>=-1e-10 and
    ↪ md["bounds_Tmin_Tmax_Cmin_Cmax"][3]<2.550000001 for _,md in datasets.values())
1630 gates = {
1631     "input_and_model_tests":all(md["model_checks"]["pass"] for _,md in datasets.values()),
1632     "spatial_fields_2e-5":max(spatial_T,spatial_C,physical_C_error)<2e-5,
1633     "temporal_fields_2e-5":max(temporal_T,temporal_C)<2e-5,
1634     "spatial_event_1s":abs(ma["event_s"]-mb["event_s"])<1.,
1635     "temporal_event_1s":abs(ma["event_s"]-mc["event_s"])<1.,
1636     "mass_relative_1e-6":all(md["mass_relative_error"]<1e-6 for _,md in datasets.values()),
1637     "heat_equation_residual_1e-6":all(md["heat_equation_residual_max"]<1e-6 for _,md in
    ↪ datasets.values()),
1638     "physical_bounds":bounds_pass,
1639     "strict_report_all_checked_grids":all(v<THRESHOLD for v in maxima.values()),
1640     "strict_report_empirical_margin":maxima[production]+empirical_margin<THRESHOLD,
1641     "event_within_observed_radius":t_rep<=259200.,
1642     "table_four_decimals":table_changed==0,
1643 }
1644 summary = {"version":"q4-baseline-v1","model":"q4-model-v1","production":production,
    ↪ "result_status":"PUBLISHED",
1645     "critical_s":critical,"critical_h":critical/3600., "report_s":t_rep,"report_h":t_rep/3600.,
1646     "report_radius_cm":float(inputs.radius(t_rep)*100),"report_maxima":maxima,"same_time_critic
    ↪ al_maxima":critical_max,
1647     "empirical_C_margin":empirical_margin,"event_spatial_difference_s":abs(ma["event_s"]-mb["ev
    ↪ ent_s"])),
1648     "event_time_difference_s":abs(ma["event_s"]-mc["event_s"])),
1649     "max_spatial_T_C":[spatial_T,spatial_C],"max_temporal_T_C":[temporal_T,temporal_C],
1650     "max_output_physical_C_difference":physical_C_error,"table_changed_four_decimal_cells":tabl
    ↪ e_changed,
1651     "checks":gates,"pass":all(gates.values()),"runs":{k:md for k,(_,md) in datasets.items()},
1652     "heat_validation_scope":"weighted equation residual, not full multiphase energy
    ↪ conservation",
1653     "strict_margin_scope":"empirical grid/time differences, not a rigorous PDE error bound"}
1654 (RESULTS_DIR/"q4_validation.json").write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding=
    ↪ "utf-8")
1655 if not summary["pass"]:
1656     raise RuntimeError(f"Q4 output blocked: {gates}")
1657 np.savez_compressed(RESULTS_DIR/"q4_solution.npz",times_s=tt,xi=a["xi"],Y=YY,
1658     radius_m=radii,physical_radii_m=fixed,C_physical=C_phys,
1659     table_times_s=table_times,table_C=table_C,critical_state=at_critical[production],r
    ↪ eport_state=at_report[production])
1660 def safe(v):
1661     return None if not np.isfinite(v) else float(v)
1662 rows = [[float(t)]+[safe(v) for v in row] for t,row in zip(tt[1:],C_phys[1:])]
1663 payload = {"sheet":"Sheet1","headers":[" 时间/s\\距离/cm"]+[float(r*100) for r in fixed]+[" 药材表面"],
1664     "rows":rows,"report_h":t_rep/3600.}
1665 (OUT/"excel_payload.json").write_text(json.dumps(payload,ensure_ascii=False,allow_nan=False),encoding=
    ↪ "utf-8")
1666 with (RESULTS_DIR/"q4_table6.csv").open("w",newline="",encoding="utf-8-sig") as f:
1667     writer=csv.writer(f);writer.writerow(["时间/h"]+[f"{r*100:g} cm" for r in
    ↪ table_fixed]+["药材表面","表面半径/cm"])
1668     for t,row,rr in zip(table_times,table_C,table_R):
1669         writer.writerow([f"{t/3600:.4f}"]+[f"" if np.isnan(v) else f"{v:.4f}" for v in
    ↪ row]+[f"{rr*100:.6f}"])
1670 with (RESULTS_DIR/"q4_radius_output.csv").open("w",newline="",encoding="utf-8-sig") as f:
1671     writer=csv.writer(f);writer.writerow([" 时间/s"," 半径/cm"]);writer.writerows(zip(tt,radii*100))
1672 plt=setup_mpl();FIGURES_DIR.mkdir(exist_ok=True)
1673 fig,ax=plt.subplots(figsize=(6.2,3.8))

```

```

1674 ax.plot(tt/3600,YY[:,n],label=" 中心 (全域最大值)")
1675 ax.plot(tt/3600,YY[:, -1],label=" 表面")
1676 ax.axhline(.15,color="0.3",linestyle="--",label=" 达标阈值")
1677 ax.axvline(critical/3600,color="0.6",linestyle=":")
1678 ax.set(xlabel=" 时间 / h",ylabel=" 干基含水率 / (kg/kg)",xlim=(0,t_rep/3600))
1679 ax.legend(frameon=False);fig.tight_layout()
1680 fig.savefig(FIGURES_DIR/"q4_C_history.pdf");fig.savefig(FIGURES_DIR/"q4_C_history.png",dpi=160);plt.cla()
    ↪ ose(fig)
1681 fig,ax=plt.subplots(figsize=(6.2,3.8))
1682 for t,i,rr in zip(table_times,table_ids,table_R):
1683     ax.plot(a["xi"]*rr*100,YY[i,n:],label=f"{t/3600:.2f} h")
1684 ax.set(xlabel=" 到药材中心的距离 / cm",ylabel=" 干基含水率 / (kg/kg)")
1685 ax.legend(frameon=False,ncol=3,fontsize=7);fig.tight_layout()
1686 fig.savefig(FIGURES_DIR/"q4_C_profiles.pdf");fig.savefig(FIGURES_DIR/"q4_C_profiles.png",dpi=160);plt.close(fig)
    ↪ close(fig)
1687 # Small visible table companion; this is a result record, not paper prose.
1688 headers=[" 时间/h","0 cm","0.5 cm","1.0 cm","1.5 cm","2.0 cm"," 表面"," 表面半径/cm"]
1689 lines=["# 第四附表6 (基线结果, 待审) ","", "| "+"|".join(headers)+"|",
    ↪ "| "+"|".join(["---"]*len(headers))+"|"]
1690 for t,row,rr in zip(table_times,table_C,table_R):
1691     lines.append("| "+"|".join([f"{t/3600:.4f}"]+["—" if np.isnan(v) else f"{v:.4f}" for v in
    ↪ row]+[f"{rr*100:.6f}"])+"|")
1692 lines += ["", "—表示该固定半径在当前药材区域之外。表面列对应各时刻真实外半径。末行以未舍入值通过严格阈值检查;
    ↪ 四位显示可能仍为0.1500。"]
1693 (REPORTS_DIR/"Q4_TABLE6.md").write_text("\n".join(lines)+"\n",encoding="utf-8")
1694 print(json.dumps({k:summary[k] for k in ("critical_h","report_h","report_radius_cm","report_maxima","e"
    ↪ vent_spatial_difference_s","pass")},ensure_ascii=False),flush=True)
1695
1696
1697 if __name__=="__main__":
1698     main()
1699 '''
1700
1701 def prepare():
1702     code_dir = PACKAGE_ROOT / '_runtime' / f'Q{QUESTION}' / 'code'
1703     code_dir.mkdir(parents=True, exist_ok=True)
1704     for name, source in SOURCES.items():
1705         (code_dir / (name + '.py')).write_text(source, encoding='utf-8')
1706     sys.path.insert(0, str(code_dir))
1707     for folder in ('results', 'figures', 'reports'):
1708         (PACKAGE_ROOT / folder).mkdir(exist_ok=True)
1709     os.environ.setdefault('MPLBACKEND', 'Agg')
1710     for name in SOURCES:
1711         importlib.import_module(name)
1712
1713
1714 def main():
1715     parser = argparse.ArgumentParser(description=__doc__)
1716     parser.add_argument('--check', action='store_true', help='Check imports and original inputs without
    ↪ solving')
1717     parser.add_argument('--export-only', action='store_true', help='Export existing validated NPZ arrays;
    ↪ does not solve the model')
1718     args = parser.parse_args()
1719     prepare()
1720     from utils import load_oven_table
1721     load_oven_table()
1722     if QUESTION == 4:
1723         from problem4 import Inputs
1724         Inputs()
1725     if args.check:
1726         print(f'Q{QUESTION}: imports and input checks passed', flush=True)
1727         return
1728     from reproduction_export import export_saved
1729     if args.export_only:

```

```

1730     export_saved(PACKAGE_ROOT, QUESTION)
1731     return
1732 if QUESTION == 1:
1733     import problem1
1734     problem1.main()
1735 elif QUESTION == 2:
1736     import problem2
1737     problem2.main()
1738 elif QUESTION == 3:
1739     # The original delivery gate compares Q3 with independently computed Q2.
1740     if not (PACKAGE_ROOT / 'results' / 'q2_solution.npz').exists():
1741         print('Q3: computing Q2 first for the original overlap check', flush=True)
1742         import problem2
1743         problem2.main()
1744         export_saved(PACKAGE_ROOT, 2)
1745     import q3_closeout
1746     q3_closeout.main(['--no-xlsx'])
1747 else:
1748     import problem4
1749     import verify_q4_results
1750     for level, tight in ((2, False), (4, False), (4, True), (8, False), (16, False), (8, True)):
1751         problem4.run(level, tight)
1752         verify_q4_results.main()
1753     export_saved(PACKAGE_ROOT, QUESTION)
1754     print(f'Q{QUESTION}: full recomputation and Excel export completed', flush=True)
1755
1756
1757 if __name__ == '__main__':
1758     main()

```